Kvantni celični avtomati in kvantno procesiranje

Uredila dr. Miha Janež in prof. dr. Miha Mraz

januar 2026

Predgovor

S povečevanjem gostote integriranih vezij postajajo tranzistorji vse manjše nanoelektronske naprave. Želeno delovanje tranzistorjev velikosti le nekaj nanometrov pa pričnejo ovirati kvantni pojavi in sile med elektroni. Zato so se pojavile zamisli o novih platformah, ki jih ti pojavi ne le ne bi ovirali, pač pa bi jih platforme izkoristile za procesiranje. Poleg tega lahko nove nanoelektronske naprave prinesejo še več prednosti pred današnjimi računalniki, npr. večjo gostoto, manjšo porabo energije, hitrejše delovanje itd.

Ena od takih predlaganih platform so kvantni celični avtomati (Quantum Cellular Automata - QCA). Namesto tranzistorjev vsebujejo celice z elektroni, katerih razporeditev znotraj celice določa logično stanje. Procesiranje se izvede z ustrezno razporeditvijo QCA celic, pri čemer se izkoriščata Coulombova sila in kvantno tuneliranje. Do sedaj so bile fizično implementirane le manjše QCA strukture z majhnim številom celic, ker je praktično težko izdelati QCA celice nanometrske velikosti. Možno pa je načrtovati in optimizirati večja QCA vezja s pomočjo računalniškega simulatorja.

V zadnjem času je zaradi praktične implementacije veliko pozornosti posvečene kvantnemu procesiranju z osnovnimi gradniki kvantnimi biti. Superpozicija in kvantna prepletenost omogočata novo paradigmo procesiranja, ki obljublja eksponentno pohitritev nekaterih algoritmov glede na klasične računalnike. Izgradnjo skalabilnega kvantnega računalnika trenutno otežuje zahteven nadzor neizoliranega sistema kvantnih bitov. Vendar predstavljene implementacije, množica razvitih kvantnih algoritmov in protokolov, dostopni računalniški simulatorji in možnost uporabe manjših fizičnih kvantnih računalnikov preko spleta dopuščajo preučevanje in razvoj področja.

Tako QCA kot kvantnemu procesiranju je skupna reverzibilnost. Fizično reverzibilno procesiranje brez izgube informacij se približa minimalni možni porabi energije. Kvantno procesiranje je inherentno reverzibilno, saj informacije v kvantnem bitu ni mogoče izbrisati. Primerna zasnova QCA vezja omogoča implementacijo fizično reverzibilne procesne platforme.

Zahvala

Avtorji posameznih poglavij so slušatelji predmeta *Nekonvencionalne platforme in metode procesiranja*, ki se je v študijskem letu 2025/2026 predaval na 2. stopnji univerzitetnega študija Računalništva in informatike na Fakulteti za računalništvo in informatiko Univerze v Ljubljani. Vsem študentom se zahvaljujeva za izkazani trud, ki so ga vložili v svoje prispevke.

dr. Miha Janež in prof. dr. Miha Mraz, Ljubljana, januar 2026

Kazalo

1 Kompaktna in učinkovita zasnova polnega seštevalnika-odštevalnika v QCA 17
Jakob Jerše, Nejc Krajšek, Nejc Škrbec
1.1 Uvod 17
1.1.1 Osnove QCA 18
1.1.2 Osnovne QCA logične strukture 18
1.1.3 Polni seštevalnik-odštevalnik 20
1.2 Metode 23
1.2.1 Prva implementacija polnega seštevalnika/odštevalnika . . 23
1.2.2 Druga implementacija polnega seštevalnika/odštevalnika 24
1.2.3 Tretja implementacija: Reverzibilni seštevalnik/odštevalnik 26
1.2.4 Primerjava in analiza rezultatov 28
1.3 Zaključek 29
1.4 Doprinos avtorjev 29

2 Optimizacija pretočnosti prometa z uporabo kvantnega računalnika D-Wave 31
Filip Gros, Andraž Klobučar, Marko Horvat
2.1 Uvod 31
2.2 Podjetje D-Wave Systems 32
2.2.1 D-Wave Advantage2 32
2.3 Reševanje QUBO problemov na kvantnem računalniku D-Wave 2X 34
2.3.1 Arhitektura kvantnega računalnika D-Wave 2X 34
2.3.2 Chimera graf in njegova fizična struktura 34
2.3.3 QUBO in problem nahrbtnika 35
2.3.4 Prehod iz QUBO v Isingov model 37
2.3.5 Kvantno žarenje 38
2.3.6 Rekonstrukcija rešitve 38
2.3.7 Algoritem qbsolv 39
2.4 Primer optimizacije pretoka prometa 41

2.4.1 Predstavitev problema . . . 41
2.4.2 Prevedba problema v kvantni problem . . . 41
2.4.3 Rezultati raziskave . . . 43
2.5 Zaključek . . . 45
2.6 Doprinosi avtorjev . . . 46

3 Implementacija Shorovega algoritma v več simulatorjih in primerjava . . . 47
Filip Merkan
3.1 Uvod . . . 47
3.2 Teoretično ozadje . . . 48
3.2.1 Kvantni biti in kvantna vrata . . . 48
3.2.2 Shorov algoritem . . . 49
3.2.3 Kvantna Fourierjeva transformacija . . . 50
3.2.4 Modularna eksponentiacija . . . 50
3.2.5 Iskanje periode in zvezni ulomki . . . 50
3.3 Metode . . . 51
3.3.1 Izbrani simulatorji . . . 51
3.3.2 Metodologija implementacije . . . 51
3.3.3 Implementacijski pristop . . . 52
3.3.4 Testiranje in validacija . . . 53
3.4 Rezultati . . . 53
3.4.1 Implementacija algoritma . . . 53
3.4.2 Simulacije z različnimi vhodnimi podatki . . . 54
3.4.3 Analiza kompleksnosti . . . 55
3.4.4 Primerjava simulatorjev . . . 56
3.5 Zaključek . . . 58
3.6 Razprava . . . 59
Doprinosi avtorjev . . . 59

4 Analiza ter implementacija Groverjevega algoritma . . . 61
Bernard Kučina, Dejan Jarc, Jure Zupančič, Maja Razinger
4.1 Uvod . . . 61
4.2 Osnove kvantnega računalništva . . . 62
4.2.1 Kvatni bit - qubit . . . 62
4.2.2 Kvantna superpozicija in interferenca . . . 62
4.3 Groverjev algoritem . . . 62
4.3.1 Opis algoritma . . . 63
4.3.2 Geometrijska interpretacija . . . 64
4.3.3 Analiza algoritma . . . 66
4.3.4 Implementacija algoritma . . . 67
4.3.5 Razširjeni Groverjev algoritem . . . 69
4.4 Zaključek . . . 70
4.5 Doprinos avtorjev . . . 70

5 Pregled uporabe kvantnih algoritmov v medicinih, logistiki in finančnem sektorju 71
Leon Alessio, Tilen Anzeljc, Timotej Blažič
5.1 Uvod 71
5.2 Klasifikacija kvantnih algoritmov za optimizacijo 72
5.2.1 Čisti kvantni algoritmi 72
5.2.2 Variacijski kvantni algoritmi 72
5.2.3 Kvantno navdihnjeni algoritmi (QIA) 73
5.2.4 Hibridni algoritmi (HA) 73
5.2.5 Kvantne nevronske mreže (QNN) 73
5.3 Logistika 73
5.3.1 Optimizacija poti vozil 74
5.3.2 Praktična uporaba kvantnega žarjenja za Adjuster Routing Problem pri optimizaciji poti vozil 75
5.3.3 Načrtovanje logističnega omrežja 76
5.3.4 Upravljanje skladišč 77
5.4 Finančni sektor 78
5.4.1 Optimizacija portfelja 78
5.4.2 Analiza tveganja 80
5.4.3 Vrednotenje finančnih derivatov 81
5.5 Medicina 82
5.5.1 Molekularne simulacije za načrtovanje zdravil 82
5.5.2 Rekonstrukcija in analiza medicinskih slik 84
5.5.3 Genomska in klinična analiza za personalizirano medicino 85
5.6 Zaključek 86
5.7 Doprinosi avtorjev 86

6 Predstavitev in primerjava različnih tipov kvantnih računalnikov . . . 87
Marko Delić
6.1 Metodologija 87
6.2 Tipi Kvantnih Računalnikov 88
6.2.1 Superprevodna Vezja 89
6.2.2 Ujeti Ioni 92
6.2.3 Primerjava 95
6.3 Zaključek 96

7 Quantum Walk Implementation in Qrisp 97
Jure Dežman, Aljaž Justin, Alen Leban, Matjaž Madon
7.1 Introduction 97
7.2 Related Works 98
7.3 Quantum walk 98
7.3.1 Discrete time quantum walk 99
7.3.2 Continuous-time quantum walk 101
7.4 Methods 102
7.4.1 Quantum circuits 103
7.4.2 Environment 104

7.4.3 Qrisp . . . 105
7.5 Experiments . . . 105
7.6 Results . . . 105
7.7 Discussion . . . 106

8 Implementacija in primerjava polnih seštevalnikov v kvantnih celičnih avtomatih . . . 109
Klemen Kavčič, Tomi Trošt, Pia Sotlar
8.1 Uvod . . . 109
8.1.1 Reverzibilno procesiranje . . . 109
8.1.2 QCA celica . . . 110
8.1.3 Osnovni gradniki QCA struktur . . . 111
8.1.4 XOR in 3XOR vrata . . . 112
8.1.5 Ura v QCA . . . 112
8.1.6 Križanje vodil . . . 113
8.2 Polni seštevalnik . . . 113
8.2.1 SUM . . . 114
8.2.2 Carry . . . 114
8.3 Pregled sorodnih del . . . 115
8.3.1 Ključni parametri primerjave . . . 115
8.3.2 Pregled izbranih obstoječih rešitev . . . 116
8.3.3 Grafična predstavitev obravnavanih implementacij . . . 117
8.4 Primerjalna analiza . . . 118
8.5 Naša implementacija . . . 119
8.5.1 Prva implementacija . . . 120
8.5.2 Druga implementacija . . . 120
8.6 Rezultati . . . 121
8.7 Zaključek . . . 121

9 Analiza in implementacija Simonovega algoritma . . . 125
Matevž Crček in Maks Klemenčič
9.1 Uvod . . . 125
9.2 Opis problema . . . 125
9.3 Algoritem za klasične računalnike . . . 126
9.4 Simonov algoritem . . . 127
9.4.1 Kvantni del Simonovega algoritma . . . 127
9.4.2 Klasično procesiranje Simonovega algoritma . . . 129
9.5 Implementacija Simonovnega algoritma . . . 131
9.6 Rezultati . . . 133
9.7 Zaključek . . . 134
9.8 Doprinosi avtorjev . . . 134

10 QCA-Based 4-to-2 Priority Encoder: Majority Logic vs. Reversible Toffoli Design 135
Stefan Krstevski and Žan Seničar
10.1 Introduction 135
10.2 Basic QCA Building Blocks 136
10.2.1 QCA Cell 136
10.2.2 Clocking and Signal Propagation 137
10.2.3 Inverter and 3-Input Majority Gate 137
10.2.4 Toffoli Gate 138
10.3 Digital Circuits Using Universal Gates 139
10.3.1 4-to-2 Encoder 139
10.3.2 4-to-2 Priority Encoder 139
10.4 Methods 140
10.4.1 QCADesigner 141
10.4.2 Metrics 141
10.5 Implementation of 4-to-2 Encoders Using QCA Gates 141
10.5.1 Example: 4-to-2 Encoder Using Majority Gates (Single Layer) 142
10.5.2 Example: Reversible 4-to-2 Priority Encoder with Toffoli Gates 143
10.6 Conclusion 145

11 Rekonstrukcija 3-vhodnih AND/OR QCA struktur in simulacija v QCADesigner-E 147
Nik Bažato
11.1 Uvod 147
11.2 Teoretično ozadje 148
11.2.1 Osnovni koncepti QCA 148
11.2.2 Majority gate 148
11.2.3 Omejitve večinskih struktur 148
11.3 Metodologija 149
11.3.1 Uporabljena programska oprema 149
11.3.2 Postopek rekonstrukcije 149
11.3.3 Simulacijske nastavitve 150
11.4 Rezultati simulacij 150
11.4.1 Pravilnost logičnega delovanja 150
11.4.2 Energijska disipacija: predlagana vrata 150
11.4.3 Energijska disipacija: kaskadirana majority vrata 152
11.4.4 Primerjava energijske učinkovitosti 153
11.4.5 Neujemanje rezultatov z referenčnim člankom 153
11.5 Zaključek 154

12 Implementacija kvantnih celičnih avtomatov z uporabo QCA Designer in QuTiP 155
Aljaž Kern, Nejc Drašler
12.1 Uvod 155
12.2 QCA Designer in QuTiP 156
12.3 Pregled QCA struktur 156
12.3.1 QCA celica v QCA Designer 156
12.3.2 QCA celica v QuTip 157
12.3.3 Implementacija osnovnih QCA struktur 158
12.4 Implementacije QCA vezij 162
12.4.1 Prvo QCA vezje 162
12.4.2 Drugo QCA vezje (1-bitni polni seštevalnik) 165
12.5 Primerjava 168
12.6 Zaključek 168
12.7 Doprinosi avtorjev 168

13 Implementacija XOR vrat ter polovičnega in polnega seštevalnika . . 169
Kristjan Volk in Matevž Jecl
13.1 Uvod 169
13.2 Metode 170
13.2.1 Uporabljena orodja 170
13.2.2 Predlagana QCA-vezja 170
13.2.3 Postopek implementacije in simulacije 171
13.2.4 Metrike za primerjalno analizo 171
13.3 Rezultati 171
13.3.1 2-vhodna vrata XOR 172
13.3.2 Polovični seštevalnik (*half-adder*) 173
13.3.3 Polni seštevalnik (*full-adder*) 175
13.3.4 Primerjalna analiza rezultatov 177
13.4 Zaključek 177
13.5 Doprinosi avtorjev 178

14 Analiza algoritma za računanje delnih vsot in uteženih delnih vsot amplitud kvantnih stanj 179
Domen Kastelic in Jaka Kovačič
14.1 Uvod 179
14.2 Zapis podatkov 180
14.3 Izhod algoritma 180
14.4 Algoritem za seštevanje delne vsote 181
14.4.1 Pravilnost algoritma 181
14.4.2 Primer delovanja 184
14.4.3 Uporaba za integracijo 185
14.5 Algoritem za računanje uteženih delnih vsot 187
14.6 Kompleksnost 187
14.7 Zaključek 187
14.8 Doprinosi avtorjev 189

15 Implementacija D-latch tolerantnega na napake in s Fredkinovimi vrati v QCA 191
Manca Vavpotič in Blaž Grabnar
15.1 Uvod 191
15.2 Metode 192
15.2.1 QCA 192
15.2.2 D-latch 193
15.2.3 D-latch z multiplekserjem 193
15.2.4 D-latch s Fredkinovimi vrati 194
15.2.5 QCA napake 194
15.3 Implementacija in rezultati 195
15.3.1 Implementacija D-latch z Fredkinovimi vrati 195
15.3.2 Implementacija D-latch z multiplekserjem tolerantnega na napake 197
15.3.3 Testiranje napak 198
15.3.4 Rezultati 199
15.4 Zaključek 200
15.5 Doprinos avtorjev 200

16 Analiza in implementacija algoritma Bernstein-Vazirani 201
Andraž Hribernik, Timotej Kroflič, Jan Rutar
16.1 Uvod 201
16.2 Metode 202
16.2.1 Formalna definicija problema 202
16.2.2 Klasična rešitev 202
16.2.3 Kvantni algoritem 203
16.2.4 Implementacija Bernstein-Vazirani algoritma 206
16.3 Rezultati 209
16.4 Zaključek 209
16.5 Doprinosi avtorjev 210

17 Zasnova in analiza pretvornika iz Grayeve v binarno kodo (G2B) v tehnologiji QCA 211
Javornik Anže in Matej Medved
17.1 Uvod 211
17.2 Teoretična izhodišča in metode 212
17.2.1 QCA celica in vodilo 212
17.2.2 Logična vrata 212
17.2.3 Ura in sinhronizacija 212
17.2.4 Metodologija načrtovanja 213
17.2.5 Logika G2B pretvorbe in serijska odvisnost 213
17.3 Načrtovanje in implementacija 214
17.3.1 Implementacija XOR vrat 214
17.3.2 2-bitni G2B pretvornik 214
17.3.3 3-bitni G2B pretvornik 216
17.4 Dvosmerni pretvornik z izbiro SEL (B2G/G2B) 218

17.5 Rezultati in diskusija 221
17.5.1 Simulacija delovanja 221
17.5.2 Analiza karakteristik lastnega vezja 221
17.6 Zaključek 222

18 Implementacija in analiza Shorovega algoritma za faktorizacijo števil 225
Blaž Grilj, Ana Poklukar
18.1 Uvod 225
18.2 Metode 226
18.2.1 Problem faktorizacije celih števil 226
18.2.2 Teoretično ozadje Shorovega algoritma 226
18.2.3 Opis implementacije 227
18.3 Rezultati 230
18.3.1 Vpliv izbire števila a 230
18.3.2 Vpliv števila ponovitev 231
18.3.3 Primerjava idealnega simulatorja in pravega kvantnega procesorja 232
18.3.4 Primerjava s klasičnimi metodami faktorizacije 233
18.4 Zaključek 234
18.5 Doprinosi avtorjev 235

19 Analiza enobitnih primerjalnikov v QCA 237
Maks Popović
19.1 Uvod 237
19.1.1 Enobitni primerjalnik 237
19.1.2 QCA 238
19.2 Metode 241
19.2.1 Vezje 1 241
19.2.2 Vezje 2 242
19.2.3 Vezje 3 242
19.3 Rezultati 243
19.4 Zaključek 245
19.5 Doprinosi avtorjev 246

20 Analiza in implementacija reverzibilnega mrežnega stikala na podlagi kvantnih celičnih avtomatov 247
Jan Novak Domijan
20.1 Uvod 247
20.2 Vezja 247
20.2.1 Fredkinova vrata 248
20.2.2 Križno stikalo, 2:1 multiplekser in 1:2 demultiplekser . . . 249
20.2.3 Mrežno stikalo 249
20.3 Metodologija 250
20.4 Rezultati 252
20.5 Zaključek 254

21 Primerjava kvantnih pristopov za problem N-kraljic 255
Rudi Val Arčon, Lara Anžur
21.1 Uvod 255
21.2 Teoretično ozadje 256
21.2.1 CSP in SAT 256
21.2.2 Groverjev algoritem 257
21.2.3 W-stanja 258
21.3 Metode 258
21.3.1 Problem *N*-kraljic kot CSP 258
21.3.2 Metoda 1: Uporaba SAT in Groverjevega algoritma 259
21.3.3 Metoda 2: Direktno kvantno CSP kodiranje 261
21.4 Implementacija metod in primerjava rezultatov 262
21.4.1 Knjižnice in izvorna koda 262
21.4.2 Omejitev problema 263
21.4.3 Rezultati metode 1: SAT–Grover 263
21.4.4 Rezultati metode 2: Direktno kvantno CSP kodiranje 264
21.4.5 Primerjava metod 266
21.5 Zaključek 268

22 Zasnova in analiza kvantno-celičnega reverzibilnega komparatorja . 269
Žane Bučan, Stjepan Sazonov, Gregor Bučar, Lan Pavletič
22.1 Uvod 269
22.2 QCA 270
22.2.1 Osnove 270
22.2.2 Vodila 270
22.2.3 Vrata 271
22.2.4 Ura 271
22.2.5 Pomen 272
22.3 Boolova specifikacija 1 bitnega komparatorja 272
22.4 Prvi postkus implementacije 1-bitnega komparatorja 273
22.5 Drugi postkus implementacije 1-bitnega komparatorja 275
22.6 Realizacija 2-bitnega komparatorja 277
22.7 Zaključek 280
Appendix 280

23 Primerjava simulatorjev za implementacijo Shorovega algoritma . . . 281
Katarina Perman, David Goljat
23.1 Uvod 281
23.2 Kvantni računalniki 282
23.3 Shorov algoritem 282
23.3.1 Klasični del 282
23.3.2 Kvantni del 283
23.4 Implementacija 283
23.4.1 Qiskit 283
23.4.2 Cirq 287
23.5 Doprinosi avtorjev 288

24 Seštevanje v kvantnih računalnikih z uporabo QCADesignerja-E . . . 289
Mitja Kocjančič
24.1 Uvod . . . 289
24.2 QCA Osnove . . . 290
24.2.1 Celična linija . . . 290
24.3 Klasično seštevanje . . . 291
24.3.1 Vrata AND . . . 291
24.3.2 Vrata OR . . . 291
24.3.3 Vrata XOR . . . 291
24.4 Kvantno seštevanje . . . 292
24.4.1 QCA Negator . . . 292
24.4.2 QCA majoritetna vrata . . . 292
24.5 Seštevalnik . . . 293
24.5.1 Polovični seštevalnik . . . 293
24.5.2 Polni seštevalnik . . . 294
24.6 Metode . . . 294
24.6.1 Implementacija z orodjem QCADesignerE . . . 294
24.6.2 Implementacija z Pythonom in Qiskit . . . 296
24.7 Resultati . . . 296
24.8 Zaključek . . . 298

25 Foundational Quantum Algorithms: Grover, Deutsch–Jozsa and Shor . . . 301
Miguel Serra
25.1 Introduction . . . 301
25.2 Methods . . . 302
25.2.1 Grover's Search Algorithm . . . 302
25.2.2 The Deutsch–Jozsa Algorithm . . . 303
25.2.3 Shor's Factoring Algorithm . . . 303
25.3 Results . . . 304
25.4 Discussion . . . 306
25.5 Conclusion . . . 306

Literatura . . . 307

Poglavje 1
Kompaktna in učinkovita zasnova polnega seštevalnika-odštevalnika v QCA

Jakob Jerše, Nejc Krajšek, Nejc Škrbec

Povzetek V seminarski nalogi smo analizirali in simulirali tri implementacije polnih seštevalnikov-odštevalnikov v QCA. Glavni članek [1], na podlagi katerega smo zasnovali nalogo, predstavlja kompakten in energijsko učinkovit polni seštevalnik-odštevalnik ter ga primerja z drugimi že obstoječimi implementacijami. Izmed teh smo sami izbrali še dve implementaciji in preverili pravilnost delovanja, pri čemer sta bila glavna kriterija dostopnost referenciranih člankov ter njihova kompleksnost in praktičnost simulacije dizajna v okolju QCADesigner.

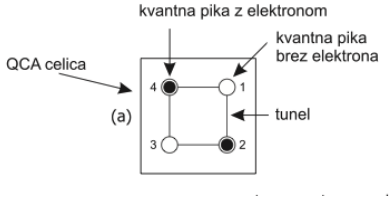
1.1 Uvod

Polni seštevalnik je eden osnovnih gradnikov digitalnih vezji in ga je mogoče enostavno realizirati s klasično CMOS tehnologijo, ki pa se sooča z omejitvami zaradi visoke porabe energije in fizičnih omejitev pri načrtovanju digitalnih vezji. Kvantni celčni avtomati (QCA) so ena od predlaganih nanotehnologij, ki ponuja nov pristop k načrtovanju digitalnih vezij z edinstvenimi lastnostmi, kot so visoka gostota integracije, nizka poraba energije in hitrost delovanja.

Polni seštevalnik sprejme tri binarne vrednosti: seštevanca in prenos iz prejšnjega koraka, ter vrne vsoto ter izhodni prenos, medtem ko polni odštevalnik odšteje eno število od drugega z upoštevanjem prenosa (posode) iz prejšnjega koraka, pri čemer vrne razliko in ustrezen prenos. Polni seštevalnik-odštevalnik pa elegantno združuje funkcionalnosti obeh vezij v enotno strukturo, ki omogoča gladko izvajanje obeh operacij.

1.1.1 Osnove QCA

Kvantni celični avtomati (QCA) predstavljajo revolucionarno paradigmo procesnih struktur, ki temelji na zakonih kvantne fizike in omogoča implementacijo logičnih operacij brez električnega toka. Osnovna enota QCA je kvantna celica, kvadratne oblike z štirimi kvantnimi pikami v kotih, ki imajo pozitiven električni naboj. V celico sta ujeta dva elektrona, ki se zaradi Coulombovih odbojnih sil postavita v eno od dveh diagonalnih leg, kar ustvari polarizacijo $P = +1$ ali $P = -1$, interpretirano kot logični vrednosti 1 ali 0. Kvantne pike so povezane s tuneli, ki omogočajo tuneliranje elektronov, pri čemer lega elektronov v eni celici vpliva na sosednje prek elektrostatičnih sil.



Slika 1.1: Kvantna celica

1.1.2 Osnovne QCA logične strukture

V *Quantum-dot Cellular Automata* (QCA) so osnovni gradniki logike:

- **Negator (NOT):** Negator obrne vhodni signal. Če je vhod $X = 1$, je izhod $X' = 0$ in obratno. V QCA se to doseže z ustrezno geometrijsko postavitvijo celic.

$$X' = \mathrm{NOT}(X) \tag{1.1}$$

  Primer negatorja v QCA je prikazan na sliki 1.2.

- **Majoritetna vrata (M):** 3-vhodna majoritetna vrata vračajo logično vrednost, ki je enaka večini vhodov S, X_1 in X_2. To je osnova za gradnjo AND, OR in drugih logičnih funkcij.

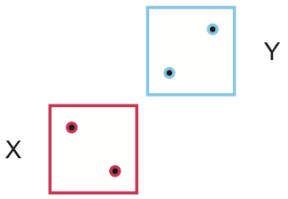
$$M(X_1, X_2, S) = X_1X_2 + X_1S + X_2S \tag{1.2}$$

  Posebni primeri:

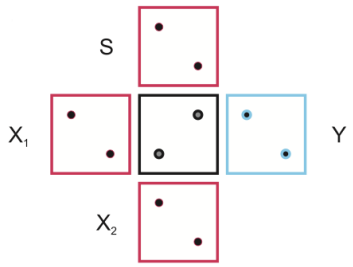
$$M(X_1, X_2, 0) = X_1X_2 \quad \text{(AND funkcija)} \tag{1.3}$$
$$M(X_1, X_2, 1) = X_1 + X_2 \quad \text{(OR funkcija)} \tag{1.4}$$

To lahko vidimo tudi v pravilnostni tabeli 1.1. Primer majoritetnih vrat v QCA je prikazen na sliki 1.3.

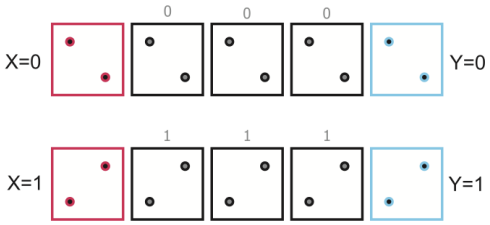
- **QCA vodilo (wire):** QCA celice so poravnane tako, da polarizacija (logični signal) prehaja iz ene celice na drugo. S tem se signali prenašajo med elementi. Primer vodila v QCA je prikazen na sliki 1.4.



Slika 1.2: Realizacija negatorja s QCA.



Slika 1.3: Relizacija majoritetnih vrat s QCA vrata.



Slika 1.4: Realizacija vodila v QCA.

Tabela 1.1: Pravilnostna tabela za 3-vhodna majoritetna vrata $M(A,B,C)$.

S	X_1	X_2	$M(X_1,X_2,S)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

1.1.3 Polni seštevalnik-odštevalnik

Polni seštevalnik

Polni seštevalnik je logično vezje, ki sprejme tri dvojiška števila: seštevanca A in B ter prenos C_{in} iz prejšnjega koraka ter vrne dve vrednosti, vsoto S in prenos C_{out}. Predstavimo ga lahko tudi z enačbama 1.5. Osnovni enobitni seštevalnik lahko ponazorimo tudi z logično shemo vezja na sliki 1.5 ali pa resničnostno tabelo 1.2. V svetu QCA lahko pri implementaciji polnega seštevalnika uporabimo majoritetna vrata, s katerimi lahko nadomestimo XOR operacijo. Tako lahko seštevalnik implementiramo zgolj z uporabo majoritetnih vrat in negatorjev kot so to storili avtorji članka [2]. Primer sheme takega seštevalnika je na sliki 1.6.

$$\text{SUM} = A \oplus B \oplus C_{in}, \quad C_{out} = AB + C_{in}(A \oplus B) \tag{1.5}$$

Zgornji enačbi lahko prevedemo v QCA implementacijo, kjer uporabimo 3-vhodna *majoritetna* (M) vrata in negatorje (NOT). Prenos C_{out} je enostavno izveden z enimi majoritetnimi vrati:

$$C_{out} = M(A,B,C_{in}) = AB + AC_{in} + BC_{in}. \tag{1.6}$$

Za izračun SUM najprej definiramo

$$X = A \oplus B, \tag{1.7}$$

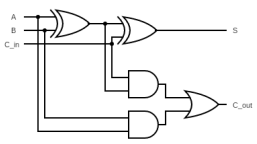
kar lahko v QCA izvedemo s kombinacijo M in NOT vrat:

$$X = M(A,B',0) + M(A',B,0). \tag{1.8}$$

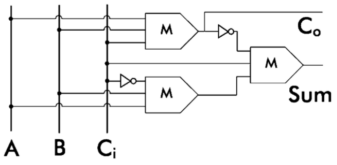
Nato izračunamo SUM kot

$$\text{SUM} = X \oplus C_{in} = M(X,C'_{in},0) + M(X',C_{in},0), \tag{1.9}$$

pri čemer X' in C'_{in} predstavljata negirana signala. Ta klasična struktura uporablja 5 majoritetnih vrat in 4 negatorje. Potrebno je opozoriti, da je primer na sliki 1.6 že optimiziran polni seštevalnik, čigar optimizacija je opisana v članku [2].



Slika 1.5: Shema vezja enobitnega polnega seštevalnika.



Slika 1.6: Shema optimiziranega QCA polnega seštevalnika. Slika je vzeta iz članka [2].

Tabela 1.2: Resničnostna tabela enobitnega polnega seštevalnika.

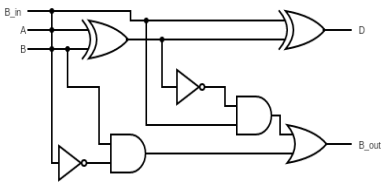
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Polni odštevalnik

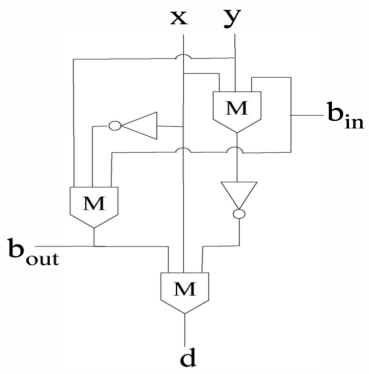
Polni odštevalnik je logično vezje, ki izvede odštevanje dveh dvojiških števil z upoštevanjem prenosa iz prejšnjega koraka. Sprejme tri vhode: bit A (minuend), bit B (subtrahend) ter prenos B_{in} ter vrne razliko D in B_{out}. Funkcionalnost lahko opišemo z enačbama 1.11. Osnovni enobitni odštevalnik ponazorimo z logično shemo vezja na sliki 1.7 ali z resničnostno tabelo 1.3. V QCA lahko podobno kot pri seštevalniku implementiramo tudi odštevalnik, kot so to npr. storili avtorji članka [3] in je prikazan na sliki 1.8. Tukaj ne bomo izpeljevali enačb za prevod v QCA, saj se prevodi izvajajo na enak način, kot smo prikazali v sekciji *Polni seštevalnik*.

$$D = A \oplus B \oplus B_{in}, \tag{1.10}$$

$$B_{out} = A'B + (A \oplus B)' B_{in} \tag{1.11}$$



Slika 1.7: Shema vezja enobitnega polnega odštevalnika.



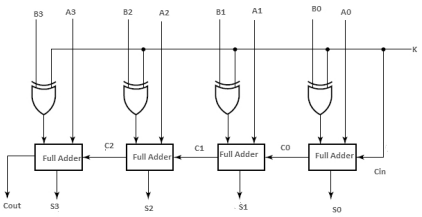
Slika 1.8: Shema optimiziranega QCA polnega odštevalnika. Slika je vzeta iz članka [3].

Tabela 1.3: Resničnostna tabela enobitnega polnega odštevalnika.

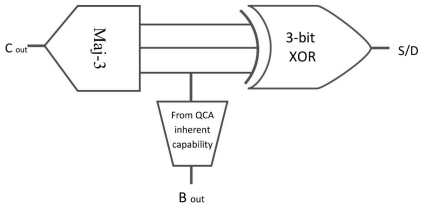
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Polni seštevalnik-odštevalnik

Polni seštevalnik-odštevalnik v QCA tehnologiji je kompaktno in energetsko učinkovito vezje, ki združuje funkcionalnosti polnega seštevalnika in odštevalnika v enotni strukturi. V najbolj preprosti obliki binarnega seštevalnika-odštevalnika združimo seštevalnik in odštevalnik, med njima pa preklapljamo z nastavljanjem vrednosti spremenljivke K ($K = 0$ za seštevanje, $K = 1$ za odštevanje). Vezje polnega seštevalnika-odštevalnika za n-bitno binarno število temelji na n polnih seštevalnikih, pri čemer bite drugega operanda skupaj s K transformiramo preko XOR vrat in nato pošljemo v polni seštevalnik skupaj z istoležečim bitom drugega operanda, kot je vidno na shemi 1.9. Tudi polne seštevalnike-odštevalnike je seveda mogoče implementirati v QCA paradigmi. Eden od takih primerov je seštevalnik-odštevalnik iz članka [1], ki si ga bomo natančneje pogledali v nadaljevanju te naloge, njegovo vezje, vidno na sliki 1.10 pa sestavljajo XOR vrata, majoritetna vrata in negatorji.



Slika 1.9: Shema vezja 4-bitnega polnega odštevalnika. Slika vzeta iz [4].



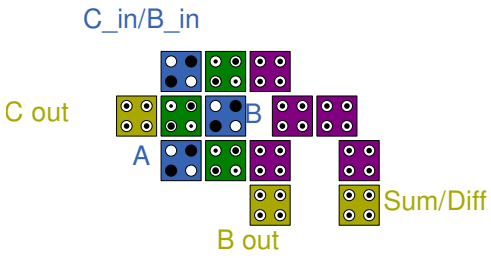
Slika 1.10: Shema optimiziranega QCA polnega seštevalnika-odštevalnika. Slika je vzeta iz članka [1].

1.2 Metode

V nadaljevanju se osredotočamo na implementacijo in analizo treh različnih arhitektur polnih seštevalnikov in odštevalnikov z uporabo QCA. Za verifikacijo predlaganih vezij smo uporabili simulacijsko okolje QCADesigner, pridobljene rezultate pa smo kritično ovrednotili glede na teoretična izhodišča in primerljive študije iz literature."

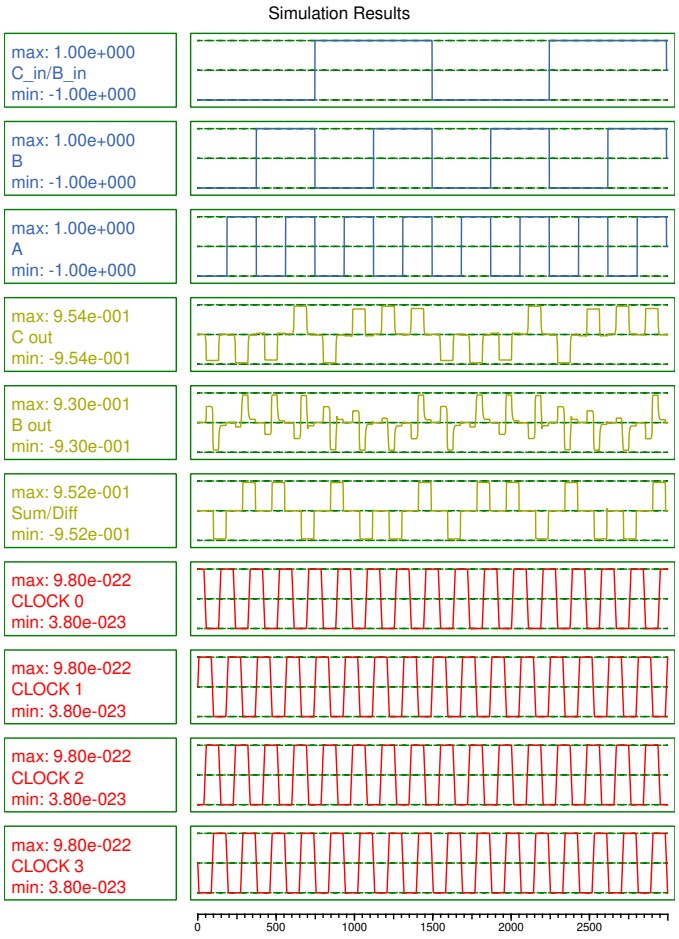
1.2.1 Prva implementacija polnega seštevalnika/odštevalnika

Najprej smo z orodjem QCADesigner implementirali kompakten polni seštevalnik in odštevalnik iz članka [1]. Postavitev celic je predstavljena na sliki 1.11. Za realizacijo smo uporabili le 14 QCA celic in dve urini fazi, kar predstavlja izjemno optimizirano in površinsko učinkovito strukturo. Vhodni celici A in B predstavljata operanda, celica C_{in}/B_{in} pa vhodni prenos oziroma izposojo. Vezje ima tri izhodne celice: Sum/Diff za rezultat operacije, C_{out} za izhodni prenos pri seštevanju in B_{out} za izhodno izposojo pri odštevanju.



Slika 1.11: Realizacija kompaktnega seštevalnika/odštevalnika s QCA

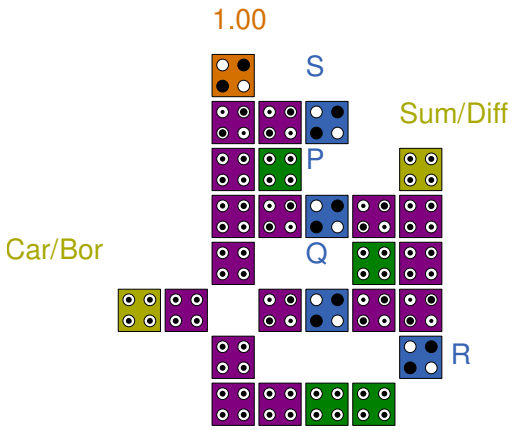
Rezultati simulacije so prikazani na sliki 1.12. Primerjava teh rezultatov s pravilnostno tabelo potrjuje, da vezje pravilno opravlja obe aritmetični funkciji hkrati. Signali so stabilni, pri čemer izhod Sum/Diff pravilno preklaplja glede na vhodne vrednosti, ločena izhoda C_{out} in B_{out} pa pravilno indicirata prenos oziroma izposojo v ustrezni logični kombinaciji.



Slika 1.12: Rezultati simulacije prve implementacije s QCA

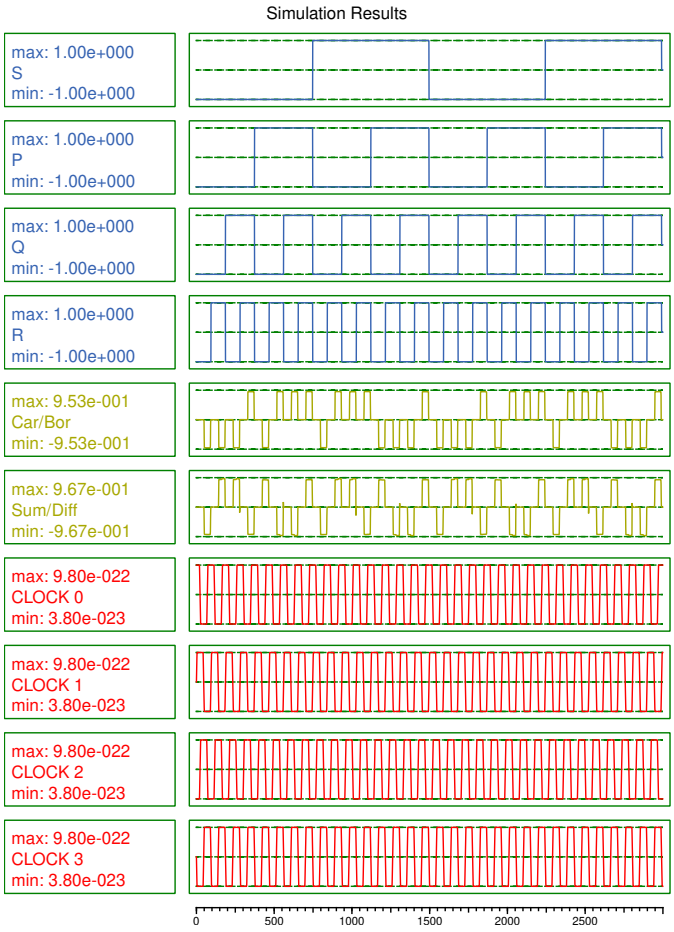
1.2.2 Druga implementacija polnega seštevalnika/odštevalnika

Za drugo različico implementacije smo izbrali vezje iz članka [5], ki temelji na robustni uporabi večinskih vrat. Postavitev celic je predstavljena na sliki 1.13. Struktura je v primerjavi s prvo implementacijo prostornejša, uporablja pa jasno ločene urine cone za zanesljiv prenos informacij. Vhodi so označeni s S, P, Q in R, pri čemer fiksna polarizacija 1.00 služi za nastavitev delovanja logičnih vrat. Izhod Sum/Diff podaja rezultat, izhod Car/Bor pa informacijo o prenosu ali izposoji.



Slika 1.13: Realizacija robustnega seštevalnika na osnovi večinskih vrat

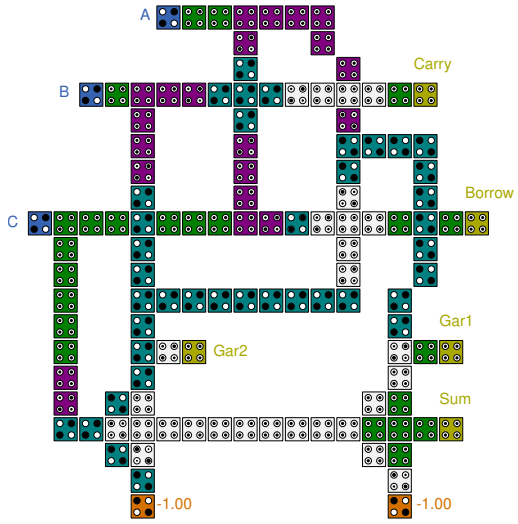
Rezultati simulacije, prikazani na sliki 1.14, kažejo izjemno stabilnost signalov. Izhodne vrednosti dosegajo visoko stopnjo polarizacije (okoli 9.67×10^{-1}), kar nakazuje na odlično odpornost vezja proti šumu in motnjam. Ujemanje s pričakovano logiko je popolno v vseh kombinacijah vhodnih vektorjev.



Slika 1.14: Rezultati simulacije druge implementacije s QCA

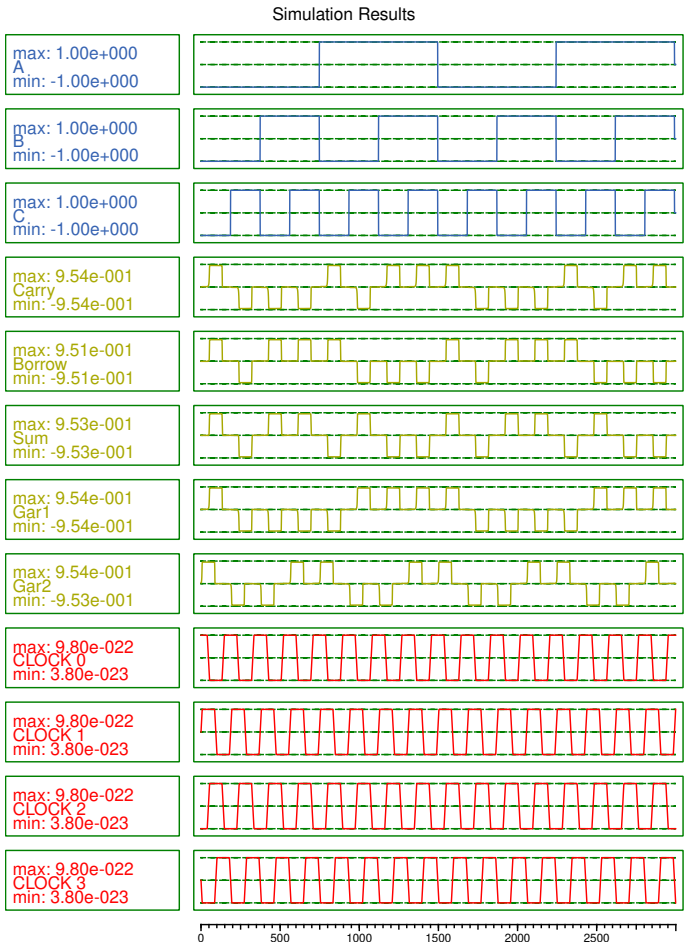
1.2.3 Tretja implementacija: Reverzibilni seštevalnik/odštevalnik

Za tretjo različico smo implementirali vezje iz članka [6], ki predstavlja reverzibilni polni seštevalnik in odštevalnik (RFAS). Postavitev celic je predstavljena na sliki 1.15. Gre za kompleksnejšo strukturo, ki vključuje križanja signalov v eni ravnini. Vhodi A, B in C se preslikajo v enako število izhodov, kar je zahteva reverzibilne logike. Poleg funkcionalnih izhodov *Sum*, *Carry* in *Borrow* vezje vsebuje še dva neuporabna (garbage) izhoda, označena z *Gar*1 in *Gar*2, ki sta nujna za ohranitev reverzibilnosti operacij.



Slika 1.15: Realizacija reverzibilnega seštevalnika/odštevalnika (RFAS)

Rezultati simulacije so prikazani na sliki 1.16. Analiza valovnih oblik potrjuje pravilno delovanje vseh logičnih izhodov. Posebnost te implementacije je, da so tudi neuporabni izhodi ($Gar1, Gar2$) stabilni in sinhronizirani z ostalimi signali, kar potrjuje pravilno zasnovo brez izgube informacij. Vezje uspešno realizira hkratni izračun vsote, prenosa in izposoje znotraj iste strukture.



Slika 1.16: Rezultati simulacije reverzibilne implementacije s QCA

1.2.4 Primerjava in analiza rezultatov

Na podlagi izvedenih simulacij in analize topologij smo primerjali vse tri implementacije polnih seštevalnikov/odštevalnikov. Primerjava ključnih parametrov je zbrana v tabeli 1.4.

Prva implementacija se izkaže kot najučinkovitejša z vidika porabe prostora, saj zaseda minimalno število celic (14) in dosega najnižjo latenco. Primerna je za aplikacije, kjer je ključna gostota integracije.

Druga implementacija, ki temelji na klasični logiki večinskih vrat, zahteva večjo površino, vendar rezultati simulacije kažejo na najvišjo stabilnost signalov (najvišja dosežena polarizacija). To vezje je najbolj primerno za sisteme, kjer je prioriteta robustnost delovanja pred samo velikostjo vezja.

Tretja implementacija predstavlja poseben razred vezij, ki temeljijo na reverzibilni logiki. Čeprav je struktura kompleksnejša in vsebuje dodatne neuporabne izhode (t.i. *Garbage outputs*), je to edina implementacija, ki teoretično omogoča delovanje z minimalno disipacijo energije. Simulacija je potrdila, da vezje pravilno ohranja informacijsko celovitost signalov skozi celoten proces obdelave.

Tabela 1.4: Primerjava implementiranih vezij

Lastnost	**Impl. 1**	**Impl. 2**	**Impl. 3**
Tip vezja	Kompakten	Robusten (MV)	Reverzibilen
Število celic	14	45	120
Latenca	0.5 cikla	1.0 cikel	1.25 cikla
Max. Polarizacija	≈ 0.952	≈ 0.967	≈ 0.954

1.3 Zaključek

V sklopu seminarske naloge smo uspešno analizirali in simulirali tri različne topologije polnih seštevalnikov-odštevalnikov v tehnologiji kvantnih celičnih avtomatov (QCA). Rezultati simulacij v okolju QCADesigner so potrdili pravilnost delovanja vseh treh vezij in razkrili ključne kompromise pri načrtovanju. Medtem ko prva implementacija izstopa po izjemni prostorski učinkovitosti in nizki latenci, druga ponuja večjo stabilnost signala in robustnost, tretja pa z uporabo reverzibilne logike naslavlja problem energetske disipacije. Zaključimo lahko, da tehnologija QCA ponuja obetavne rešitve za preseganje omejitev trenutne CMOS tehnologije, pri čemer je izbira specifične arhitekture odvisna od ciljev načrtovanja, najsi gre za gostoto integracije ali minimalno porabo energije.

1.4 Doprinos avtorjev

Vsi avtorji so enakovredno prispevali k pregledu in izbiri literature, pripravi poročila in predstavitve ter k načrtovanju vezij v QCA Designerju, pri čemer smo večino dela izvedli skupaj, bodisi v živo bodisi prek spleta.

Poglavje 2
Optimizacija pretočnosti prometa z uporabo kvantnega računalnika D-Wave

Filip Gros, Andraž Klobučar, Marko Horvat

Povzetek

V seminarski nalogi obravnavamo uporabo kvantnega računalnika podjetja D-Wave Systems za optimizacijo pretočnosti prometa. V prvem delu predstavimo razvoj sistemov D-Wave skozi čas. Nadaljujemo s podrobno obravnavo arhitekture sistema D-Wave 2X, specifičnosti topologije Chimera ter opisi teoretičnega ozadja, ki vključuje matematično modeliranje v obliki QUBO (Quadratic Unconstrained Binary Optimization), pretvorbo v Isingov model in kvantno žarenje. Podrobno preučimo hibridni algoritem qbsolv za reševanje večjih QUBO problemov. Praktično delovanje teh metod ilustriramo na primeru raziskave optimizacije prometnih poti, ki temelji na podatkih iz Pekinških taksijev. Rezultati obravnavane študije kažejo, da lahko kvantni žarilniki v realnem času uspešno prerazporedijo prometne tokove in zmanjšajo zastoje bolj učinkovito kot trenutne hevristične metode, vendar njihova praktična uporaba zaradi trenutnih strojnih in modelnih omejitev ostaja omejena.

2.1 Uvod

Sodobna družba se sooča z vse večjo potrebo po obdelavi ogromnih količin podatkov in optimizaciji kompleksnih sistemov, od logistike in financ do upravljanja energentov in prometa. Mnogi izmed teh problemov spadajo v razred NP-težkih problemov, kjer čas iskanja optimalne rešitve na klasičnih računalnikih z velikostjo problema eksponentno narašča. Ko se približujemo fizikalnim omejitvam Moorovega zakona, postaja iskanje alternativnih računskih pristopov nujno. Ena bolj obetavnih tehnologij na tem področju je kvantno računalništvo.

V kontekstu optimizacije izstopa pristop kvantnega žarenja (ang. *quantum annealing*), ki ga komercialno razvija podjetje D-Wave Systems [7]. Za razliko od kvantnih računalnikov na osnovi logičnih vrat, ki so namenjeni splošnemu računanju, so kvantni žarilniki namenske naprave, zasnovane za iskanje globalnega minimuma

energijske funkcije [8]. To jih naredi posebej primerne za reševanje kombinatoričnih optimizacijskih problemov, kot so problem trgovskega potnika, problem nahrbtnika ali problem usmerjanja vozil [9].

V tej seminarski nalogi se osredotočamo na pregled uporabe te tehnologije za optimizacijo pretočnosti prometa. Prometni zastoji predstavljajo velik globalni izziv in povzročajo gospodarsko škodo ter onesnaževanje okolja[10]. Dinamična narava prometa zahteva rešitve v realnem času, kar je za klasične algoritme pri velikem številu vozil pogosto neizvedljivo.

2.2 Podjetje D-Wave Systems

Podjetje D-Wave Quantum Inc., ustanovljeno leta 1999 v Burnabyju (Kanada), velja za pionirja komercialne uporabe kvantnih računalnikov — naprav, ki temeljijo na principu kvantnega žarenja (ang. quantum annealing) in so usmerjene v reševanje optimizacijskih problemov. Ena izmed prvih izvedenk njihovega sistema je bilo javno demonstrirano že leta 2007 (projekt "Orion", 16 qubitov)[7] in kasneje nadgrajeno do 28-qubitnega modula v istem letu.

Leta 2011 je D-Wave predstavil sistem D-Wave One, opremljen s 128 qubiti, ki velja za prvi komercialno dostopen kvantni računalnik. Razvoj se je hitro nadaljeval in že leta 2013 je podjetje predstavilo D-Wave Two z 512 qubiti, kar je predstavljalo pomemben mejnik v zmogljivosti kvantnega žarenja.

Leta 2015 je sledila nadgradnja v obliki D-Wave 2X 2.3.1, procesorja z več kot 1000 razpoložljivimi qubiti, osnovanega na Chimera grau. Čeprav je čip fizično vseboval 2048 qubitov, je bila zaradi proizvodnih variacij in lastnosti nadprevodnih vezij funkcionalna približno polovica, med 1090 in 1150 qubiti. V primerjalnih testih[7] se pri njem niso osredotočali na doseganje t. i. "kvantne prednosti", temveč na primerjavo konstantnih časovnih faktorjev glede na klasične algoritme. Pri splošnih optimizacijskih nalogah je QPU (kvantna procesna enota) računalnika D-Wave 2X dosegel približno 15-kratno pohitritev glede na visokozmogljive enojedrne CPU-je — kar bi po analogiji ustrezalo zmogljivosti približno 30 klasičnih procesorskih jeder pri dobro paraleliziranih nalogah.

Nadaljnje generacije — na primer D-Wave 2000Q ($\approx$ 2 000 qubitov) in sistem D-Wave Advantage (5 760+ qubitov, nov Pegasus graf) — so dodatno razširile možnosti za optimizacijske in paralelne procese.

2.2.1 D-Wave Advantage2

Advantage2™ predstavlja najnovejšo, šesto generacijo kvantnega računalnika podjetja D-Wave, uradno predstavljeno leta 2025. Sistem vključuje več kot 4.400 qubi-

tov [1] in temelji na novi arhitekturni topologiji (grafu) Zephyr™, ki zagotavlja do 20-krat večjo povezljivost qubitov v primerjavi s starejšimi generacijami (Chimera in Pegasus). Povečana povezljivost omogoča učinkovitejše modeliranje kompleksnih optimizacijskih nalog, saj lahko več qubitov neposredno sodeluje v problemu z manj potrebe po minor-embedding postopkih.



Slika 2.1: Kvantni računalnik Advantage2, predstavljen javnosti 20.5.2025.

V primerjavi s prejšnjim sistemom Advantage prinaša Advantage2 pomembne izboljšave v zmanjšanju šuma (približno 75 % manj), kar neposredno vpliva na bolj stabilno delovanje in podaljšano koherenco, ki je približno dvojno daljša od predhodne generacije. Procesor deluje v izjemno nadzorovanih pogojih blizu absolutne ničle, podobno kot prejšnje generacije, vendar s še bolj dovršenim nadzorom nad kvantnim okoljem, kar izboljšuje zanesljivost reševanja kvantnih optimizacijskih problemov.

Energetska učinkovitost ostaja presenetljivo konstantna kljub večjemu številu qubitov in večji računski zmogljivosti — celoten sistem porabi približno 12,5 kW, kar je približno enako kot pri prejšnjih generacijah. D-Wave Advantage2 je dostopen tako prek oblaka (platforma Leap[11]) kot tudi v različici za namestitev pri

[1] Število fizičnih qubitov (npr. 5 760 pri sistemu D-Wave Advantage) ne pomeni, da je toliko qubitov dejansko programirljivih ali neposredno uporabnih pri reševanju problema; zaradi omejene povezljivosti in potrebe po *minor-embeddingu* je efektivno število logičnih qubitov bistveno manjše. Zato lahko novejša arhitektura Advantage2, kljub manjšemu skupnemu številu fizičnih qubitov, zaradi bistveno večje povezljivosti (z uporabo Zephyr grafa) in manjšega šuma dejansko omogoča reševanje večjih ali kompleksnejših problemov.

naročniku (on-premises), kar ga uvršča med komercialno najbolj dostopne kvantne sisteme na svetu.

Z vidika aplikacij je Advantage2 zasnovan predvsem za reševanje kompleksnih problemov optimizacije, razporejanja in simulacij. Zaradi svoje arhitekture se izkaže za primernega za področja, kot so logistika, bioinformatika, finančna analiza in tudi modeli pretočnosti prometa, kar bomo kasnje podrobneje raziskali. D-Wave poroča o pomembnih napredkih, vključno z demonstracijami, ki kažejo potencialno obliko kvantne prednosti pri simulaciji magnetnih materialov — nalog, ki so za klasične sisteme izjemno zahtevne ali neizvedljive.

2.3 Reševanje QUBO problemov na kvantnem računalniku D-Wave 2X

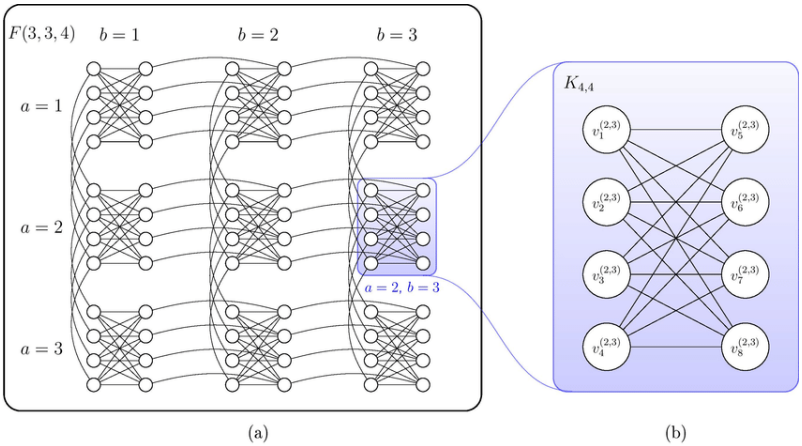
2.3.1 Arhitektura kvantnega računalnika D-Wave 2X

Arhitektura kvantnega računalnika D-Wave 2X temelji na nadprevodnih tokovnih krogih, ki pri izjemno nizkih temperaturah delujejo kot kvantni biti (qubiti). Sistem je hlajen na temperature približno 15 mK, kar je zgolj nekaj stotink stopinje nad absolutno ničlo. Tako nizke temperature so nujne, saj kvantna stanja nadprevodnih qubitov izjemno občutljivo reagirajo na toplotni in elektromagnetni šum. Že en sam dodatni fonon ali foton lahko povzroči neželen prehod med energijskimi stanji, kar bi uničilo koherenco ter s tem pravilno izvedbo kvantnega žarenja [7].

Qubit v D-Wave sistemih ni tradicionalni spin elektrona, temveč makroskopski kvantni objekt – nadprevodni tok v Josephsonovi zanki (tokovni qubit). [12] Kljub temu se obnaša matematično enako kot spinski sistem z dvema osnovnima stanjema, zato lahko Hamiltonian naprave zapišemo v obliki Isingovega modela.

2.3.2 Chimera graf in njegova fizična struktura

Qubiti v D-Wave 2X niso poljubno vezani. Razporejeni so v mrežo osnovnih celic, imenovanih *Chimera cells*, kjer vsaka celica tvori bipartiten graf $K_{4,4}$. V posamezni celici so štiri ”vodoravni”in štirje ”navpični”qubiti, ki so med seboj popolnoma bipartitno povezani. Celice so nato medsebojno povezane v dvodimenzionalno mrežo, kar tvori fizično topologijo procesorja [13].



Slika 2.2: Struktura Chimera grafa v arhitekturi D-Wave 2X. Desno je prikazana Chimera celica, levo pa njihova medsebojna povezanost. Slika povzeta po [13].

Slika 2.2 jasno prikazuje, da Chimera celice delujejo kot lokalne računske enote, ki so nato modularno povezane v celoten procesor. Kljub temu ima vsak qubit največ šest sosedov (4 znotraj celice in 2 iz istoležečih pozicij v sosednjih celicah), kar pomeni, da fizični graf ni polno povezan in zato ni možno neposredno implementirati poljubnega QUBO grafa. Problem posplošimo tako, da je ena logična spremenljivka lahko realizirana kot povezana*veriga več fizičnih qubitov*. Temu postopku pravimo *minor-embedding* [11, 14]. Omejena povezljivost je ključna omejitev, ki določa največjo velikost logičnega problema, ki ga lahko učinkovito rešimo na D-Wave 2X.

Povezave med qubiti so realizirane z *nastavljivimi induktivnimi sklopitvami* – tunelirnimi Josephsonovimi elementi (ang. tunable couplers), ki omogočajo magnetno interakcijo med qubitoma. Z nadzorovanjem magnetnega fluksa v sklopitvenem elementu je mogoče programirati interakcijske uteži J_{ij}:

$$J_{ij} > 0 \Rightarrow \text{feromagnetna povezava}, \qquad J_{ij} < 0 \Rightarrow \text{antiferomagnetna povezava}.$$

Programiranje uteži J_{ij} pomeni fizično prilagajanje jakosti induktivne sklopitve. Omejitve v razponu J_{ij} so posledica fizikalnih omejitev nadprevodnih elementov.

2.3.3 QUBO in problem nahrbtnika

D-Wave kvantni računalniki rešujejo probleme v obliki *Quadratic Unconstrained Binary Optimization* (QUBO). Splošna oblika QUBO je:

$$\min_{x_i \in \{0,1\}} \left(\sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j \right).$$

V tej obliki so spremenljivke x_i binarne odločitvene spremenljivke, ki lahko zavzamejo le vrednosti 0 ali 1, matrika Q pa vsebuje vse koeficiente, ki določajo energijo problema. Diagonalni elementi Q_{ii} predstavljajo linearne prispevke posameznih spremenljivk, medtem ko nediagonalni elementi Q_{ij} predstavljajo kvadratične interakcijske člene med pari spremenljivk. Cilj QUBO problema je najti binarni vektor x, ki minimizira to kvadratično energijsko funkcijo.

Ekvivalentno lahko QUBO zapišemo v kompaktni matrični obliki:

$$\min_{x \in \{0,1\}^n} x^\top Q x,$$

kjer je $x = (x_1, x_2, \ldots, x_n)^\top$. Ta zapis združi linearne in kvadratne člene v eno samo matriko.

Za ilustracijo uporabimo preprost problem nahrbtnika [15]. Natančnejša preslikava problema v QUBO je predstavljena v viru [16].

Naj bo:

- v_i vrednost predmeta,
- w_i masa predmeta,
- $x_i \in \{0,1\}$ odločitev izbire.
- C nosilnost nahrbtnika,

Cilj je maksimizirati vrednost:

$$\max \sum_i v_i x_i$$

kar zaradi minimizacijske narave žarenja prepišemo kot:

$$\min -\sum_i v_i x_i.$$

Omejitev mase prepišemo s penalitetnim členom:

$$P \left(\sum_i w_i x_i - C \right)^2.$$

Celoten QUBO je tako:

$$\min_x \left(-\sum_i v_i x_i + P \left(\sum_i w_i x_i - C \right)^2 \right).$$

Razširitev kvadrata neposredno zapolni matriko Q.

2.3.4 Prehod iz QUBO v Isingov model

QUBO je potrebno prevesti v Isingov model [17], saj D-Wave uporablja spinske spremenljivke $s_i \in \{-1,+1\}$. Uporabimo preslikavo:

$$x_i = \frac{1-s_i}{2}$$

in dobimo Isingovo obliko:

$$H(s) = \sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j,$$

ki je neposredni vhod za kvantno žarenje. V primeru, da je Isingova oblika problema večja od zmožnosti kvantenga procesorja uporabimo algoritem *qbsolv*, katerega podrobno predstavimo v poglavju 2.3.7.

Za primer problema nahrbtnika dobimo z uporabo preslikave v spinske spremenljivke naslednjo Isingovo obliko:

$$H(s) = \sum_i \left(-\frac{v_i}{2} + \frac{P}{2} w_i \left(\sum_j w_j - 2C \right) \right) s_i + \sum_{i<j} \left(\frac{P}{4} w_i w_j \right) s_i s_j + konst.$$

V tej enačbi imajo posamezni členi naslednji pomen:

- **Linearni člen** ($h_i s_i$ v splošni enačbi) predstavlja vpliv posamezne izbire predmeta. Višja vrednost v_i prispeva k nižji energiji stanja $s_i = -1$, kar ustreza izbiri predmeta ($x_i = 1$). Členi z utežmi w_i dodajo penalizacijo za prekomerno težo prek kapacitete C.
- **Interakcijski člen** ($J_{ij} s_i s_j$ v splošni enačbi) izvira iz kvadratnega penalitetnega člena definiranega v 2.3.3. Predstavlja medsebojni vpliv parov predmetov; če sta oba izbrana (kar prinese $s_i = s_j = -1$), se skupni prispevek k skupni masi kaznuje s faktorjem $Pw_i w_j$.
- **Konstanta** ne vpliva na optimizacijo energije, zato jo prištejemo ko se je algoritem že izvedel.

Tako dobimo Isingov Hamiltonian za naš problem nahrbtnika. Sledi preslikava v topologijo fizičnega kvantnega procesorja. Kot že omenjeno, fizični Chimera graf nima popolne povezljivosti. S pomočjo *minor-embeddinga* logične spinske spremenljivke preslikamo v fizične qubite in njihove povezave:

1. **Veriženje qubitov:** Logična spremenljivka je reprezentirana kot množica fizičnih qubitov, povezanih s *feromagnetnim členom*, ki zagotavlja, da delujejo kot ena sama spremenljivka:

$$J_{\text{chain}} \sum_{(u,v)\in\text{chain}} s_u s_v,$$

kjer je $J_{\text{chain}} < 0$ dovolj velik, da poravna qubite v verigi.
2. **Mapiranje interakcij:** Ko je logična spremenljivka zgrajena kot veriga, se interakcije J_{ij} med dvema spremenljivkama realizirajo na vsaj eni fizični povezavi med verigama.
3. **Minimizacija uporabe qubitov:** Cilj embeddinga je najti verige, ki so čim krajše in porabijo čim manj fizičnih virov.

Minor-embedding tako predstavlja most med abstraktnim QUBO in dejansko povezljivostjo čipa.

2.3.5 Kvantno žarenje

Kvantno žarenja je proces, pri katerem sistem postopoma preide iz začetnega Hamiltoniana H_0 v Isingov Hamiltonian H_p za dani problem. Časovna evolucija sistema je opisana z izrazom:

$$H(t) = A(t)H_0 + B(t)H_p,$$

kjer je:

- H_0 začetni Hamiltonian s transversalnim poljem, ki ustvarja superpozicijo,
- H_p Hamiltonian za naš problem,
- $A(t)$ padajoča, $B(t)$ naraščajoča funkcija v intervalu $t \in [0, T]$.

Funkciji $A(t)$ in $B(t)$ zvezno nadzorujeta prispevek posameznih Hamiltonianov, pri čemer v začetku ($t = 0$) velja $A(0) \gg B(0)$, proti koncu ($t = T$) pa $A(T) \ll B(T)$. Če je sprememba $H(t)$ dovolj počasna (ang. adiabatic condition), sistem ostane v bližini osnovnega stanja trenutnega Hamiltoniana. Ko transversalno polje postopoma izgine in postane H_p dominanten, se sistem razvije proti stanju z minimalno energijo ciljnega Isingovega Hamiltoniana. V strojni opremi D-Wave to ustreza nadzorovanemu spreminjanju magnetnih tokov v nadprevodnih zankah, kar vpliva na tuneliranje med energetskimi stanji.

2.3.6 Rekonstrukcija rešitve

Po koncu žarenja se konfiguracija spinov $\{s_i\}$ izmeri. Meritev povzroči kolaps kvantnega stanja, kar da eno diskretno spinsko konfiguracijo. Spinske spremenljivke nato preslikamo v binarne QUBO spremenljivke z enačbo iz 2.3.4 in dobimo en rezultat. Ker je kvantno žarenje stohastičen postopek, se tipično izvede več tisoč ponovitev (ang. *reads*). Končna rešitev se običajno izbere na enega od dveh načinov:

- konfiguracija z **najnižjo energijo**, pridobljena med meritvami,
- statistična porazdelitev energijsko ugodnih konfiguracij, kar omogoča vpogled v približke ali več optimalnih rešitev.

To predstavlja rešitev izvornega optimizacijskega problema.

2.3.7 Algoritem qbsolv

Večji QUBO oziroma Isingovi problemi pogosto presegajo število logičnih spremenljivk, ki jih je mogoče hkrati vložiti v topologijo fizičnega kvantnega procesorja. Algoritem `qbsolv` [18] rešuje ta izziv z uporabo hibridnega klasično–kvantnega pristopa, pri katerem veliki QUBO razdeli na manjše podprobleme (t. i. *subQUBOs*), katere kvantni računalnik lahko izvede. Jedro algoritma temelji na dveh komplementarnih postopkih: kvantnem iskanju globalno različnih kandidatnih rešitev in klasičnem **tabu** preiskovanju (ang. tabu search), ki hitro izboljša trenutno lokalno rešitev.[19] Algoritem iterativno izbere podmnožico najbolj vplivnih spremenljivk, ostale pa *zamrzne* na trenutno najboljše znane vrednosti. Na ta način se problem lokalno zoži na manjši subQUBO, ki ga je mogoče rešiti na D-Wave sistemu. Rezultati podproblema se nato vpišejo nazaj v globalno kandidatno rešitev, s čimer lahko algoritmu uspe pobegniti iz lokalnih minimumov. Celoten proces poteka iterativno, dokler ni dosežena stabilnost ali pa je preseženo število dovoljenih ponovitev.

Celoten algoritem je podan v naslednji psevdokodi:

Algoritem 1 Partitioning algorithm implemented by `qbsolv`

Input: QUBO instance
V_{best} is the lowest value found to date
Q_{best} is the solution bit vector corresponding to the lowest value so far
index is the indices of the bits in the solution, sorted from most to least impact on value

Get initial estimate of minimum value and backbone
$Q_{\text{tmp}} \leftarrow$ random 0/1 vector
$(V_{\text{best}}, Q_{\text{best}}) \leftarrow \mathit{TabuSearch}(QUBO, Q_{\text{tmp}})$
$\mathit{index} \leftarrow \mathit{OrderByImpact}(QUBO, Q_{\text{best}})$
passCount $\leftarrow 0$
$Q_{\text{tmp}} \leftarrow Q_{\text{best}}$
while passCount $<$ numRepeats **do**
    for $i = 0$; $i <$ fraction$\cdot\mathit{Size}(QUBO)$; $i+=$ subQuboSize **do**
        # select subQubo with other variables clamped
        subQubo $\leftarrow \mathit{Clamp}(QUBO, Q_{\text{tmp}}, \mathit{index}[i : i+ \text{ subQuboSize } -1])$
        $(V_{\text{sub}}, Q_{\text{sub}}) \leftarrow \mathit{DWaveSearch}$(subQubo)
        # project onto full solution
        $Q_{\text{tmp}}[\mathit{index}[i : i+ \text{ subQuboSize } -1]] \leftarrow Q_{\text{sub}}$
    end for
    $(V, Q_{\text{new}}) \leftarrow \mathit{TabuSearch}(QUBO, Q_{\text{tmp}})$
    $\mathit{index} \leftarrow \mathit{OrderByImpact}(QUBO, Q_{\text{new}})$
    if $V < V_{\text{best}}$ **then**
        $V_{\text{best}} \leftarrow V$
        $Q_{\text{best}} \leftarrow Q_{\text{new}}$
        passCount$\leftarrow 0$
    else if $V == V_{\text{best}}$ **then**
        $Q_{\text{best}} \leftarrow Q_{\text{new}}$
        passCount++
    else
        passCount++
    end if
    $Q_{\text{tmp}} \leftarrow Q_{\text{new}}$
end while
Output: $V_{\text{best}}, Q_{\text{best}}$

Algoritem izkorišča različne prednosti obeh pristopov: D-Wave kvantni žarilnik je učinkovit pri preskakovanju v globalno oddaljena področja energijske krajine, medtem ko tabu iskanje omogoča visoko natančnost in hitro lokalno optimizacijo. S tem pristopom `qbsolv` omogoča reševanje QUBO modelov, ki so zaradi omejene povezljivosti ali velikosti preveliki za neposredno izvedbo na kvantnem procesorju.

D-Wave je `qbsolv` ponujal kot lastno odprtokodno orodje za razbijanje velikih QUBO modelov, vendar njegovo izvajanje ni bilo povsem avtomatsko — uporabnik je moral eksplicitno izbrati uporabo kvantnega procesorja in zagotoviti embedding. Danes je `qbsolv` označen kot zastarel, njegovo vlogo pa so prevzeli D-Wavovi hibridni solverji, zlasti ogrodje `dwave-hybrid` [20] in oblačne hibridne storitve v okolju *Leap*, ki celoten proces razdeljevanja, embeddinga in reševanja velikih QUBO instanc izvajajo avtomatizirano.

2.4 Primer optimizacije pretoka prometa

V članku *Traffic Flow Optimization Using a Quantum Annealer*, so avtorji raziskovali možnost uporabe kvantnih žarilnikov za optimizacijo prometnega toka v mestu[10]. Njihov cilj je bil podatke pretvoriti v ustrezno QUBO obliko in problem optimizirati s kvantnim pristopom, kar bi omogočilo reševanje takšnih problemov v realnem času.

2.4.1 Predstavitev problema

Zastoji na cestah so vsakodnevni pojav in povzročajo manjšo učinkovitost prometne infrastrukture v času povečanega prometa, ko večje število avtomobilov uporablja iste cestne segmente na poti do svojega cilja. Problem optimizacije pretoka predpostavi, da obstajajo alternativne poti, ki bi jih vozniki lahko izbrali, da pridejo do cilja. Cilj reševanja problema optimizacije pretoka prometa je zmanjšati število in velikosti zastojev v celotnem sistemu, tako da vsak voznik izbere eno od alternativnih poti.

Optimizacija pretoka prometa je kombinatorični problem, saj ima vsak udeleženec v prometu na voljo več alternativnih poti do končnega cilja. Kompleksnost problema tako s številom avtov narašča eksponentno. Z naraščajočo kompleksnostjo problema, klasični optimizacijski algoritmi postanejo neučinkoviti. Ker mora tak algoritem v resničnem svetu delovati konstantno in ponujati odločanje v realnem času, avtorji članka priporočajo uporabo kvantnih žarilnikov (angl. quantum annealer), ki so prav zasnovani za reševanje kompleksnih kombinatoričnih problemov.

Avtorji članka so delovanje algoritma preizkusili na naboru podatkov T-Drive, ki vsebuje poti 10.357 taksijev v Pekingu, zajetih v enem tednu, in predstavlja približno 9 milijonov prevoženih kilometrov. Avtorji so se v svojem delu omejili le na ožji nabor 418 vozil, ki so potovala od središča mesta do pekinškega letališča in obratno. Cilj njihovega algoritma je tako postalo hkratno preusmerjanje 418 avtomobilov po alternativnih poteh, da se zmanjšata število in velikost zastojev.

2.4.2 Prevedba problema v kvantni problem

Da bi avtorji zastavljeni problem lahko obravnavali s kvantnim žarilnikom, so ga morali preoblikovati v obliko problema QUBO. Tako preoblikovan problem so lahko nato reševali z minimizacijo $x^T Qx$, kjer je x vektor binarnih spremenljivk in Q zgornje trikotna matrika realnih števil.

Vsak avtomobil, ki je vključen v simulacijo, ima na voljo 3 različne alternativne poti po katerih lahko pride do cilja. Algoritem za vsak avtomobil izbere eno od njegovih možnih poti, tako da zmanjša količino zastojev v celotnem sistemu. Avtorji so vsakemu avtu dodelili binarno spremenljivko q_{ij}, kjer i predstavlja i-ti avto in

j predstavlja j-to pot. Vrednost spremenljivke za izbrano pot je enaka 1, ostale spremenljivke, ki predstavljajo neizbrane poti, pa imajo vrednost 0.

$$q_{ij} = \begin{cases} 1, & \text{če ima avto } i \text{ dodeljeno pot } j \\ 0, & \text{sicer} \end{cases}$$

V redkih primerih preusmeritev za določene avte ni bila mogoča, zato so te avti imeli za eno od poti določeno fiksno vrednost 1, ta vrednost pa se je še vedno upoštevala pri optimizaciji poti ostalih avtomobilov.

Vsaka pot je bila predstavljena kot zaporedje cestnih segmentov, ključni cilj zastavljenega algoritma pa je bila minimizacija števila avtomobilov, ki so bili hkrati dodeljeni vsakemu segmentu ceste v nekem časovnem intervalu. Avtorji so vsakemu segmentu dodelili binarno spremenljivko q_{ij}, ki je predstavljala ali j-ta pot avta i vsebuje segment s ali ne.

Ceno zastoja za segment s so nato definirali, kot

$$cost(s) = (\sum_{q_{ij} \in B_s} q_{ij})^2,$$

kjer je B_s množica vseh uporabljenih poti, ki vključujejo segment s.

Morali so še upoštevati omejitev, da mora avto imeti dodeljeno točno eno pot. Zato so uvedli omejitev za vsak avto i:

$$(\sum_{j \in \{1,2,3\}} q_{ij} - 1)^2 = 0$$

Tej omejitvi so dodali še faktor λ, katerega vrednost je morala biti dovolj velika, da je bila kršitev omejitve vedno kaznovana bolj kot pa povečanje zastojev ob dodelitvi določene poti avtu. Za vrednost faktorja λ so vzeli maksimalno število segmentov, ki si jih nek avto i deli s potmi drugih avtov.

Končna cenovna funkcija je bila:

$$\sum_{s \in S} cost(s) + \lambda \sum_i (\sum_j q_{ij} - 1)^2.$$

Z razširitvijo cenovne funkcije na linearne in kvadrate člene ter z upoštevanjem preslikave $I(q_{ij})$, ki binarni spremenljivki q_{ij} dodeli indeks v matriki Q, je matrika imela na diagonali linearne člene, na mestih nad diagonalo pa kvadratne člene. Kvadratni členi so predstavljali interakcije med dvema odločitvama. Tako so na diagonali prišteli $+1$ za vsak avto i, ki je imel pot, ki je vključevala ta cestni segment in $-\lambda$ zaradi omejitve ene poti na en avto. Ostalim elementom so dodali $+2$ za pare avtomobilov, katerih pot je vključevala isti cestni segment in $+2\lambda$ med pari poti istega avtomobila.

Ena od omejitev, ki so jo avtorji morali upoštevati, je omejena povezanost med kubiti v D-Wave QPU. Povezava med dvema spremenljivkama je možna le v primeru, da obstaja neposredna fizična povezava med njihovimi kubiti. Zato so morali uporabiti postopek minor-embedding, opisanega v poglavju 2.3.4. Ker pa je obrav-

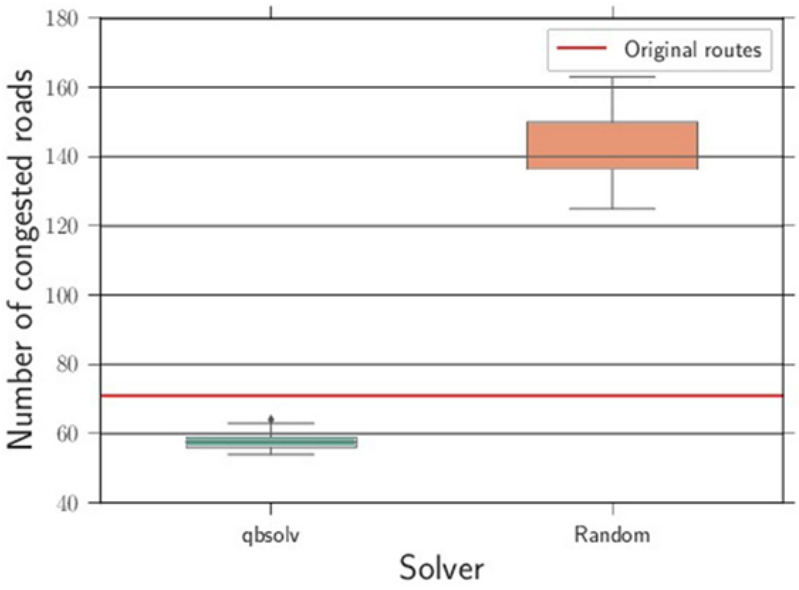
navanje primera s 418 avti in 3 alternativnimi potmi zahtevalo 1.254 logičnih spremenljivk, so morali problem reševati s pomočjo algoritma qbsolv, ki smo ga opisali v poglavju 2.3.7.

2.4.3 Rezultati raziskave

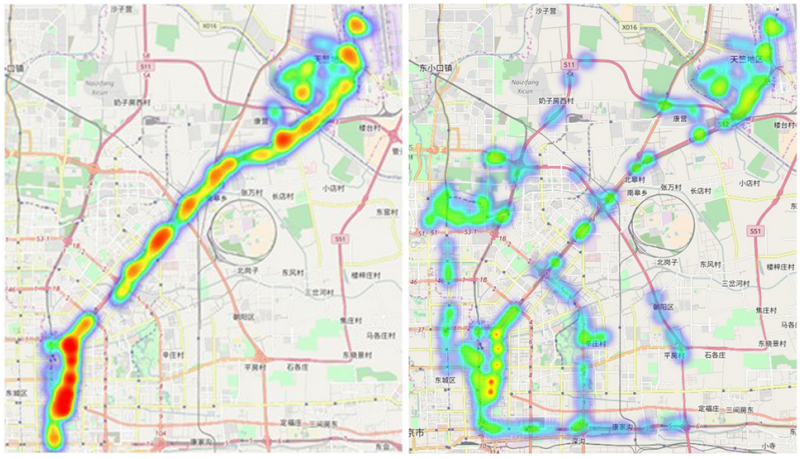
Končni cilj avtorjev je bil prevedba opisanega problema na kvantni problem in reševanje z uporabo kvantnega žarilnika. Prav tako je algoritem moral proizvesti dobre rešitve v realnem času. Kakovost rešitve so ocenili tako, da so za vsak cestni segment izračunali število poti avtomobilov, ki si ta segment delijo. Mejo, ki določa, ali je na segmentu nastal zastoj ali ne so označili s spremenljivko $N_{intersections}$ in za svoj primer so izbrali $N_{intersections} = 10$.

Svojo rešitev so primerjali z naključno dodelitvijo alternativnih poti avtomobilom. Svoj algoritem z uporabo algoritma qbsolv in naključno dodelitev so na primeru 418 avtomobilov zagnali 50-krat. V vseh 50. ponovitvah je opisan algoritem uspešno razrešil zastoje. V primerjavi z naključnim razporejanjem je bolje optimiziral dodelitev alternativnih poti, saj je bilo število in velikost zastojev precej manjša, kar je razvidno iz grafa 2.3. Tudi v primerjavi z originalnimi potmi avtomobilov, kjer je bilo manjše število poti zelo obremenjeno, je algoritem uspešno razporedil poti. Razliko v zastojih med originalnimi potmi in optimiziranimi potmi, prikazuje slike 2.4.

Algoritem so zaganjali na deljeni enoti QPU, dostopni preko oblačnih storitev, zato so bili časi izvajanja zelo različni. Za čas izvajanja so izbrali minimalen čas, ker so menili, da ta najbolj ponazori dejanski čas izvajanja algoritma, ki ni odvisen od obremenitve QPU s strani drugih uporabnikov. Najmanjši čas izvajanja je bil torej 22 sekund.



Slika 2.3: Graf porazdelitev števila zastojev po dodelitvi alternativnih poti avtomobilom. Na levi je prikazana porazdelitev za rešitev s kvantnim žarilnikom, na desni pa porazdelitev po naključni razdelitvi. Rdeča črta predstavlja število zastojev v originalni dodelitvi poti v naboru podatkov. Graf je iz članka [10].



Slika 2.4: Slika prikazuje primerjavo med neoptimizirano dodelitvijo poti in dodelitvijo optimizirano s kvantnim žarilnikom. Na levi je prikazana neoptimizirana pot, na desni pa optimizirana. Slika je iz članka [10].

Avtorji so obravnavali poenostavljeno obliko problema pretoka prometa. V svojem pristopu so uporabili manjšo podmnožico avtomobilov in poti ter niso upoštevali drugih udeležencev v prometu. Prav tako v optimizaciji niso upoštevali nobenega drugega dejavnika, kakor minimizacijo zastojev na cestnih segmentih. Alternativne poti, ki jih algoritem dodeli avtomobilu, bi lahko v resničnem svetu precej povečale čas potovanja ali pa vključevale cestnine, česar zasnovan algoritem ni upošteval v svojih odločitvah.

Omenjeni algoritem so avtorji iz skupine Volkswagen in podjetja D-Wave leta 2019 nadgradili ter implementirali v projektu Quantum Shuttle [21]. V tem projektu so preizkusili učinkovitost algoritma za optimizacijo poti 9 avtobusov, ki so prevažali obiskovalce od in do konference Web Summit. V tem primeru so upoštevali tudi aktivno stanje prometa v mestu in omejili celoten sistem tako, da poti, ki bi preveč podaljšale čas potovanja ali pa vključujejo ulice, premajhne za avtobus, niso upoštevali. Po besedah avtorjev je bil to prvi primer komercialne aplikacije, ki za reševanje kritične naloge v realnem času uporablja kvantni računalnik. S tem projektom so dokazali, da ta implementacija za reševanje problema optimizacije pretoka prometa, deluje v kompleksnem mestnem prometu v realnem svetu.

2.5 Zaključek

V nalogi smo obravnavali kvantne žarilnike podjetja D-Wave in njihov potencial pri reševanju optimizacijskih problemov, s poudarkom na optimizaciji pretočnosti prometa. Predstavili smo temeljne lastnosti arhitekture kvantnih računalnikov D-Wave, vključno s topologijo Chimera. Opisali smo postopek formulacije optimizacijskih problemov v QUBO obliko, preslikavo v Isingov model ter postopek minor-embeddinga, ki je nujen zaradi omejene fizične povezljivosti qubitov. Zaradi teh omejitev ostaja pomemben hibridni pristop, zlasti algoritem `qbsolv`, ki omogoča razdelitev večjih problemov na obvladljive podprobleme.

Na praktičnem primeru optimizacije prometa v Pekingu smo povzeli, kako je mogoče problem izbire alternativnih poti preoblikovati v QUBO in ga optimizirati s kvantnim žarilnikom. Rezultati so pokazali zmanjšanje števila in velikosti zastojev ter možnost izvajanja v realnem času. Vendar je model poenostavljen: obravnaval je omejeno število vozil, ni upošteval časa potovanja, cestnin, drugih udeležencev prometa ali geografskih omejitev, ki bi bile v realnem okolju ključnega pomena.

Projekt Quantum Shuttle pa je pokazal, da je mogoče kvantno žarenje uporabiti tudi v realnem prometnem sistemu, kjer so bile upoštevane dodatne omejitve in dejansko stanje prometa. Čeprav taka uporaba za zdaj ostaja omejena na manjše, natančno definirane primere in temelji na tesnem prepletu kvantnih in klasičnih metod, predstavlja pomemben korak k praktični uporabi kvantne optimizacije v kompleksnejših prometnih scenarijih v prihodnosti.

Trenutno kvantni žarilniki še ne nadomeščajo klasičnih optimizacijskih metod, vendar že predstavljajo učinkovito dopolnitev pri specifičnih nalogah. Nadaljnji razvoj v povezljivosti qubitov, zmanjševanju šuma ter napredku hibridnih kvantno-

klasičnih pristopov nakazuje, da bi se kvantna optimizacija v prihodnjih generacijah naprav lahko postopno uveljavila kot pomemben del orodij za upravljanje vse bolj kompleksnih prometnih sistemov.

2.6 Doprinosi avtorjev

Filip Gros: Poglavja 1.2 do 1.3.7
Andraž Klobučar: Poglavje 1.4
Marko Horvat: povzetek, uvod, zaključek, popravki dokumenta, priprava predstavitve

Poglavje 3
Implementacija Shorovega algoritma v več simulatorjih in primerjava

Filip Merkan

Povzetek V tem seminarju predstavljam implementacijo Shorovega algoritma v dveh različnih simulatorjih kvantnih računalnikov: Qiskit in Cirq. Shorov algoritem je kvantni algoritem za faktorizacijo celih števil, ki ima eksponentno manjšo časovno kompleksnost kot najboljši klasični algoritmi. Implementiral sem algoritem v obeh simulatorjih, analiziral njegovo delovanje z različnimi vhodnimi podatki ter primerjal prednosti in slabosti posameznih simulatorjev. Rezultati kažejo razlike v enostavnosti implementacije, hitrosti simulacije ter funkcionalnostih, ki jih posamezni simulatorji ponujajo.

Poročilo vsebuje teoretično ozadje Shorovega algoritma, opis izbranih simulatorjev, metodologijo implementacije, rezultate simulacij z različnimi vhodnimi podatki ter analizo kompleksnosti in primerjavo simulatorjev.

3.1 Uvod

Kvantno računalništvo predstavlja revolucionarno paradigmo procesiranja, ki izkorišča kvantne pojave, kot sta superpozicija in kvantna prepletenost, za reševanje določenih problemov eksponentno hitreje kot klasični računalniki [22]. Kljub temu da fizični kvantni računalniki še niso v splošni uporabi, omogočajo simulatorji kvantnih računalnikov raziskovalcem in razvijalcem načrtovanje, implementacijo in analizo kvantnih algoritmov.

Problem faktorizacije celih števil je eden najpomembnejših problemov v teoretičnem računalništvu in kriptografiji. Klasični algoritmi za faktorizacijo, kot sta kvadratno sito in metoda eliptičnih krivulj, imajo eksponentno časovno kompleksnost, kar zagotavlja varnost RSA kriptosistema, ki temelji na predpostavki, da je faktorizacija težka.

V tem seminarju se osredotočam na implementacijo in analizo Shorovega algoritma [23, 24]. Shorov algoritem je eden najpomembnejših kvantnih algoritmov, saj dokazuje, da lahko kvantni računalniki eksponentno hitreje faktorizirajo cela

števila kot klasični računalniki. To ima pomembne posledice za kriptografijo, saj ogroža varnost RSA šifriranja, ki temelji na težavnosti faktorizacije. Algoritem je bil predstavljen leta 1994 in predstavlja enega prvih praktičnih primerov, kjer kvantni računalniki dosegajo eksponentno pohitritev glede na klasične metode.

Cilj dela je:

- Implementirati Shorov algoritem v simulatorjih kvantnih računalnikov, predvsem v Qiskit in Cirq
- Analizirati delovanje algoritma z različnimi vhodnimi podatki (majhna cela števila)
- Primerjati prednosti in slabosti različnih simulatorjev
- Analizirati časovno in prostorsko kompleksnost algoritma
- Oceniti praktične omejitve implementacije zaradi števila potrebnih kubitov

3.2 Teoretično ozadje

3.2.1 Kvantni biti in kvantna vrata

Kvantni bit (kubit) je osnovna enota kvantne informacije. Za razliko od klasičnega bita, ki je lahko le v stanju 0 ali 1, lahko kubit obstaja v superpoziciji obeh stanj:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{3.1}$$

kjer sta α in β kompleksni amplitudi, ki zadoščata pogoju $|\alpha|^2 + |\beta|^2 = 1$.

Kvantna vrata so unitarne transformacije, ki delujejo na kubite. Med osnovnimi kvantnimi vrati so:

- Paulijeva vrata (X, Y, Z): X vrata izvajajo NOT operacijo, Y in Z vrata izvajajo rotacije okoli ustreznih osi
- Hadamardovo vrata (H): ustvarjajo superpozicijo, preslikajo $|0\rangle$ v $(|0\rangle + |1\rangle)/\sqrt{2}$ in $|1\rangle$ v $(|0\rangle - |1\rangle)/\sqrt{2}$
- CNOT vrata: kontrolirana NOT operacija, ki negira tarčni kubit, če je kontrolni kubit v stanju $|1\rangle$
- Rotacijska vrata (R_x, R_y, R_z): izvajajo rotacije okoli osi x, y ali z za določen kot

Kvantna vrata morajo biti unitarne, kar pomeni, da mora veljati $U^\dagger U = I$, kjer je $U^\dagger$ hermitska konjugirana matrika vrat U, I pa identitetna matrika. Ta lastnost zagotavlja reverzibilnost kvantnih operacij in ohranitev verjetnosti.

3.2.2 Shorov algoritem

Shorov algoritem [23, 24] je kvantni algoritem za faktorizacijo celih števil. Problem faktorizacije je: dano sestavljeno liho število N, poišči praštevili p in q, tako da $N = p \cdot q$.

Algoritem deluje v dveh glavnih korakih:

3.2.2.1 Klasični del

Najprej izberem naključno število a, kjer $1 < a < N$ in $\gcd(a,N) = 1$. Če $\gcd(a,N) \neq 1$, sem že našel faktor. Nato poiščem red r funkcije $f(x) = a^x \bmod N$, torej najmanjše pozitivno število r, za katerega velja $a^r \equiv 1 \pmod{N}$.

3.2.2.2 Kvantni del

Kvantni del algoritma uporablja kvantno Fourierjevo transformacijo (QFT) za iskanje reda r. Algoritem sestavljajo naslednji koraki:

1. Inicializacija: pripravimo superpozicijo vseh možnih vrednosti x v registru
2. Modularna eksponentiacija: izračunamo $a^x \bmod N$ in shranimo rezultat v drugi register
3. Kvantna Fourierjeva transformacija: uporabimo QFT na prvem registru
4. Meritve: izmerimo prvi register in dobimo vrednost, ki je blizu večkratniku N/r
5. Klasični postprocessing: z uporabo zveznih ulomkov poiščem r iz izmerjene vrednosti

Ko poznam red r, lahko z verjetnostjo vsaj $1/2$ najdem faktor N kot $\gcd(a^{r/2} \pm 1, N)$, če je r sodo in $a^{r/2} \not\equiv \pm 1 \pmod{N}$.

3.2.2.3 Kompleksnost

Shorov algoritem ima časovno kompleksnost $O((\log N)^3)$, kar je eksponentno hitreje od najboljših klasičnih algoritmov za faktorizacijo, ki imajo kompleksnost $O(e^{(\log N)^{1/3}})$.

Prostorska kompleksnost je $O(\log N)$ kubitov, kar pomeni, da za faktorizacijo 1024-bitnega števila potrebujem približno 2048 kubitov, kar je še vedno preveč za današnje kvantne računalnike.

3.2.3 Kvantna Fourierjeva transformacija

Kvantna Fourierjeva transformacija (QFT) je kvantna različica diskretne Fourierjeve transformacije. Za n kubitov preslika bazično stanje $|j\rangle$ v:

$$QFT|j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i jk/2^n} |k\rangle \tag{3.2}$$

QFT je ključna komponenta Shorovega algoritma in se izvaja z uporabo Hadamardovih vrat in kontroliranih rotacijskih vrat. Implementacija QFT za n kubitov zahteva $O(n^2)$ kvantnih vrat, kar je eksponentno hitreje kot klasična FFT, ki zahteva $O(n2^n)$ operacij.

3.2.4 Modularna eksponentiacija

Modularna eksponentiacija $a^x \bmod N$ je ključna operacija v Shorovem algoritmu. V kvantnem vezju jo implementiramo z uporabo kontroliranih množenj. Za izračun $a^x \bmod N$ uporabimo binarni razvoj eksponenta x:

$$x = x_0 2^0 + x_1 2^1 + \ldots + x_{n-1} 2^{n-1} \tag{3.3}$$

Nato izračunamo:

$$a^x \bmod N = a^{x_0 2^0} \cdot a^{x_1 2^1} \cdot \ldots \cdot a^{x_{n-1} 2^{n-1}} \bmod N \tag{3.4}$$

Vsak faktor $a^{2^k} \bmod N$ lahko izračunamo z zaporednimi kvadriranji, kar zahteva $O(\log N)$ kontroliranih množenj.

3.2.5 Iskanje periode in zvezni ulomki

Po meritvi kvantnega registra dobimo vrednost, ki je blizu večkratniku N/r, kjer je N velikost registra in r red funkcije. Za iskanje r iz izmerjene vrednosti uporabimo algoritem zveznih ulomkov (continued fractions).

Algoritem zveznih ulomkov pretvori racionalno število v zaporedje celih števil, ki predstavljajo približke. Če je izmerjena vrednost y/N blizu k/r za neko celo število k, lahko z uporabo zveznih ulomkov najdem r.

3.3 Metode

3.3.1 Izbrani simulatorji

Za implementacijo algoritma sem izbral naslednje simulatorje:

3.3.1.1 Qiskit

Qiskit je odprtokodni okvir za delo s kvantnimi računalniki, ki ga razvija IBM [25]. Omogoča načrtovanje, simulacijo in izvajanje kvantnih algoritmov na IBM-ovih kvantnih računalnikih ali lokalnih simulatorjih. Qiskit ponuja:

- Enostavno API za gradnjo kvantnih vezij
- Integracijo z IBM Quantum Experience
- Različne simulatorje (statevector, qasm, unitary)
- Optimizacijo vezij
- Vgrajene implementacije standardnih algoritmov

3.3.1.2 Cirq

Cirq je odprtokodni okvir za kvantno računalništvo, ki ga razvija Google. Namenjen je predvsem za delo z NISQ (Noisy Intermediate-Scale Quantum) napravami. Cirq ponuja:

- Natančno kontrolo nad kvantnimi vezji
- Optimizacijo za Google-ove kvantne procesorje
- Simulacijo z upoštevanjem šuma
- Integracijo z TensorFlow Quantum

3.3.2 Metodologija implementacije

Implementacijo Shorovega algoritma sem razdelil na naslednje korake:

3.3.2.1 Analiza zahtev

Najprej sem analiziral zahteve algoritma:

- Število kubitov: za faktorizacijo N potrebujem približno $2\log_2 N$ kubitov za kvantni del
- Modularna eksponentiacija: implementacija funkcije $a^x \bmod N$
- Kvantna Fourierjeva transformacija: implementacija QFT

- Klasični postprocessing: algoritem za iskanje reda iz izmerjenih vrednosti

3.3.2.2 Implementacija v Qiskit

V Qiskit sem uporabil:

- `QuantumCircuit` za gradnjo vezij
- Vgrajene funkcije za modularno eksponentiacijo (če so na voljo)
- `QFT` iz Qiskit-ove knjižnice
- `AerSimulator` za simulacijo

3.3.2.3 Implementacija v Cirq

V Cirq sem uporabil:

- `Circuit` za gradnjo vezij
- `cirq.ops` za kvantna vrata
- Vlastno implementacijo modularne eksponentiacije
- `Simulator` za simulacijo

Simulatorji so implementirani v ločenih datotekah v mapi `code/`: `shor_qiskit.py` za Qiskit in `shor_cirq.py` za Cirq. Implementacije uporabljajo splošno implementacijo modularne eksponentiacije za vse testirane vrednosti N, kar omogoča pošteno primerjavo simulatorjev.

Za testiranje sem izbral majhna sestavljena števila ($N = 15$ in $N = 21$), da lahko algoritem izvedem z razumnim številom kubitov. Za vsako vrednost N sem testiral z različnimi vrednostmi a: za $N = 15$ sem uporabil $a = 7$ in $a = 11$, za $N = 21$ pa $a = 2$ in $a = 8$. Za vsako kombinacijo sem:

- Izračunali potrebno število kubitov: $n = \lceil 2\log_2 N \rceil$
- Uporabili vrednost a z $\gcd(a,N) = 1$ (za $N = 15$: $a = 7$ in $a = 11$; za $N = 21$: $a = 2$ in $a = 8$)
- Implementiral celoten algoritem v vsakem simulatorju
- Izvedel večkratne simulacije za statistično analizo
- Primerjal rezultate med simulatorji

3.3.3 Implementacijski pristop

Za pošteno primerjavo simulatorjev sem uporabil enak pristop za vse testirane vrednosti N:

- Splošna implementacija modularne eksponentiacije z uporabo matričnih vrat za vse vrednosti N
- Enaka struktura vezja za oba simulatorja

- Enako število kubitov za iste vrednosti N

Ta pristop omogoča direktno primerjavo zmogljivosti simulatorjev brez vpliva različnih optimizacij.

3.3.4 Testiranje in validacija

Za validacijo implementacije sem uporabil:

- Testne primere z znanimi rezultati: faktorizacija $N = 15$ (faktorja: 3 in 5) z $a = 7$ in $a = 11$, $N = 21$ (faktorja: 3 in 7) z $a = 2$ in $a = 8$
- Primerjavo rezultatov med različnimi simulatorji: preverjanje, da vsi simulatorji dajo enake rezultate
- Analizo verjetnostnih porazdelitev izhodnih stanj: preverjanje, da so izmerjene vrednosti blizu pričakovanim večkratnikom N/r
- Statistično analizo: vsak testni primer sem izvedel 5-krat in analiziral uspešnost ter čas izvajanja

3.4 Rezultati

3.4.1 Implementacija algoritma

3.4.1.1 Implementacija v Qiskit

V Qiskit sem implementiral Shorov algoritem z uporabo vgrajenih funkcij in lastnih implementacij. Kvantno vezje sestavljajo naslednje komponente:

- Inicializacija superpozicije z Hadamardovimi vrati
- Modularna eksponentiacija z kontroliranimi vrati
- Kvantna Fourierjeva transformacija
- Meritve

Implementacija v Qiskit vključuje naslednje glavne komponente:

1. **Inicializacija**: Uporabimo Hadamardova vrata na prvem registru za ustvarjanje superpozicije. V Qiskit to implementiramo z zanko, ki dodaja Hadamardova vrata na vsak kubit v registru.
2. **Modularna eksponentiacija**: Implementiramo kontrolirana množenja za izračun $a^x \bmod N$. Za vsak bit eksponenta uporabimo kontrolirano množenje z ustreznim faktorjem.
3. **QFT**: Uporabimo vgrajeno funkcijo `QFT` iz Qiskit-ove knjižnice ali implementiramo lastno verzijo z Hadamardovimi vrati in kontroliranimi rotacijami.
4. **Meritve**: Izmerimo prvi register in dobimo vrednost za postprocessing.

Qiskit omogoča enostavno vizualizacijo vezij z metodo `circuit.draw()`, kar je koristno za razumevanje in debugiranje implementacije.

3.4.1.2 Implementacija v Cirq

V Cirq sem implementiral algoritem z uporabo osnovnih gradnikov. Glavne razlike v primerjavi z Qiskit:

- Drugačen način gradnje vezij: v Cirq gradimo vezja z dodajanjem operacij na specifične kubite
- Različna sintaksa za kontrolirana vrata: Cirq uporablja `cirq.CNOT` ali `cirq.ControlledGate`
- Možnost optimizacije za specifične procesorje: Cirq omogoča optimizacijo vezij za Google-ove procesorje
- Natančnejša kontrola nad časovno zaporedjem operacij

Implementacija v Cirq zahteva več ročnega dela, vendar ponuja več kontrole nad vezjem. Cirq omogoča tudi simulacijo z upoštevanjem šuma, kar je koristno za oceno delovanja na realnih napravah.

3.4.2 Simulacije z različnimi vhodnimi podatki

Za vsako testno število sem izvedel večkratne simulacije in analiziral rezultate. V naslednjih podpoglavjih predstavljam rezultate za posamezne testne primere.

3.4.2.1 Faktorizacija $N = 15$

Za faktorizacijo $N = 15$ potrebujem 8 kubitov. Testiral sem z $a = 7$ in $a = 11$, kjer je $\gcd(7,15) = 1$ in $\gcd(11,15) = 1$. Pričakovani red funkcije $f(x) = 7^x \bmod 15$ je $r = 4$, saj velja $7^4 \equiv 1 \pmod{15}$, za $a = 11$ pa je red $r = 2$.

Oba simulatorja sta uspešno faktorizirala $N = 15$ z 100% uspešnostjo in našla faktorja 3 in 5. Povprečni čas izvajanja za Qiskit je bil 0.21 s, za Cirq pa 0.007 s.

3.4.2.2 Faktorizacija $N = 21$

Za faktorizacijo $N = 21$ potrebujem 9 kubitov. Testiral sem z $a = 2$ in $a = 8$, kjer je $\gcd(2,21) = 1$ in $\gcd(8,21) = 1$. Pričakovani red za $a = 2$ je $r = 6$, za $a = 8$ pa $r = 2$.

Oba simulatorja sta uspešno faktorizirala $N = 21$ z 100% uspešnostjo in našla faktorja 3 in 7. Povprečni čas izvajanja za Qiskit je bil 3.32 s (z velikimi razlikami med različnimi vrednostmi a), za Cirq pa 0.009 s.

Tabela 3.1: Rezultati simulacij z različnimi vhodnimi podatki

N	a	Pričakovani faktorji	Qiskit	Cirq
15	7	3, 5	3, 5 (100%)	3, 5 (100%)
15	11	3, 5	3, 5 (100%)	3, 5 (100%)
21	2	3, 7	3, 7 (100%)	3, 7 (100%)
21	8	3, 7	3, 7 (100%)	3, 7 (100%)

Tabela 3.2: Primerjava zmogljivosti simulatorjev

Metrika	Qiskit	Cirq
Povprečni čas ($N = 15$)	0.21 s	0.007 s
Povprečni čas ($N = 21$)	3.32 s	0.009 s
Skupni povprečni čas	1.76 s	0.008 s
Uspešnost	100%	100%
Enostavnost uporabe	Visoka	Srednja
Dokumentacija	Odlična	Dobra

3.4.3 Analiza kompleksnosti

3.4.3.1 Teoretična kompleksnost

Shorov algoritem ima:

- Časovno kompleksnost: $O((\log N)^3)$ kvantnih operacij
- Prostorsko kompleksnost: $O(\log N)$ kubitov
- Verjetnost uspeha: vsaj $1/2$ za vsak poskus

3.4.3.2 Praktična kompleksnost

V praksi je kompleksnost odvisna od:

- Implementacije modularne eksponentiacije: ta zahteva $O(\log N)$ kontroliranih operacij
- Implementacije QFT: ta zahteva $O((\log N)^2)$ vrat
- Števila ponovitev: zaradi verjetnostne narave algoritma lahko potrebujem več poskusov

Za $N = 15$ potrebujem približno 8 kubitov, za $N = 21$ pa približno 9 kubitov.

3.4.3.3 Meritve zmogljivosti

Izmeril sem:

- Število kvantnih vrat v vezju za različne vrednosti N
- Čas izvajanja simulacije v posameznem simulatorju
- Uspešnost algoritma (verjetnost pravilne faktorizacije)

Izmeril sem naslednje metrike:

- **Število kvantnih vrat**: Za $N = 15$ vezje vsebuje približno 200-300 vrat, odvisno od implementacije. Število vrat narašča približno kvadratno z N.
- **Čas simulacije**: Za $N = 15$ je povprečni čas simulacije v Qiskit 0.21 s, v Cirq pa 0.007 s. Za $N = 21$ je povprečni čas v Qiskit 3.32 s (z velikimi razlikami med različnimi vrednostmi a: od 0.73 s do 5.91 s), v Cirq pa 0.009 s. Cirq je torej približno 30-krat hitrejši za $N = 15$ in približno 370-krat hitrejši za $N = 21$. Razlika v hitrosti je posledica dejstva, da Cirq direktno uporablja matrična vrata brez potrebe po transpilaciji, medtem ko Qiskit zahteva dekompozicijo in transpilacijo vezja pred simulacijo.
- **Uspešnost algoritma**: Oba simulatorja sta dosegla 100% uspešnost za vse testne primere. To pomeni, da sta oba simulatorja zanesljiva za faktorizacijo majhnih števil.
- **Poraba pomnilnika**: Simulacija zahteva $O(2^n)$ pomnilnika, kjer je n število kubitov. Za $N = 15$ (8 kubitov) to pomeni približno 256 kompleksnih števil, za $N = 21$ (9 kubitov) pa približno 512 kompleksnih števil.

Te meritve kažejo, da je praktična uporaba Shorovega algoritma za večja števila omejena z eksponentno rastjo potrebnega pomnilnika in časa simulacije.

3.4.4 Primerjava simulatorjev

3.4.4.1 Qiskit

Prednosti:

- Odlična dokumentacija in velika skupnost uporabnikov
- Enostavna integracija z IBM Quantum Experience
- Vgrajene implementacije standardnih algoritmov
- Dobra podpora za Python
- Boljša integracija z realnimi kvantnimi napravami

Slabosti:

- Odvisnost od Python ekosistema
- Za kompleksnejše optimizacije lahko zahteva dodatno znanje
- Počasnejša simulacija zaradi potrebe po transpilaciji vezij

3.4.4.2 Cirq

Prednosti:

- Natančna kontrola nad kvantnimi vezji
- Optimizacija za Google-ove procesorje
- Možnost simulacije z upoštevanjem šuma
- Integracija z TensorFlow Quantum za kvantno strojno učenje
- Znatno hitrejša simulacija zaradi direktne uporabe matričnih vrat

Slabosti:

- Manjša skupnost uporabnikov kot Qiskit
- Zahtevnejša za začetnike

3.4.4.3 Splošna primerjava

Obe platformi (Qiskit in Cirq) omogočata uspešno implementacijo Shorovega algoritma. Izbira je odvisna od:

- Predhodnega znanja programerja: Qiskit je najlažji za začetnike
- Ciljne platforme (IBM, Google): če želimo izvajati na fizičnih napravah, je smiselno izbrati ustrezen simulator
- Zahtev glede optimizacije: Cirq ponuja največ kontrole nad vezji

3.4.4.4 Primerjava hitrosti

Za majhna števila ($N \leq 21$) je Cirq znatno hitrejši od Qiskit. Povprečni čas izvajanja je v Cirq približno 220-krat krajši (0.008 s proti 1.76 s). Razlika je posledica različnih pristopov k simulaciji: Cirq direktno uporablja matrična vrata brez potrebe po transpilaciji, medtem ko Qiskit zahteva dekompozicijo in transpilacijo vezja v osnovna vrata pred simulacijo. Ta razlika postane še bolj izrazita za večja števila N, kjer Qiskit potrebuje več časa za dekompozicijo večjih unitarnih matrik.

3.4.4.5 Primerjava enostavnosti uporabe

Qiskit ima najboljšo dokumentacijo in največjo skupnost uporabnikov, kar olajša iskanje pomoči in primerov. Cirq zahteva več ročnega dela, vendar ponuja več kontrole nad kvantnimi vezji.

3.5 Zaključek

V tem seminarju sem uspešno implementiral Shorov algoritem v simulatorjih kvantnih računalnikov, predvsem v Qiskit in Cirq. Oba simulatorja sta omogočila uspešno faktorizacijo majhnih sestavljenih števil ($N = 15$ in $N = 21$) z 100% uspešnostjo.

Glavne ugotovitve:

- Qiskit in Cirq omogočata implementacijo Shorovega algoritma, vendar z različnimi pristopi in sintakso
- Qiskit je najbolj prijazen za začetnike zaradi odlične dokumentacije in velike skupnosti
- Cirq je znatno hitrejši za simulacije (približno 220-krat hitrejši v naših testih) zaradi direktne uporabe matričnih vrat brez transpilacije
- Oba simulatorja dosežeta 100% uspešnost za testirana števila
- Praktična omejitev je število kubitov - za večja števila N postane simulacija izjemno počasna ali nemogoča, predvsem v Qiskit zaradi časa potrebnega za transpilacijo

Omejitve naše implementacije:

- Implementiral sem algoritem le za majhna števila zaradi omejitev simulatorjev
- Praktična uporaba za velika števila bi zahtevala fizične kvantne računalnike z več tisoč kubiti
- Nisem upošteval kvantnega šuma, ki bi vplival na rezultate na realnih napravah

Možnosti za nadaljnje delo:

- Optimizacija implementacije modularne eksponentiacije: raziskovanje učinkovitejših metod za implementacijo modularne eksponentiacije, ki bi zmanjšale število potrebnih vrat
- Implementacija na fizičnih kvantnih računalnikih (IBM, Google, Microsoft): testiranje algoritma na realnih kvantnih napravah in analiza vpliva šuma
- Analiza vpliva kvantnega šuma na uspešnost algoritma: raziskovanje, kako kvantni šum vpliva na verjetnost uspešne faktorizacije
- Primerjava z optimiziranimi verzijami algoritma: implementacija in primerjava različnih optimiziranih verzij Shorovega algoritma
- Razširitev na večja števila z uporabo naprednejših tehnik: raziskovanje metod za faktorizacijo večjih števil z uporabo hibridnih pristopov
- Implementacija kvantnega popravljanja napak: raziskovanje, kako kvantno popravljanje napak vpliva na uspešnost algoritma
- Primerjava z drugimi kvantnimi algoritmi za faktorizacijo: raziskovanje alternativnih pristopov k faktorizaciji

3.6 Razprava

Implementacija Shorovega algoritma v več simulatorjih je pokazala, da sta Qiskit in Cirq primerna za ta namen, vendar z različnimi prednostmi. Qiskit je najbolj prijazen za začetnike in ponuja odlično dokumentacijo, Cirq pa ponuja največ kontrole in možnosti optimizacije.

Praktična omejitev implementacije je število kubitov, ki jih lahko simuliram. Za večja števila N postane simulacija izjemno počasna ali nemogoča zaradi eksponentne rasti potrebnega pomnilnika. To poudarja pomen razvoja fizičnih kvantnih računalnikov za praktično uporabo Shorovega algoritma.

Kljub omejitvam simulatorjev, implementacija algoritma v več simulatorjih omogoča boljše razumevanje algoritma in primerjavo različnih pristopov k implementaciji. To znanje je koristno za prihodnje delo z kvantnimi algoritmi in razvoj novih optimizacij.

Doprinosi avtorjev

Seminarsko nalogo sem v celoti izdelal Filip Merkan.

Poglavje 4
Analiza ter implementacija Groverjevega algoritma

Bernard Kučina, Dejan Jarc, Jure Zupančič, Maja Razinger

Povzetek V seminarski nalogi obravnavamo Groverjev kvantni algoritem. Najprej predstavimo temeljne koncepte kvantnega računalništva, med drugim kvantni bit, superpozicijo. Nato podrobneje opišemo Groverjev algoritem, njegovo iterativno strukturo ter geometrijsko interpretacijo kot rotacijo stanja proti označenemu vektorju. Sledi praktična implementacija algoritma s pomočjo knjižnice Qiskit, kjer prikažemo delovanje algoritma na konkretnem primeru. Na koncu povzamemo še nekatere razširitve osnovnega algoritma, ki omogočajo uporabo v primerih z več rešitvami ali pri neznanem številu označenih elementov.

4.1 Uvod

V klasičnih računalniških sistemih je iskanje specifičnega zapisa v nestrukturirani zbirki podatkov z N elementi izjemno neučinkovito, saj zahteva povprečno pregledovanje velikega dela vseh zapisov, kar pomeni, da imajo takšni algoritmi časovno zahtevnost reda $\mathcal{O}(N)$ oziroma binarni iskalni algoritmi, ki imajo časovno zahtevnost $\mathcal{O}(\log N)$.

Kvantni računalniki pa omogočajo tudi izvajanje več operacij hkrati, temu pojavu pravimo *kvantna superpozicija*. S tem pojavom se odpirajo možnosti za bistveno učinkovitejše reševanje določenih problemov, ki so za klasične algoritme izjemno zahtevni.

Primer algoritma, ki izkorišča kvantno superpozicijo in s tem omogoča hitrejše iskanje je **Groverjev algoritem**, ki ga je leta 1996 predstavil Lov K. Grover [26]. Ta algoritem omogoča iskanje v nestrukturirani zbirki podatkov v samo $\mathcal{O}(\sqrt{N})$ korakih, kar predstavlja kvadratno pospešitev v primerjavi s klasičnimi pristopi.

4.2 Osnove kvantnega računalništva

Kvantno računalništvo združuje matematiko, fiziko in računalništvo ter tako predstavlja nov pristop k obdelavi informacij. Medtem ko klasični računalniki temeljijo na bitih, kvantni računalniki uporabljajo **kvantne bite (qubite)**, ki lahko zaradi **superpozicije** in **prepletenosti** obstajajo v več stanjih hkrati.

4.2.1 Kvatni bit - qubit

Kvantni bit ali **qubit** je osnovna enota kvantnega računalništva. Medtem ko klasični bit lahko zavzame samo eno od dveh možnih vrednosti, 0 ali 1, kvantni bit omogoča, da je sistem hkrati v stanju 0 *in* v stanju 1, kar imenujemo **superpozicija**. [27]

Matematično stanje enega qubita zapišemo kot:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{4.1}$$

kjer sta α in β kompleksni števili, ki določata verjetnost, da bo ob meritvi rezultat 0 oziroma 1, pri čemer velja:

$$|\alpha|^2 + |\beta|^2 = 1. \tag{4.2}$$

4.2.2 Kvantna superpozicija in interferenca

Kvantna superpozicija je lastnost kvantnega računalništva, ki omogoča, da kvantni sistem obstaja v več možnih stanjih hkrati, dokler ne izvedemo meritve. To kvantnim računalnikom omogoča, da obdelujejo več vhodnih stanj hkrati in s tem izvajajo operacije nad ogromnim številom kombinacij v enem samem koraku, kar pa omogoča večjo računsko moč. [27]

Kvantna interferenca je pojav, pri katerem se valovne funkcije različnih kvantnih stanj medsebojno prepletajo. V kvantnem sistemu delci ne obstajajo kot točke, temveč kot valovne funkcije, ki opisujejo verjetnost njihovih možnih stanj. Takrat ko se valovi med seboj ojačajo, govorimo o *konstruktivni interferenci*, ko pa se med seboj izničijo, pa o *destruktivni interferenci* [28].

4.3 Groverjev algoritem

Groverjev algoritem je kvantni algoritem za iskanje ene označene rešitve v nesortiranem seznamu velikosti N s časovno kompleksnostjo $O(\sqrt{N})$. Algoritem izkorišča

kvantno superpozicijo in interferenco, da postopoma poveča verjetnost pravilne rešitve.

4.3.1 Opis algoritma

Jedro Groverjevega algoritma temelji na zaporedju dveh kvantnih operacij, ki skupaj tvorita iterativni postopek povečanja verjetnosti pravilne rešitve: *orakelj*, ki označuje ciljni elelement in *difuzijski operator*, ki odraža amplitude okoli povprečja [29].

4.3.1.1 Priprava začetne superpozicije

Algoritem se začne z začetnim stanjem $|0\rangle^{\otimes n}$. Nad tem stanjem se uporabijo Hadamardova vrata $H^{\otimes n}$, s čimer se pripravi enakomerna superpozicija vseh možnih rešitev:

$$|\psi_0\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle. \tag{4.3}$$

4.3.1.2 Groverjeva iteracija

Vsaka iteracija algoritma vsebuje dve osnovni operaciji, ki skupaj tvorita t. i. *Groverjev operator*.

Orakelj operacija.

Orakelj O_f je črna skrinjica, ki implementira funkcijo $f(x)$, pri čemer velja $f(x) = 1$ za iskano rešitev in $f(x) = 0$ sicer. Orakelj spremeni stanje, ki ustreza iskani rešitvi po pravilu:

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle, \tag{4.4}$$

kar pomeni, da stanje pravilne rešitve dobi negativno fazo (multiplikacijo z -1), medtem ko ostala stanja ostanejo nespremenjena. [30, 27]

Difuzijski operator.

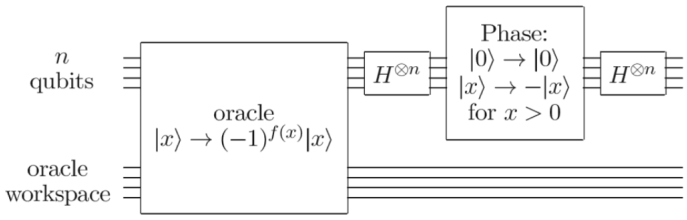
Druga operacija, imenovana *difuzijski operator*, okrepi okrepi amplitudo označenega stanja. Matematično jo opišemo kot:

$$H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n} = 2|\psi\rangle\langle\psi| - I. \tag{4.5}$$

kjer je $|\psi\rangle$ enakomerno utežena superpozicija vseh stanj 4.3. [27].

Celotno Groverjevo iteracijo tako zapišemo kot:

$$G = (2|\psi\rangle\langle\psi| - I)O_f. \tag{4.6}$$



Slika 4.1: Shematski prikaz ene iteracije Groverjevega algoritma, sestavljene iz orakelj operatorja 4.4 in difuzijskega operatorja 4.5.

4.3.1.3 Potek algoritma

Celoten Groverjev algoritem poteka v naslednjih korakih:

1. Priprava začetne superpozicije $|\psi_0\rangle$ z uporabo Hadamardovih vrat.
2. Večkratna ponovitev Groverjeve iteracije 4.6 , približno $\frac{\pi}{4}\sqrt{N}$-krat, da se amplituda označenega stanja maksimira.
3. Meritev končnega stanja, pri čemer je verjetnost pravilne rešitve skoraj enaka 1.

[26, 29].

4.3.2 Geometrijska interpretacija

Analizo delovanja Groverjevega algoritma lahko poenostavimo tako, da namesto N-razsežnega Hilbertovega prostora obravnavamo zgolj dvorazsežni podprostor. Ta podprostor določata dva ortogonalna vektorja: vektor $|a\rangle$ predstavlja iskano stanje, vektor $|e\rangle = \sum_{x\neq a} \frac{1}{\sqrt{N-1}}|x\rangle$ pa združuje vsa ostala stanja. Začetno stanje v tem podprostoru lahko predstavimo kot linearno kombinacijo $|a\rangle$ in $|e\rangle$:

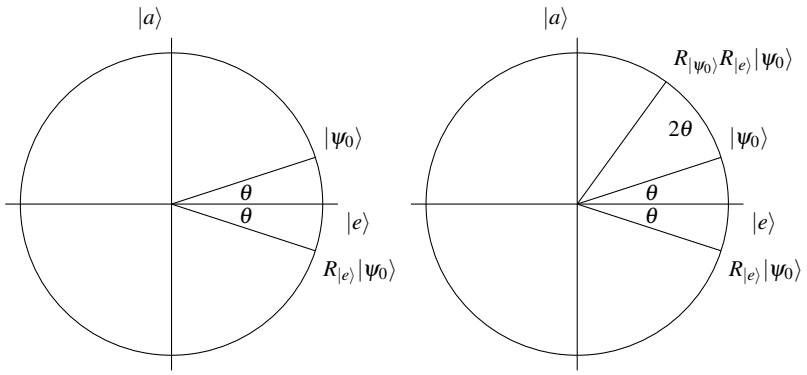
$$|\psi_0\rangle = sin(\theta)|a\rangle + cos(\theta)|e\rangle, \tag{4.7}$$

kjer je kot θ določen s prekrivanjem med začetnim in iskanim stanjem:

$$sin(\theta) = \langle a|\psi_0\rangle = \frac{1}{\sqrt{N}}. \tag{4.8}$$

Za velike N je kot θ zelo majhen, kar pa pomeni da je začetno stanje $|\psi_0\rangle$ skoraj popolnoma ortogonalno iskani rešitvi $|a\rangle$. Cilj algoritma je zavrteti vektor stanja iz začetne pozicije čim bližje stanju $|a\rangle$.

Operaciji oraklja in difuzijskega operatorja v definiranem dvorazsežnostnem prostoru predstavljata različni zrcaljenji vektorja stanja. Operacija oraklja zrcali vektor zažetnaga stanja $|\psi_0\rangle$ preko $|e\rangle$, oziroma preko hiperravnine, ortogonalne na $|a\rangle$. Rezultat označimo kot $R_{|e\rangle}|\psi_0\rangle$. Difuzijski operator nato prezrcali stanje preko začetnega stanja $|\psi_0\rangle$. Novo stanje označimo kot $R_{|\psi_0\rangle}R_{|e\rangle}|\psi_0\rangle$. Skupen rezultat teh dveh zrcaljenj je rotacija vektorja stanja za kot 2θ v smeri proti iskanemu stanju $|a\rangle$ [31]. Slika 4.2 prikazuje zrcaljenji, ki predstavljata en korak Groverjeve iteracije.



Slika 4.2: Geometrijska ponazoritev ene iteracije Groverjevega algoritma. (Levo) Začetno stanje $|\psi_0\rangle$. (Levo) Stanje po zrcaljenju zaradi oraklja (zrcaljenje preko osi $|e\rangle$). (Desno) Končno stanje po zrcaljenju zaradi difuzijskega operatorja (zrcaljenje preko osi $|\psi_0\rangle$). (Slika povzeta po [31])

Groverjeva iteracija se ponovi tolikokrat, da se stanje čim bolj približa iskanemu stanju. Vemo, da vsaka iteracija zavrti stanje za 2θ. Število rotacij k bo optimalno takrat, ko bo skupni kot zasuka čim bližje $\frac{\pi}{2}$. Število rotacij k lahko izračunamo kot:

$$\frac{\pi}{2} \approx k \cdot 2\theta \Longrightarrow k \approx \frac{\pi}{4\theta}. \tag{4.9}$$

Za velike N velja $sin(\theta) \approx \theta$. Ker vemo da je $sin(\theta) = \frac{1}{\sqrt{N}}$, dobimo:

$$k \approx \frac{\pi}{4}\sqrt{N}. \tag{4.10}$$

4.3.3 Analiza algoritma

Algoritem 2 opisuje konstrukcijo orakla. Najprej za vsako označeno stanje poiščemo vse položaje kubitov z vrednostjo 0 in na te kubite apliciramo X vrata. Nato izvedemo večkratno kontrolirana Z vrata ter na koncu ponovno apliciramo X vrata, s čimer razveljavimo pomožne transformacije. Na ta način efektivno zgradimo orakel, ki ga lahko zapišemo z naslednjim predpisom:

$$|x\rangle \mapsto \begin{cases} -|x\rangle, & x \text{ je označeno,} \\ |x\rangle, & \text{sicer.} \end{cases} \tag{4.11}$$

Algoritem 2 Konstrukcija orakla za množico označenih stanj

```
1: function GroverOracle(oznacena_stanja)
2:     n ← dolžina(oznacena_stanja[0]) // Število kubitov.
3:     qc ← novoKvantnoVezje(n)
4:     for all oznaceno_stanje ∈ oznacena_stanja do
5:         rev_target ← obrniNiz(oznaceno_stanje)
6:         S_ničle ← { i ∈ {0,...,n−1} | (rev_target)_i = 0 }
7:         X(qc, S_ničle) // Apliciramo X vrata.
8:         MCMT(Z, n−1, 1) // Večkratno kontrolirana Z vrata.
9:         X(qc, S_ničle) // Razveljavitev pomožnih X vrat.
10:    end for
11:    return qc
12: end function
```

Algoritem 3 opisuje generiranje Groverjevega vezja za izbrano število kubitov. Celotna implementacija opisanih algoritmov je na voljo v GitHub repozitoriju [32].

Algoritem 3 Konstrukcija Groverjevega vezja

```
1: Naj bo n stevilo kubitov.
2: Naj bo M stevilo oznacenih stanj.
3: Naj bo N = 2^n
4: function GroverCircuit(oznacena_stanja)
5:     O_f ← GroverOracle(oznacena_stanja) // Konstrukcija orakla, algoritem 2.
6:     G ← GroverOperator(oracle) // Groverjev operator (orakel + difuzija).
7:     qc ← novoKvantnoVezje(n)
8:     k ← ⌊ (π/4) √(N/M) ⌋ // Optimalno število iteracij.
9:     qc ← qc ∘ G^k // Kompozitum k-kratnega Groverjevega operatorja.
10:    return qc
11: end function
```

4.3.4 Implementacija algoritma

V nadaljevanju implementiramo primer kvantnega vezja za Groverjev algoritem za $N = 8$, pri čemer obravnavamo funkcijo

$$f(x) = \begin{cases} 0, & x \in \{0,1,3,4,5,6,7\}, \\ 1, & x = 2. \end{cases} \tag{4.12}$$

Za realizacijo potrebujemo qubitni register za hranjenje kvantne besede dolžine tri. Najprej pripravimo superpozicijo za $N = 8$:

$$|\psi\rangle = H^{\otimes 3}|0\rangle^{\otimes 3} = H^3|000\rangle = \frac{1}{2\sqrt{2}}|000\rangle + \frac{1}{2\sqrt{2}}|001\rangle + \cdots + \frac{1}{2\sqrt{2}}|111\rangle. \tag{4.13}$$

Nato izvedemo orakelsko operacijo, ki pri vrednosti $f(010_2) = 1$ spremeni predznak amplitude v superpoziciji:

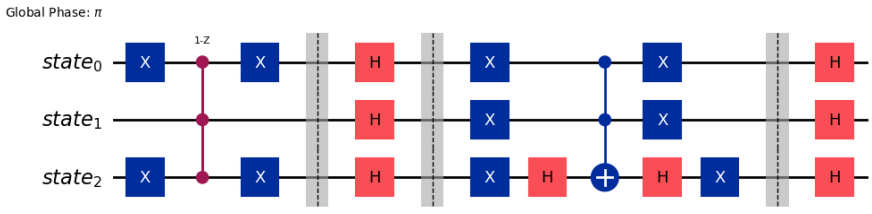
$$|\psi\rangle = \frac{1}{2\sqrt{2}}|000\rangle + \frac{1}{2\sqrt{2}}|001\rangle - \frac{1}{2\sqrt{2}}|010\rangle + \cdots + \frac{1}{2\sqrt{2}}|111\rangle. \tag{4.14}$$

Nato sledi *Groverjeva difuzija*, ki poveča amplitudo pri označenem stanju in zmanjša amplitude pri ostalih stanjih. Po *Groverjevi difuziji* dobimo naslednjo superpozicijo:

$$|\psi\rangle = \frac{1}{4\sqrt{2}}|000\rangle + \frac{1}{4\sqrt{2}}|001\rangle + \frac{5}{4\sqrt{2}}|010\rangle + \cdots + \frac{1}{4\sqrt{2}}|111\rangle. \tag{4.15}$$

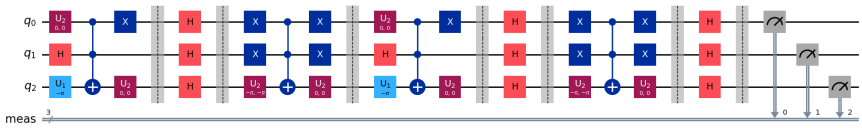
Opazimo, da je po prvi iteraciji verjetnost, da izmerimo napačen rezultat, še vedno razmeroma velika. Koraka izračuna orakla in Groverjeve difuzije moramo ponoviti približno šestkrat, da izboljšamo verjetnost izmerjenega rezultata.

Implementacija Groverjevega algoritma je izvedena v programskem jeziku Python z uporabo knjižnic `qiskit` in `qiskit_aer`. Algoritem lahko izvajamo na IBM-ovih kvantnih računalnikih. Za realizacijo Groverjevega operatorja potrebujemo osem Hadamardovih vrat in šest vrat CNOT, kot je razvidno na sliki 4.3.



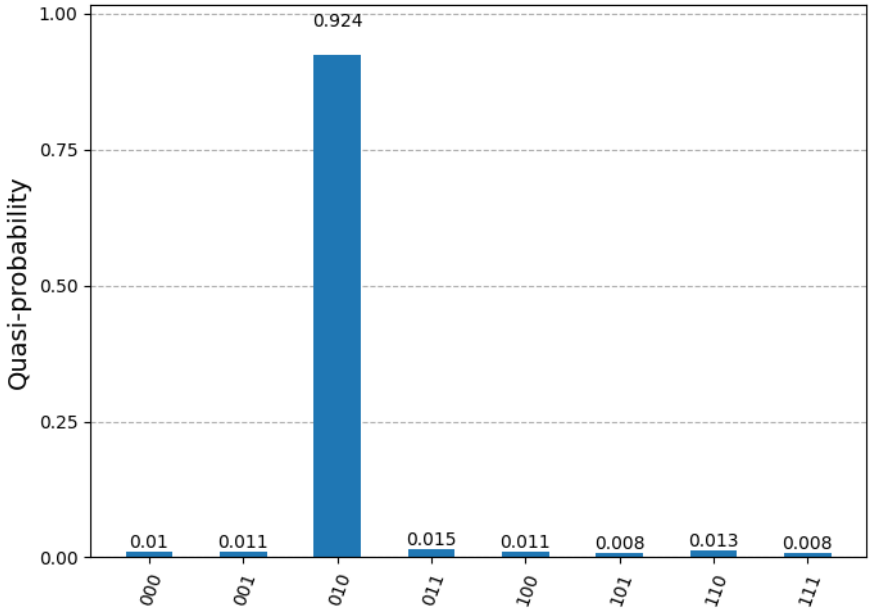
Slika 4.3: Implementacija vezja za Groverjev operator.

Algoritem smo testirali na klasičnem in kvantnem računalniku. Na sliki 4.4 je prikazano vezje Groverjevega algoritma, optimizirano za simulacijo na klasičnem računalniku.



Slika 4.4: Optimizirano vezje Groverjevega algoritma za simulacijo na klasičnem računalniku.

Za simulacijo kvantnega vezja smo uporabili simulator `AerSimulator`. Na sliki 4.5 so prikazane verjetnosti izmerjenih stanj pri 1000 ponovitvah.



Slika 4.5: Rezultati klasične simulacije Groverjevega algoritma.

4.3.5 Razširjeni Groverjev algoritem

Razširjeni Groverjev algoritem predstavlja poljubno izmed predlaganih izboljšav osnovnega algoritma. V osnovnem Groverjevem algoritmu predpostavljamo, da je vrednost parametra M, ki predstavlja število rešitev v preiskovalnem prostoru velikosti N, enaka $M = 1$. Groverjev algoritem obravnava tudi prostore z več rešitvami $M > 1$, če poznamo točno število M. V tem primeru na izhodu ne pričakujemo samo ene unikatne rešitve, temveč eno izmed M rešitev. Spremeni se tudi število izvedenih iteracij algoritma iz

$$k \approx \frac{\pi}{4}\sqrt{N} \tag{4.16}$$

iteracij na

$$k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}} \tag{4.17}$$

iteracij [33].

Glavni problem, ki ga mora rešiti razširjeni Groverjev algoritem, je primer $M > 0$ za neznano vrednost števila rešitev M. Tedaj ni možno izračunati točnega števila iteracij $k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}$. V primeru prevelikega števila iteracij za nek $M \geq N/2$ bi osnovni Groverjev algoritem propadel tako, da verjetnost uspeha postopno oscilira navzdol (angl. overshooting) [34]. To je razvidno iz izraza $\theta = \arcsin\left(2\sqrt{M(N-M)}/N\right)$, kjer vrednost θ pada v primeru $\frac{N}{2} \leq M \leq N$ [27]. Posledično začne število potrebnih iteracij za iskanje rešitve drastično naraščati.

Razširjeni Groverjev algoritem lahko, če ni znano ali velja $M \geq N/2$, poveča iskalni prostor z dodatnimi N elementi, ki ne vsebujejo rešitev [27]. Posledično zdaj manj kot polovica vseh elementov v iskalnem prostoru predstavlja iskane rešitve. To dosežemo z dodajanjem dodatnega qubita v iskalni indeks, kar podvoji število preiskovanih elementov na $2N$. Nova orakeljska funkcija O' nato označi element samo v primeru, da je rešitev, dodatni qubit pa nastavi na 0. Opisani iskalni problem ima samo M rešitev izmed $2N$ elementov, zato je za O' potrebno le $k = \pi/4\sqrt{2N/M}$ klicev, torej je to $\mathcal{O}(\sqrt{N/M})$ klicev nad $\mathcal{O}$ [27].

Razširjeni Groverjev algoritem lahko v primeru neznanega $M \leq N/2$ uspešno najde rešitev s kvantnim štetjem rešitev. Če lahko hitro najdemo število rešitev, lahko prav tako hitro najdemo tudi rešitev (kljub neznanemu M). V algoritmu BHMT so to dosegli z združevanjem idej Groverjevega in Shorovega algoritma za izvajanje amplitudne ocene za ugotavljanje M nad problemom približnega štetja (ugotavljanje $x \in M$, da bo $f(x) = 1$). Rezultati analize algoritma nakazujejo, da algoritem izvede $\mathcal{O}(\varepsilon^{-1}\sqrt{N/M})$ klicev (glede na napako ε). [35]

Ena izmed možnih rešitev je tudi razširitev Groverjevega algoritma tako, da izvajamo delno iskanje po blokih, kjer iščemo samo delni naslov rešitve [36]. Prvotni prostor velikosti N bi razdelili na K blokov, kjer je blok velik $b = N/K$. Nato naključno izberemo nek blok in nad njim izvedemo iskanje po klasičnem Groverjevem algoritmu. Povprečno število iteracij pade iz $N/2$ na $(N-b)/2$. Ker potrebujemo za Groverjev algoritem $\frac{\pi}{4}\sqrt{\frac{N}{M}}$ iteracij, bo rezultat odvisen od izbranega števila blokov

K. Potrebujemo tudi optimalne vrednosti j_1 in j_2, na podlagi katerih izvajamo j_1 globalnih in nato j_2 lokalnih (delnih) iskanj, kateri sledi še ena navadna Groverjeva iteracija [36].

4.4 Zaključek

Groverjev algoritem predstavlja pomemben primer kvantne prednosti pri iskanju, saj s kvadratno pospešitvijo presega klasične pristope. Njegova učinkovitost izhaja iz uporabe superpozicije in interferenčnih učinkov, kar potrdi tako matematična analiza kot praktična implementacija. Kljub omejitvam današnjih kvantnih naprav ostaja algoritem ključen gradnik prihodnjih kvantnih aplikacij,razširitve pa omogočajo njegovo uporabo tudi v bolj splošnih primerih.

4.5 Doprinos avtorjev

Ime in priimek	**Doprinos avtorja**
Bernard Kučina	Implementacija in analiza Groverjevega algortima
Dejan Jarc	Opis razširjenega Groverjevega algoritma
Jure Zupančič	Opis matematičnega ozadja algoritma
Maja Razinger	Pisanje uvoda, opis kvatnih osnov, ter opis Groverjevega algoritma.

Poglavje 5
Pregled uporabe kvantnih algoritmov v medicinih, logistiki in finančnem sektorju

Leon Alessio, Tilen Anzeljc, Timotej Blažič

Povzetek Hiter napredek kvantnega računalništva ponuja alternativne rešitve za reševanje kompleksnih problemov, ki presegajo zmogljivosti klasičnih računalniških sistemov. Ta članek podaja pregled raziskav in uporabe kvantnih algoritmov na treh pomembnih področij gospodastva: logistiki, financah in medicini. Podrobneje so analizirani konkretni primeri uporabe, kot so optimizacija poti vozil in skladišč, optimizacije finančnih portfeljev ter molekularne simulacije za odkrivanje novih zdravil. S pregledom aktualnih raziskav primerov članek osvetljuje, kako te tehnologije prihodnosti naslavljajo NP-težke probleme.

5.1 Uvod

Hiter tehnološki napredek in eksponentno naraščanje količine podatkov v moderni družbi ustvarjata vedno zahtevnejše računske izzive. Številni problemi v industriji in akademskih vodah spadajo v razred NP-težkih problemov, kjer se klasični računalniški sistemi kljub izboljšavam strojne opreme soočajo s svojimi fizikalnimi omejitvami. V tem kontekstu se kvantno računalništvo ponuja kot potencialna alternativa, ki z izkoriščanjem pojavov kvantne fizike odpira nove možnosti za reševanje problemov, ki so bili doslej obravnavani kot neobvladljivi.

V tem članku je podan pregled uporabe kvantnih algoritmov na treh pomembnih področjih gospodarstva: logistiki, finančnem sektorju in medicini. Čeprav so popolnoma delujoči kvantni računalniki s popravljanjem napak še niso sedanjost, se v vmesnem času (doba NISQ) pospešeno razvijajo variacijski, hibridni in kvantno navdahnjeni pristopi, ki že danes kažejo potencial za praktično uporabo.

V nadaljevanju najprej predstavljamo klasifikacijo kvantnih algoritmov, ki služi kot teoretični okvir za razumevanje različnih metodologij reševanja problemov. Osrednji del članka je namenjen podrobni obravnavi implementacije teh algoritmov v praksi. V logistiki se osredotočamo na optimizacijo poti vozil in upravljanje skladišč, v finančnem sektorju na optimizacijo portfeljev, analizo tveganj in vredno-

tenje finančnih derivatov, v medicini pa na molekularne simulacije, ključne za razvoj novih zdravil. Skozi pregled aktualnih raziskav in konkretnih primerov članek osvetljuje prednosti, ki jih kvantno računalništvo prinaša pri reševanju problem na teh treh področjih.

5.2 Klasifikacija kvantnih algoritmov za optimizacijo

Klasifikacija kvantnih algoritmov omogoča jasen in pregleden vpogled v pristope, ki se uporabljajo za reševanje optimizacijskih problemov s kvantnimi računalniki. V članku Review of quantum algorithms for medicine, finance and logistics [37] so kategorije razdelili na podlagi načina uporabe kvantnih lastnosti in povezovanja s klasičnimi pristopi. Pomembno je izpostaviti, da lahko nekatere algoritme uvrstimo v več kategorij hkrati.

5.2.1 Čisti kvantni algoritmi

Algoritmi, ki v celoti temeljijo na kvantnem računanju in izkoriščajo kvantne pojave, kot sta superpozicija in prepletenost.

- **Shorov algoritem**: Uporablja pri faktorizaciji celih števil. V primerjavi s klasičnimi metodami dosega eksponentno večjo učinkovitost.
- **Groverjev**: Kvantni iskalni algoritem, uporabljen predvsem v velikih in neurejenih bazah.

5.2.2 Variacijski kvantni algoritmi

Variacijski kvantni algoritmi uporabljajo nastavljiva (parametrizirana) kvantna vezja, ki ustvarjajo različna kvantna stanja. Cilj je najti takšne parametre, ki dajo najboljši približek optimalne rešitve.

- **Variational Quantum Eigensolver (VQE)**: Izračun energij in valovnih funkcij v molekularnih kvantnih sistemih.
- **Quantum Approximate Optimization Algorithm (QAOA)** Reševanje kombinatoričnih optimizacijskih problemov.
- **Quantum Amplitude Estimation (QAE)**: Uporablja se za ocenjevanje amplitud kvantnih stanj, pogosto v kombinaciji z drugimi podprogrami.

5.2.3 Kvantno navdihnjeni algoritmi (QIA)

Klasični algoritmi, ki posnemajo principe kvantne mehanike. Izvajajo se izključno na klasični strojni opremi.

- **Quantum-Inspired Genetic Algorithm (QIGA)**: Genetski algoritem, ki uporablja načela kvantne mehanike.
- **Quantum Evolutionary Algorithm (QEA)**: Uporablja biološko evolucijo, hkrati pa izkorišča kvantno računanje za pospešitev procesa iskanja.
- **Quantum Particle Swarm Optimization (QPSO)**: Simulira populacijo kvantnih delcev, ki se gibljejo v prostorih rešitev, s ciljem najti globalni minimum funkcije.
- **Quantum Walk Optimization Algorithm (QWOA)**: Uporablja kvantne sprehode za reševanje kombinatoričnih optimizacijskih problemov.

5.2.4 Hibridni algoritmi (HA)

Algoritmi, ki kombinirajo kvantno in klasično računanje ter izkoriščajo prednosti obeh pristopov.

- **Hybrid Quantum-Classical Genetic Algorithm (HQGA)**: Genetski in kvantni algoritem, ki združuje načela genetskih algoritmov s kvantnimi pristopi.
- **Quantum-Assisted Simulated Annealing (QASA)**: Algoritem, ki izboljšuje klasično metodo simuliranega žarjenja z vključitvijo kvantnih principov, kot so kvantno tuneliranje in superpozicija.

5.2.5 Kvantne nevronske mreže (QNN)

Algoritmi, ki uporabljajo kvantne pristope in nevronske mreže za reševanje problemov iz strojnega učenja in optimizacijskih problemov.

5.3 Logistika

Področje logistike se nenehno srečuje z optimizacijskimi problemi izjemne kompleksnosti, ki so ključni za znižanje operativnih stroškov in povečanje splošne učinkovitosti. Logistični procesi se hitro razvijajo, kar pa pomeni tudi višji nabor podatkov. To zahteva obdelavo velikega števila kombinatoričnih odločitev, kar je razlog za vpeljavo kvantnih in kvantno navdahnjenih optimizacijskih algoritmov. V logistiki se raziskave na tem področju običajno razdelijo na tri glavna področja. Za

vsako od teh področij so bili preučeni različni pristopi in metode, pri čemer so bili izpostavljeni ključni prispevki in primeri uporabe [37].

5.3.1 Optimizacija poti vozil

Optimizacija poti vozil je izziv logistike, kjer se je pojavilo največ raziskav o uporabi kvantnih in kvantno navdahnjenih metod. Klasični VRP (angl. Vehicle Route Planning) zajema optimizacijo poti za potrebe znižanja stroškov prevoza vozil. Cilj problema je zasnovati zaporedje več poti skozi več lokacij strank, pri tem pa upoštevati dodatne omejitve, ki se lahko pojavijo. [38] Podobni problemi, ki izhajajo iz VRP obsegajo:

- **Problem trgovskega potnika (angl . Travelling Salesman Problem):** Iskanje najkrajše poti, ki obišče vsa vozlišča natanko enkrat in se vrne na začetno točko.
- **Kapacitativni VRP (CVRP):** Dodaja omejitve glede nosilnosti vozil, kar zahteva optimizacijo ne le poti, temveč tudi razporeditve tovora.
- **VRP s časovnimi okni (VRPTW):** Vključuje omejitve, da mora dostava priti v določen časovni interval. Problem zahteva usklajevanje poti in časovnih omejitev.
- **Trajnostni VRP:** Minimizira porabo emisij in upošteva dodatne okoljske kriterije.
- **Ostali primeri:** VRP z droni - omejitve dosega in nevarna območja, optimizacija poti cenilcev, heterogeni VRP - različne fizične karakteristike vozil.

Na tem področju je bilo opravljenih največ raziskav:

- **Čisti kvantni algoritmi:** Prilagojene različice Groverjevega algoritma omogočajo vzorčenje omejenih rešitev in znatno zmanjšanje iskalnega prostora. Uporabni za hitro identifikacijo obetavnih poti pri problemih, kot je usmerjanje in polnjenje električnih vozil [39].
- **Variacijski kvantni algoritmi:** QAE in VQE se uporabljajo za iskanje optimalnih poti pri večini zgoraj omenjenih problemov. Izkaže se, da so te metode precej bolj učinkovite v primerjavi s klasičnimi. Na primer Srinivasan in sodelavci (2018) so razvili QAE za problem trgovskega potnika in dosegli kvadratno pospešitev v primerjavi s klasičnimi “brute-force“ metodami [40].
- **Kvantno-inspirirani algoritmi:** Kvantno-inspirirani algoritmi so bili uspešno uporabljeni predvsem za posebne VRP. Quantum-Inspired Immune Clonal Algorithm za CARP, z kvantimi metodami dokazuje hitrejšo konvergenco k optimalni rešitvi. Velja omeniti izboljšavo Quantum-Behaved Pigeon-Inspired Optimisation. S kvantnimi pristopi je bilo omogočeno izogibanje prezgodnji konvergenci in ujetosti v lokalnem optimumu, kar izboljša načrtovanje poti brezpilotnih letalnikov. [41, 42].
- **Hibridni algoritmi:** Najbolj razširjena je uporaba kvantno-inspiriranega tuneliranja s klasičnim simuliranim žarjenjem. Pristop se je izkazal za učinkovitega tudi pri reševanju novodefiniranih praktičnih problemov, kot je Adjuster Routing Problem (MORI & FURUKAWA, 2023) [43].

- **Kvantne nevronske mreže:** Correll in sodelavci (2018) so razvili algoritem za minimizacijo časa vožnje tovrnjakov pri kapacitativnem VRP. Omrežje uči vzorce optimalnega dodeljevanja poti in je sposobno generirati rešitve primerljive s človeškimi strategijami [44].

5.3.2 Praktična uporaba kvantnega žarjenja za Adjuster Routing Problem pri optimizaciji poti vozil

Avtorja Mori in Furukawa [43] sta preučevala Adjuster Routing Problem (ARP), specializirano različico problema usmerjanja vozil (VRP), ki obravnava dodeljevanje in načrtovanje poti zavarovalniških cenilcev škode po naravnih nesrečah. ARP vključuje dodatne omejitve, kot so različni tipi stavb, kompetence cenilcev, časovna okna in omejen delovni čas, kar problem uvršča med NP-težke kombinatorične naloge. Zaradi teh lastnosti je primeren za reševanje s kvantnim žarjenjem, kjer je mogoče omejitve natančno kodirati v QUBO obliko.

5.3.2.1 Dvofazni pristop

Za obvladovanje kompleksnosti sta avtorja ARP razdelila na dve fazi. S tem sta ločita prostorsko razporeditev nalog od časovne optimizacije in zagotovila manjšo izrabo kvantnih virov:

1. **Gručenje**: Določitev, katere stavbe pripadajo posameznim cenilcem.
2. **Usmerjanje**: Optimizacija vrstnega reda obiskov znotraj vsake gruče, z upoštevanjem časovnih oken in potovalnih časov.

5.3.2.2 QUBO formulacija

V članku je ARP formuliran z dvema ločenima QUBO modeloma, za gručenje in usmerjanje. Na primer za gručenje avtorja uporabita QUBO člen, ki omejuje prekoračitev največjega dovoljenega števila stavb, dodeljenih posameznemu cenilcu:

$$H_{\text{num}} = \sum_{a} \left(\sum_{\omega} x_{\omega,a} - N_a \right)^2, \tag{5.1}$$

pri čemer a označuje posameznega cenilca, ω stavbo, $x_{\omega,a}$ binarno spremenljivko, ki je 1, če je stavba ω dodeljena cenilcu a, in N_a največje dovoljeno število stavb za cenilca a. Če je cenilec preobremenjen ali ima premalo dodeljenih stavb, člen prispeva večjo energijo, kar vodi sistem k ustreznejši razporeditvi. Podobni QUBO modeli so uporabljen tudi za ostale omejitve v obeh fazah in nato združeni v fazi žarjenja.

5.3.2.3 Hamiltonova funkcija

Kvantno žarjenje poteka nad časovno odvisnim Hamiltonianom

$$\hat{H}(t) = A(t)\hat{H}_q + B(t)\hat{H}_{\text{problem}}, \tag{5.2}$$

kjer $\hat{H}_{\text{problem}}$ predstavlja uporabljen QUBO model, katerega osnovno stanje ustreza optimalni rešitvi. Hamiltonian $\hat{H}_q$ uvaja kvantne fluktuacije, ki sistemu omogočajo raziskovanje prostora rešitev in prehajanje čez lokalne minime. Funkciji $A(t)$ in $B(t)$ postopno zmanjšata vpliv fluktuacij in povečata vpliv problemnega Hamiltoniana, tako da sistem ob koncu doseže energijsko najnižjo konfiguracijo, ki predstavlja rešitev ARP.

5.3.2.4 Numerični rezultati

Za končno testiranje sta bila uporabljena dva reševalca iz D-Wave Ocean SDK:

- **Neal**: klasično simulirano žarjenje,
- **hybrid_v2**: hibridni kvantno-klasični reševalec.

Rezultati kažejo, da hybrid_v2 dosledno dosega nižje energijske vrednosti, predvsem pri večjih instancah. Pri vseh testnih primerih se izkaže za stabilnejšega in najde rešitve z nižjo energijo v obeh fazah. To potrjuje, da je kvantno žarjenje primeren pristop za reševanje logističnih problemov z večplastnimi kompleksnimi omejitvami, kot je ARP.

5.3.3 Načrtovanje logističnega omrežja

Naslednje pomembno področje logistike zajema načrotvanje logističnih omrežij:

- **Optimizacija lokacij distribucijskih centrov in logističnih vozlišč:** Problem rešuje urbanizirana območja z negotovim (stohastičnim) povpraševanjem, kjer se konfiguracija vozlišč prilagaja spremenljivim potrebam prebivalstva. Glavni cilj je minimizacija stroškov prevoza.
- **Postavitev tovarn in konfiguracija omrežja:** Vključuje načrtovanje fizične postavitve tovarn in povezav v omrežju. Upošteva se spremembe povpraševanja in logistične omejitve.
- **Optimizacija pretoka blaga in implementacija globalnih navigacijskih sistemov:** Problem zahteva učinkovito usklajevanje pretoka materiala in informacij po omrežju. V praksi to pomeni integracija globalnih navigacijskih sistemov za hitrejšo izvedbo transportnih operacij.

Za reševanje zgoraj opisanih problemov sta se izkazali kot učinkoviti dve skupini algoritmov, kvantno-inspirirani in hibridni:

- **Kvantno-inspirirani algoritmi:** Jiang in sodelavci so uporabili QPSO za zasnovo zelenega multimodalnega logističnega omrežja v urbanih območjih z negotovim povpraševanjem. Z vgrajeno metodo zaporednih povprečij so izboljšali stabilnost rešitev [45]. Shokry in Youssef so razvili Quantum Fingerprint Algorithm za natančnejšo lokalizacijo v geografskih območjih. Algoritem primerja podobnosti med lokacijami, ki so kodirane kot kvantna reprezentacija "prstni odtis" [46].
- **Hibridni algoritmi:** Malviya in sodelavci so uporabili QASA za optimizacijo lokacij distribucijskih centrov pri paketni dostavi. Uporabili Kerberos vzorčevalnik, dostopen na D-Wave [47].

5.3.4 Upravljanje skladišč

Zadnje področje logistike predstavlja upravljanje skladišč. NP - težki problemi zajemajo:

- **Nadzor zalog:** Cilj je uravnotežiti zaloge, stroške in razpoložljivost virov v skladišču. Sem spada znan problem Bin Packing Problem (BPP) in njegove več dimenzionalne različice.
- **Optimizacija pobiranja naročil in razporeditve prostora:** Osredotoča se na organizacijo skladišč, da se skrajša čas pobiranja naročil, zmanjša razdalja prevoženih poti in poveča kapaciteta skladiščnih lokacij.
- **Maksimalizacija dobička in izrabe virov:** Vključuje načrtovanje kompleksnih skladiščnih sistemov z omejitvami na delovno silo, vozila, skladiščne kapacitete in dostavne poti, pri čemer cilj ostaja maksimizirati dobiček, izkoristek razpoložljivih sredstev in zmanjšati izgube zaradi neizpolnjenega povpraševanja ali zamud.

Raziskave na področju upravljanja skladišč:

- **Čisti kvantni algoritmi:** Avtorji so uvedli kvantiziran algoritem iteracije politik za reševanje problema nadzora zalog, kjer je najzahtevnejši del reševanje velikih linearnih sistemov. Uporabili so HHL (Harrow–Hassidim–Lloyd) kvantni algoritem za optimizirano reševanje linearnih sistemov [48].
- **Variacijski kvantni algoritmi:** Avtorji so reševali problem minimizacije prevoženih razdalj pri pobiranju naročil v skladiščih. Kvantne pristope so integrirali v robotiko, in sicer na Raspberry Pi 4 [49].
- **Kvantno-inspirirani algoritmi:** Algoritmi QEA, QPSO in pristopi s kvantnim tuneliranjem so bili aplicirani na probleme, kot so Bin Packing Problem, Disassembly Line Balancing Problem in Joint Replenishment Problem. Njihova uporaba omogoča zmanjšanje stroškov, izboljšano izrabo prostora in učinkovitejše uravnavanje zalog.
- **Hibridni algoritmi:** Bozhedarov in sodelavci so s kvantnimi pristopi reševali Bin Packing Problem, uporaben za polnjenje izrabljenega jedrskega goriva v globoka odlagališča [50].

5.4 Finančni sektor

Finančni sektor je eno izmed najbolj konkurenčnih področij gospodarstva. V njem imajo že desetletja pomembno vlogo analitični modeli in algoritmi, ki podpirajo odločanje na podlagi ogromnih količin podatkov in hitro spreminjajočih se razmer. Področje se sooča z izrazito kompleksnimi izzivi, katerih uspešno reševanje močno vpliva na celotno gospodarstvo, pa tudi na življenje ljudi, ki niso neposredno vključeni v finančni sektor.
Kompleksnost teh problemov spodbuja raziskave in uporabo alternativnih računalniških pristopov za njihovo reševanje. Eno izmed najbolj obetavnih področij je kvantno računalništvo. V zadnjem desetletju je bilo razvitih več različnih kvantnih algoritmov za finančni sektor, predvsem za reševanje problemov, kot so optimizacija portfeljev, analiza tveganja in ocenjevanje vrednosti finančnih derivatov.
Pregledni članek [37] sistematično razvrsti raziskave na področju kvantnih algoritmov in njihovi uporavi v finančnem sektorju. V tem poglavju bomo predstavili ključne kvantne algoritme, ki se uporabljajo za reševanje treh različnih problemov v finančnem sektorju.

5.4.1 Optimizacija portfelja

Portfelj v finančnem smislu je zbirka finančnih instrumentov, kot so delnice, obveznice, opcije in drugi produkti, ki jih ima v lasti posameznik ali institucija. Glavni cilj upravljanja portfelja je maksimirati donosnost ob upoštevanju določene stopnje tveganja. Ta cilj predstavlja osrednji izziv problema optimizacije portfelja, iskanje optimalne kombinacije naložb, ki omogoča najboljše razmerje med donosom in tveganjem glede na določene omejitve.
Problem je matematično definiral Harry Markowitz že leta 1952 in hkrati tudi predstavil Markowitzev model [51]. Model določa optimalne uteži posameznih naložb v portfelju z uporabo kvadratne optimizacije, katere cilj je maksimirati pričakovani donos ob določeni ravni tveganja, pri čemer tveganje meri varianca ali standardni odklon donosov portfelja.
Kljub svoji učinkovitosti se klasični pristopi, kot je Markowitzev model hitro soočajo z omejitvami, kadar portfelji vključujejo veliko število sredstev ali kompleksne omejitve, kot so regulativne zahteve, omejitve likvidnosti ali dodatni kriteriji tveganja. Takšne situacije problem optimizacije pretvorijo v obliko, ki je klasificirana kot NP-težek problem. To pomeni, da se s povečanjem števila naložb računska zahtevnost povečuje eksponentno, s čimer hitro preseže zmogljivosti klasičnih računalnikov za reševanje problema v sprejemljivem času.
Prav zaradi omenjenih omejitev klasičnih pristopov se pojavlja potreba po alternativnih metodah, ki bi omogočale učinkovito reševanje optimizacijskih problemov velikih in kompleksnih portfeljev. V zadnjih nekaj letih je bilo razvitih več kvantnih algoritmov za reševanje problema optimizacije portfelja:

- Yalovetzky in sod. so razvili NISQ-HHL [52] (hibridno različico HHL algoritma) za reševanje manjših problemov optimizacije portfelja na trenutnih Noisy Intermediate-Scale Quantum napravah.
- Mugel in sod. [53] so implementirali različne metode kot so kvantno ohlajanje (Quantum annealing), VQA (Variacijski kvantni algoritem) in tenzorske mreže za dinamično optimizacijo portfelja, pri čemer so uporabili podatke o dnevnih cenah 52 finančnih sredstev za obdobje 8 let.
- Herman in sod. [54] so preizkusili pristop z uporabo QAOA (Kvantni približni optimizacijski algoritem) za reševanje problema optimizacije portfelja.
- Cadavid in sod. so leta 2023 razvili DCQO (Digitized-Counterdiabatic Quantum Optimization) algoritem za optimizacijo portfelja [55]. Pristop so preizkusili na realnem primeru portfelja s 20 sredstvi.
- Slate in sod. (2021) [56] so uporabili pristop s QWOA (Quantum Walk Optimisation Algorithm), ki omogoča znatno boljšo učinkovitost pri iskanju visokokakovostnih rešitev problema optimizacije portfelja v primerjavi z nekaterimi predhodnimi deli.

Podrobneje bomo opisali implementacijo tenzorskih mrež s strani Mugel in sod. Namreč ta metoda je v večini primerov dosegla najboljše rezultate v primerjavi z ostalimi metodami njihove raziskave.

Avtorji so se lotili reševanja problema dinamične optimizacije portfelja. Reševanje tega klasičnega optimizacijskega problema zahteva določitev uteži, vektor $\vec{w}$, različnim sredstvom, vektor $\vec{q}$, skozi neko časovno obdoblje, predstavljeno s $t = t_i, t_{i+1}, ..., t_j$. Donos portfelja je definiran s spodnjo enačbo:

$$\text{Donos} = \sum_{t=t_i}^{T} q_t \, \vec{w}_t$$

Stopnja tveganja portfelja je pa definirana z enačbo:

$$\text{Risk} \equiv \tfrac{1}{2} \sum_{t=t_i}^{t_f} \vec{w}_t^T \, \Sigma_t \, \vec{w}_t$$

, pri čemer je Σ_t kovariančna matrika zgodovinskih donosov sredstev. V praksi pogosto velja, da je stopnja investicije čez čas fiksna, kar pomeni, da lahko uteži normaliziramo.
Algoritem mora najti optimalno pot uteži naložb skozi čas, tako da minimizira spodnjo Hamiltonovo enačbo:

$$H = \sum_{t=t_i}^{t_j} \left[-q_t^T w_t + \tfrac{\gamma}{2} w_t^T \Sigma_t w_t + \rho (\mathbf{u}^T w_t - 1)^2 \right]$$

, kjer del:

- $-q_t^T w_t$ maksimizira donos,
- $\frac{\gamma}{2} w_t^T \Sigma_t w_t$ minimizira stopnjo tveganja glede na podan parameter γ, ki predstavlja naklonjenost k tveganju
- $\rho (\mathbf{u}^T w_t - 1)^2$ predstavlja omejitev, da je vsota uteži enaka 1.

Kvaliteto rešitve algoritma, pot uteži, ki predstavlja portfelj skozi čas se kvantificira z uporabo Sharpeovega razmerja [57]:

$$\text{Sharpe} \equiv \frac{\sum_{t=t_i}^{t_j} q_t^T w_t}{\sqrt{\sum_{t=t_i}^{t_j} w_t^T \Sigma_t w_t}}$$

, ki predstavlja donos, glede na stopnjo tveganja, kar pomeni, da višja kot je vrednost, večja je donosnost portfejla glede na stopnjo tveganja, medtem ko manjša vrednost predstavlja manjšo donosnost, negativna pa izgubo.
Tu je predstavljena osnovna verzija tega problema, avtorji so dodali še upoštevanje cen posameznih transakcij.

Mugel in sod. so pri njihovem pristopu z uporabo tenzorskih mrež najprej diskretizirali prostor rešitev tako, da so za vsako sredstvo q_i celoten interval razdelili na K diskretnih nivojev. Tako če imamo N sredstev, je v nekem časovnem trenutku možnih K^N stanj, kar je dimenzija fizičnega indeksa tenzorja A_t. Uporabili so nato Stanje produkta matrik, MPS (Matrix product state), enodimenzionalno strukturo, ki jo sestavljajo tenzorji A_i. Vsak tenzor A_i je sestavljen iz fizičnega indeksa, ki predstavlja možne konfiguracije uteži in povezave dimenzije vrednosti veznega parametra D s prejšnjim stanjem A_{i-1} ter naslednjim stanjem A_{i+1}. Na ta način je število parametrov v MPS polinomialno namesto eksponentno kot v ostalih metodah. Hamiltonovo funkcijo, ki so jo minimizirali so pretvorili v kvantni Hamiltonov operator $\mathscr{H}$. Definirajo enočlenske operatorje za donosnost in tveganje v določenem času t_i, ki delujejo le na enem tenzorju verige ter dvočlenske operatorje za cene transakcij, ki povezujejo dva člena skupaj, saj en tenzor vpliva na drugega. Za iskanje optimalnega stanja so uporabili tehniko Evolucije v imaginarnem času (iTE), ki konvergira stanje MPS v osnovno štanje minimalne energije", tako da začne v naključnem stanju in nato v vsakem koraku algoritma aplicira operator evolucije, s katerim zmanjša prispevek slabim rešitvam. Po nekaj korakih stanje v MPS konvergira v osnovno stanje. Nato se optimalna pot uteži sestavi tako da se za vsak čas t_i določi konfiguracija w_t z največjo verjetnostjo v osnovnem stanju. Pridobljena rešitev se nato kvantificira z uporabo Sharpeovega razmerja.

5.4.2 Analiza tveganja

Analiza tveganja v finančnem sektorju predstavlja proces ocenjevanja možnosti, da določene naložbe ali portfelji utrpijo izgubo, ter kvantificiranja velikosti teh potencialnih izgub. Obstaja več različnih vrst tveganj, med najbolj pomembnimi so:

- Tržna tveganja (Market Risk): Povezana so z nihaji cen finančnih instrumentov zaradi sprememb na trgu, kot so spremembe obrestnih mer, valutnih tečajev ali cen delnic in obveznic.
- Kreditna tveganja (Credit Risk): Nastanejo, kadar posojilojemalec ali protistranka ne izpolni svojih obveznosti, kar lahko vodi do izgube za finančno institucijo.
- Likvidnostna tveganja (Liquidity Risk): Pojavijo se, ko finančni instrumenti ne morejo biti prodani ali pretvorjeni v gotovino brez znatnega vpliva na ceno.

- Operativna tveganja (Operational Risk): Vključujejo tveganja, ki izhajajo iz napak v procesih, sistemih ali človeških napak.

Pred vsako naložbeno odločitvijo se običajno izvede določena oblika analize tveganja, ki pomaga oceniti, kako bi ta naložba lahko reagirala na različne tržne razmere. Tako lahko investitorji in finančne institucije predvidijo potencialne izgube, primerjajo tveganje med različnimi sredstvi in sprejemajo odločitve, ki so skladne z njihovo toleranco do tveganja.

Klasični pristopi za analizo tveganja vključujejo metode, kot so Value at Risk (VaR), Condition Value at Risk (CVaR), Monte Carlo simulacije itd. Čeprav so ti pristopi široko uporabljeni, se pri zelo kompleksnejših problemih hitro soočajo z omejitvami kot so eksponentna rast računske zahtevnosti ali potreba po ogromnih količinah simulacijskih podatkov.

V zadnjem desetletju se aktivno raziskuje uporaba kvantnih algoritmov za analizo različnih tveganj, med katerimi so:

- Stamatopoulos in sod. (2022) so razvili kvantni gradientni algoritem za oceno tržnega tveganja finančnih derivatov [58].
- Woerner in Egger sta že leta 2019 razvila kvantni algoritem za oceno amplitude za izračun VaR in CVaR, ki doseže skoraj kvadratno pohitritev v primerjavi z Monte Carlo simulacijami [59].
- Egger in sod. (2021) so predstavili kvantni algoritem za oceno kreditnega tveganja, ki omogoča učinkovitejšo oceno v primerjavi s klasičnimi Monte Carlo simulacijami [60].
- Dri in sod. (2023) so razvili izboljšano različico kvantnega algoritma za analizo kreditnega tveganja, ki upošteva več sistemskih dejavnikov tveganja za vsako sredstvo, omogoča uporabo realnih podatkov [61].

5.4.3 Vrednotenje finančnih derivatov

Finančni derivati so pogodbe, katerih vrednost je odvisna od gibanja nekega drugega sredstva, kot so delnice, obveznice itd. Sami po sebi nimajo lastne vrednosti, temveč predstavljajo dogovor o prihodnjih obveznostih, katerih velikost določa sprememba vrednosti drugega sredstva. Med najpogosteje uporabljenimi izvedenimi finančnimi instrumenti so opcije, terminske pogodbe in zamenjave obrestnih mer. Najpogosteje se uporabljajo za zavarovanje pred tržnimi tveganji ali za špekulativno trgovanje.

Moderni pristopi za vrednotenje finančnih derivatov vključujejo analitične metode, kot na primer Black–Scholesov model [62], in numerične metode. Podobno kot pri ostalih problemih v finančnem sektorju se uporabljajo tudi Monte Carlo simulacije.

V zadnjih letih se raziskave usmerjajo tudi v uporabo kvantnih algoritmov za vrednotenje finančnih derivatov, kot so:

- Rebentrost in sod. so že leta 2018 razvili kvantni algoritem za izvajanje Monte Carlo simulacij s ciljem vrednotenja finančnih derivatov [63].
- Kubo in sod. (2022) so predlagali pristop variacijske kvantne simulacije za vrednotenje večsredstvenih derivatov. [64].
- Chakrabarti in sod. (2021) so predlagali pristop, ki uporablja prednaučena variacijska kvantna vezja za vrednotenje finančnih derivatov, s čimer znatno zmanjšajo porabo računalniških virov [65].
- Kaneko in sod. (2022) so predstavili dve implementaciji kvantnih algoritmov z uporabo modela lokalne volatilnosti, kjer je volatilnost odvisna od cene in časa. [66].

5.5 Medicina

Kvantno računalništvo se v medicini pojavlja v različnih vlogah, od simulacije molekul in načrtovanja zdravil do rekonstrukcije medicinskih slik ter analize genomskih in kliničnih podatkov. Pregledni članek [37] sistematično razvrsti obstoječe kvantne algoritme po področjih uporabe in v poglavju o zdravstvu predstavi več tipičnih razredov problemov (molekularne simulacije, medicinsko slikanje, klinično odločanje), skupaj z uporabo kvantnih, variacijskih algoritmov in kvantnega strojnega učenja.

Pri interpretaciji teh problemov in njihovem pomenu za biomedicino se opiramo tudi na novejše domensko usmerjene preglede, ki obravnavajo vlogo kvantnega računalništva v biomedicinskih raziskavah, zdravstveni obravnavi in personalizirani medicini [67, 68]. Ti članki dodatno osvetlijo, zakaj so posamezni problemi računsko zahtevni (npr. eksponentna rast konformacijskega prostora pri prepogibanju proteinov, slabo pogojevani inverzni problemi pri CT/MRI rekonstrukciji ali visoka dimenzionalnost in heterogenost genomskih ter kliničnih podatkov) ter kje kvantni algoritmi potencialno prinašajo prednost pred klasičnimi pristopi.

5.5.1 Molekularne simulacije za načrtovanje zdravil

Molekularne simulacije (prepogibanje proteinov, molekularno umeščanje, optimizacija zaporedij) so ključne za razumevanje bolezni in načrtovanje zdravil, a so izjemno računsko zahtevne. Število možnih konformacij proteina in možnih načinov vezave liganda na tarčni protein narašča eksponentno, energijske površine pa imajo ogromno lokalnih minimumov, zato klasične metode (MD, Monte Carlo, heuristike) hitro postanejo neizvedljive ali zahtevajo močno poenostavitev modela [67, 37].

Kvantni algoritmi te probleme prevajajo v optimizacijske (QUBO/Ising) ali lastno-vrednostne probleme, ki jih rešujejo variacijski pristopi (VQE, QAOA, DC-QAOA) in kvantne oziroma kvantno-inspirirane metode strojnega učenja. S tem izkoriščajo superpozicijo in interferenco za bolj učinkovito preiskovanje konfor-

macijskega prostora ter bolj naravno opisujejo kvantno-kemijske interakcije, kar je posebej obetavno za zgodnjo fazo odkrivanja zdravil (rangiranje kandidatov, ocena energij vezave, optimizacija struktur) [37, 69].

5.5.1.1 QAOA formalizacija problema molekularnih simulacij

Probleme prepogibanja proteinov in molekularnega umeščanja lahko obravnavamo kot diskretno optimizacijo nad binarnimi konfiguracijami $\mathbf{z} \in \{0,1\}^n$, kjer objektna funkcija $C(\mathbf{z})$ predstavlja (diskretizirano) energijo konfiguracije. Zapišemo jo kot vsoto lokalnih členov

$$C(\mathbf{z}) = \sum_{\alpha=1}^{m} C_\alpha(\mathbf{z}), \tag{5.3}$$

pri čemer posamezni C_α opisujejo prispevke kontaktnih potencialov in kazni za neveljavne konformacije. QAOA nad tem problemom pripravi parametrizirano kvantno stanje

$$|\gamma,\beta\rangle = U_B(\beta_p)U_C(\gamma_p)\cdots U_B(\beta_1)U_C(\gamma_1)\,|s\rangle\,, \tag{5.4}$$

kjer sta γ in β vektorja variacijskih parametrov, $U_C(\gamma)$ in $U_B(\beta)$ pa enotska operatorja, ki uveljavljata energijski in mešalni del algoritma [70]. Kakovost dobljenih konformacij merimo z pričakovano vrednostjo energije

$$F_p(\gamma,\beta) = \langle\gamma,\beta|\,\hat{C}\,|\gamma,\beta\rangle\,, \tag{5.5}$$

pri čemer z optimizacijo parametrov (γ,β) maksimiziramo F_p (oziroma minimiziramo energijo), meritev stanja $|\gamma,\beta\rangle$ v računski bazi pa z visoko verjetnostjo vrne nizkoenergijske konformacije, ki ustrezajo stabilnim strukturam proteina [70].

- Fingerhuth in sod. (2018) formualirajo problem prepogibanja proteinov na mreži kot optimizacijo nad Isingovim modelom in ga rešujejo z QAOA. Študirajo, kako se povečuje število izvedljivih operacij in pokažejo, da se z naraščajočim številom nivojev QAOA izboljšuje kakovost dobljenih konformacij [71].
- Robert in sod. (2021) uporabijo VQE za model prepogibanja polimerne verige na mreži. Primerjajo časovno zahtevnost in natančnost s klasičnimi pristopi ter pokažejo, da VQE ob danem številu qubitov omogoča krajši čas izračuna pri primerljivi kakovosti rešitev [72].
- Mustafa in sod. (2022) sistematično primerjajo VQE in QAOA na več biokemijskih primerih prepogibanja proteinov. Rezultat je, da VQE pri teh instancah dosega bolj stabilne energije in boljše konformacije kot QAOA, kar je pomembno za izbiro algoritma na konkretni kvantni platformi [73].
- Ding in sod. (2023) predstavijo QAOA (DC-QAOA) in ga aplicirajo na molekularno umeščanje, vključno z biološko pomembnimi sistemi, kot so kompleksi SARS-CoV-2 Mpro. DC-QAOA doseže bolj natančne in biološko relevantne konfiguracije za umeščanje kot standardni QAOA [74].
- Mensa in sod. (2023) obravnavajo problem odkrivanja novih zdravil kot klasifikacijski problem in uporabijo QSVM na kvantno-inspiriran način. Pokažejo iz-

boljšanje performančnih metrik v primerjavi z nekaterimi klasičnimi modeli ter izpostavijo, kako kvantni prostor značilk (*feature space*) pomaga pri ločevanju aktivnih in neaktivnih spojin [75].

5.5.2 Rekonstrukcija in analiza medicinskih slik

Pri CT, MRI in drugih modalitetah je treba iz omejenega, šumnega nabora meritev rekonstruirati visoko-dimenzionalne slike. Gre za klasičen inverzni problem, ki je pogosto slabo pogojevan, zato zahteva iterativne optimizacijske metode, ki so pri visokih ločljivostih in nizkih dozah sevanja zelo računsko drage. Poleg rekonstrukcije pride še faza avtomatske analize (segmentacija tumorjev, klasifikacija lezij, detekcija COVID-19 itd.), kjer kombinacija velikih slikovnih podatkov in omejenega števila označenih primerov oteži učenje robustnih modelov [76, 37].

Kvantni in kvantno-inspirirani algoritmi se pojavljajo tako v rekonstrukciji kot v analizi slik. Kvantno-podprte rekonstrukcijske sheme (QIR, QACT, kvantno-inspirirane optimizacije) ciljajo na boljši kompromis med dozo in kakovostjo slike z učinkovitejšim reševanjem rekonstrukcijskih optimizacijskih problemov. Kvantni modeli strojnega učenja (QNN, QCNN, hibridni HNN, QSVM) pa se uporabljajo za klasifikacijo in segmentacijo medicinskih slik, kjer izkoriščajo kvantni prostor značilk, da z manj parametri dosegajo primerljivo ali boljšo natančnost na CT, MRI, rentgenskih in drugih slikah, posebej pri manjših zbirkah podatkov [37, 77].

- Kiani in sod. (2020) predlagajo čisto kvantni algoritem za rekonstrukcijo medicinskih slik, ki teoretično doseže eksponentno pospešitev operacij v primerjavi s klasičnimi metodami. Predstavijo tudi kvantne postopke za ekstrakcijo informacij iz kvantno rekonstruirane slike [78].
- Jun (2023) zapiše problem CT rekonstrukcije kot optimizacijo, ki jo rešuje QAOA. Algoritem je zasnovan tako, da je združljiv z različnimi tipi CT geometrij in potencialno izboljša kakovost rekonstrukcije pri manjšem številu projekcij [79].
- Huflage in sod. (2023) so uvedli kvantno-iterativni rekonstrukcijski algoritem (QIR) za ultra visoko ločljivostni CT s fotonskim štetjem. Rezultati kažejo zmanjšanje šuma in boljše razmerje kontrast–šum v kostnem in mišičnem tkivu [80].
- Zhang in sod. (2022) uporabijo QWPSO za segmentacijo možganskih tumorjev na medicinskih slikah. Algoritem je posebej zasnovan za izolacijo zabrisanih regij in se izkaže za učinkovitega pri tipičnih nalogah segmentacije [81].
- Parisi in sod. (2022) predlagajo kvantno navdahnjeni aktivacijski funkciji QReLU in m-QReLU za konvolucijske nevronske mreže pri klasifikaciji medicinskih slik. Na sedmih podatkovnih zbirkah za odkrivanje Parkinsonove bolezni in COVID-19 pokažejo, da takšni modeli presegajo devet klasičnih aktivacijskih funkcij glede točnosti, natančnosti, priklica in F1-mere [82].

- Khan in sod. (2024) uporabijo QCNN za klasifikacijo možganskih tumorjev. Eksperimenti kažejo visoko natančnost in hitrejšo konvergenco učenja v primerjavi s klasičnimi modeli [83].

5.5.3 Genomska in klinična analiza za personalizirano medicino

Genomska analiza in obdelava velikih biomedicinskih zbirk sta tipična primera problemov z ogromnim prostorom stanj. Število možnih poravnav in filogenetskih dreves narašča eksponentno, baze genomskih podatkov pa so masivne, kar otežuje hitro in natančno analizo na klasičnih sistemih. Na ravni kliničnega odločanja želi natančna medicina kombinirati genomske, klinične in druge podatke za napoved tveganja, diagnozo in izbiro terapije, a se srečuje z visoko dimenzionalnostjo, heterogenostjo podatkov in omejenim številom dobro označenih primerov [37, 76].

Kvantni algoritmi tukaj ciljajo na pospešitev iskanja in optimizacije (npr. Groverjevi algoritmi za genomske probleme, kvantno-inspirirani optimizatorji za filogenetska drevesa) ter na izboljšanje klasifikacije v visoko-dimenzionalnih kliničnih podatkih preko QML (QSVM, QNN, hibridni modeli). Prednosti, ki jih prinašajo ti algoritmi so naslednje: hitrejše in natančnejše obdelave genomskih podatkov, napoved odziva na terapijo, stratifikacija tveganja ter podpora personaliziranemu zdravljenju [37, 68, 84].

V skladu z Hoogland in sod. [85] individualiziran učinek zdravljenja (ITE) za bolnika z značilkami x_i definiramo kot

$$\delta(x_i) = \mathbb{E}\big(Y_{a=1} \mid X = x_i\big) - \mathbb{E}\big(Y_{a=0} \mid X = x_i\big). \tag{5.6}$$

Ta količina formalno opisuje problem personalizirane medicine: cilj je za vsakega bolnika oceniti $\delta(x_i)$ ter na tej osnovi izbrati zdravljenje, ki maksimizira pričakovani izid.

- Kaur in sod. (2022) predstavijo kvantni algoritem, ki temelji na QWOA, za diagnostiko depresije. QWOA doseže izboljšano napovedno zmogljivost pri klasifikaciji pacientov na depresivne in nedepresivne v primerjavi z nekaterimi klasičnimi heuristikami [86].
- Sarkar in sod. (2021) predlagajo algoritem za obdelavo in rekonstrukcijo genomskih zaporedij, ki temelji na prilagojeni različici Groverjevega iskanja. Cilj je kvadratno pospešiti postopke sestavljanja DNA [87].
- Niraula in sod. (2021) razvijejo kvantni algoritem, ki podpira odločanje pri prilagodljivi radioterapiji, zlasti pri oceni doze za posameznega pacienta. Algoritem optimizira oceno doze med zdravljenjem in tako podpira bolj personalizirano terapijo [88].
- Moradi in sod. (2022) predlagajo dva čisto kvantna algoritma za napovedne naloge na realnih kliničnih podatkih in ju preizkusijo na IBM kvantni strojni opremi. Rezultati kažejo obetavno napovedno učinkovitost in demonstrirajo izvedljivost kvantnih metod na dejanskih medicinskih podatkih [89].

- Nijaguna in sod. (2023) uvedejo QFFA (Quantum Fruit Fly Algorithm) za klasifikacijo v medicinski diagnostiki na velikih naborih podatkov. Pokažejo izboljšano učinkovitost klasifikacije pri kompleksnih, visoko-dimenzionalnih medicinskih atributih [90].
- Gupta in sod. (2022) uporabijo VQC za napoved diabetesa iz kliničnih podatkov. Model poveča napovedno moč v primerjavi z izbranimi klasičnimi metodami in služi kot orodje za zgodnje odkrivanje bolezni [91].

5.6 Zaključek

Kvantni algoritmi že danes ponujajo smiselne pristope k NP-težkim optimizacijskim problemom v logistiki, financah in medicini. Z razdelitvijo pristopov na čiste kvantne, variacijske, kvantno navdahnjene, hibridne algoritme in kvantne nevronske mreže dobimo enoten okvir za primerjavo med teoretičnim kvantnim pospeškom in praktično izvedljivostjo na realni strojni opremi.

V logistiki se kvantno navdihnjeni in hibridni pristopi uporabljajo za optimizacijo poti vozil in načrtovanje omrežij, v financah za optimizacijo portfeljev, vrednotenje derivatov in oceno tveganja, v medicini pa predvsem za molekularne simulacije in analizo medicinskih podatkov. Večina rezultatov je za zdaj omejena na manjše primere in simulatorje, ključni izzivi pa ostajajo učinkovito kodiranje podatkov, robustnost na šum ter skaliranje na realne industrijske instance. Kljub temu se kaže, da bodo z napredkom strojne opreme in programskih orodij hibridni in kvantno navdahnjeni algoritmi postali pomemben dopolnilni del optimizacijskih orodij v logistiki, financah in medicini.

5.7 Doprinosi avtorjev

Delo smo si razdelili tako, da si je vsak avtor izbral svoje področje ter ter pripravil ustrezno poglavje. Tilen Anzeljc je pokril področje logistike, Leon Alessio finančni sektor, Timotej Blažič pa medicino.

Poglavje 6
Predstavitev in primerjava različnih tipov kvantnih računalnikov

Marko Delić

Povzetek Moorov zakon zapoveduje, da se vsaki dve leti število tranzistorjev na integriranem vezju podvoji, kar pomeni, da tranzistorji postajajo vse manjše nanoelektronske naprave. Želeno delovanje takšnih tranzistorjev pričnejo ovirati kvantni pojavi ter sile med elektroni. Zato so v zadnjem času tehnološki velikani začeli razvijati ter raziskovati koncept kvantnih računalnikov, ki bi tovrstne ovire ne le odpravili, pač pa tudi izkoristili za hitrejše delovanje ter manjšo porabo energije. Obstaja več različnih fizičnih sistemov kvantnega računalništva, a ni povsem jasno katera tehnologija se bo na koncu dokazala kot najučinkovitejša. Zato bom v tem poglavju predstavil dva vodilna pristopa, ju primerjal ter izpostavil njune ključne izzive.

6.1 Metodologija

Pri pripravi seminarske naloge sem uporabljal princip sistematičnega pregleda literature. Za iskanje virov sem uporabljal spletni iskalnik znanstvene literature `Google Scholar` (`https://scholar.google.com/`), ki omogoča hiter izbor vsebinsko ustreznih, uveljavljenih in primernih virov na podlagi števila citatov ter leta izdaje vira. Področje kvantnega računalništva vključuje veliko naprednih fizikalnih konceptov, zato sem zahtevnejše dele literature preverjal z dodatnimi razlagami in pregledi podpornih virov.

Na podlagi zbranih informacij sem oblikoval strukturo naloge, ki omogoča primerjavo različnih tehnik zasnove kvantnih računalnikov, predstavi vsako izmed tehnik ter poudarja njihove ključne prednosti in omejitve.

6.2 Tipi Kvantnih Računalnikov

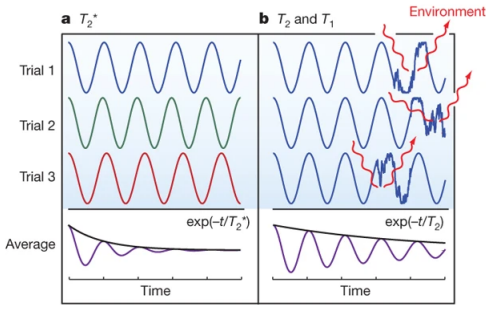
Vsi tipi kvantnih računalnikov si delijo skupen kritičen nabor zahtev, ki jih pogosto opredeljujemo kot *closed box* zahteve; Delovanje kvantnega računalnika mora biti izolirano od okolja. Že majhna količina informacij, ki uhaja iz sistema lahko zmoti ranljive kvantno-mehanične valove od katerih je odvisen sam računalnik, kar povzroči destruktiven proces imenovan `dekoherenca` (izguba informacij sistema v okolju). [92]

Koherenčni čas je torej trajanje dokler `kubit` (abstraktna kvantno-informacijska enota) ohranja svoje kvantne lastnosti - zlasti `superpozicijo` in `prepletenost` preden jih izgubi zaradi interakcije z okoljem. V splošnem poznamo dva tipa konherenčnega časa:

T_1 `Čas relaksacije`; Čas, v katerem kubit izgubi energijo in preide iz vzbujenega v osnovno stanje

T_2 `Čas defaziranja`; Čas, v katerem se ohrani fazna informacija valovne funkcije kubita (čas ohranjanja fazne koherence)

[93]

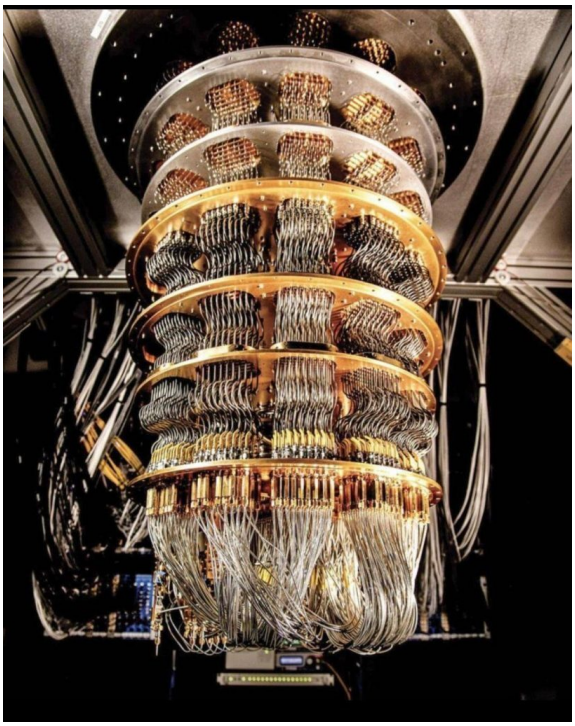


Slika 6.1 Na sliki je prikazan vpliv dekoherence na `oscilacijo` (kvantno nihanje); Del 1a prikazuje, da ima oscilator (naš kubit) v vsakem poizkusu malo drugačno frekvenco. Ko te poizkuse povprečimo, se valovanje navidezno zaduši. Ta pojav merimo s časom T_2*, ki predstavlja čas defaziranja v večih poizkusih. V delu 1b pa je prikazano kaj se dogaja v enem samem poizkusu. Okolje povzroča naključne fazne anomalije, ki kvarijo koherenco oscilatorja. To dušenje opisuje čas T_2 oziroma čas defaziranja. Poleg tega pa sistem sčasoma izgublja energijo in preide v osnovno stanje po času T_1. [92]

V nadaljevanju bom podrobneje predstavil vodilni platformi oziroma tipa kvantnih računalnikov, ki na različne fizikalne načine realizirata abstraktni kubit ter mu dovoljujeta kvantne operacije (`kvantna logična vrata`), superpozicijo, prepletenost in meritve. Ne glede na uporabljeni pristop pa morajo kvantni procesorji zagotavljati zadostno izolacijo od okolja, saj je ravno koherentno ohranjanje kvantnega stanja pogoj za izvajanje kvantnih operacij.

6.2.1 Superprevodna Vezja

Kvantni pojavi so zaradi mikroskopske velikosti težavni za observacijo, kar močno oteži implementacijo kvantnih logičnih vrat in kubitov. En način, da naredimo kvantne učinke observabilne je uporaba superprevodnih vezij, kjer definiramo makroskopske induktorje (označevali bomo z L) in kondenzatorje (označevali bomo s C). Pri tovrstnih sistemih je ključnega pomena, da ves čas ohranjamo nizke temperature.



Slika 6.2: Kvantni računalnik tipa superprevodnih vezij.

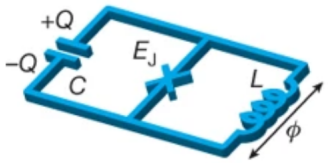
6.2.1.1 Realizacija Kubita

Nosilci naboja pri superprevodnih vezjih so elektroni, ki se pri nizkih temperaturah povežejo v takoimenovane *Cooper pare*. Ti pari skupaj tvorijo enotno kvantno stanje brez upora in s skupno fazo. Ta faza (označevali bomo s Φ) skozi induktor in naboj na kondenzatorju (označevali bomo s Q) sta osnovni kvantni spremenljivki, ki določata energijska stanja sistema, izmed katerih izberemo dve, ki omogočata superpozicijo kubita.

Energijsko stanje takšnega sistema je tako določeno s:

- potenicalno enerijo
$$\frac{\Phi^2}{2L} \tag{6.1}$$
- kinetično energijo
$$\frac{Q^2}{2C} \tag{6.2}$$

Takšen sistem je znan kot kvantni harmonični oscilator, kjer so energijska stanja enakomerno razporejena. Za izdelavo kubita pa potrebujemo `anharmoničnost` energijskih stanj, saj lahko le tako izberemo dve specifični, ki bosta predstavljali vrednosti kubita. Anharmoničnost uvedemo z *Josephsonovim stikom* [1], ki doda kosinusni člen, znan kot `Josepshonovo energijo` (označevali bomo z E_J), v potencialno energijo do tedaj parabolične oblike. To torej omogoča določitev dveh najnižjih energijskih nivojev, ki definirata kubit.



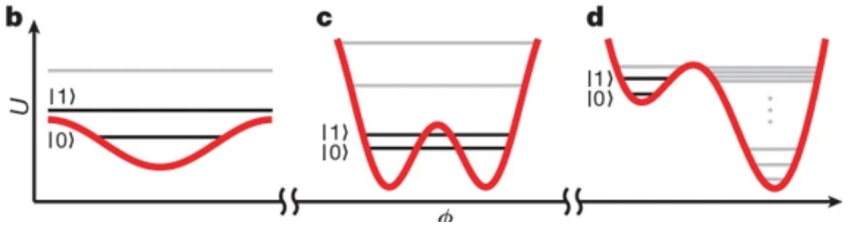
Slika 6.3: Superprevodna zanka, ki realizira kubit

Obstajajo trije osnovni tipi superprevodnih kubitov, ki se razlikujejo predvsem po obliki potenciala (Slika 6.4) in razmerju med Josephsonovo energijo E_J in kinetično energijo E_C, kjer je
$$E_C = \frac{e^2}{2C} \tag{6.3}$$
kar predstavlja energijo naboja posameznega elektrona na kondenzatorju.

- `Nabojni kubit`: temelji na kvantizaciji naboja posameznih Cooper parov. Pri tem tipu ni zaprte superprevodne zanke, saj ni induktorja, potencial pa ima obliko kosinusa z minimumom pri ničelni fazi.
- `Tokovni kubit`: uporablja tok, ki kroži v superprevodni zanki, kjer sta energijska minimuma povezana z dvema nasprotnima smerema toka. Induktor je pogosto zamenjan z nizom Josepshonovih stikov. Potencial ima obliko dvojne vdolbine.
- `Fazni kubit`: faza v vezju je ujeta v eni vdolbini potenciala, kjer se kubit definira z dvema najnižjima energijskima nivojema.

[1] Josepshonov stik je tanka izolacijska plast iz Al_2O_3, ki povezuje dva dela superprevodnega vezja iz Al.

Vsak tip ima različne prednosti glede občutljivosti na šume, koherence in hitrosti operacij, kar omogoča prilagoditev specifičnim zahtevam implementacije. [92]



Slika 6.4: Potencialna energija *U(Φ)* (rdeča linija) in energijski stopnji kubita (črni liniji), za nabojni kubit (b), tokovni kubit (c) in fazni kubit (d)

6.2.1.2 Merjenje, branje in manipulacija Kubitov

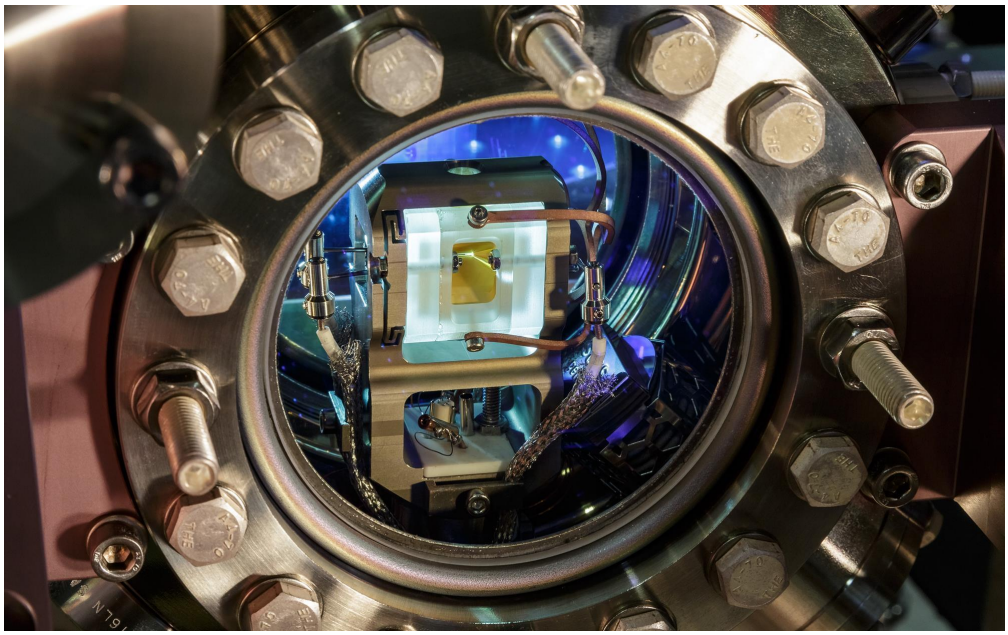
Za branje stanja kubita se pogosto uporablja preklapljanje *Josephsonovega stikala*, ki deluje kot pragovni detektor med dvema osnovnima stanjema kubita. Druga obetavna metoda je *QND* (quantum non-demolition) branje z rezontaorjem, kjer stanje kubita vpliva na fazni zamik elektromagnetnega valovanja v prenosni liniji, kar omogoča hitro (desetinke nanosekund) in natančno merjenje (okoli 95% natančnost).

Kubiti v superprevodnih vezjih so med sabo povezani z induktorji ali kondenzatorji, kar omogoča enostavno implementacijo osnovnih kvantnih logičnih vrat. Za kompleksnejše arhitekture pa so uporabljani prilagodljivejši načini povezovanja, kjer se interakcije med kubiti uravnava z nastavljivim vezjem (*tunable coupler*), ki omogoča vklop in izklop povezave.

Kljub makroskopskemi implementaciji superprevodnih kubitov, so koherence zadnje čase pomembno izboljšane s časovnim konstantama T_1 in T_2 6.2 v mikrosekundah, kar presega časovne omejitve manipulacij in branja. Vendar pa je potrebno bolje razumeti in odpraviti vzroke dekoherence, kar zahteva nadaljnje inženirske izboljšave materialov in vezij na mikroskopski ravni.

6.2.2 Ujeti Ioni

Kvantni računalniki temelječi na tehnologiji ujetih ionov izkoriščajo izvrstne koherenčne lastnosti določenih energijskih stanj atomov. Zaradi dolgotrajnih časov koherence (T_1 in T_2 trajata več sekund) tovrstni kubiti predstavljajo eno izmed najbolj zanesljivih implementacij kvantnih računalnikov.



Slika 6.5: Kvantni računalnik tipa ujetih ionov.

6.2.2.1 Realizacija Kubita

Pri arhitekturi ujetih ionov kubit izvira iz notranjih energijskih stanj enega samega atoma, ki mu odvzamemo elektron tako da postane ion. Ioni imajo zelo lepo urejeno energijsko strukturo, iz katere izbermo dve stabilni energijski stanji (osnovno (označevali ga bomo z g) in vzbujeno (označevali ga bomo z e)), ki predstavljata vrednosti kubita. Stabilnost ionske energijske strukture je razlog za izjemno dolge koherenčne čase T_1 in T_2. 6.2

Ion pa se v prostoru giblje, zato ga moramo prisilita, da stoji na mestu, saj drugače dekoherira takoj. To je storjeno s cilindričnimi elektrodami, ki tvorijo *Paulovo past*[2]

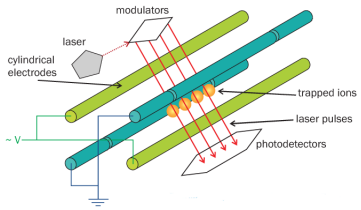
[2] Paulova past je konstrukt iz več elektrod, ki skupaj ustvarjajo električno polje s katerim stabiliziramo ione.

tako, da tvorijo spreminjajoče se električno polje, ki stabilizira ion v prostoru. V osnovi ima ion tako notranja stanja (izmed katerih izberemo dve za realizacijo superpozicije) in vibracijsko stanje (označevali ga bomo z n).

Vendar to še ni dovolj za želeno koherenco, saj se ion kljub temu še preveč giblje na mestu oziroma se "*trese*"in ima previsoko vibracijsko stanje, zato ga ponavadi ohladimo na enega izmed sledečih načinov:

1. *Dopplerjevo hlajenje*: Z lasersko svetlobo ion ohladimo do nekaj miljonink kelvina
2. *Sideband ohlajevanje*: Z lasersko rezonanco znižujemo vibracijsko stanje

Tako dosežemo vibracijsko stanje $n = 0$, ki je osnova za kubitne operacije.



Slika 6.6: Na sliki so prikazani ioni, ujeti v Paulovo past torej v tem primeru štiri elektrode. Skozi ione pa pulzi laserjev, ki ion ohladijo, in jih tako popolnoma stabiliziramo

Obstajata dva osnovna tipa ionskih kubitov, ki se v glavnem razlikujeta v velikosti ter koherenčnem času.

- `Optični kubit`: uporablja dva notranja atomska stanja, ki sta ločena z optičnim (vidnim ali UV) prehodom. To omogoča izjemno hitre kubitske operacije z laserskimi pulzi, vendar imajo ta stanja nekoliko krajše koherenčne čase, ker so bolj občutljiva na laserski šum.
- `Hiperfini kubit`: temelji na dveh hiperfinih stanjih, ki sta ločeni z mikrovalovno frekvenco. Ta stanja so izjemno stabilna in imajo zelo dolge koherenčne čase, pogosto več sekund, njihova manipulacija pa poteka z mikrovalovi ali laserskimi prehodi.

[94]

6.2.2.2 Merjenje, branje in manipulacija Kubitov

Ko imamo ion v mirujočem stanju (torej (g, 0) ali (e, 0)), lahko z laserskimi operacijami nad njim izvajamo kubitne operacije. Lasersko svetlobo nastavljamo na različne frekvence, s čimer dostopamo do različnih prehodov v ionski energijski strukturi:

- `Carrier prehod`: Tukaj je laser nastavljen na energijsko razliko med osnovnim stanjem g in vzbujenim stanjem e. Ta prehod spreminja samo notranje stanje iona;
$$(g,0) \leftrightarrow (e,0) \tag{6.4}$$
- `Red sideband prehod`: Laser je nastavljen na nižjo frekvenco kot pri *Carrier prehodu*. Pri prehodu se spremeni notranje stanje iona ter zniža se vibracijsko stanje;
$$(g,1) \leftrightarrow (e,0) \tag{6.5}$$
- `Modri sideband prehod`: Tukaj je laser nastavljen na višjo frekvenco kot pri *Carrier prehodu*. Pri prehodu se spremeni notranje stanje iona ter poveča se vibracijsko stanje;
$$(g,0) \leftrightarrow (e,1) \tag{6.6}$$

Ponavadi imamo v pasti več ionov, ki se oblikujejo v verigo zaradi *Colombovega zakona*[3]. Takšni ioni se obnašajo kot skupen mehanski sitem, tako je vibracijsko stanje skupno celotni verigi. To pomeni, da laserski pulz, usmerjen na en ion spremeni celotno vibracijsko stanje verige. Ta lastnost omogoča tvorbo kvantne prepletenosti med kubiti in gradnjo kvantnih logičnih vrat.

Stanje ionskega kubita izmerimo tako, da usmerimo laser na določen prehod, kjer bo ion svetlobo odbijal, če je v osnovnem stanju g, oziroma bo svetlobo absorbiral, če je v vzbujenem stanju e. Tako lahko meritev prevedemo v preprosto razlikovanje med svetlim in temnim signalom, natančnost le te pa presega 99.9%. [94]

[3] Colombov zakon je v fiziki zakon, ki podaja, kako sila med dvema točkastima električnima nabojema pojema z razdaljo.

6.2.3 Primerjava

Spodaj prikazana tabela prikazuje primerjavo kvantnih računalnikov superprevodnega vezja ter ujetih ionov glede na pet značilk.

Prva značilka je `hitrost kubitnih operacij` pri kateri pred vsemi ostalimi tipi prevladujejo superprevodna vezja, saj je manipulacija z električnimi signali izredno hitra, medtem ko so operacije pri ujetih ionih nekoliko počasnejše, saj laserski pulzi terjajo več časa.

Druga primerjana lastnost je `stopnja napak`. Tukaj iztopajo kvantni računalniki ujetih ionov, saj omogočajo izredno dobro natančnost meritev zaradi izredno dolge koherence in trivialnega pogoja svetlosti in temnosti ter stabilno izvajanje operacij zaradi močne prepletenosti met kubiti verige 6.2.2.2. Superprevodna vezja pa so močno občutljiva na šum okolja, kar hitro povzroči dekoherenco ter s tem viša stopnjo napak.

Tretja značilka, ki sem jo primerjali je `Skalabilnost`. Ta je višja pri superprevodnih vezjih, saj uporabljajo makroskopske komponente ter močno spominjajo na običajna elektronska vezja, kar skupaj omogoča enostavno implementacijo ter nizko kompleksnost infrastrukture sistema (z izjemo infrastrukture hlajenja)6.2.1. Med tem ko je infrastruktura sistema pri ujetih ionih zelo kompleksna zaradi uporabe laserjev na mikroskopskem nivoju prav tako pa je z laserji težavno obvladovati veliko število ionov posamezno 6.2.2.

Četrta lastnost za primerjavo je `Energijska poraba`. Ta je pri superprevodnih vezjih ogromna saj, za konstrukcijo Cooperjevih parov ohranjamo temperature skoraj na absolutni ničli, takšni hladilni sistemi pa so zelo potrošni in dragi 6.2.1.1. Seveda so zelo potrošni vsi kvantni računalniki vendar pa hlajenje ujetih ionov zahteva mnogo manj energije.

Zadnja lastnost, ki sem jo primerjal je `Industrijska podpora`. Superprevodna vezja imajo tu močno prednost, ker temeljijo na tehnologiji, ki je združljiva z že zelo razvito industrijo, zato je razvoj hiter in dobro finančno podprt. Pri ujetih ionih je podpora manjša, saj tehnologija zahteva laserske in optične sisteme, ki jih je težje standardizirati in množično izdelovati.

Tabela 6.1: Tabela prikazuje primerjavo kvantnih računalnikov superprevodnega vezja ter ujetih ionov.

Lastnost	**Superprevodna vezja**	**Ujeti ioni**
Hitrost operacij	Zelo visoka	Nižja
Stopnja napak	Višja	Zelo nizka
Skalabilnost	Visoka	Nižja
Energijska poraba	Zelo visoka	Nižja
Industrijska podpora	Dobro podprto	Manj podprto

6.3 Zaključek

V poročilu sem predstavil dva vodilna pristopa realizacije kvantnih računalnikov; superprevodna vezja in ujete ione. Obe tehnologiji temeljita na istih principih vendar ju ločujejo velike razlike v fizikalni implementaciji, koherenčnih časih, hitrostih operacij ter infrastrukturi.

Oba tipa kvantnih računalnikov sta obetavna in se nenehno razvijata. Danes še ni jasno, katera tehnologija bo dolgoročno najbolj primerna za univerzalno kvantno računalništvo.

Chapter 7
Quantum Walk Implementation in Qrisp

Jure Dežman, Aljaž Justin, Alen Leban, Matjaž Madon

Abstract In this paper, we explain what quantum walks are, how they operate and review several key areas in which they are used. We start with a review of existing literature, followed by some basic definitions of certain types of quantum walks. Following that, we describe methods used in the experiments. At the end we also demonstrate the unique features that distinguish them from classical random walks on a cyclic graph, 2-dimensional torus grid graph, and a binary tree. We discuss the plotted probability distributions and describe the implementation of used quantum circuits. Finally, we discuss a few open problems about quantum walks.

7.1 Introduction

Quantum computers are progressing at a rapid pace and are becoming practical tools rather than merely conceptual ideas. As a result, increasingly powerful algorithms are being developed to solve new problems that were previously considered intractable, as well as to enhance and accelerate existing classical methods. One of the fundamental building blocks of many such quantum algorithms is the *quantum walk*.

A random walk are often used to model unpredictable or stochastic behavior. Random walks are widely used in algorithms and computer science because they help explore complex spaces without requiring complete knowledge of the structure. Their ability to balance exploration and randomness makes them powerful tools for solving problems in networks and machine learning.

Quantum walks originate from classical random walks but incorporate principles of quantum mechanics, giving rise to behaviors and properties not achievable in classical systems. These properties open up a wide range of new possibilities in algorithm design, optimization, and information processing.

7.2 Related Works

Quantum walks have proven to be a powerful theoretical and practical tool in the domain of quantum computing. Their applications span a wide range of areas, including quantum machine learning. For example, the paper *Towards interpretable quantum machine learning via single-photon quantum walks* [95] introduces a variational approach to quantize Projective Simulation [96], a reinforcement learning model specifically designed for interpretable artificial intelligence. The motivation behind this work is to overcome the limitations and opacity of classical neural networks and conventional variational quantum algorithms [97], particularly in scenarios where transparent and explainable decision-making is required.

Another important application of quantum walks appears in the study of the element distinctness problem and its generalizations. In the paper *Quantum walk algorithm for element distinctness* [98], the authors present a quantum algorithm capable of finding two identical items among N given items more efficiently than classical approaches.

Quantum walk–based search algorithms have also been studied extensively. The work *Quantum Walk Based Search Algorithms* [99] outlines how discrete-time quantum walks can be used to quantize classical Markov chains. It also presents Grover's search algorithm and several quantum walk search algorithms developed by Ambainis [98], Szegedy [100], Magniez [101] et al., framing them as quantum analogues to classical search procedures. Similarly, the paper *Spatial search by quantum walk* [102] demonstrates how spatial search can be accelerated using Grover's algorithm, while *Faster quantum-walk algorithm for the two-dimensional spatial search* [103] uses quantum walks to achieve a significant improvement in two-dimensional spatial search. On special kinds of graph, quantum walk can also achieve exponential speedup in finding a certain vertex [104].

Quantum walks have also inspired novel approaches in computer vision. For instance, the work *Quantum walk search based edge detection of images* [105] proposes an edge detection algorithm based on quantum walks, aiming to enhance one of the most fundamental tasks in image processing.

Overall, quantum walk–based algorithms continue to show significant potential and have been adapted in many ways to improve or accelerate existing computational methods. The topic is well established and continues to receive substantial attention from the scientific community, as documented in several comprehensive books and reviews [106, 107, 104].

7.3 Quantum walk

A Quantum Walk (QW) is defined as the quantum mechanical generalization of the Classical Random Walk (CRW). This emerging field is inspired by the success of discrete random walks in classical algorithm development. The core objective of

QWs is to exploit fundamental quantum mechanical principles to achieve computational power that surpasses classical computing limits.

QWs utilize quantum effects such as superposition, interference, and entanglement. In a classical random walk, a particle (walker) moves stochastically according to specific probability distributions. Conversely, in a quantum walk, the particle evolves in a superposition of possible states, exploring multiple possible paths concurrently via quantum amplitudes.

The evolution of a QW is unitary, reversible, and deterministic during the steps themselves, in contrast to the inherent randomness of classical Markov chains. The probabilistic nature in a QW only arises when a final measurement is performed, which collapses the quantum state.

The distinct nature of quantum evolution grants QWs significantly faster spreading dynamics compared to CRWs. This difference is quantified by how the variance (σ^2), which measures the spread of the probability distribution, grows with time (t) or the number of steps (n), as seen in 7.1. This quadratic acceleration in spreading is a primary motivation for employing QWs in algorithm design, particularly for search problems [108, 109].

Table 7.1: Characteristic of CRW and QW

Characteristic	Classical Random Walk (CRW)	Quantum Walk (QW)
Evolution	Stochastic, step-by-step probability.	Unitary, amplitude superposition, interference.
Variance (σ^2) Growth	Linear with time: $\sigma^2 \propto t$.	Quadratic with time: $\sigma^2 \propto t^2$.
Speed/Spread	Diffusive spread.	Ballistic spread (quadratically faster).

Quantum walks are broadly categorized by their time evolution: Discrete-Time Quantum Walk (DTQW) and Continuous-Time Quantum Walk (CTQW).

7.3.1 Discrete time quantum walk

DTQW is defined by the application of a corresponding evolution operator only at discrete time steps. The DTQW structure exhibits fundamental physical significance. The evolution equations describing the discrete-time quantum walk have been shown to be structurally similar or mathematically identical to the Dirac equation (a relativistic quantum mechanical equation) [109]. This contrasts with the CTQW, which is typically derived from and behaves similarly to the non-relativistic Schrödinger equation [109].

Both CTQW and DTQW models has been proven capable of universal quantum computation [108]. Thus, the DTQW can be regarded as a foundational primitive for

quantum computation, providing a viable framework for implementing any quantum algorithm.

Within the domain of DTQWs, several prominent models exist, distinguishable primarily by whether they require an auxiliary "coin" space and how the evolution operator is constructed.

7.3.1.1 Coin-based quantum walk

The coin-based model is the generally accepted standard form of the DTQW. This model utilizes the structure of the classical random walk, where a "coin flip" dictates the next step.

This QW requires two distinct coupled quantum systems residing in a total Hilbert space H_T:

- Position Space (H_S): Defines the possible locations (vertices) of the walker.
- Coin Space (H_C): A finite-dimensional auxiliary space (often a single qubit) used to define the direction or transition rule for the next step. The total state lives in the tensor product space $H_T = H_C \otimes H_S$ [109, 110].

With that we can create a unitary evolution operator U defined by $U = S(C \otimes I_S)$, that for a single step its composed of two successive unitary operations, which are repeatedly iterated. The two unitary transformations are:

- **Shift operator** (S): A unitary operation that moves the walker to a new position depending on two coin states $|\uparrow\rangle$ and $|\downarrow\rangle$, so the shift operator can be defined by $S = |\uparrow\rangle\langle\uparrow| \otimes \sum_i |i-1\rangle\langle i| + |\downarrow\rangle\langle\downarrow| \otimes \sum_i |i+1\rangle\langle i|$.
- **Coin operator** ($C \otimes I_S$): It is applied to the coin space to put the coin into a superposition of direction states. Common examples include the Hadamard coin C_H or the Grover coin C_G [108], they are defined by $C_H = \frac{\sqrt{2}}{2}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $C_G = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

Probability distributions produces by different coins will be demonstrated in the experiments section.

7.3.1.2 Szegedy quantum walk

The Szegedy Quantum Walk (SQW) [100] is an important development in the field of quantum walks, characterized by being a *coinless* model derived directly from classical random walks. When in 2004 Szegedy proposed a new method to quantize an arbitrary time-homogeneous classical Markov chain, defined by a transition probability matrix $P = (P_{i,j})$ [100, 111]. Instead of acting on an augmented space including a physical "coin" the SQW acts on an enlarged Hilbert space corresponding to the edges or directed transitions of the graph, $H_T \propto C^N \otimes C^N$.

The evolution operator W is constructed from the quantization of the stochastic matrix P, using position states defined over the transitions: $|\Psi_j\rangle = \sum_i \sqrt{P_{i,j}}|i\rangle|j\rangle$. By defining a unitary operator based on this quantization, SQW determines the movement of a walker on an enlarged symmetric bipartite graph corresponding to the classical transitions.

This formalism directly generalizes previous search algorithms, including Grover's algorithm, which can be viewed as a specific instance of an SQW on a uniformly connected graph.

7.3.1.3 Staggered quantum walk

The Staggered Quantum Walk (StQW) [112] is another notable coinless discrete model. This model defines its evolution by partitioning the graph structure into a series of *tessellations*, where each tessellation is composed of polygons whose union covers all vertices of the graph [108].

The dynamics involve a "staggered movement" implemented by reflection operators associated with these tessellations. This model can also be formulated using Hamiltonians (the continuous-time approach) derived from these reflection operators.

7.3.2 Continuous-time quantum walk

A Continuous-Time Quantum Walk (CTQW) is defined by quantum evolution governed by a time-independent Hamiltonian, where transitions occur continuously over time rather than at discrete steps. This model was originally formulated by Farhi and Gutmann as a direct quantum analogue of continuous-time classical Markov processes.

In CTQW, the system evolves according to the Schrödinger equation $i\hbar\frac{d}{dt}|\psi(t)\rangle = H|\psi(t)\rangle$, where the Hamiltonian H is typically taken to be the adjacency matrix or Laplacian of the graph. This contrasts with the coin-based mechanism of discrete-time walks and provides a fundamentally different approach to quantum-walk-based algorithms [113].

The relationship between CTQW and Discrete-Time Quantum Walk (DTQW) has been extensively studied, with Childs demonstrating that while the two models exhibit different behaviors on certain graphs, they can be formally related through appropriate mathematical mappings [114]. This connection reveals the underlying unity in quantum walk formulations despite their apparent differences. CTQWs have found particular utility in quantum search algorithms and spatial search problems, where their continuous nature often leads to simpler analytical treatments compared to discrete-time counterparts.

7.3.2.1 Chiral continuous-time quantum walk

A chiral continuous-time quantum walk is a variant of the CTQW in which time-reversal symmetry is deliberately broken by introducing complex phase factors (imaginary edge weights) into the graph's Hamiltonian. Zimborás *et al.* [115] showed that such "chiral" walks—implemented by adding directed phase biases while preserving overall Hermiticity—enable directional bias in the unitary evolution, allowing one to either enhance or suppress transport probabilities relative to a standard (time-symmetric) quantum walk. In essence, a chiral CTQW gives the walker different transition amplitudes depending on the direction of travel, something not possible in a regular CTQW on an undirected graph. This approach has been used to achieve controlled transport on networks, for example by improving energy transfer efficiency or realizing quantum routing, thanks to the ability to break the symmetric flow of probability amplitude while still maintaining quantum coherence.

7.3.2.2 Quantum stochastic walk

The quantum stochastic walk (QSW) is another important CTQW-based model that incorporates environmental decoherence into the walk's dynamics. Introduced by Whitfield, Rodríguez-Rosario, and Aspuru-Guzik [116] as a generalization of both classical and quantum walks, a QSW evolves according to a Lindblad master equation rather than a strictly unitary Schrödinger equation. This means the evolution includes both unitary (quantum) and non-unitary (dissipative) transition components, effectively combining a continuous-time quantum walk with a classical random walk. In fact, the QSW framework encompasses the ordinary CTQW and the classical random walk as special cases, representing a continuous interpolation between them. By tuning decoherence parameters, one can transition smoothly from purely quantum transport (no decoherence) to classical diffusive behavior (strong decoherence). Quantum stochastic walks thus provide a versatile tool for studying quantum walks under realistic noisy conditions, and they have been used to explore how limited decoherence can sometimes enhance transport or enable new algorithmic applications in quantum networks.

7.4 Methods

In the following experiments, we will run the quantum walk algorithm in different settings. The algorithm will be written in quantum circuits.

7.4.1 Quantum circuits

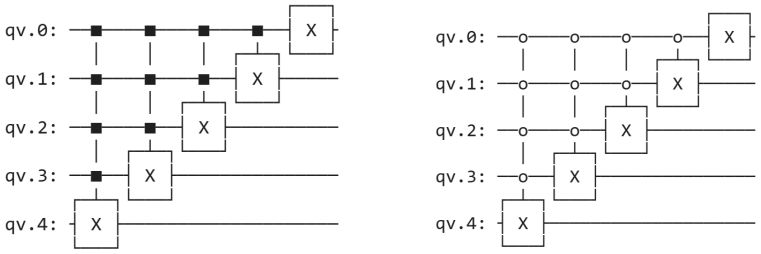
The quantum circuits used in the following experiments will be explained here more in depth.

7.4.1.1 Increment and decrement circuits

When a quantum register is interpreted as an unsigned integer, the increment and decrement circuits are very simple. For an n-qubit register, n CNOT gates (with multiple controls) are used in series that flip each qubit depending on the values of lower significant ones. The representations of both circuits are shown in Figure 7.1.

For incrementing, each qubit (in order from lowest to highest significant) is flipped if all lower significant qubits are in state $|1\rangle$. On the other hand, a decrement circuits flips each qubit if all lower significant qubits are in the state $|0\rangle$.

When all qubits are in the state $|1\rangle$, the next increment will flip all qubits back to the $|0\rangle$ state, which wraps around the maximal value back to the smallest one ("overflow" of an unsigned integer). When decrementing, the same happens, but in the other direction - qubits in the $|0\rangle$ state map to $|1\rangle$ state.



(a) The increment circuit.

(b) The decrement circuit.

Fig. 7.1: Increment and decrement quantum circuits for 5-qubit quantum registers. The dark control points correspond to the state $|1\rangle$.

If we're promised that the input register will represent an even number, an even simpler circuit can be designed for incrementing by simply negating the least significant qubit. Decrement circuit can also be simplified this way, but when promised an odd number as input.

An example of the simplified increment circuit will be shown in the experiment of a quantum walk on a binary tree.

7.4.1.2 Multiplication by 2

Multiplication of a binary number by powers of 2 can be quickly performed by bit shifting. The same behavior can be replicated on a quantum circuits using 2-qubit swap gates. The following circuit however does the shifting in a cyclic way to reuse the existing qubits, which causes incorrect results when the multiplication overflows the register. An example of a circuit is shown in Figure 7.2.

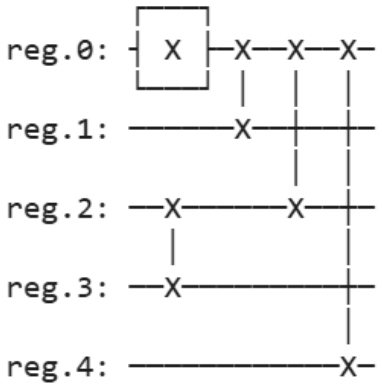


Fig. 7.2: Quantum circuit for multiplying a 4-qubit quantum register by 2. The X gate at the top-most qubit encodes input number 1.

7.4.2 Environment

The following experiments were conducted in a simulated environment, provided by the Qrisp Python library. Behind the scenes, all written Python code is compiled into a quantum circuit with gates, and simulated when taking measurements. For simplicity, the number of shots for each final measurement was left to be determined by the library (which is around 1 million for simple circuits), to get the probability distribution of results with high precision.

All code was written in a Python notebook via Google Colab. The code is available in the supplementary material.

7.4.3 Qrisp

One of the most popular libraries for programming quantum circuits is IBM's Qiskit. It covers all the essential circuit elements, and provides many quantum algorithms.

Despite being the most widely used library for its task, we chose a lesser known library, Qrisp [117], that tries to narrow the gap between writing low-level code for constructing quantum circuits using quantum gates and classical python programming. It also provides a simple way of simulating quantum circuits locally. Although it performs barely any low-level optimizations, the compiler can exploit knowledge of high-level algorithm structure.

7.5 Experiments

The simplest demonstration of a quantum walk can be performed on a cycle graph. 5 qubits were used to encode the position of the walker on a cycle graph with 2^5 vertices, and 1 qubit for the coin register. The walker was placed at position 2^{n-1} to start in the middle of the $[0, 2^{n-1} - 1]$ range, but could have been placed anywhere since the graph is a cycle. The coin register was initialized in two ways: in a biased way with the state $H|0\rangle$, and an unbiased way by adding an S-gate afterwards ($SH|0\rangle$). After 32 steps, the position register was measured. At each step, a Hadamard gate is used as the coin operator.

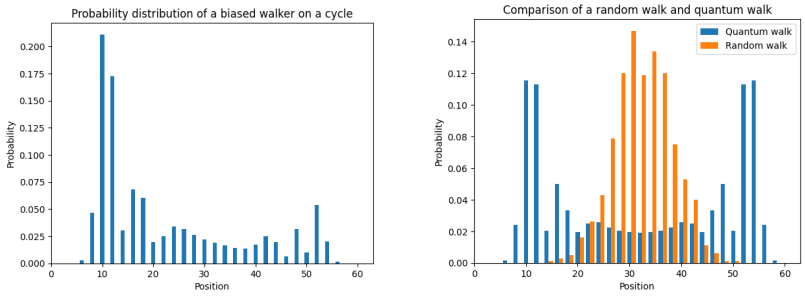
Another experiment was performed on a square grid graph, where two 5-qubit position registers were used to encode the "x" and "y" coordinates of the walker, and a 2-qubit coin register. The walker is allowed to move in one direction at each step, where possible directions are encoded as the basis states of the coin register ($|00\rangle, |01\rangle, |10\rangle$ and $|11\rangle$). The rest of the circuit follows the same logic as the one for a cycle graph.

The last experiment was a quantum walk on a binary tree. One 3-qubit position register was used as the position of the walker in a binary tree according to the indexing of a level-order traversal from left to right (where index [1] is in the root, indices [2, 3] in the first level, [4, 5, 6, 7] in the second level, etc.). The 1-qubit coin register decided whether the walker moves to the left or to the right subtree. After 3 steps (proportional to the height of the tree), the position register was measured.

As a bonus, a quantum walk on a square graph was also tested using the Grover coin to see, how the probability distributions differ with the Hadamard coin.

7.6 Results

For the quantum walk on a cycle graph, Figure 7.3 shows the probability distribution of the walker after the final measurement. When using the biased coin, the distribution is shifted to one side. The direction of the direction of bias depends on the



(a) A biased quantum walk.

(b) Quantum walk and random walk comparison.

Fig. 7.3: Probability distributions of (a) biased quantum walk with the coin initialized to $H|1\rangle$ and (b) a comparison of a quantum walk and a random walk between the amount of spread after 32 steps.

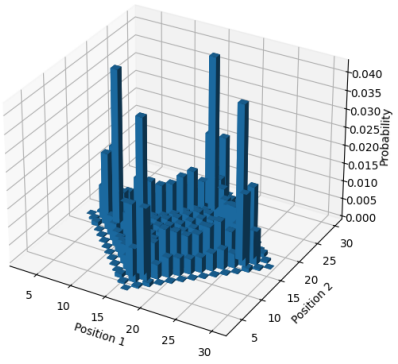
initial state of the coin qubit: if $|1\rangle$, then the bias is to the left, if $|0\rangle$, the bias would be to the right. In the plot (b), we can see the quicker spread of a quantum walk compared to a random walk, which gives the quantum walk an edge at e.g. finding a marked vertex in a graph quicker.

In the experiment on a 2-dimensional torus grid graph, the quicker spread is also observed in the quantum walk plot in Figure 7.4a. The produced distribution shows four peaks, which correspond to the peaks of the walker on a cycle graph, but combined with a new dimension. At first however, you might expect the distribution to have a more circular shape, since a random walker produces normal distribution that looks bell-shaped (described as ”circular”) by following the same ”walking rule” of making a steps in single directions at a time. But considering that the two qubits of coin register aren't entangled in any way, it makes sense that the result is just a product of their distributions but on perpendicular axis. A more circular shaped distribution (Figure 7.4c) can be obtained using a Grover coin.

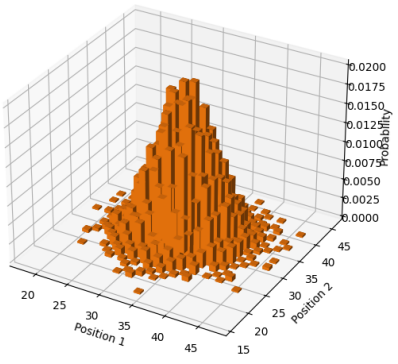
For the quantum walk on a binary tree, the probability distribution was measured after traversing the whole height of the tree. The probability values are printed out at the bottom of Figure 7.5. At each step, the distribution was uniform across all positions within the same level, and 0 everywhere else.

7.7 Discussion

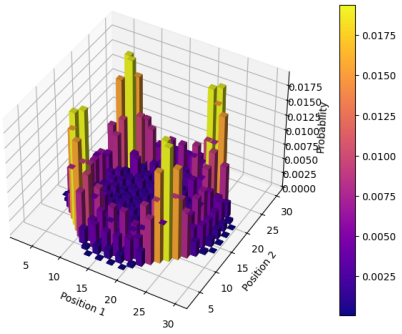
In the experiments, we showed how to implement examples of discrete time quantum walks on certain graphs and plotted the results. The natural but challenging next step is to generalize this process to arbitrary graphs. There are many open problems regarding properties of (continuous) quantum walks. Three classes of problems were highlighted by Coutinho et. al. [118] and are as follows:



(a) Quantum walk + Unbiased Hadamard coin



(b) Random walk



(c) Quantum walk + Grover coin (after 16 steps)

Fig. 7.4: Comparison of probability distributions of a quantum and random walk on a 2-dimensional torus grid graph. As a bonus, Grover coin's distribution was also plotted.

1. State transfer - how to transfer quantum states between two parts of a quantum computer, where local interactions are modeled as edges of a graph. This is a key ingredient to building scalable architectures that simplifies the design and therefore reduces opportunities for errors to arise [119].
2. Instantaneous uniform mixing - if a random walker starts on any vertex of a graph and after time t ends up at any other vertex with uniform distribution, such graph is said to have uniform mixing. For a quantum walker, if the distribution is uniform at an instant time t, then the graph has instantaneous uniform mixing. The question is, which graphs have this property for continuous-time quantum walks [120]?

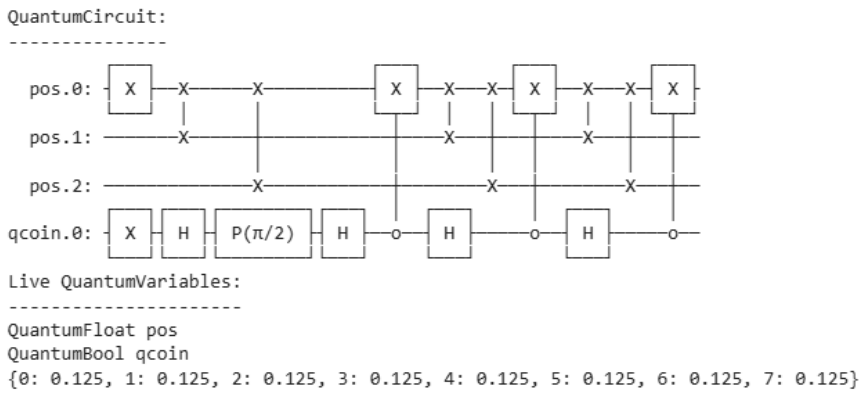


Fig. 7.5: Quantum circuit of a quantum walk on a binary tree. The probability distribution is shown at the bottom.

3. Average mixing matrix - contrary to random walks, quantum walks don't reach a stable stationary distribution. An average mixing matrix is a distribution that a quantum walk adheres to on average, over time (a "replacement" for stationary distribution), and the open problems about the properties of the matrix and connections with the underlying graph [121].

In conclusion, quantum walks have proven to be a powerful tool, and solving complex tasks and work in this field will bring even more improvements to existing algorithms and bring quantum computers and their dominance even closer to reality.

Poglavje 8
Implementacija in primerjava polnih seštevalnikov v kvantnih celičnih avtomatih

Klemen Kavčič, Tomi Trošt, Pia Sotlar

Povzetek Quantum-dot Cellular Automata (QCA) predstavlja energijsko učinkovito alternativo CMOS tehnologiji. V seminarski predstavimo obstoječe QCA polne seštevalnike ter jih primerjamo glede na število celic, površino, zakasnitev (delay) in strošek (cost). Nato razvijemo dve lastni implementaciji polnega seštevalnika, pri čemer je druga verzija optimizirana z zmanjšanim številom celic, zmanjšano zakasnitvijo in zmanjšanimi stroški.

8.1 Uvod

Potreba po zmanjševanju velikosti vezij je privedla do tega, da se tehnologija CMOS približuje fizikalnim mejam. Posledično se je pozornost usmerila na nanotehnologije in ena izmed najbolj obetavnih je tehnologija *Quantum-dot Cellular Automata* (QCA). Podpira gradnjo digitalnih vezij z nizko porabo energije, visoko hitrostjo in visoko gostoto [122].

8.1.1 Reverzibilno procesiranje

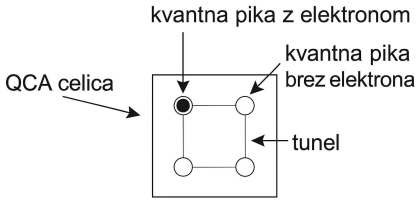
Pri nepovratnem procesiranju prihaja do izgube informacij in posledično velike porabe energije, pri čemer narašča količina toplote, ki jo ustvarjajo integrirana vezja. Landauer je pokazal, da vsak izgubljen bit informacije povzroči razpršitev energije v obliki toplote v višini $kTln2$ joulov. To pomeni, da procesne moči takšnih vezij ne moremo v nedogled povečevati.

Pri reverzibilnem procesiranju pa se izhodne informacije lahko pretvorijo nazaj v vhodne, kar preprečuje izgubo podatkov oziroma toplote. V teoriji bi bila poraba energije ničelna, saj bi sistem omogočal stodstotno porabo energije. Takšen sistem pa zahteva vrata, ki imajo enako število vhodov kot izhodov.

Tehnologija QCA omogoča izjemno nizko porabo energije, saj ne temelji na toku elektronov, ampak prerazporejanju le-teh. Hkrati podpira tudi reverzibilno logiko, zato je trenutno najprimernejši kandidat za trajnostne računske rešitve [123].

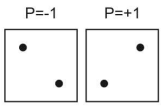
8.1.2 QCA celica

Delovanje QCA deluje na zakonih kvantne fizike, temelj pa predstavlja kvantna ali QCA celica. QCA celica je dvodimenzionalne površinske kvadratne oblike in v vsakem oglišču se nahaja kvantna pika s pozitivnim nabojem. V celico ujamemo dva elektrona, ki se lahko nahajata v eni izmed štirih kvantnih pik in lahko prehajata preko štirih tunelov, ki se nahajajo med pikami (vidni na sliki 8.1). Elektron glede na prisotnost odbojnih sil iz okolice teži k prehodu v piko, ki maksimizira njegovo energetsko stabilno lego na osnovi Coulombovih fizikalnih zakonov. S tem so odbojne sile med njim in drugim elektronom (ter elektroni iz okolice) minimalne.



Slika 8.1: QCA celica

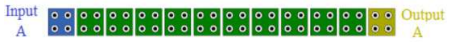
Elektrona se postavita v eno izmed dveh diagonalnih leg, s čimer dobimo sistem z dvema stabilnima stanjema 8.2 (P=+1 oziroma 1 in P=-1 oziroma 0) [124].



Slika 8.2: Dva stabilna stanja elektronov v QCA celici

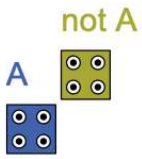
8.1.3 Osnovni gradniki QCA struktur

Signali se v kvantnih vezij prenašajo preko vodil. Zaradi vpliva odbojnih sil, se stanje ene celice prenese na stanje sosednjih celic. To omogoča propagacijo vrednosti logičnih spremenljivk. Vodilo je predstavljeno na sliki 8.3 [124].



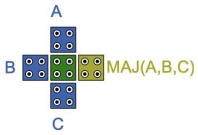
Slika 8.3: QCA vodilo

V QCA logiki obstajata dve osnovni vrsti vrat - negator in majoritetna vrata. Najbolj enostavna struktura za izvedbo logične funkcije NOT je diagonalna postavitev celic, saj odbojne sile povzročijo nasprotno polarizacijo. Na sliki 8.4 je predstavljena struktura negatorja [124].



Slika 8.4: Negator

Majoritetna vrata imajo tri vhode in en izhod, ki sprejme tisto polarizacijo, ki prevladuje. Če je ena izmed vhodnih celic logična vrednost 0 (oziroma polarizacija -1) vrata predstavljajo logično funkcijo AND. Če pa je ena izmed vhodnih celic logična vrednost 1 (polarizacija 1) vrata predstavljajo logično funkcijo OR. Na sliki 8.5 so predstavljena majoritetna vrata [125].



Slika 8.5: Majoritetna vrata

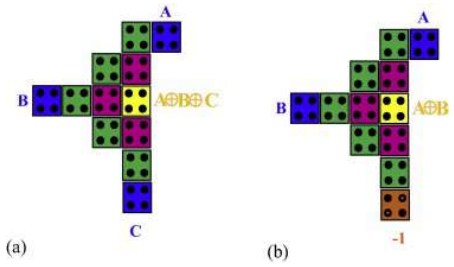
8.1.4 XOR in 3XOR vrata

Z osnovnimi gradniki, predstavljenimi v 8.1.3 lahko zgradimo kompleksnejše strukture za realizacijo vrat, kot sta XOR in 3XOR vrata.

XOR vrata omogočajo implementacijo seštevanja bitov in temeljijo na logični relaciji $A \oplus B = (\overline{A}B) + (A\overline{B})$.

3XOR oziroma CNOT vrata imajo 3 vhode, pri čemer eden deluje kot kontrolni vhod. V primeru polnega seštevalnika ta vhod predstavlja prenos (carry) iz prejšnje faze seštevanja.

Pri naši implementaciji polnega seštevalnika smo uporabili XOR in 3XOR realizaciji iz članka [125], prikazani na sliki 8.6.



Slika 8.6: Predpostavljena (a) 3XOR vrata in (b) XOR vrata

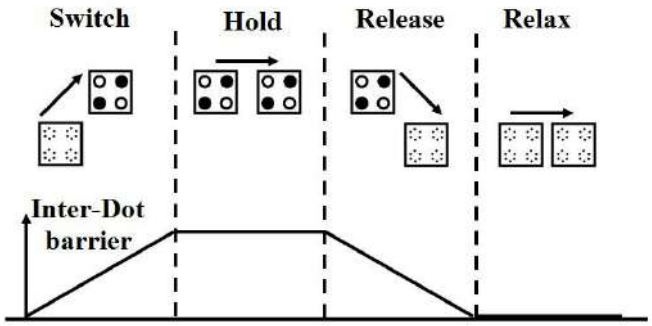
8.1.5 Ura v QCA

Vpeljava ure za usmerjanje toka je ključna za preprečevanje trčenja oziroma konflikta dveh podatkovnih vrednosti. Ura določa, kdaj je katera celica aktivna in kdaj preide v stabilno stanje. To izvedemo tako, da začasno zaklenemo dinamiko v posameznih notranjih celicah, kar pomeni da je prehajanje celice v stabilno stanje onemogočeno [124].

Ura v kontekstu QCA pomeni električno polje, ki deluje nad določenim četverokotnim področjem celic oziroma nad tuneli med kvantnimi pikami. Zaklepanje in odklepanje celic je torej zapiranje in odpiranje pregrad med tuneli, po katerih prehajajo elektroni. Ura je sestavljena iz štirih faz, ki skupaj tvorijo eno urino periodo:

- **Switch faze**, v kateri se pregrade zapirajo in celice prehajajo v polarizirana stanja.
- **Hold faze**, v kateri so pregrade zaprte in celice ne morejo prehajati v drugo polarizirano stanje.
- **Release faze**, v kateri se pregrade odpirajo in elektroni začnejo prehajati.

- **Relax faze**, v kateri elektroni lahko prosto prehajajo ker so pregrade odprte [124][126].



Slika 8.7: Prikaz štirih faz ure v QCA [126]

8.1.6 Križanje vodil

Križanje vodil v QCA je ključno za prenos dveh signalov, ki ne smeta imeti medsebojnega vpliva na polarizacijo. Za to obstaja več tehnik:

- **Koplanarno križanje**, kjer uporabljamo dve vrsti vodil, običajnega in alternirajočega. To omogoči, da vodili delujeta neodvisno in se križata v isti ravnini. Največja težava je v implementaciji, kjer lahko ob premiku celice pride do medsebojnega vpliva.
- **Večplastno križanje**, ki zahteva uporabo več plasti, nameščenih ena na drugo. Uporabimo lahko enake vrste celic, če je razdalja dovolj velika, da ne pride do vpliva.
- **Koplanarno križanje ob upoštevanju interferenc med fazami ure**, kjer izkoristimo dejstvo da celice v fazah Release in Relax nimajo vpliva na sosednje celice. Celice v fazi Hold se lahko križajo s celicami v fazi Relax, celice v fazi Switch pa s celicami v fazi Release brez da bi vplivale na polarizacijo [122].

8.2 Polni seštevalnik

Polni seštevalnik je logično vezje, ki sešteje tri bitne vhode, in proizvede dvobitni izhod (dva izhoda po en bit). Dva vhoda sta A in B, to sta bita, ki ju želimo seštevati, tretji vhod pa je C (v določeni literaturi tudi C-in), ki je prenesen bit od prejšnjega

seštevanja (če je zaporedno sestavljeno več polnih seštevalnikov v večbitnih sistemih). Izhoda sta vsota (SUM) in prenesen bit - Carry (v literaturi tudi C-out), ki se prenaša v naslednji seštevalnik.

VHOD			**IZHOD**	
A	**B**	**C**	**Sum**	**Carry**
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tabela 8.1: Logična tabela polnega seštevalnika

SUM in CARRY lahko neodvisno predstavimo z uporabo naslednje logike [127]:

8.2.1 SUM

Logična tabela za SUM je predstavjena kot:

$$S = A'B'C + A'BC' + AB'C' + ABC$$

Ker drži: $A'B + AB' = A \oplus B$ lahko izraz poenostavimo v:

$$S = C(A \oplus B)' + C'(A \oplus B)$$

Končna poenostavitev izraza je:

$$\boxed{S = A \oplus B \oplus C}$$

Torej lahko SUM najlažje dobimo s trojnimi XOR vrati.

8.2.2 Carry

Iz logične tabele lahko vidimo, da dobimo CARRY z izrazom:

$$CARRY = A'BC + AB'C + ABC' + ABC$$

, ki se poenostavi v:

$$CARRY = AB(C' + C) + C(A'B + AB')$$

Ker drži: $C' + C = 1$ and $A'B + AB' = A \oplus B$, je končna poenostavitev enaka:

$$\boxed{CARRY = AB + C(A \oplus B)}$$

8.2.2.1 Aternativni zapis

Carry izraz lahko alternativno predstavimo tudi majoritetna vrata, ki vrnejo večinsko vrednost vhodov. To nam je posebej koristno pri QCA tehnologiji, saj so majoritetna vrata pogosto uporabljena [127].

8.3 Pregled sorodnih del

Polni seštevalniki (Full Adders – FA) so eden ključnih gradnikov pri načrtovanju aritmetičnih enot v QCA tehnologiji. Ker vsak napreden računarski sistem potrebuje zanesljive in čim bolj kompaktne aritmetične osnovne enote, je optimizacija polnega seštevalnika že dolgo zanimiva raziskovalna tema. Pri QCA vezjih so izzivi nekoliko drugačni kot pri klasičnih CMOS implementacijah: pomembni so število uporabljenih celic, vrsta križanja žic, površina postavitve in zakasnitev skozi urne cone.

Zaradi teh posebnosti se v literaturi pojavlja več različnih pristopov, ki vsak na svoj način poskuša zmanjšati kompleksnost ali izboljšati delovanje vezja. V nadaljevanju podamo pregled najbolj citiranih in vplivnih rešitev, ki so služile kot izhodišče tudi za naše delo.

8.3.1 Ključni parametri primerjave

Pri analizi obstoječih QCA polnih seštevalnikov so najpogosteje uporabljene naslednje metrike:

- **število celic** – pove kompleksnost in posledično energijsko učinkovitost,
- **vrsta žičnega križanja** – koplanarna ali alternative, kar vpliva na stabilnost signala,
- **površina (μm^2)** – manjša površina pomeni kompaktnejšo implementacijo,
- **zakasnitev (clock cycles)** – pomembno za hitrost izvajanja aritmetike,
- **ocenjena cena (cost)** – kombinacija več parametrov, s katero avtorji izražajo celotno učinkovitost implementacije.

Ti parametri omogočajo enostavno primerjavo različnih pristopov in tudi jasno pokažejo, kako se je razvoj skozi čas izboljševal.

8.3.2 Pregled izbranih obstoječih rešitev

V sledečih podpoglavjih predstavljamo hiter pregled implementacij zbranih v članku [128].

8.3.2.1 Tougaw in Lent (1994)

Tougaw in Lent sta predstavila enega prvih konceptualnih QCA polnih seštevalnikov. Njuna rešitev temelji na osnovni uporabi koplanarnega križanja žic in zahtevala relativno veliko število celic (192). Takratna tehnologija je bila šele v začetni fazi, zato je bila postavitev precej obsežna, zakasnitev pa visoka (14 urnih ciklov). Kljub temu ta dizajn pogosto predstavlja referenčno točko, saj je prvi jasno pokazal, kako je mogoče seštevalnik sploh realizirati v QCA okolju.[129]

8.3.2.2 Vetteth in sodelavci (2002)

Približno desetletje kasneje je Vetteth s skupino predstavil izboljšano verzijo polnega seštevalnika. Čeprav je bila implementacija bolj robustna, je zaradi uporabe za izgradnjo carry look-ahead adder (CLA) število celic naraslo na 292. Tudi tukaj je uporabljeno coplanar križanje, zakasnitev pa je ostala enaka kot pri Tougaw–Lent rešitvi (14 ciklov). Glavna prednost te izvedbe je bila predvsem stabilnost delovanja, vendar na račun povečane površine in kompleksnosti.[130]

8.3.2.3 Wang in sodelavci (2003)

Wangov polni seštevalnik predstavlja pomemben preskok v smeri optimizacije, ki so ga dosegli z uporabo boljšega algoritma za implementacijo seštevalnika. Avtorji so uspeli zmanjšati število celic na 105, kar je skoraj polovica prvotne vrednosti. Površina implementacije je bila s tem bistveno manjša (0.17 μm^2), zakasnitev pa je padla na samo 4 cikle. Čeprav je tudi ta rešitev uporabljala coplanar križanja, se je izkazala za precej bolj učinkovito. Prav zaradi razmerja med majhno kompleksnostjo in hitro izvedbo je ta dizajn postal ena najpogosteje omenjenih izboljšav v literaturi. [131]

8.3.2.4 Joy et al. (2021)

Joy in sodelavci so predstavili coplanar izvedbo polnega seštevalnika, ki uporablja 61 celic in zasede zelo majhno površino (0.07 μm^2). Glavna prednost njihovega pristopa je kratka zakasnitev, saj se signal propagira skozi samo dva urna cikla. Dizajn je zasnovan kot izboljšava prejšnjih kompaktnih izvedb, pri čemer so avtorji veliko pozornosti namenili zmanjševanju nepotrebnih medceličnih povezav. Rezultat je do-

bro uravnotežen seštevalnik, ki kljub coplanar križanju dosega nizko kompleksnost in razmeroma dobro stabilnost delovanja. [132]

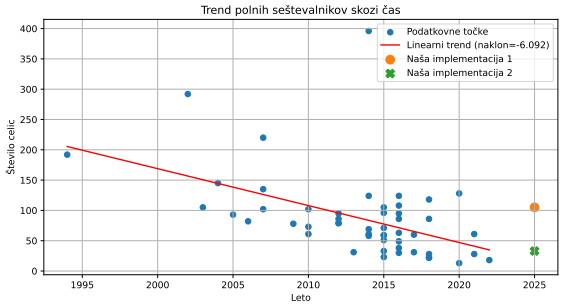
8.3.2.5 Seyedi in Navimipour (2021)

Avtorja sta leta 2021 predstavila multilayer strukturo s samo 28 celicami. To predstavlja enega izmed bolj kompaktnih seštevalnikov v literaturi. Majhna površina (0.01 μm^2) in zmerna zakasnitev (3 cikli) naredita dizajn zanimiv predvsem za aplikacije, kjer sta ključna gostota in minimalna raba celic. Z uporabo multilayer križanja sta se izognila omejitvam coplanar pristopa, kar je omogočilo bolj učinkovito razporeditev signalnih poti ter s tem tudi nižjo vrednost "cost" metrike. [133]

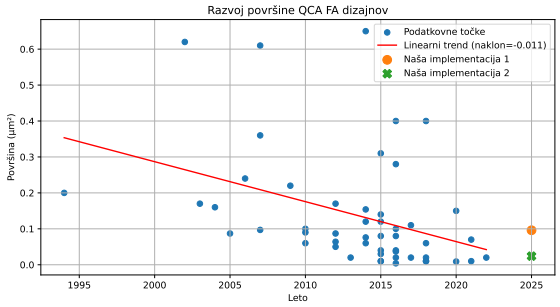
8.3.2.6 Seyedi in Jafari Navimipour (2022)

Leta 2022 sta Seyedi in Jafari Navimipour predstavila še bolj optimizirano multilayer različico, ki uporablja samo 18 celic. To je ena najmanjših kompleksnosti, ki jo najdemo med QCA polnimi seštevalniki. Površina (0.02 μm^2) ostaja zelo majhna, zakasnitev pa znaša le 2 cikla. V primerjavi s prejšnjimi rešitvami je razvoj usmerjen predvsem v zmanjšanje števila celic ob ohranjanju pravilnega delovanja. Takšne arhitekture so posebej uporabne za večje aritmetične enote, kjer se prištevalniki pojavijo v velikem številu. [134]

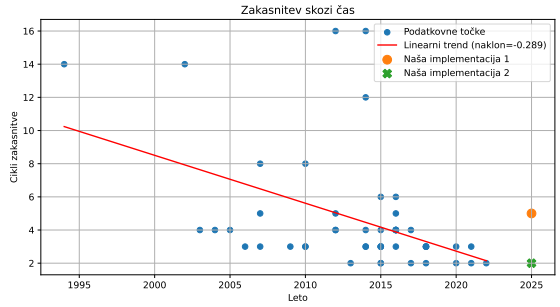
8.3.3 Grafična predstavitev obravnavanih implementacij



Slika 8.8: Prikaz števila celic v QCA polnih seštevalnikih skozi čas. Vidimo jasen trend zmanjševanja kompleksnosti, kar odraža napredek v načrtovanju arhitektur.



Slika 8.9: Razvoj površine QCA FA implementacij skozi leta. Površina se praviloma zmanjšuje, kar kaže na optimizacije postavitve celic in manjše izgube zaradi žičenja.

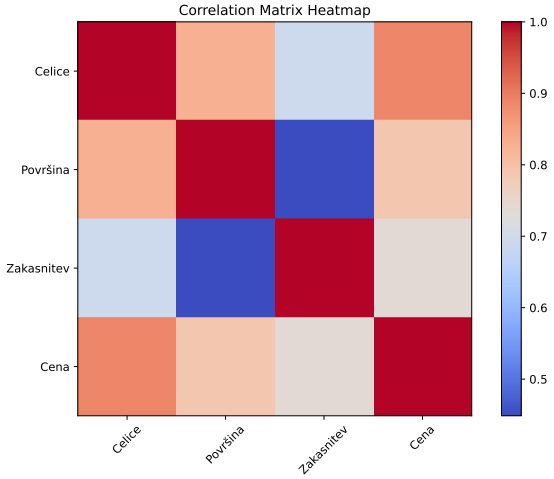


Slika 8.10: Zakasnitev v taktnih ciklih skozi čas. Opazimo postopno izboljševanje, saj novejši dizajni uvajajo krajše signalne poti in učinkovitejše strukture za prenos polarizacije.

8.4 Primerjalna analiza

Za boljše razumevanje razvoja QCA polnih seštevalnikov je bila izvedena primerjalna analiza obstoječih rešitev, ki pokriva obdobje skoraj treh desetletij. Pri tem smo se osredotočili na ključne metrike, ki se najpogosteje uporabljajo v literaturi: število celic (kompleksnost), površino, zakasnitev ter strošek, ki združuje vpliv vseh parametrov.

Rezultati jasno kažejo, da so se skozi čas pojavili izraziti trendi optimizacije. Število celic in posledično površina dizajnov se zmanjšujeta, kar je razvidno iz grafov 8.8 in 8.9. To je predvsem posledica novih arhitektur, boljše uporabe večplastnega žičenja ter učinkovitejšega razporejanja logičnih vrat. Podoben trend je opazen tudi pri zakasnitvi, kjer novejši dizajni izkoriščajo krajše signalne poti ter boljšo sinhronizacijo faz, kar je prikazano v grafu 8.10.



Slika 8.11: Korelacijska matrika med izbranimi parametri (število celic, površina, zakasnitev in strošek). Najmočnejše korelacije so pričakovano med številom celic, površino in stroškom.

Korelacijska analiza (slika 8.11) potrdi pričakovane povezave med parametri. Najmočnejši vpliv imajo število celic, površina in strošek, kar je logično, saj se metrika *cost* navadno neposredno izračuna iz teh vrednosti.

Kljub temu je pri interpretaciji rezultatov potrebna previdnost. Različni avtorji uporabljajo nekoliko različne arhitekturne pristope, optimizacijske strategije ter simulacijska orodja, ter metrike ocenjevanja, kar lahko vpliva na končne vrednosti posameznih parametrov. Zato grafi in statistika dobro prikažejo splošni razvoj skozi čas, vendar je za natančno razumevanje razlogov za izboljšave vedno potrebno pregledati izvorne članke in analizirati specifične oblikovne odločitve vsake implementacije posebej.

8.5 Naša implementacija

Odločili smo se zasnovati lastno implementacijo in si pri tem nismo pomagala z drugimi implementacijami, ki smo jih odkrili in primerjali v poglavju pregled literature. Sledili smo osnovni logiki polnega seštevalnika.

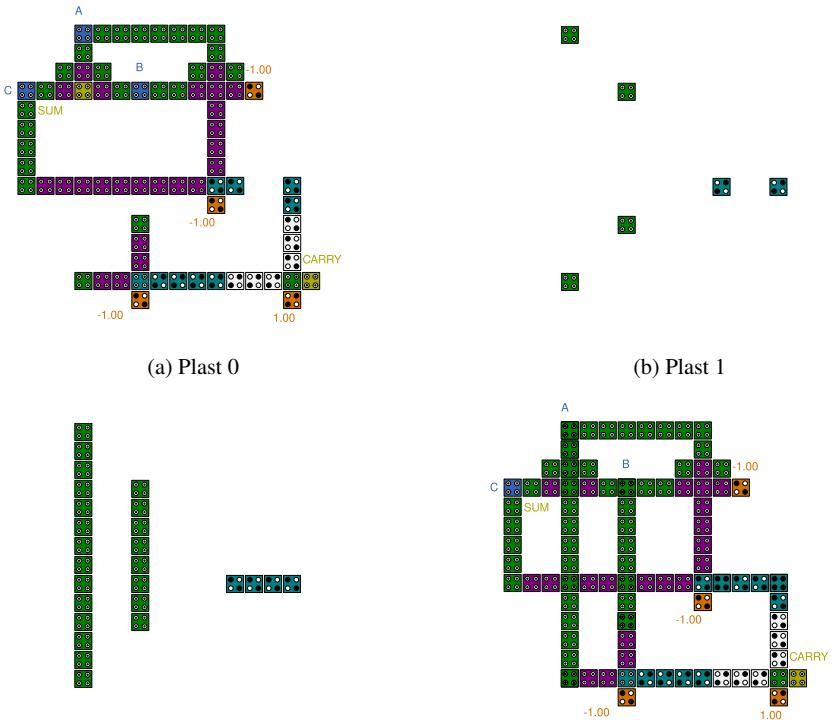
8.5.1 Prva implementacija

V prvi implementaciji 8.12 smo uporabili poenostavne logične izraze za SUM in CARRY opisane v poglavju polni seštevalnik. SUM dobimo s trojnimi XOR vrati, ki takoj vrnejo SUM logične vrednosti. CARRY ima bolj zahteven logični izraz, najprej smo uprabaili XOR vrata sestavljena iz CNOT, tako, da smo dobili A xor B, nato smo ta rezultat vodili do majoritetnih vrat z vrednostjo -1, kar predstavlja AND vrata, kjer smo dodali še C signal in tako dobili desno polovico enačbe $C(A \oplus B)$. Posebej smo izvedli še AND operacijo med A in B, in to operacijo združili s prej dobljenim signalom z majoritetnimi vrati z vrednostjo 1, kar predstavlja OR vrata. Tako smo dobili celotno enačbo $CARRY = AB + C(A \oplus B)$ in s tem CARRY izhod.

Velika ovira je bilo vodenje signala, saj ni mogoče povezati vseh vrat brez križanja signalov, pri čemer je prihajalo do motenj, zato smo morali uporabit več plasti. Pri prenašanju signala na daljše razdalje je prihajalo tudi do šumov in izgube signala zaradi česar izhodni signal ne ustreza popolnoma definicijskemu primeru signala z vrednostjo 0 (8.13), so pa očitne razlike med vrednostmi 0 in 1, kar pomeni, da se signal prenaša.

8.5.2 Druga implementacija

Ker nismo bili zadovoljni z izstopnim signalom, smo se odločili narediti novo iteracijo naše implementacije. Želeli smo si čimbolj poenostaviti vezje in skrajšati razdalje po katerih vodimo signale. Signal SUM smo že dobili z enimi 3XOR vrati, kar je že poenostavljena verzija dveh XOR vrat, zato smo se posvetili poenostavljanju CARRY izhoda, ki uporablja dvojna AND vrata in ena OR vrata. Ugotovili smo, da je možno CARRY signal zapisati kot majoritetna vrata s tremi vhodi, kar je zelo koristno, saj se v tehnologiji kvantnih celic uporabljajo majoritetna vrata s konstanto za tako AND kot tudi OR vrata. Konstanto smo nadomestili s tretjim signalom in tako znatno poenostavili vezje, tako v površini, kot plasteh in tudi številu uporabljenih celic, kar se vidi iz slike 8.14. Druga iteracija tako deluje na le eni plasti, in uporablja le dvoje vrat, ena 3XOR vrata in ena majoritetna vrata. Izhodni signal na drugi iteraciji ima veliko manj šuma, signal za vrednost 0 ima polariteto -1, in signal za 1 ima polariteto 1, kar je tudi vidno na sliki 8.15.



(a) Plast 0

(b) Plast 1

(c) Plast 2

(d) Združene Plasti

Slika 8.12: QCA polni seštevalnik plasti: (a) Plast 0, (b) Plast 1, (c) Plast 2, (d) Združeni

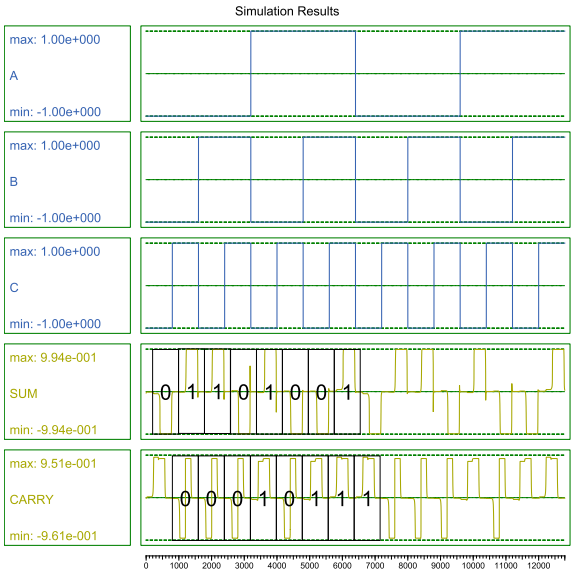
8.6 Rezultati

Rezultati v tabeli 8.2 kažejo, da smo z drugo implemetacijo dosegli uporabo manjšega števila celic, manjše površine, pri čemer pride tudi do manjše zakasnitve in manjšega stroška [1].

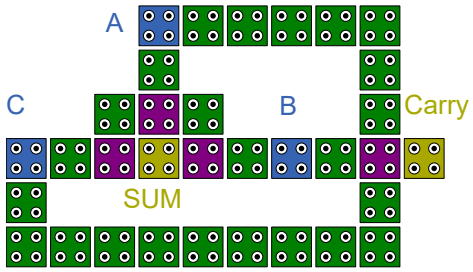
8.7 Zaključek

V seminarski smo raziskali različne implementacije polnega seštevalnika v QCA. Primerjava že obstoječih rešitev je pokazala izrazit trend optimizacije, pri čemer se

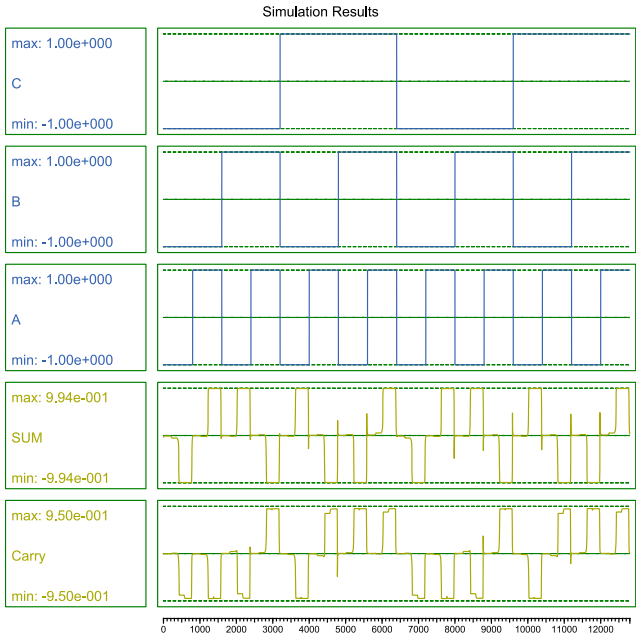
[1] V članku [128] so ocenjene cene zračunane na dva načina, zato jih težko primerjamo med sabo. Nekateri so ceno računali z normalizirano zakasnitvijo ($\frac{zakasnitev}{4} * povrsina$), nekateri pa ne ($zakasnitev * povrsina$).



Slika 8.13: Simulacija prve implementacije



Slika 8.14: Druga implementacija



Slika 8.15: Simulacija druge implementacije

Metrika	**Prva implementacija**	**Druga implementacija**
Število celic	105	33
Površina (μm^2)	0.096	0.024
Zakasnitev (clock cikli)	5	2
Vrsta križanja	večplastno	koplanarno
Strošek	0.12	0.012

Tabela 8.2: Primerjava obeh implementacij polnega seštevalnika

zmanjšujeta število celic in površina. Opazna je tudi optimizacije pri zakasnitviji in stroških. Razvili smo tudi dve lastni implementaciji z uporabo kompleksnejših vrat. Prva verzija je sledila klasični logični strukturi, vendar je zaradi večplastnega križanja prišlo do zmanjšane stabilnosti in povečanega števila celic ter večje zakasnitve. V drugi implementaciji smo uporabili minimalno število vrat ter koplanarno križanje, pri čemer smo izboljšali vse metrike: manj celic, manjša površina, krajša zakasnitev, manjša cena in stabilnejši izhod.

Doprinos Avtorjev

Delo smo si razdelili sledeče:

- Klemen Kavčič - Implementacija obeh vezij in opis polnega seštevalnika
- Tomi Trošt - Pregled, interpretacija in analiza sorodnih ter predstavitev teh z grafi.
- Pia Sotlar - Predstavitev QCA in osnovnih gradnikov vezij, pomoč pri prvi implementaciji

Poglavje 9
Analiza in implementacija Simonovega algoritma

Matevž Crček in Maks Klemenčič

Povzetek V tem poročilu obravnavamo Simonov problem, enega ključnih zgodovinskih primerov kvantnega pospeška, ki prikazuje, kako lahko kvantni algoritmi izkoristijo lastnosti kvantne superpozicije in interferenc za učinkovitejše reševanje problemov kot klasični postopki. Predstavimo osnovno idejo problema, njegovo matematično ozadje ter kvantni algoritem, ki omogoča izračun skritega niza z eksponentno manj koraki kot pri klasičnem pristopu. Poleg teoretičnega pregleda vključimo tudi praktične meritve delovanja kvantnega vezja, analiziramo dobljene rezultate in ovrednotimo vpliv števila meritev na natančnost rekonstrukcije skritega niza.

9.1 Uvod

Simonov problem [135] je zgodovinsko pomemben primer kvantnega algoritma, ki demonstrira eksponenten pospešek pred klasičnimi metodami. V središču problema je funkcija z lastnostjo, da za nek neznan binarni niz s velja, da je funkcijska vrednost enaka za vhoda x in $x \oplus s$. Z uporabo kvantnega računalnika lahko niz s določimo z bistveno manj poizvedbami v primerjavi s klasičnim pristopom. Ideje, ki izhajajo iz Simonovega algoritma, so vplivale tudi na razvoj Shorovega algoritma, ki ga lahko uporabimo za zlom šifriranja RSA [136].

9.2 Opis problema

Dana je črna škatla oziroma orakel, ki realizira funkcijo iz bitnega niza dolžine n v bitni niz dolžine m

$$f : \{0,1\}^n \to \{0,1\}^m, m \geq n, \tag{9.1}$$

kjer je f bodisi injektivna funkcija (kjer se vsak vhodni niz preslika v različno izhodno vrednost), bodisi dvovredna funkcija (kjer se dva vhodna niza preslikata v isto izhodno vrednost), pod dodatnim pogojem, da obstaja nek skrivni bitni niz s dolžine n:

$$f(x) = f(y) \quad \text{natanko tedaj, ko} \quad y = x \oplus s, \tag{9.2}$$

kjer $\oplus$ označuje bitni ekskluzivni ali (XOR).

Cilj je ugotoviti vrednost skrivnega bitnega niza s s čim manj evalvacijami funkcije f.

Zgornjo definicijo problema lahko rahlo poenostavimo zaradi lastnosti $\oplus$. Če je $y = x$, potem ima $f(y) = f(x)$ in $y = x \oplus s$ edino rešitev $s = 0^n$. Torej za injektivne funkcije velja, da imajo vrednost s enako 0^n. Če upoštevamo dejstvo, da $a \oplus b \oplus b = a$ za vse a in b, sledi $y = x \oplus s \iff y \oplus x = s$. Lahko opazimo tudi, da v primeru, da je f dvovrednostna funkcija, potem mora biti periodična s periodo $s : f(x) = f(x \oplus s)$.

Tabela 9.1: Primer dvovrednostne funkcije za $n = 3$, $m = 4$ in $s = 110$.

x	$f(x)$
000	1000
001	0010
010	1110
011	0000
100	1110
101	0000
110	1000
111	0010

Iz zgornje tabele lahko razberemo, da gre za dvovrednostno funkcijo, ker se vsaka različna izhodna vrednost pojavi dvakrat in za vse vrednosti x in y z enako izhodno vrednostjo velja $y = x \oplus 110$, torej $s = 110$. V tem primeru je $m = 4$, tj. dolžina izhodnih nizov je 4, lahko bi bila poljubno število ≥ 3.

9.3 Algoritem za klasične računalnike

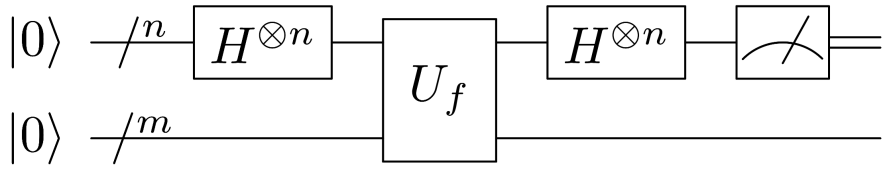
Ta problem je zelo težko rešiti s klasičnimi računalniki, tudi če uporabimo naključnost in sprejmemo majhno verjetnost napake. Intuicija za to težavnost je precej preprosta: če želimo problem rešiti klasično, moramo najti dva različna vhoda x in y, za kateri velja $f(x) = f(y)$. V funkciji f ni nujno nobene strukture, ki bi nam pomagala pri iskanju takšnega para vhodov; natančneje, o funkciji f (o tem, kaj počne) lahko izvemo nekaj šele takrat, ko za dva različna vhoda dobimo enak izhod. Če bi evalvirali funkcijo f z naključnimi vhodi, bi morali presikati $\Omega(2^{n/2})$ različnih vhodov, preden bi z dovolj veliko verjetnostjo našli par, na katerem ima

f enak izhod, skladno z rojstnodnevnim paradoksom. Če bi hoteli dobiti odgovor s 100% verjetnostjo, bi morali preiskati več kot polovico ($2^{n-1}+1$) možnih vhodnih nizov.

9.4 Simonov algoritem

Simonov algoritem je sestavljen iz dveh delov - večkratne evalvacije kvantnega vezja in nato klasičnega procesiranja dobljenih rezultatov. V podpoglavju 9.4.1 je opisan kvantni del, v 9.4.2 pa klasično procesiranje.

9.4.1 Kvantni del Simonovega algoritma



Slika 9.1: Spošna shema za kvantno vezje za vhode z dolžino n in poizvedbo z m qubiti.

Po izvedbi prve plasti Hadamardovih vrat na zgornjih n kubitih postane stanje

$$\frac{1}{2^n}\sum_{x\in\Sigma^n}|0^m\rangle|x\rangle. \tag{9.3}$$

Ko izvedemo operator U_f, se izhod funkcije f XOR-a na ničelno stanje spodnjih m kubitov, zato postane stanje

$$\frac{1}{2^n}\sum_{x\in\Sigma^n}|f(x)\rangle|x\rangle. \tag{9.4}$$

Ko izvedemo drugo plast Hadamardovih vrat, dobimo naslednje stanje, pri čemer uporabimo enako formulo za delovanje plasti Hadamardovih vrat kot prej:

$$\frac{1}{2^n}\sum_{x\in\Sigma^n}\sum_{y\in\Sigma^n}(-1)^{x\cdot y}|f(x)\rangle|y\rangle. \tag{9.5}$$

Zanimajo nas verjetnosti, da meritve vrnejo vsak možen niz $y\in\Sigma^n$. Po pravilih za analizo meritev vemo, da je verjetnost $p(y)$ za dobitev niza y enaka

$$p(y) = \left\| \frac{1}{2^n} \sum_{x \in \Sigma^n} (-1)^{x \cdot y} |f(x)\rangle \right\|^2 . \tag{9.6}$$

Da bi bolje razumeli te verjetnosti, potrebujemo nekaj dodatne notacije. Najprej je zaloga (angl. *range*) funkcije f množica vseh njenih izhodnih nizov:

$$\text{range}(f) = \{f(x) : x \in \Sigma^n\}. \tag{9.7}$$

Drugič, za vsak niz $z \in \text{range}(f)$ lahko množico vseh vhodnih nizov, ki se preslikajo v z, zapišemo kot $f^{-1}(\{z\})$:

$$f^{-1}(\{z\}) = \{x \in \Sigma^n : f(x) = z\}. \tag{9.8}$$

Množico $f^{-1}(\{z\})$ imenujemo predslika množice $\{z\}$ pod funkcijo f. Predsliko katerekoli množice lahko definiramo analogno, torej kot množico vseh elementov, ki se preslikajo v to množico. (Te notacije ne smemo zamenjevati s funkcijskim inverzom, ki morda ne obstaja.)

Z uporabo te notacije lahko razbijemo vsoto v izrazu za verjetnosti:

$$p(y) = \left\| \frac{1}{2^n} \sum_{z \in \text{range}(f)} \left(\sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right) |z\rangle \right\|^2 . \tag{9.9}$$

Vsak niz $x \in \Sigma^n$ se pojavi natanko enkrat v teh dveh vsotah — v bistvu jih le razvrščamo v ustrezne koše glede na izhodni niz $z = f(x)$ ter nato seštevamo po vsakem košu posebej.

Zdaj lahko izračunamo evklidsko normo na kvadrat:

$$p(y) = \frac{1}{2^{2n}} \sum_{z \in \text{range}(f)} \left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 . \tag{9.10}$$

Da poenostavimo izraz, poglejmo vrednost

$$\left| \sum_{x \in f^{-1}(\{z\})} (-1)^{x \cdot y} \right|^2 \tag{9.11}$$

za poljuben $z \in \text{range}(f)$.

Če velja $s = 0^n$, je funkcija f injektivna in ima vsaka množica $f^{-1}(\{z\})$ natanko en element. V tem primeru ima izraz 9.11 vrednost 1.

Če pa velja $s \neq 0^n$, ima vsaka množica $f^{-1}(\{z\})$ natanko dva elementa. Če izberemo poljuben $w \in f^{-1}(\{z\})$, je drugi element $w \oplus s$. Zato dobimo:

$$\left|(-1)^{w\cdot y}+(-1)^{(w\oplus s)\cdot y}\right|^2=|(-1)^{w\cdot y}(1+(-1)^{s\cdot y})|^2=|1+(-1)^{y\cdot s}|^2=\begin{cases}4 & y\cdot s=0,\\ 0 & y\cdot s=1.\end{cases} \tag{9.12}$$

Sedaj lahko nadaljujemo ločeno za dva primera:

Primer 1: $s=0^n$.

V tem primeru je funkcija f injektivna, zato je v range(f) natanko 2^n elementov. Dobimo:

$$p(y)=\frac{1}{2^{2n}}\cdot 2^n=\frac{1}{2^n}. \tag{9.13}$$

Meritev vrne niz $y\in\Sigma^n$ enakomerno naključno.

Primer 2: $s\neq 0^n$.

V tem primeru je funkcija f dvovredna (two-to-one), zato je v range(f) natanko 2^{n-1} elementov. Iz zgornje formule dobimo:

$$p(y)=\frac{1}{2^{2n}}\sum_{z\in\text{range}(f)}\left|\sum_{x\in f^{-1}(\{z\})}(-1)^{x\cdot y}\right|^2=\begin{cases}\frac{1}{2^{n-1}} & \mathbf{y}\cdot\mathbf{s}=0,\\ 0 & \mathbf{y}\cdot\mathbf{s}=1.\end{cases} \tag{9.14}$$

Z drugimi besedami, rezultat meritve je enakomerno naključen niz iz množice

$$\{y\in\Sigma^n:\mathbf{y}\cdot\mathbf{s}=0\}, \tag{9.15}$$

ki vsebuje 2^{n-1} nizov. Ker je $s\neq 0^n$, ima natanko polovica binarnih nizov dolžine n skalarni produkt 1 (modulo 2) s s, druga polovica pa 0.

9.4.2 Klasično procesiranje Simonovega algoritma

Iz zgornje analize vemo kakšne so verjetnosti mogočih izidov meritev, ko zaženemo kvantno vezje Simonovega algoritma. Tako lahko iz dejanske distribucije meritev izračunamo najbolj verjetno vrednost s. Recimo, da vezje zaženemo neodvisno k-krat, pri čemer izberemo $k=n+10$. Ta izbira ni nujna, lahko izberemo tudi več (ali manj), odvisno od verjetnosti napake, ki jo želimo tolerirati. Za $k=n+10$ je verjetnost pravilne rekonstrukcije niza s večja od 99.9%.

Po k ponovitvah dobimo nize $y^{(1)},y^{(2)},\ldots,y^{(k)}\in\Sigma^n$. Tu indeks v oklepajih označuje zaporedno številko meritve, kateri pripada niz. Vsak niz lahko zapišemo kot

$$y^1 = y^{(1)}_{n-1} \dots y^{(1)}_0, \quad y^2 = y^{(2)}_{n-1} \dots y^{(2)}_0, \quad \dots, \quad y^k = y^{(k)}_{n-1} \dots y^{(k)}_0. \tag{9.16}$$

Na osnovi izmerjenih nizov oblikujemo matriko M dimenzij $k \times n$ z binarnimi vrednostmi:

$$M = \begin{pmatrix} y^{(1)}_{n-1} & \cdots & y^{(1)}_0 \\ y^{(2)}_{n-1} & \cdots & y^{(2)}_0 \\ \vdots & \ddots & \vdots \\ y^{(k)}_{n-1} & \cdots & y^{(k)}_0 \end{pmatrix}. \tag{9.17}$$

Na tej točki niza s še ne poznamo. Toda predstavljajmo si, da ga poznamo, in iz njegovih bitov $s = s_{n-1} \dots s_0$ sestavimo stolpčni vektor

$$\mathbf{v} = \begin{pmatrix} s_{n-1} \\ \vdots \\ s_0 \end{pmatrix}. \tag{9.18}$$

Če izračunamo produkt matrike in vektorja modulo 2:

$$M\mathbf{v} = \begin{pmatrix} \mathbf{y_1} \cdot \mathbf{s} \\ \mathbf{y_2} \cdot \mathbf{s} \\ \vdots \\ \mathbf{y_k} \cdot \mathbf{s} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \tag{9.19}$$

dobimo ničelni vektor. Torej je vektor $\mathbf{v}$, ki predstavlja niz s, vedno v ničelnem prostoru matrike M pri aritmetiki modulo 2. To velja tako v primeru $s = 0^n$ kot v primeru $s \neq 0^n$. Ničelni vektor je vedno v jedru matrike M, v primeru $s \neq 0^n$ pa se mu pridruži še vektor, ki ustreza nizu s.

Ostaja vprašanje: ali je v ničelnem prostoru matrike M še kakšen drug vektor poleg tistih, ki predstavljajo 0^n in s? Verjetnost za to hitro pada, ko povečujemo k. Če izberemo $k = n + 10$, bo z verjetnostjo več kot 99.9% ničelni prostor vseboval točno vektorja za 0^n in s. Na splošno, če vzamemo $k = n + r$, kjer je r poljubno pozitivno celo število, potem bo verjetnost, da sta to edina vektorja v jedru, vsaj $1 - 2^{-r}$.

Nišči prostor matrike M modulo 2 lahko učinkovito izračunamo z Gaussovo eliminacijo, ki deluje enako kot pri aritmetiki z realnimi ali kompleksnimi števili, le da računamo modulo 2.

Če sta vektorja, ki ustrezata 0^n in s, edina v jedru matrike (kar velja z visoko verjetnostjo), lahko iz dobljenega opisa ničelnega prostora enostavno rekonstruiramo niz s.

9.5 Implementacija Simonovnega algoritma

Za praktično predstavitev Simonovega algoritma smo razvili interaktivni program v jezuku Python s pomočjo knjižnice za kvantno računalništvo Qiskit[137]. Programu podamo željen skriven niz s, iz katerega se zgradi kvantno vezje z ustreznim oraklom, nato se vezje simulira $n+10$-krat na lokalnemu kvantnemu simulatorju, na koncu pa se na meritvah izvede še klasično procesiranje za izračun s. Program ustvarjeno kvantno vezje in rezulate simulacije shrani v obliki png slik. Slike 9.2, 9.3 in 9.4 prikazujejo celotno Python kodo. Specifike implementacije so opisane s komentarji v kodi.

```
from qiskit import QuantumCircuit
from qiskit.transpiler import generate_preset_pass_manager
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
from qiskit_aer.primitives import SamplerV2 as Sampler

from scipy.linalg import null_space
import numpy as np

# Zgradi kvantno vezje, ki deluje kot Simonov orakel za skrivno \
+ vrednost s
def simon_orakel(s: str) -> QuantumCircuit:
    n = len(s)
    s = s[::-1] # Obrnemo vrstni red bitov za lažjo iteracijo
    orakel_vezje = QuantumCircuit(n*2)
    # Povežemo bite iz vhodnega registra z delovnem registrom
    for i in range(n):
        orakel_vezje.cx(i, i+n)
    prvi_s = s.find('1')  # index prvega neničelnega bita v s
    # Če ima s bit 1, povežemo qbit prvi_s s tem qbitom v \
+ delovnem registru
    for q in range(n):
        if s[q] == '1':
            orakel_vezje.cx(prvi_s, q + n)
    return orakel_vezje
```

Slika 9.2: Koda za izdelavo vezja orakla.

```
# Zgradi vezje: vhodni register dolžine n in delovni register \
+ dolžine n
def zgradi_vezje(s: str) -> QuantumCircuit:
    n = len(s)
    simon_vezje = QuantumCircuit(n * 2, n)
    simon_vezje.h(range(n)) # Hadamard na vhodni register pred \
+ vhodom v orakel
    simon_vezje.barrier() # Črtkana črta za ločenje orakla
    simon_vezje = simon_vezje.compose(simon_orakel(s)) # Dodamo \
+ simonov orakel
    simon_vezje.barrier() # Črtkana črta za ločenje orakla
    simon_vezje.h(range(n)) # Hadamard na register dolžine n po \
+ izhodu iz orakla
    simon_vezje.measure(range(n), range(n)) # Izmerimo vhodni \
+ register
    simon_vezje.draw('mpl', filename="simon_vezje.png")
    return simon_vezje

def simuliraj_vezje(vezje: QuantumCircuit, st_ponovitev: int) \
+ -> np.array:
    sampler = Sampler()
    pass_manager = generate_preset_pass_manager(3, \
+ AerSimulator())
    isa_circuit = pass_manager.run(vezje.copy())
    job = sampler.run([(isa_circuit, None, st_ponovitev)]) # \
+ Poženemo naše vezje na lokalnem simulatorju
    meritve = job.result()[0].data['c'].array
    u, c = np.unique(meritve, return_counts=True)
    plot_histogram(dict(zip(u, c / len(u))), \
+ filename="meritve_histogram.png")
    return meritve
```

Slika 9.3: Koda za izdelavo celotnega kvantnega vezja in simulacijo.

```
# Spremeni število v seznam bitov dolžine n npr. 010110 -> \
+ [0,1,0,1,1,0]
def spremeni_v_seznam_bitov(stevilo: int, n: int) -> list[int]:
    biti = []
    for i in range(n):
        if stevilo & (1 << (n - i - 1)) != 0:
            biti.append(1)
        else:
            biti.append(0)
    return biti

def izracunaj_s(meritve: np.array, n: int) -> str:
    M = np.array([spremeni_v_seznam_bitov(x, n) for x in \
+ meritve]) # Meritve damo zaporedno v M
    nicelni_prostor = np.array(np.round(np.abs(null_space(M))), \
+ dtype=int) # Izračunamo ničelni prostor
    if nicelni_prostor.shape[1] > 1:
        raise Exception("Ničelni prostor ima več rešitev. \
+ Posksite ponovno simulirati.")
    if nicelni_prostor.shape[1] == 0:
        return "0" * n # Ničelni prostor ima samo ničelni \
+ vektor. s je lahko samo 0.
    return "".join([str(x[0]) for x in nicelni_prostor]) # \
+ Vrnemo niz 0 in 1

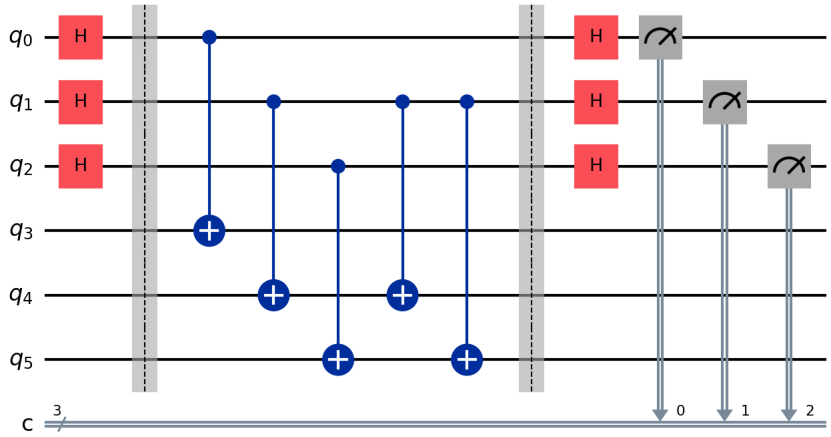
def main():
    while True:
        print("Vpiši s (niz 0 in 1):")
        s = input()
        n = len(s)
        st_meritev = n + 10
        vezje = zgradi_vezje(s)
        meritve = simuliraj_vezje(vezje, st_meritev)
        izracunan_s = izracunaj_s(meritve, n)
        print(f"Izračunan s: {izracunan_s}. {"Pravilno" if \
+ izracunan_s == s else "Napačno" }")

if __name__ == "__main__":
    main()
```

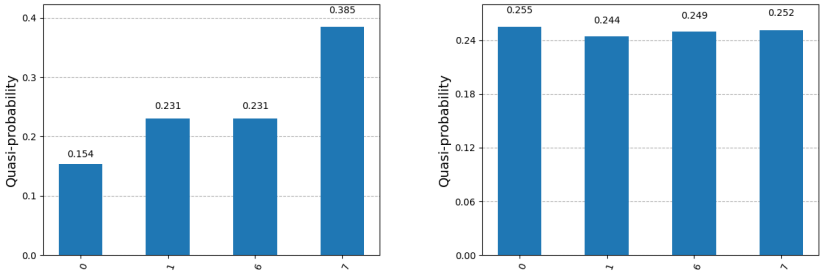
Slika 9.4: Koda za klasični del agoritma.

9.6 Rezultati

Oglejmo si primer, kjer je $s = 110$. Pripadajoče vezje je prikazano na sliki 9.5.



Slika 9.5: Kvantno vezje za $s = 110$.



(a) 13 meritev

(b) 10000 meritev

Slika 9.6: Hitogram meritev vezja z $s = 110$.

Če poženemo simulacijo, dobimo meritve, ki so prikazane na sliki 9.6. Pri 13 meritvah je distribucija neuravnotežena, ker gre za premajhen vzorec, pri 10000 meritvah pa lahko jasno vidimo, da so vse vrednosti meritev enako verjetne. Opazimo lahko tudi, da so prisotne samo 4 vrednosti meritev, kar je polovica 8 možnih vrednosti. Kot smo izpostavili v podpoglavju 9.4.2 so to vrednosti, kjer je skalarni

produkt $\mathbf{y} \cdot \mathbf{s} = 0$ modulo 2. V obeh primerih po klasičnem procesiranju meritev dobimo pravilen rezultat $s = 110$.

9.7 Zaključek

V poročilu smo predstavili Simonov problem in kvantni algoritem, ki omogoča določitev skritega niza s z eksponentno manj poizvedbami kot klasični pristop. Analiza meritev je pokazala, da se natančnost rekonstrukcije niza izboljšuje z večjim številom meritev, kar je skladno s teoretičnimi pričakovanji o kvantnem algoritmu. Praktični primeri potrjujejo, da kvantni postopki uspešno izkoriščajo interferenco za pridobivanje linearnih omejitev, iz katerih lahko s klasičnimi metodami določimo iskani niz. Simonov algoritem tako ostaja pomemben primer, ki jasno prikazuje prednosti kvantnega računanja ter njegovo vlogo pri razumevanju in razvoju naprednejših kvantnih algoritmov, kot je npr. Shorov algoritem.

9.8 Doprinosi avtorjev

Maks Klemenčič je raziskal Simonov problem in kvantni del algoritma ter napisal ta poglavja v poročilu. Matevž Crček je raziskal klasični del Simonovega algoritma, naredil implementacijo in napisal preostanek poročila.

Chapter 10
QCA-Based 4-to-2 Priority Encoder: Majority Logic vs. Reversible Toffoli Design

Stefan Krstevski and Žan Seničar

Abstract Classical CMOS technology is reaching fundamental limits in scaling and energy efficiency. As a result, alternative computing methods that do not rely on current flow through transistors are being explored. Quantum-dot cellular automata (QCA) represent one such approach by encoding binary information in the spatial arrangement of electrons within nanoscale cells. Reversible logic is another promising direction, aiming to reduce energy dissipation by preserving information during computation. This seminar explores how QCA and reversible logic can be combined in the design of a 4-to-2 priority encoder. We review the basic QCA components, the conventional operation of 4-to-2 encoders and priority encoders, and the use of the Toffoli gate as a universal reversible gate. Two QCA implementations are presented: a single-layer majority-gate encoder and a reversible encoder based on QCA Toffoli gates. Both circuits are simulated in QCADesigner and evaluated using standard metrics such as cell count, circuit area, number of layers, and clock zones. The results highlight a clear trade-off between implementation efficiency and logical reversibility.

10.1 Introduction

In recent years, CMOS technology has reached physical limitations in device scaling and energy efficiency. As transistor dimensions approach the nanometre scale, problems such as leakage current, parameter variability and high power density become increasingly difficult to manage. This has motivated the development of alternative computing technologies that operate beyond conventional transistor-based logic.

Quantum-dot cellular automata (QCA) represent one such approach. Instead of current flow, QCA encodes binary information through the charge configuration of electrons in nanoscale quantum-dot cells. Logical operations are performed by electrostatic interactions between neighbouring cells, enabling potentially high integration density and low power dissipation [138].

A second promising direction is *reversible logic*. According to Landauer's principle, every irreversible bit operation dissipates at least $k_B T \ln 2$ of energy. Reversible circuits avoid this energy loss by maintaining a one-to-one mapping between inputs and outputs. The Toffoli gate is a standard universal reversible gate and is commonly used for constructing larger reversible structures [139].

The focus of this seminar is the *4-to-2 priority encoder*, a widely used component in control systems such as interrupt controllers and bus arbitration [140, 141]. We implement and compare two QCA-based versions: (1) a compact majority-gate design, and (2) a reversible design based on QCA Toffoli gates. Both are simulated in QCADesigner and evaluated using standard performance metrics.

10.2 Basic QCA Building Blocks

In this section we briefly review the QCA concepts needed to understand our encoder implementations: the QCA cell, the clocking scheme, basic logic gates and the Toffoli gate as an example of reversible logic in QCA [138, 139].

10.2.1 QCA Cell

In QCA, binary information is stored in nanoscale square cells. Each QCA cell consists of four quantum dots positioned at the corners of a square and two mobile electrons that can tunnel between these dots. Due to Coulomb repulsion, the electrons naturally arrange themselves in opposite corners of the cell in order to minimise their potential energy. This leads to two stable charge configurations: one along the main diagonal and one along the opposite diagonal. These two configurations correspond to logical values "0" and "1".

The logical state of a cell is quantified using its polarization P, which takes values close to $+1$ or -1 depending on the electron configuration. When multiple QCA cells are placed in close proximity, the polarization of one cell can influence its neighbours through electrostatic interaction. This enables information transfer along a chain of cells and allows the construction of logic gates using purely physical interactions.

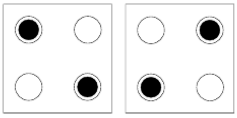


Fig. 10.1: QCA cell consisting of four quantum dots and two electrons.

10.2.2 Clocking and Signal Propagation

To achieve directed signal flow and stable operation, QCA layouts use a special clocking mechanism. The surface is divided into four clock zones, each driven by a phase-shifted control signal. The clock locally raises and lowers the tunnelling barriers inside the cells and cycles through four phases:

- **Switch**: barriers rise, and the cell becomes polarised according to its inputs;
- **Hold**: barriers remain high, the cell keeps its polarization and drives the next zone;
- **Release**: barriers start to fall and the cell loses its stored information;
- **Relax**: barriers are low and the cell is in a neutral state.

Neighbouring zones are shifted by 90°, so signals travel through the layout like a wave. The number of distinct clock zones on the critical path between inputs and outputs is therefore a good approximation of the circuit latency.

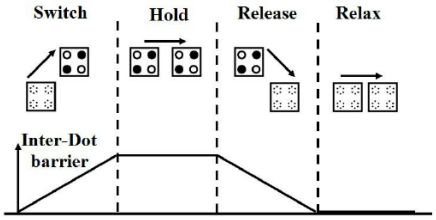


Fig. 10.2: The four phases of the QCA Clock.

10.2.3 Inverter and 3-Input Majority Gate

Two basic logic primitives can be implemented directly with small cell arrangements: the inverter and the 3-input majority gate.

If two cells are placed diagonally, electrostatic repulsion favours opposite polarizations. This configuration realises a logical inverter (NOT gate) and can also be used to route signals around corners.

The 3-input majority gate is the main QCA logic element. It consists of a central cell coupled to three input cells and one output cell. The output polarization follows the majority of the three inputs and is described by

$$M(a,b,c) = ab + ac + bc.$$

By fixing one input permanently to logical 0 or 1, the majority gate realises AND and OR, respectively. This means that arbitrary Boolean functions can be implemented using only majority gates and inverters.

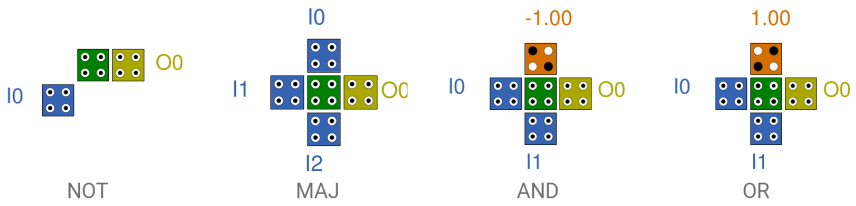


Fig. 10.3: Basic logic gates in QCA: NOT, MAJ, AND and OR.

10.2.4 Toffoli Gate

The Toffoli gate, also known as the controlled-controlled-NOT (CCNOT), is a 3-input, 3-output reversible gate. For inputs (A,B,C) the outputs are defined as

$$P = A, \qquad Q = B, \qquad R = A \cdot B \oplus C,$$

where $\oplus$ denotes the XOR operation. In words, the third bit is flipped only when both control bits A and B are 1, while the first two bits pass through unchanged. Because the mapping between inputs and outputs is bijective, the gate is logically reversible and forms a universal building block for classical reversible computation [139].

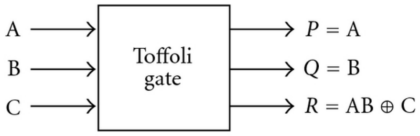


Fig. 10.4: Block diagram of the Toffoli gate.

In QCA, Toffoli gates can be constructed from majority gates, inverters and supporting XOR structures. These realizations are typically more complex than simple majority-based logic, but they enable the construction of reversible QCA circuits.

10.3 Digital Circuits Using Universal Gates

Before presenting the QCA implementations, we first review how 4-to-2 encoders and 4-to-2 priority encoders are realised with classical logic gates. The Boolean behaviour defined in this section is used directly as the specification for our QCA designs [140, 141].

10.3.1 4-to-2 Encoder

A 4-to-2 encoder has four input lines I_0, I_1, I_2, I_3 and two output lines O_1, O_0. The circuit assumes that *exactly one input is high* at any time. When one input is active, the encoder outputs its binary index.

The truth table is shown in Table 10.1. For example, if only I_2 is high, the output is $O_1O_0 = 10$.

Table 10.1: Truth table of the standard 4-to-2 encoder.

I_3	I_2	I_1	I_0	O_1	O_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

In conventional CMOS, such an encoder can be implemented with AND, OR and NOT gates, or equivalently using universal NAND or NOR gates.

10.3.2 4-to-2 Priority Encoder

In many applications the assumption of a single active input is too restrictive. A 4-to-2 *priority encoder* allows multiple inputs to be high at the same time and assigns a fixed priority to each input line:

$$I_3 \succ I_2 \succ I_1 \succ I_0.$$

If several inputs are active simultaneously, the encoder outputs the index of the highest-priority active input. In addition, a *validity signal* V indicates whether at least one input is active. In our QCA implementations we realise all three outputs: O_1, O_0 and V.

The behaviour is summarised in Table 10.2. The symbol "X" denotes a *don't care* value that does not influence the outputs in that row.

A compact Boolean description of the three outputs is:

Table 10.2: Truth table of the 4-to-2 priority encoder [140].

I_3	I_2	I_1	I_0	O_1	O_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

$$V = I_0 \vee I_1 \vee I_2 \vee I_3, \tag{10.1}$$

$$O_1 = I_2 \vee I_3, \tag{10.2}$$

$$O_0 = (\overline{I_2} \wedge I_1) \vee I_3. \tag{10.3}$$

These equations are used as the starting point for the QCA layouts in Sect. 10.5, both for the majority-based encoder and for the reversible Toffoli-based encoder.

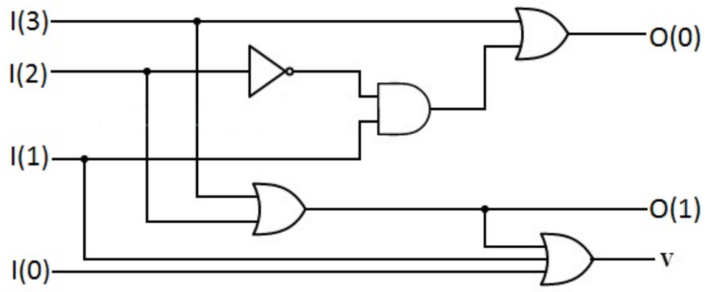


Fig. 10.5: Gate-level realisation of a 4-to-2 priority encoder (adapted from [140]).

10.4 Methods

The comparison of our two 4-to-2 QCA encoders was carried out in two steps. First, we reviewed existing designs of priority encoders and reversible QCA structures in the literature and selected several recent articles that report promising results [141, 139]. Second, we implemented our own QCA layouts in QCADesigner, verified their functional behaviour and evaluated them according to a common set of metrics.

10.4.1 QCADesigner

QCADesigner is a widely used tool for the design and simulation of quantum-dot cellular automata circuits [142]. It allows users to place cells on a canvas, assign clock zones and run time-domain simulations. Two simulation engines are available:

- the *Coherence Vector* engine, suitable for analysing thermal effects and energy dissipation in small circuits;
- the *Bistable Approximation* engine, which is faster and commonly used for functional verification of larger layouts.

In this work we used the Bistable Approximation engine with default parameters for all simulations. For each encoder implementation, we simulated all relevant input combinations and inspected the resulting waveforms to confirm that the outputs matched the truth table in Table 10.2.

10.4.2 Metrics

To compare the practicality of different QCA layouts we use the following standard metrics:

- **Cell count**: total number of QCA cells in the layout. Fewer cells generally imply lower complexity and lower energy consumption.
- **Circuit area** (in μm^2): area occupied on the QCADesigner canvas. Smaller area is beneficial for dense integration.
- **Number of layers**: some QCA designs use multiple physical layers to implement wire crossovers. Since multilayer fabrication is still challenging, single-layer designs are preferred.
- **Number of clock zones**: number of distinct clock phases on the critical path from inputs to outputs. This is a good approximation of the propagation delay; fewer clock zones correspond to lower latency.

These metrics will be reported for both encoder implementations in Sect. 10.5.

10.5 Implementation of 4-to-2 Encoders Using QCA Gates

Based on the Boolean equations in Sect. 10.3.2, we designed two QCA versions of the 4-to-2 priority encoder:

- a compact, single-layer encoder using only majority gates and inverters;
- a reversible encoder constructed from QCA Toffoli gates.

Both implementations use the same external interface: the four inputs $I_0, \ldots, I_3$, the two encoded outputs O_1, O_0 and the validity signal V.

10.5.1 Example: 4-to-2 Encoder Using Majority Gates (Single Layer)

The first implementation exploits the strengths of QCA majority logic. The outputs O_1 and O_0 and the validity signal V are synthesized using only 3-input majority gates and inverters, following the expressions given in Sect. 10.3.2. Fixed inputs of 0 and 1 are realised by cells with constant polarization.

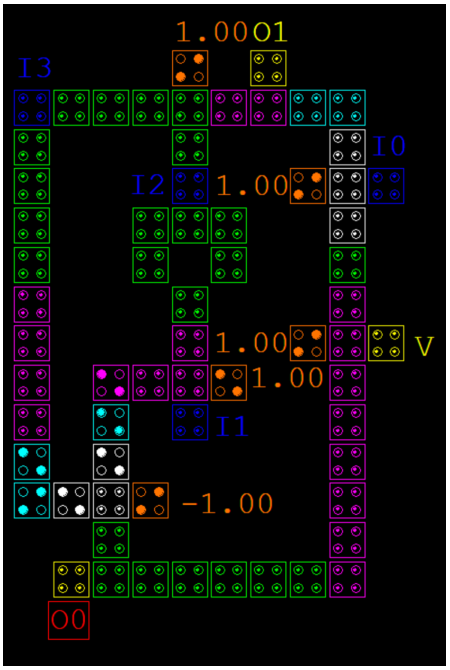


Fig. 10.6: Example 4-to-2 encoder using majority gates (single layer).

The resulting layout, shown in Fig. 10.6, is highly compact. It uses 64 cells and occupies approximately $0.07\,\mu\text{m}^2$. All cells are placed in a single physical layer and simple coplanar crossovers are sufficient for signal routing. The outputs O_1, O_0 and V are produced after roughly 1.25 clock zones along the critical path, leading to a very small propagation delay.

The main advantages of this design are:

- low cell count and small circuit area;
- single-layer implementation, which simplifies fabrication;
- very low latency due to the small number of clock zones.

The disadvantage is that the encoder is not reversible: different input patterns can map to the same output, and information is lost during computation.

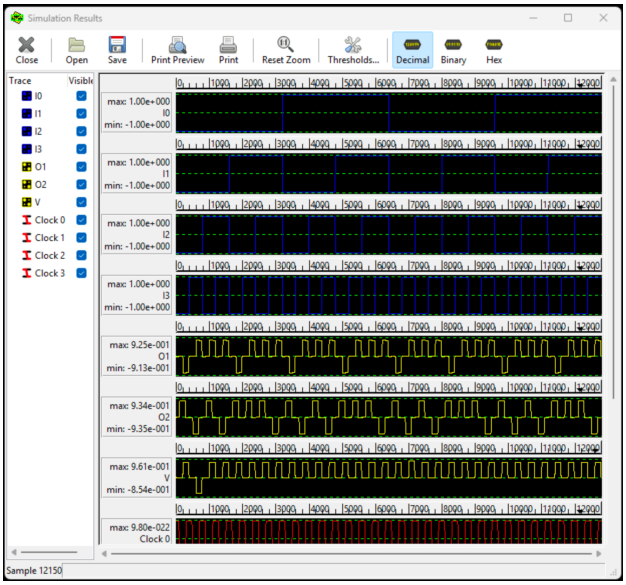


Fig. 10.7: Simulation results of the majority-gate encoder.

10.5.2 Example: Reversible 4-to-2 Priority Encoder with Toffoli Gates

The second implementation aims to realise a *reversible* 4-to-2 priority encoder in QCA. Reversible logic is interesting because, in principle, it can be operated with arbitrarily low energy dissipation if switching is sufficiently slow [139]. To achieve reversibility, the priority encoding function is embedded into a larger bijective mapping by adding ancilla inputs and garbage outputs, and then implemented using Toffoli gates.

Figure 10.8 shows the logical scheme of the reversible priority encoder constructed from Toffoli gates. The main outputs correspond to O_1 and O_0, while additional lines carry garbage outputs needed to make the mapping reversible.

In QCA we realised each Toffoli gate using a compact layout built from majority gates, inverters and XOR structures. The resulting encoder layout and its simulation results are shown in Figs. 10.9 and 10.10.

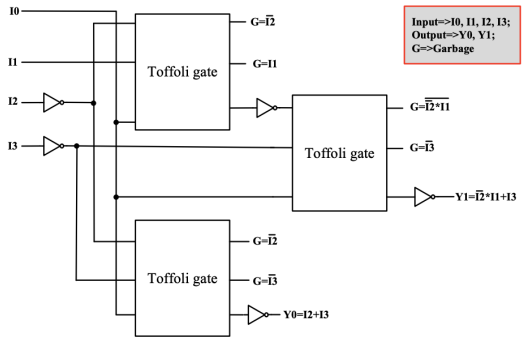


Fig. 10.8: Schematic of the reversible 4-to-2 priority encoder using Toffoli gates.

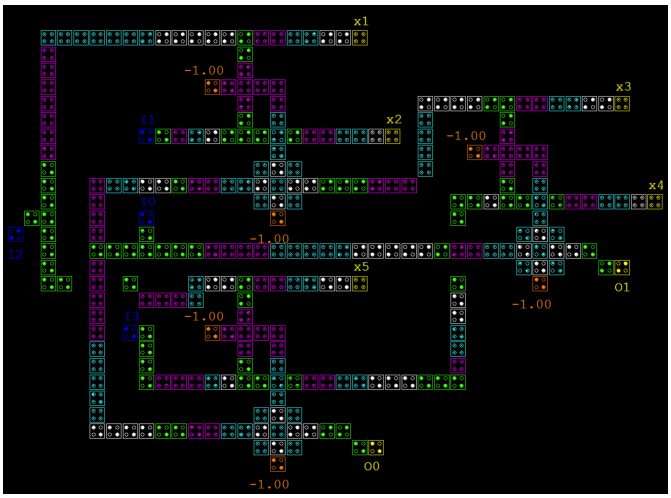


Fig. 10.9: Example and simulation reversible 4-to-2 priority encoder with Toffoli gates.

This implementation is significantly more complex than the majority-based encoder. It contains around 288 cells and occupies approximately $0.47\,\mu\text{m}^2$. Several wire crossovers and routing paths are needed to interconnect the Toffoli blocks. Signal propagation from inputs to outputs covers roughly 4.25 clock zones, resulting in a noticeably higher latency.

The main characteristics of the reversible encoder can be summarised as:

- logically reversible mapping between inputs and outputs, suitable for investigating low-dissipation computation;
- larger cell count and area due to the use of multiple Toffoli gates;

- longer propagation delay caused by a greater number of clock zones;
- more complicated routing, including several crossovers.

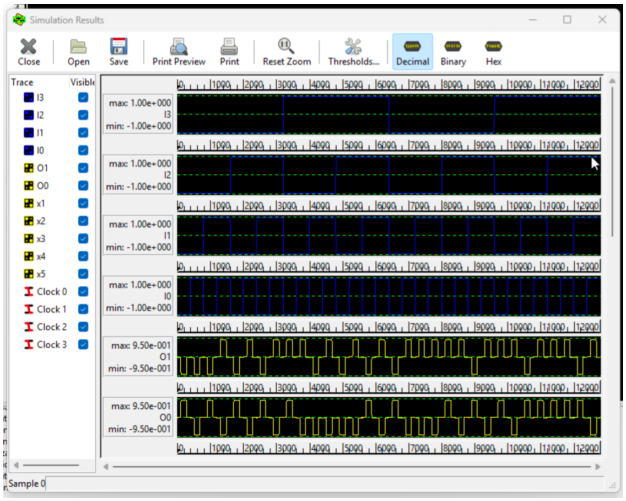


Fig. 10.10: Simulation results of the reversible encoder.

10.6 Conclusion

We have designed and analysed two QCA implementations of a 4-to-2 priority encoder: a compact single-layer encoder using majority gates and a reversible encoder built from QCA Toffoli gates. Both implementations were simulated in QCADesigner and evaluated with respect to cell count, circuit area, number of layers and clock zones.

The majority-gate encoder clearly outperforms the reversible design in terms of practicality. It uses far fewer cells, occupies a smaller area, is realised in a single layer and has a much shorter propagation delay. This makes it better suited for integration into larger QCA-based systems where density and speed are important.

The reversible encoder with Toffoli gates is conceptually appealing because it preserves information and is compatible with theoretical work on energy-efficient and reversible computing [139]. However, this comes at a substantial cost in terms of layout complexity, area and latency. For real-world applications that prioritise minimal area and delay, the majority-gate approach is therefore recommended. The Toffoli-based design is primarily useful as a case study for reversible QCA circuits and as a starting point for future optimisation.

Author Contributions

- **Stefan Krstevski**: Prepared the theoretical background and literature overview, structured and wrote the seminar in LaTeX, including QCA fundamentals, encoder theory.
- **Žan Seničar**: Implemented both QCA encoder versions (majority-gate and reversible Toffoli-based), performed simulations in QCADesigner, and prepared all schematic layouts and simulation figures.

Poglavje 11
Rekonstrukcija 3-vhodnih AND/OR QCA struktur in simulacija v QCADesigner-E

Nik Bažato

Povzetek V tem poročilu analiziramo in rekonstruiramo predlagane 3-vhodne AND/OR strukture, predstavljene v članku A. Almatrood (2025)[143], ter izvedemo simulacije z orodjem QCADesigner-E. Glavni cilj poročila je preveriti delovanje teh struktur in jih primerjati s tipičnimi rešitvami, zasnovanimi s kaskadiranjem dveh osnovnih 2-vhodnih vrat. Primerjava vključuje analizo energijske disipacije, površine vezja in zakasnitve, s čimer ocenimo prednosti in slabosti obeh pristopov pri implementaciji več-vhodnih logičnih funkcij v QCA tehnologiji.

11.1 Uvod

Quantum-dot Cellular Automata (QCA) predstavlja eno izmed obetavnih post-CMOS nanotehnologij, saj omogoča izredno visoko gostoto integracije, nizko porabo energije ter visoke hitrosti delovanja. Namesto uporabe tokovnih preklopov, kot pri CMOS tranzistorjih, QCA temelji na interakciji med naboji znotraj kvantnih pik in tako omogoča logične operacije zgolj na osnovi razporeditve elektronov.

Osnovna gradnika QCA tehnologije sta inverter in večinski člen (ang. majority gate), ki omogočata konstrukcijo kompleksnih digitalnih vezij. Kljub temu standardni večinski člen naravno podpira zgolj dve vhodni logični operaciji AND/OR, s fiksiranjem ene od vhodnih celic. V praksi to pomeni, da je za večje število vhodov treba uporabljati kaskadno povezane večinske člene, kar povečuje zakasnitev, površino in porabo energije.

V članku, ki ga obravnavamo v tem poročilu, je predstavljen nov energijsko učinkovitejši pristop k implementaciji 3-vhodnih AND/OR funkcij, ki ne temelji na večinskih členih, temveč na posebni geometrijski razporeditvi QCA celic. V tem projektu ponovno ustvarimo te strukture, jih simuliramo v QCADesigner-E ter primerjamo z tipičnimi rešitvami, zasnovanimi s kaskadiranjem dveh osnovnih 2-vhodnih vrat.

11.2 Teoretično ozadje

V tem poglavju podamo ključne koncepte, potrebne za razumevanje delovanja QCA tehnologije ter razlogov, zakaj so večvhodne strukture pomembne za zmanjšanje porabe energije in kompleksnosti vezij. Koncepti temeljijo predvsem na delih Lent et al. [144, 145, 146] ter na sodobnih pregledih QCA sinteze [147, 148].

11.2.1 Osnovni koncepti QCA

QCA logična celica je sestavljena iz štirih kvantnih pik, med katerimi sta dva elektrona, ki se zaradi Coulombovega odboja postavita v stabilni diagonalni konfiguraciji. Dve možni konfiguraciji predstavljata logični stanji $+1$ in -1. Celice vplivajo druga na drugo preko elektrostatike, kar omogoča prenos logične informacije brez toka.

11.2.2 Majority gate

Večinski člen (majority gate) je osnovni logični element, ki sprejme tri vhode in vrne logično vrednost, ki se pojavi v večini vhodov. Formalno:

$$M(a,b,c) = ab + bc + ac. \tag{11.1}$$

Fiksiranje enega vhoda omogoča izvedbo AND/OR:

- $M(a,b,-1)$ daje $a \cdot b$
- $M(a,b,+1)$ daje $a + b$

S tem lahko gradimo logična vezja, vendar ob več vhodih hitro narašča število potrebnih členov.

11.2.3 Omejitve večinskih struktur

Ker lahko majority gate neposredno obravnava le dve spremenljivki (tretji je konstanta), se za implementacijo n-vhodnih AND/OR funkcij uporabljajo kaskadne strukture. To poveča:

- število celic
- zakasnitev
- energijsko disipacijo

Alternativni pristop, predstavljen v referenčnem članku, omogoča neposredno 3-vhodno operacijo z 13 celicami.

11.3 Metodologija

V tem poglavju opišemo celoten proces rekonstrukcije QCA struktur, od analiziranja članka [143] do ponovne implementacije in simulacije z uporabo QCADesigner-E. Poseben poudarek namenimo uporabi enakih simulacijskih parametrov, da zagotovimo primerljivost rezultatov.

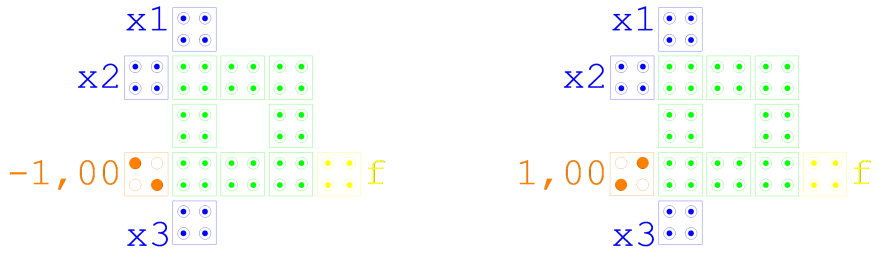
11.3.1 Uporabljena programska oprema

Za rekonstrukcijo in simulacijo QCA struktur uporabimo orodje **QCADesigner-E** (različica s podporo za energijsko simulacijo). Program omogoča simulacije z uporabo t. i. Coherence Vector pogona, ki izračuna energijsko disipacijo in stabilnost QCA struktur.

11.3.2 Postopek rekonstrukcije

Rekonstrukcija temelji na geometrijskih postavitvah celic iz članka. Za vsako strukturo izvedemo naslednje korake:

1. Ustvarimo novo QCA vezje v QCADesigner-E.
2. Ročno postavimo 8 celic v kvadratno konfiguracijo, tako kot je prikazano na spodnjih shemah 11.1.
3. Dodamo tri vhodne celice (x_1, x_2, x_3), konstantno celico (nastavljeno na $+1$ ali -1) in izhodno celico.
4. Izvedemo simulacijo za vseh osem možnih kombinacij vhodov.



(a) AND vrata (b) OR vrata

Slika 11.1: Shema AND/OR vrat

11.3.3 Simulacijske nastavitve

Simulacije so bile izvedene z enakimi parametri, kot jih navaja članek, vključno s:

- temperature: 1 K
- relaxation time: 4×10^{-15} s
- clock period: 4×10^{-12} s
- input period: 4×10^{-12} s
- time step : 4×10^{-16} s
- tip clock signala: Gauss
- Coherence Vector Engine (w/ energy)

Ti parametri omogočajo izračun disipacije energije.

11.4 Rezultati simulacij

V tem poglavju predstavimo rezultate simulacij za predlagani 3-vhodni AND/OR strukturi ter za referenčni pristop, ki temelji na kaskadiranju dvovhodnih majority vrat. Za vsako implementacijo so bile izvedene simulacije vseh osmih vhodnih kombinacij, rezultati pa vključujejo izračun energijske disipacije in preverjanje pravilnega logičnega delovanja na izhodu.

11.4.1 Pravilnost logičnega delovanja

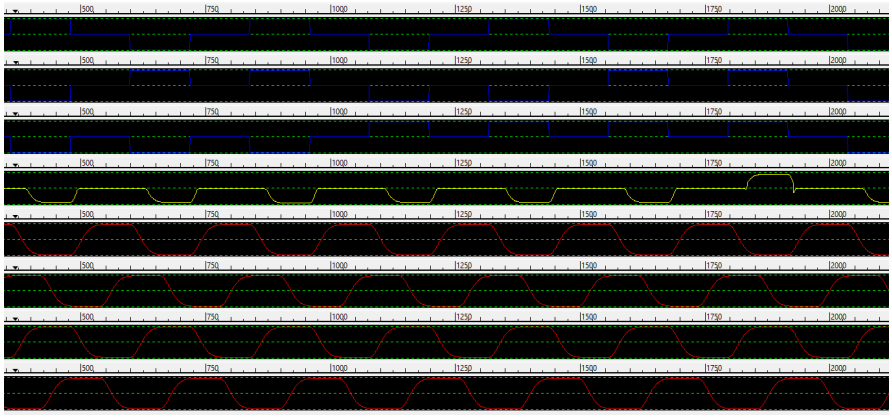
Za vse štiri strukture (predlagani AND, predlagani OR, kaskadiran AND, kaskadiran OR) smo preverili funkcionalnost z opazovanjem valovnih oblik izhodnega signala. Pri vseh vhodnih kombinacijah so izhodne valovne oblike skladne s pričakovanim delovanjem:

- predlagani AND vrača 1 izključno za vhod (111),
- predlagani OR vrača 1 za katerokoli kombinacijo, ki vsebuje vsaj en vhod 1,
- kaskadirani strukturi pravilno implementirata enakovredni funkciji, vendar z večjim številom celic in daljšo časovno potjo.

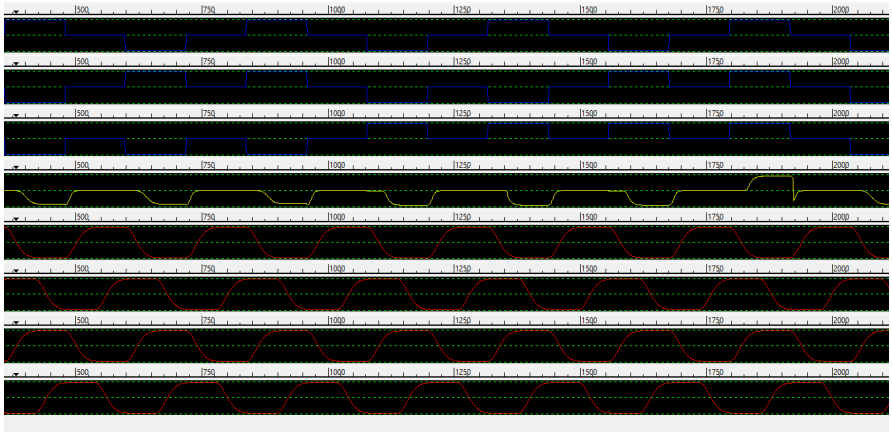
Na sliki 11.2 in 11.3 prikazujemo primera valovne oblike, s katero je bilo mogoče potrditi pravilno delovanje.

11.4.2 Energijska disipacija: predlagana vrata

Za predlagana 3-vhodna vrata smo dobili naslednje povprečne disipacije:



Slika 11.2: Primer valovne oblike predlagane strukture, ki prikazuje pravilno logično delovanje.



Slika 11.3: Primer valovne oblike strukture kaskadiranih majoriti vrat, ki prikazuje pravilno logično delovanje.

- povprečna disipacija predlaganega AND:

$$\bar{E}_{AND}^{prop} = 1.72625 \times 10^{-3} \text{ eV}$$

- povprečna disipacija predlaganega OR:

$$\bar{E}_{OR}^{prop} = 1.72625 \times 10^{-3} \text{ eV}$$

Rezultati so pričakovano simetrični, saj se strukturi med AND in OR razlikujeta le v logični konstanti, postavitev celic pa je enaka.

Tabela 11.1 prikazuje disipacijo energije za vse vhodne kombinacije pri predlaganih 3-vhodnih AND in OR vratih. Rezultati potrjujejo simetrično obnašanje obeh struktur, saj sta geometrija in število celic enaka, razlikuje se le izbrana logična konstanta. Pričakovano je disipacija višja pri vhodnih kombinacijah, kjer pride do spremembe izhodnega stanja ali večjega števila notranjih preklopov, medtem ko kombinacije, ki ohranjajo stabilno konfiguracijo celic, povzročijo nižjo porabo energije. Povprečna disipacija, izračunana iz podatkov v tabeli, znaša približno 1.73×10^{-3} eV za obe funkciji.

Vezje	000	001	010	011	100	101	110	111
Predlagan AND	1.35 meV	1.47 meV	1.66 meV	2.49 meV	1.46 meV	2.17 meV	1.02 meV	2.19 meV
Predlagan OR	2.19 meV	1.02 meV	2.17 meV	1.46 meV	2.49 meV	1.66 meV	1.47 meV	1.35 meV

Tabela 11.1: Tabela disipacije energije glede na vhode x_1, x_2 in x_3 za predlagan AND/OR

11.4.3 Energijska disipacija: kaskadirana majority vrata

Za primerjavo smo implementirali klasični pristop, pri katerem je 3-vhodna funkcija sestavljena iz dveh dvovhodnih majority vrat. Povprečni rezultati so:

- povprečna disipacija kaskadiranega AND:

$$\bar{E}_{AND}^{maj} = 5.14625 \times 10^{-4} \text{ eV}$$

- povprečna disipacija kaskadiranega OR:

$$\bar{E}_{OR}^{maj} = 5.14625 \times 10^{-4} \text{ eV}$$

Tabela 11.2 prikazuje energijsko disipacijo za vse vhodne kombinacije pri kaskadiranih dvovhodnih majority AND in OR vratih. Rezultati kažejo, da so disipacije bistveno nižje kot pri predlaganih 3-vhodnih strukturah, kar je posledica manj kompleksne topologije in manjšega števila notranjih preklopov. Vidimo lahko tudi, da se AND in OR strukturi obnašata simetrično, saj sta zasnovani z enakim številom celic in identično postavitvijo, razlikujeta pa se le v vrednosti fiksiranega vhoda. Povprečna disipacija znaša približno 5.15×10^{-4} eV, kar potrjuje energijsko prednost kaskadiranega pristopa, vendar na račun večje prostorske zahtevnosti in daljše časovne poti.

Vezje	000	001	010	011	100	101	110	111
kaskadiran AND	0.434 meV	0.562 meV	0.455 meV	0.58 meV	0.439 meV	0.797 meV	0.27 meV	0.58 meV
kaskadiran OR	0.58 meV	0.27 meV	0.797 meV	0.439 meV	0.58 meV	0.455 meV	0.562 meV	0.434 meV

Tabela 11.2: Tabela disipacije energije glede na vhode x_1, x_2 in x_3 za kaskadirana majority AND/OR vrata

11.4.4 Primerjava energijske učinkovitosti

Primerjava povprečnih disipacij jasno pokaže razliko med pristopoma:

$$\frac{\bar{E}_{prop}}{\bar{E}_{maj}} \approx \frac{1.72625 \times 10^{-3}}{5.14625 \times 10^{-4}} \approx 3.35$$

Kaskadirana dvovhodna vrata so torej približno trikrat energijsko učinkovitejša kot predlagana 3-vhodna vrata. Razlog je predvsem v tem, da predlagani pristop uporablja zgolj eno strukturo z več notranjimi preklopi, medtem ko je pri kaskadiranih vratih logična pot razdeljena na dva manj kompleksna koraka.

Kljub temu kaskadirana večinska vrata:

- imajo težje dosegljive vhode,
- zahtevajo dva fiksna vhoda (če želimo čimbolj kompaktno rešitev),
- izhodna celica kaže večjo oscilacijo polarizacije (signal ringing) pri preklopih, zlasti pri prehodu iz logične vrednosti 1 v 0.

11.4.5 Neujemanje rezultatov z referenčnim člankom

Pri primerjavi dobljenih rezultatov s podatki, predstavljenimi v izvirnem članku [143], smo opazili pomembna odstopanja v energijski disipaciji. Avtor navaja, da naj bi bila predlagana 3-vhodna struktura približno dvakrat energijsko učinkovitejša od klasične kaskadirane izvedbe, vendar naši rezultati kažejo nasprotno: izmerjena povprečna disipacija predlaganih vrat znaša približno $1.73 \times 10^{-3}\,\mathrm{eV}$, kar je okoli trikrat več od disipacije kaskadiranih majority vrat, ki znaša $5.15 \times 10^{-4}\,\mathrm{eV}$.

Prostorske zahteve se ujemajo z navedbami v članku: predlagana struktura uporablja 13 celic, medtem ko kaskadirana dvovhodna majority vrata zahtevajo 9 celic. Funkcionalno se vse strukture obnašajo pravilno, razlika se tako pojavlja izključno v energijski učinkovitosti.

Ker članek priloži tako sliko strukture kot tudi vse ključne simulacijske parametre, razlogov za odstopanje ne moremo pripisati manjkajočim podatkom. Namesto tega so možni tehnološki in numerični vzroki, vezani na implementacijo simulatorja:

- **različna interpretacija clock conov**: čeprav je geometrija pravilno povzeta, lahko majhne razlike v tem, kako simulator določi mejo med clock coni, vplivajo na število preklopnih dogodkov in posledično na izmerjeno energijo;

- **manjkajoče informacije o normalizaciji disipacije v članku**: avtor članka predstavi zgolj končne vrednosti disipacije, ne poda pa informacij o tem, ali je bila energija normalizirana na posamezen clock cikel ali preračunana iz povprečja več iteracij.

Kljub razlikam dobljeni rezultati potrjujejo pravilno logično delovanje vseh struktur. Na odstopanje energijskih vrednosti pa lahko vpliva več dejavnikov, povezanih z numerično implementacijo in robnimi pogoji simulatorja, ki jih v okviru te rekonstrukcije ni bilo mogoče neposredno preveriti ali kontrolirati.

11.5 Zaključek

V poročilu smo rekonstruirali predlagani 3-vhodni QCA AND/OR strukturi ter ju primerjali s klasičnim pristopom, ki temelji na kaskadiranju dvovhodnih majority vrat. Vse implementirane strukture so delovale pravilno z vidika logične funkcionalnosti, kar potrjuje ustreznost zasnove in rekonstrukcije. Pri analiziranju energijske disipacije pa smo opazili pomembna odstopanja od rezultatov, predstavljenih v izvirnem članku.

Medtem ko avtor navaja, da naj bi bila predlagana struktura energijsko učinkovitejša, so naši rezultati pokazali, da je njena povprečna disipacija približno trikrat večja od disipacije kaskadiranih majority vrat. Po drugi strani predlagana struktura ohranja prednosti na področju kompaktne postavitve in krajših signalnih poti, vendar teh prednosti ne spremlja nižja energijska poraba.

Neujemanje rezultatov pripisujemo predvsem interpretaciji clock con, pri čemer logična pravilnost vseh struktur ostaja nespremenjena. Kljub odstopanjem analiza ponuja verodostojen vpogled v praktično obnašanje QCA struktur in poudarja, da energijska učinkovitost močno zavisi od točne postavitve celic in simulacijskih parametrov.

Delo tako prispeva k boljšemu razumevanju kompromisov med prostorsko, energijsko in časovno učinkovitostjo QCA vezij ter potrjuje, da so natančne rekonstrukcije in dosledne simulacije ključnega pomena pri ocenjevanju novih arhitektur v QCA tehnologiji.

Poglavje 12
Implementacija kvantnih celičnih avtomatov z uporabo QCA Designer in QuTiP

Aljaž Kern, Nejc Drašler

Povzetek V seminarski nalogi sta primerjana pristopa za modeliranje kvantnih celičnih avtomatov (QCA): QCA Designer, ki omogoča enostavno grafično načrtovanje in simulacijo klasičnih QCA vezij, ter QuTiP, ki kot kvantna Python knjižnica omogoča natančno matematično modeliranje kvantnih stanj in operacij. Implementirane so bile osnovne QCA strukture (negator, majoritetna, CNOT in CCNOT vrata) ter dve kompleksnejši vezji. Obe orodji sta pravilno izvedli vse simulacije, pri čemer je QCA Designer primernejši za pregledno vizualizacijo klasičnih QCA, QuTiP pa za splošne kvantne simulacije.

12.1 Uvod

Pri razvoju kvantnih tehnologij postaja simuliranje kvantnih sistemov ključno orodje za razumevanje njihovega obnašanja, optimizacijo arhitektur ter načrtovanje eksperimentalnih postavitev. Kvantni celični avtomati (QCA) predstavljajo enega od pristopov k realizaciji računalniških elementov na kvantni ravni, zato je za raziskovalce pomembno, da imajo na voljo ustrezna programska orodja za njihovo modeliranje.

V tem poročilu primerjamo dve pogosto uporabljeni orodji: QCA Designer, specializiran program za modeliranje QCA struktur, ter QuTiP, odprtokodno Python knjižnico, namenjeno splošnemu kvantnemu modeliranju in simulacijam kvantnih sistemov. Medtem ko je QCA Designer zasnovan predvsem za klasično-obnašajoče se QCA naprave in omogoča intuitivno grafično načrtovanje, QuTiP ponuja široko paleto numeričnih metod za reševanje kvantnih enačb, kar omogoča bolj splošne in natančne kvantne simulacije.

Cilj poročila je analizirati prednosti in omejitve obeh pristopov ter oceniti, v katerih primerih je posamezno orodje primernejše. Poseben poudarek bo na uporabniški izkušnji, naboru funkcionalnosti, natančnosti simulacij in fleksibilnosti pri modeliranju kompleksnejših kvantnih sistemov.

12.2 QCA Designer in QuTiP

QCA Designer je specializirano programsko orodje za modeliranje in simulacijo kvantnih celičnih avtomatov (QCA) struktur. Razvit je bil predvsem za potrebe analize klasičnega obnašanja QCA celic. Uporabniku omogoča grafično načrtovanje QCA vezij, določanje vhodnih signalov, izračun polarnosti celic ter simulacijo propagacije informacij v vezju. Program podpira različne simulacijske načine, pri čemer so ti prilagojeni fizikalnim modelom značilnim za QCA arhitekture. Zaradi svoje preglednosti in enostavnosti uporabe je QCA Designer primeren predvsem za hitro prototipiranje, vizualizacijo vezij in analizo stabilnosti logičnih elementov v QCA tehnologiji.

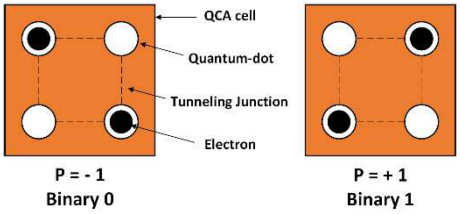
QuTiP (Quantum Toolbox in Python) je odprtokodna Python knjižnica, namenjena numeričnim simulacijam splošnih kvantnih sistemov. Omogoča delo z Hilbertovimi prostori, Hamiltoniani, kvantnimi operatorji, gostotnimi matrikami ter reševanje različnih kvantnih dinamičnih enačb. QuTiP podpira napredne funkcionalnosti, kot so kvantna odprta dinamika, kvantno upravljanje, kvantna optika, analiza koherentnosti in simulacije časovno odvisnih Hamiltonianov. Zaradi visoke prilagodljivosti je primeren za natančne simulacije, ki presegajo zmožnosti standardnih QCA orodij.

12.3 Pregled QCA struktur

V tem poglavju bomo predstavili implementacijo kvantnih celic v orodjih QCA Designer in QuTiP ter prikazali primere izvedbe izbranih struktur v obeh okoljih.

12.3.1 QCA celica v QCA Designer

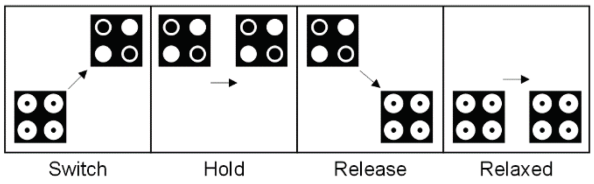
Osnovno enoto vsakega QCA predstavlja kvantna ali QCA celica. QCA celico lahko predstavimo kot dvodimenzionalno strukturo kvadratne oblike, pri čemer se v vsakem ogljišču nahaja kvantna pika. Kvantne pike so med seboj povezane preko štirih tunelov. V celici sta ujeta dva elektrona, ki se nahajata vsak v svoji kvantni piki. Elektrona se lahko znotraj kvantne celice premikata preko tunelov. Zaradi medsebojnega odboja se elektrona v kvantni celici lahko nahajata v enem izmed dveh stabilnih stanj, kar nam predstavlja logično 0 ali 1. Položaj elektronov v eni QCA celici vpliva tudi na sosednje celice. Zaradi njihovega električnega polja, se bodo elektroni v sosednjih celicah prestavili v položaj, ki je energijsko najugodnejši. Tako lahko poskrbimo za prenos informacij med QCA celicami, kar omogoča izvajanje logičnih operacij. Slika 12.1 prikazuje QCA celici v logičnih stanjih 0 in 1, kot bi se prikazali v programu QCA Designer.



Slika 12.1: Struktura QCA celice in prikaz osnovnih stanj. Pridobljeno iz [149].

Pri delovanju kvantnega celičnega avtomata lahko pride do konflikta, če dve QCA celici z različnima polarizacijama hkrati vplivata na isto notranjo celico. To težavo odpravimo z vpeljavo ure, ki poskrbi za pravilno propagacijo informacij po QCA vezju. Delovanje ure je prikazano na sliki 12.2 in je sestavljeno iz štirih faz:

- *Switch* - zapiranje pregrad med tuneli, celice iz nepolariziranih prehajajo v polarizirana stanja.
- *Hold* - pregrade med tuneli zaprte, celice so v polariziranem stanju.
- *Release* - dvigovanje pregrad med tuneli, celice iz polariziranih prehahjajo v nepolarizirana stanja.
- *Relax* - pregrade med tuneli dvignjene, celice v nepolariziranem stanju.



Slika 12.2: Prikaz delovanja ure v fazah Switch, Hold, Release in Relax. Pridobljeno iz [149].

12.3.2 QCA celica v QuTip

V QuTip lahko QCA celico predstavimo kot kvantni bit (qubit) z dvema osnovnima stanjema $|0\rangle$ in $|1\rangle$, ki ustrezata dvema možnima položajema elektronov v celici. Poljubno stanje celice $|\psi\rangle$ lahko zapišemo kot linearno kombinacijo osnovnih stanj

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{bmatrix}\alpha\\ \beta\end{bmatrix}, \quad |0\rangle = \begin{bmatrix}1\\ 0\end{bmatrix}, \quad |1\rangle = \begin{bmatrix}0\\ 1\end{bmatrix}$$

pri čemer α in β, ki sta kompleksni števili, predstavljata amplitudi, ki določata verjetnost, da bo celica ob meritvi v stanju $|0\rangle$ ali $|1\rangle$. Da zagotovimo veljavno kvantno stanje, mora veljati $|\alpha|^2 + |\beta|^2 = 1$.

Logične operacije nad qubiti v QuTip izvajamo tako, da unitarno matriko operacije pomnožimo z vektorjem kvantnega stanja sistema, ki predstavlja tenzorski produkt vhodnih qubitov. Po izvedeni operaciji, da bi dobili posamezne izhodne qubite, najprej izračunamo amplitude za stanja $|0\rangle$ in $|1\rangle$ teh qubitov, nato pa jih ustrezno normaliziramo, da zagotovimo veljavno kvantno stanje.

12.3.3 Implementacija osnovnih QCA struktur

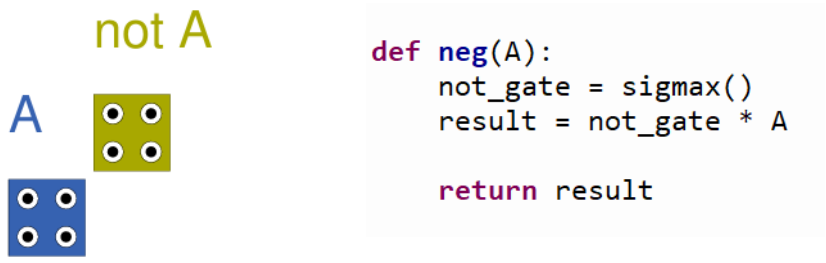
Poleg posamezne kvantne celice si oglejmo še implementacijo preostalih osnovnih QCA struktur, ki jih lahko uporabimo za gradnjo kompleksnejših vezij, kot so negator, majoritetna vrata, CNOT vrata in CCNOT vrata.

12.3.3.1 Negator

Negator predstavlja logično strukturo, ki sprejme en vhod in vrne njegovo nasprotno oziroma negirano vrednost. Pravilnostna tabela za negator je prikazana na tabeli 12.1, slika 12.3 pa prikazuje implementacijo negatorja v QCA Designer in v Pythonu z uporabo knjižnice QuTip.

A	not A
0	1
1	0

Tabela 12.1: Pravilnostna tabela za negator.



Slika 12.3: Primerjava implementacije negatorja v QCA Designerju (levo) in v QuTip (desno).

12.3.3.2 Majoritetna vrata

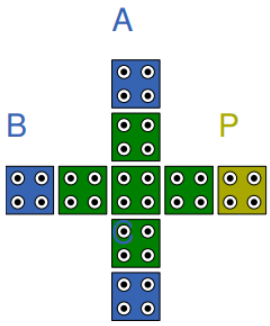
Majoritetna vrata predstavljajo eno izmed osnovnih logičnih struktur za QCA, kjer je izhod podan kot večina treh vhodov. Majoritetna vrata omogočajo implementacijo logičnih funkcij, kot sta AND in OR, in sicer kot:

$$AND(A,B) = MAJ(A,B,0)$$
$$OR(A,B) = MAJ(A,B,1)$$

Pravilnostna tabela za majoritetna vrata je prikazana na tabeli 12.2, slika 12.4 pa prikazuje implementacijo majoritetnih vrat v QCA Designer in v Pythonu z uporabo knjižnice QuTip. Ker QuTip ne ponuja vgrajene implementacije majoritetnih vrat, smo jih izdelali sami. Najprej določimo, koliko vhodnih qubitov je v stanju $|1\rangle$. Če je večina qubitov v stanju $|1\rangle$, vrata vrnejo izhodno stanje $|1\rangle$; v nasprotnem primeru pa $|0\rangle$.

A	B	C	P
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Tabela 12.2: Pravilnostna tabela majoritetnih vrat.




```
def maj(A, B, C):
    a = int(np.real(A.full()[1, 0]))
    b = int(np.real(B.full()[1, 0]))
    c = int(np.real(C.full()[1, 0]))

    if (a + b + c) >= 2:
        output = 1
    else:
        output = 0

    P = basis(2, output)

    return P
```

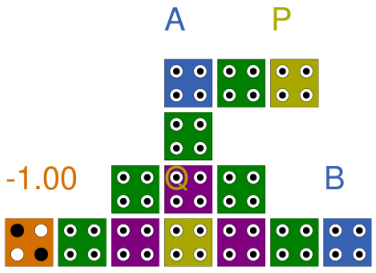
Slika 12.4: Primerjava implementacije majoritetnih vrat v QCA Designerju (levo) in v QuTip (desno).

12.3.3.3 CNOT vrata

CNOT vrata predstavljajo eno izmed pomembnejših kvantnih logičnih vrat. Vrata CNOT sprejmejo dva vhoda, pri čemer je eden izmed kontrolni vhod. V kolikor je kontrolni vhod enak 1, bodo CNOT vrata vrnila negirano vrednost drugega vhoda. Pravilnostna tabela za CNOT vrata je prikazana na tabeli 12.3, slika 12.5 pa prikazuje implementacijo majoritetnih vrat v QCA Designer in v Pythonu z uporabo knjižnice QuTip.

A	B	P (=A)	Q
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

Tabela 12.3: Pravilnostna tabela CNOT vrat.




```
from qutip.qip.operations import cnot as qutip_cnot

def cnot(A, B):
    state = tensor(A, B)
    print(state)
    cnot_gate = qutip_cnot(N=2, control=0, target=1)
    result = cnot_gate * state
    print(result)

    vec = result.full()
    c00, c01, c10, c11 = vec[:,0]

    p0 = c00 + c01
    p1 = c10 + c11
    norm_p = np.sqrt(abs(p0)**2 + abs(p1)**2)
    P = Qobj([[p0/norm_p],[p1/norm_p]])

    q0 = c00 + c10
    q1 = c01 + c11
    norm_q = np.sqrt(abs(q0)**2 + abs(q1)**2)
    Q = Qobj([[q0/norm_q],[q1/norm_q]])

    return P, Q
```

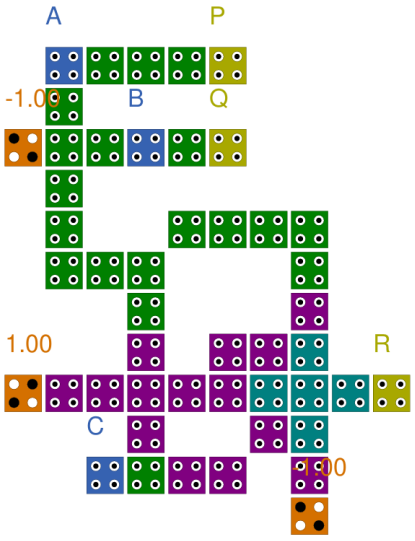
Slika 12.5: Primerjava implementacije CNOT vrat v QCA Designerju (levo) in v QuTip (desno).

12.3.3.4 CCNOT vrata

Vrata CCNOT prav tako predstavljajo eno izmed pomembnejših kvantnih logičnih vrat. Za razliko od CNOT vrat, vrata CCNOT sprejmejo 3 vhode, od katerih sta dva kontrolna. V kolikor sta oba kontrolna vhoda enaka 1, bodo CCNOT vrata vrnila negirano vrednost tretjega vhoda. Pravilnostna tabela za CCNOT vrata je prikazana na tabeli 12.4, slika 12.6 pa prikazuje implementacijo majoritetnih vrat v QCA Designer in v Pythonu z uporabo knjižnice QuTip.

A	B	C	P (=A)	Q (=B)	R
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

Tabela 12.4: Pravilnostna tabela CCNOT vrat.




```python
from qutip.qip.operations import toffoli as qutip_ccnot

def ccnot(A, B, C):
    state = tensor(A, B, C)
    ccnot_gate = qutip_ccnot(N=3, controls=[0, 1], target=2)
    result = ccnot_gate * state

    vec = result.full()
    c000, c001, c010, c011, c100, c101, c110, c111 = vec[:,0]

    p0 = c000 + c001 + c010 + c011
    p1 = c100 + c101 + c110 + c111
    norm_p = np.sqrt(abs(p0)**2 + abs(p1)**2)
    P = Qobj([[p0/norm_p], [p1/norm_p]])

    q0 = c000 + c001 + c100 + c101
    q1 = c010 + c011 + c110 + c111
    norm_q = np.sqrt(abs(q0)**2 + abs(q1)**2)
    Q = Qobj([[q0/norm_q], [q1/norm_q]])

    r0 = c000 + c010 + c100 + c110
    r1 = c001 + c011 + c101 + c111
    norm_r = np.sqrt(abs(r0)**2 + abs(r1)**2)
    R = Qobj([[r0/norm_r], [r1/norm_r]])

    return P, Q, R
```

Slika 12.6: Primerjava implementacije CCNOT vrat v QCA Designerju (levo) in v QuTip (desno).

12.4 Implementacije QCA vezij

V okviru izdelave seminarske naloge smo implementirali tudi dve QCA vezji. QCA vezji smo implementirali tako v QCA Designer in v Pythonu z uporabo knjižnice QuTip.

12.4.1 Prvo QCA vezje

Prvo implementirano QCA vezje predstavlja logično funkcijo, ki sprejme tri vhode (A, B in C) ter vrne izhoda P in Q, katera sta pridobljena po naslednjem postopku:

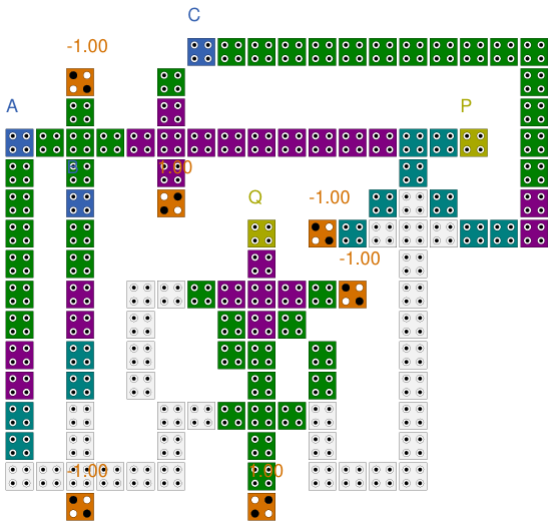
$$P = \mathrm{OR}(\mathrm{AND}(A,B),\mathrm{NOT}(C))$$
$$Q = \mathrm{CCNOT}(A,B,\mathrm{CNOT}(P,C))$$

Pravilnost implementacije smo preverili z vsemi možnimi kombinacijami vhodov, kjer pravilnostna tabela 12.5 prikazuje pričakovane rezultate.

A	**B**	**C**	**P**	**Q**
0	0	0	1	1
0	0	1	0	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

Tabela 12.5: Pravilnostna tabela prvega QCA vezja.

12.4.1.1 Implementacija



Slika 12.7: Implementacija prvega QCA vezja z uporabo QCA Designer.

```
def firstQCA(A, B, C):
    AND = maj(A, B, basis(2, 0))
    NOT_C = neg(C)
    OR = maj(AND, NOT_C, basis(2, 1))

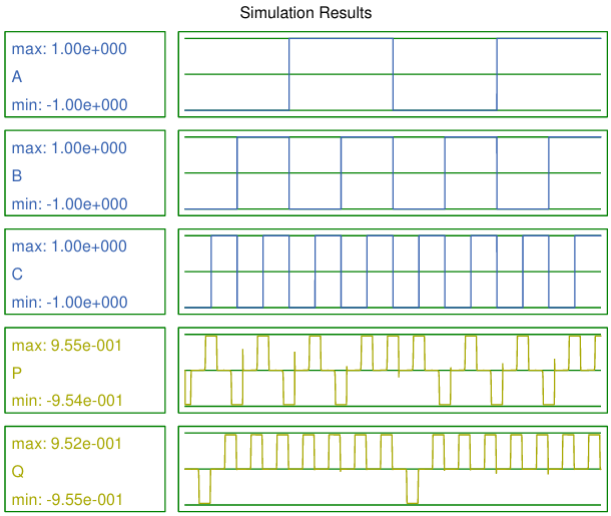
    P, Y = cnot(OR, C)
    A, B, Q = ccnot(A, B, Y)

    return P, Q
```

Slika 12.8: Implementacija prvega QCA vezja z uporabo QuTip.

12.4.1.2 Rezultati

Rezultati simulacij z uporabo QCA Designer 12.9 in QuTip 12.10 potrjujejo pravilnost delovanja QCA vezja v skladu s podano pravilnostno tabelo 12.5.



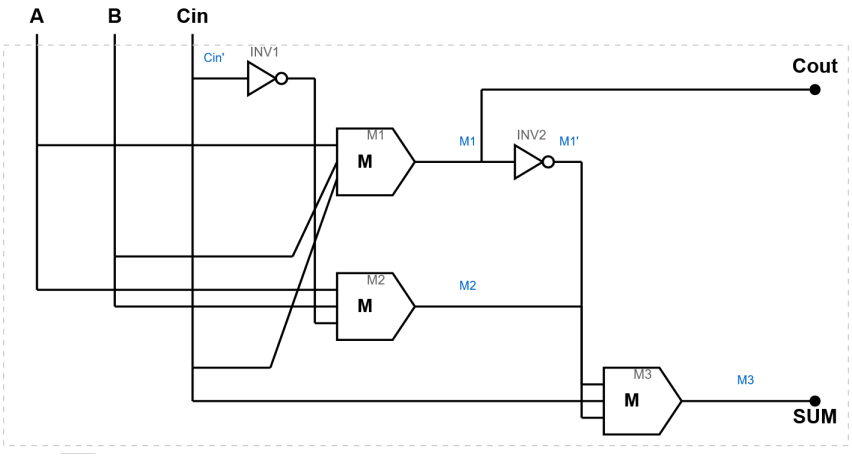
Slika 12.9: Rezultati simulacije prvega QCA vezja z uporabo QCA Designer.

```
Input: A=0, B=0, C=0
Output: P=1, Q=1
Input: A=0, B=0, C=1
Output: P=0, Q=1
Input: A=0, B=1, C=0
Output: P=1, Q=1
Input: A=0, B=1, C=1
Output: P=0, Q=1
Input: A=1, B=0, C=0
Output: P=1, Q=1
Input: A=1, B=0, C=1
Output: P=0, Q=1
Input: A=1, B=1, C=0
Output: P=1, Q=0
Input: A=1, B=1, C=1
Output: P=1, Q=1
```

Slika 12.10: Rezultati simulacije prvega QCA vezja z uporabo QuTip.

12.4.2 Drugo QCA vezje (1-bitni polni seštevalnik)

Drugo implementirano QCA vezje predstavlja 1-bitni polni seštevalnik, ki je bil pridobljen iz članka "Design and Implementation of Full Adders using QCA" [150]. Polni seštevalnik sprejme tri vhodne bite (A, B, C_{in}) in vrne dva izhodna bita: vsoto (Sum) in prenos (C_{out}). Logika polnega seštevalnika je implementirana z uporabo majoritetnih vrat in negatorjev, ki so osnovna gradnika QCA vezij.



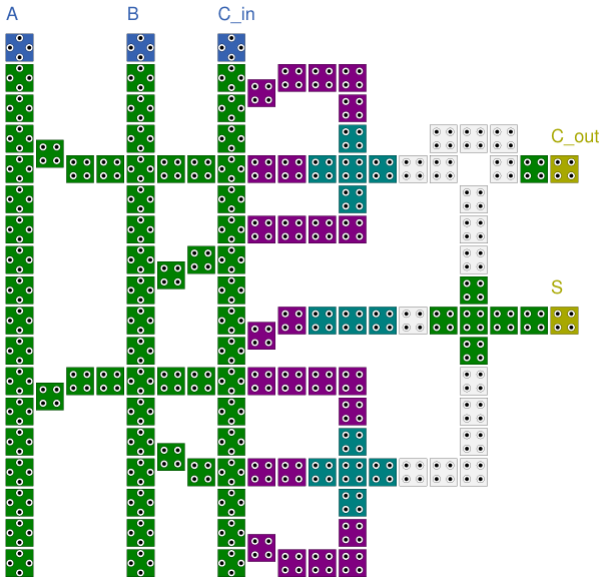
Slika 12.11: Diagram implementacije 1-bitnega polnega seštevalnika.

Pravilnost implementacije smo preverili z vsemi možnimi kombinacijami vhodov, kjer pravilnostna tabela 12.6 prikazuje pričakovane rezultate.

A	**B**	C_{in}	**Sum**	C_{out}	**Decimalno**
0	0	0	0	0	0+0+0=0
0	0	1	1	0	0+0+1=1
0	1	0	1	0	0+1+0=1
0	1	1	0	1	0+1+1=2
1	0	0	1	0	1+0+0=1
1	0	1	0	1	1+0+1=2
1	1	0	0	1	1+1+0=2
1	1	1	1	1	1+1+1=3

Tabela 12.6: Pravilnostna tabela 1-bitnega polnega seštevalnika.

12.4.2.1 Implementacija



Slika 12.12: Implementacija 1-bitnega polnega seštevalnika z uporabo QCA Designer.

```
def fullAdderQCA(A, B, Cin):
    Cin_inv = neg(Cin)

    M1 = maj(A, B, Cin)
    Cout = M1
    M1_neg = neg(M1)

    M2 = maj(A, B, Cin_inv)

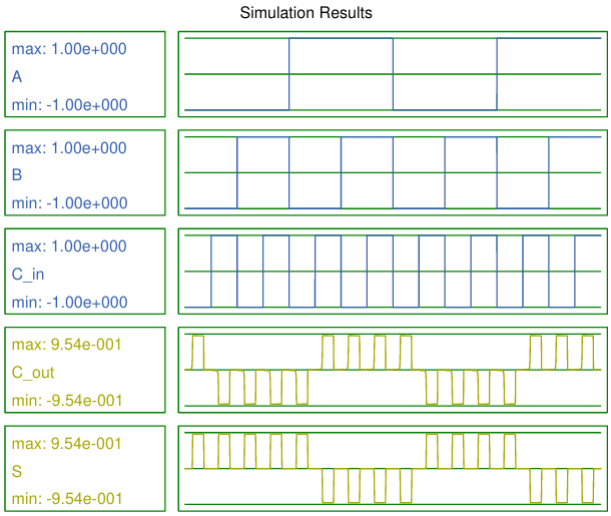
    M3 = maj(M1_neg, Cin, M2)
    Sum = M3

    return Sum, Cout
```

Slika 12.13: Implementacija 1-bitnega polnega seštevalnika z uporabo QuTip.

12.4.2.2 Rezultati

Rezultati simulacij z uporabo QCA Designer 12.14 in QuTip 12.15 potrjujejo pravilnost delovanja 1-bitnega polnega seštevalnika v skladu s podano pravilnostno tabelo 12.6.



Slika 12.14: Rezultati simulacije 1-bitnega polnega seštevalnika z uporabo QCA Designer.

```
Input: A=0, B=0, C=0
Output: Sum=0, Cout=0
Input: A=0, B=0, C=1
Output: Sum=1, Cout=0
Input: A=0, B=1, C=0
Output: Sum=1, Cout=0
Input: A=0, B=1, C=1
Output: Sum=0, Cout=1
Input: A=1, B=0, C=0
Output: Sum=1, Cout=0
Input: A=1, B=0, C=1
Output: Sum=0, Cout=1
Input: A=1, B=1, C=0
Output: Sum=0, Cout=1
Input: A=1, B=1, C=1
Output: Sum=1, Cout=1
```

Slika 12.15: Rezultati simulacije 1-bitnega polnega seštevalnika z uporabo QuTip.

12.5 Primerjava

Iz rezultatov simulacij obeh QCA vezij lahko razberemo, da sta nam obe implementaciji - implementacija v QCA Designer in v Pythonu z uporabo QuTip - dali enake in pravilne rezultate. Tako lahko uporabimo oba pristopa za implementacijo QCA vezij, vendar ima vsak svoje prednosti in slabosti.

Glavna prednost implementacije QCA vezij z uporabo QCA Designer predstavlja grafični vmesnik, ki nam ponudi enostaven pregled nad fizično arhitekturo vezja. Pri izdelavi vezja lahko QCA celice enostavno postavljamo in prilagajamo, ter s tem nadzorujemo prenašanje informacij preko vezja. Pri taki implementaciji je potrebno pravilno nastaviti urine periode posameznih celic, saj lahko njihova nepravilna nastavitev povzroči napačne ali nenatančne rezultate.

Implementacija QCA vezij z uporabo knjižnice QuTip je manj intuitivna in zahtevnejša, vendar omogoča izdelavo kompleksnejših vezij ter izvajanje naprednih kvantnih simulacij. Izdelava v programskem jeziku Python ponuja večji nadzor nad potekom simulacije, hkrati pa omogoča dodajanje lastnih funkcij in uporabo drugih knjižnic za obdelavo ter analizo podatkov.

12.6 Zaključek

V nalogi smo primerjali dva pristopa k modeliranju kvantnih celičnih avtomatov: QCA Designer in QuTiP. Rezultati so pokazali, da oba orodja zanesljivo izvajata simulacije osnovnih QCA struktur ter kompleksnejših vezij. QCA Designer se izkaže kot primernejši za vizualno, intuitivno in hitro zasnovo klasičnih QCA arhitektur, medtem ko QuTiP omogoča večjo fleksibilnost, matematično natančnost in možnost razširitve simulacij na splošne kvantne sisteme. Izbira orodja je zato odvisna predvsem od vrste problema.

12.7 Doprinosi avtorjev

- Aljaž Kern - povzetek, uvod, QCA Designer in QuTip, implementacija drugega QCA vezja, zaključek
- Nejc Drašler - pregled QCA struktur, implementacija prvega QCA vezja, primerjava, priprava osnovnih QCA struktur v Pythonu z uporabo QuTip

Poglavje 13
Implementacija XOR vrat ter polovičnega in polnega seštevalnika

Kristjan Volk in Matevž Jecl

Povzetek V tem članku bova predstavila implementacijo in analizo digitalnih logičnih vezij, zasnovanih v nanotehnologiji kvantno-celičnih avtomatov (QCA). Izhodišče najinega dela je članek avtorja V. K. Sharme "XOR gate design in QCA nanotechnology using cell translation method" [151], ki obravnava omejitve tradicionalne MOSFET-tehnologije v nanomerilu in predlaga QCA kot visoko učinkovito alternativo z manjšo porabo energije in večjo gostoto vezij. Osrednji prispevek izvirnega članka, ki ga bova preučila, je predlog zasnove logičnih vrat XOR. Ta zasnova temelji na metodi celične translacije, pri kateri polovični zamik (angl. half-distance translation) izhodnih celic omogoča realizacijo logične funkcije na optimiziran način. V sklopu najine raziskave bova implementirala tri ključna vezja, predstavljena v članku: 2-vhodna vrata XOR, polovični seštevalnik (half-adder) in polni seštevalnik (full-adder). Primerjala bova ključne metrike delovanja, kot so število celic, zakasnitev, površina vezja in celotni strošek zasnove, z rezultati v izvirnem članku.

13.1 Uvod

Kvantna celična avtomatika (QCA) predstavlja eno izmed perspektivnih nanotehnologij, ki poskušajo preseči omejitve klasične CMOS-tehnologije na ravni porabe energije, gostote integracije in hitrosti delovanja. Namesto toka nabojev QCA uporablja konfiguracijo nabojev v kvantnih pikah znotraj celice, logične funkcije pa izhajajo iz interakcij med celicami. Zaradi drugačne zasnove od klasičnih tranzistorskih vezij zahteva QCA tudi nove pristope k načrtovanju in vrednotenju digitalnih vezij, zlasti na ravni osnovnih logičnih vrat in aritmetičnih gradnikov.

V izvirnem članku Sharme [151] so predstavljena 2-vhodna XOR vrata ter iz njih izpeljana polovični in polni seštevalnik. Prednost teh vezij naj bi bila manjše število celic, manjša površina in manjša časovna zakasnitev v primerjavi z obstoječimi rešitvami.

V najinem delu se osredotočava na implementacijo teh vezij v okolju QCADesigner-E ter na primerjavo dobljenih rezultatov s tistimi iz originalnega članka. V nadaljevanju bova opisala, kako sva na novo zgradila vezja XOR, polovičnega in polnega seštevalnika. Sledila sva opisanemu konceptu in razporeditvi celic. Nato sva izvedla simulacije, v katerih sva izmerila ključne metrike (število celic, porabljeno površino, število faz ure oziroma zakasnitev ter stabilnost logičnih izhodov) in jih primerjala z rezultati iz citiranega članka.

13.2 Metode

V tem poglavju bova podrobno opisala pristop, ki sva ga uporabila pri implementaciji, simulaciji in analizi QCA-vezij. Najin cilj je bil čim bolj natančno replicirati zasnove, predstavljene v članku V. K. Sharme, ter primerjati rezultate.

13.2.1 Uporabljena orodja

Za načrtovanje in simulacijo vezij sva uporabila program QCADesigner-E, ki je uveljavljeno in široko dostopno orodje za raziskave na področju nanotehnologije kvantno-celičnih avtomatov. To orodje omogoča grafično načrtovanje vezij z natančnim postavljanjem posameznih QCA celic, konfiguracijo vhodnih signalov in nastavljanjem urnih con.

13.2.2 Predlagana QCA-vezja

Najina raziskava se osredotoča na implementacijo treh ključnih vezij, ki jih avtor predlaga z uporabo metode celične translacije:

1. **2-vhodna vrata XOR:** Osnovni logični gradnik, katerega inovativna zasnova temelji na polovičnem zamiku (9 nm) dveh izhodnih celic. Po navedbah avtorja ta zasnova omogoča delovanje z le 10 celicami in zakasnitvijo 0.25.
2. **Polovični seštevalnik (*half-adder*):** Vezje, sestavljeno iz predlaganih vrat XOR za izračun vsote in 3-vhodnih večinskih vrat za izračun prenosa.
3. **Polni seštevalnik (*full-adder*):** Najkompleksnejše vezje, zgrajeno na osnovi dveh XOR stopenj in večinskih vrat, ki omogoča seštevanje treh bitov. Zasnova je enoplastna in ne vključuje križnih povezav.

13.2.3 Postopek implementacije in simulacije

Najin delovni proces je potekal v naslednjih korakih:

1. **Replikacija vezij:** Vsako od zgoraj naštetih vezij sva zgradila v QCADesigner-E. Pri tem sva dosledno sledila postavitvi celic, vhodov, izhodov in urnih con, kot so prikazane v shemah referenčnega članka.
2. **Nastavitev simulacijskih parametrov:** Uporabila sva bistabilno aproksimacijo QCA s simulacijskimi parametri iz referenčnega članka (Tabela 1), vključno s številom vzorcev (12 800), radijem vpliva (65 nm) in vrednostmi ure (*clock high/low*).
3. **Izvedba simulacij:** Za vsako vezje sva nato izvedla simulacije v okolju QCADesigner.

13.2.4 Metrike za primerjalno analizo

Uspešnost in učinkovitost implementiranih vezij sva ovrednotila na podlagi istih kvantitativnih metrik, kot so uporabljene v referenčnem članku (Tabela 2). Te metrike so:

- **Število celic (*cell count*):** Skupno število QCA celic, uporabljenih v vezju. Manjše število pomeni večjo gostoto in nižje stroške izdelave.
- **Zakasnitev (*latency*):** Čas, ki ga signal potrebuje za pot od vhoda do izhoda. Meri se v številu faz ure, pri čemer ena faza ustreza zakasnitvi 0.25. Nižja zakasnitev pomeni hitrejše delovanje.
- **Površina vezja (*layout area*) [μm^2]:** Celotna površina, ki jo vezje zaseda. Izračuna se kot produkt števila horizontalnih in vertikalnih celic, pomnožen s površino ene celice (18 nm $\times$ 18 nm).
- **Strošek zasnove (*design cost*):** Sintetična metrika, ki združuje zgornje parametre in služi kot celovita ocena učinkovitosti zasnove. Izračuna se po formuli

$$\textit{Strošek zasnove} = (\textit{število celic}) \times (\textit{zakasnitev}) \times (\textit{površina vezja}). \quad (13.1)$$

S pomočjo teh metrik bova v naslednjem poglavju predstavila primerjalno tabelo, ki bo najine rezultate primerjala s tistimi iz članka in s tem ugotovila uspešnost najine implementacije.

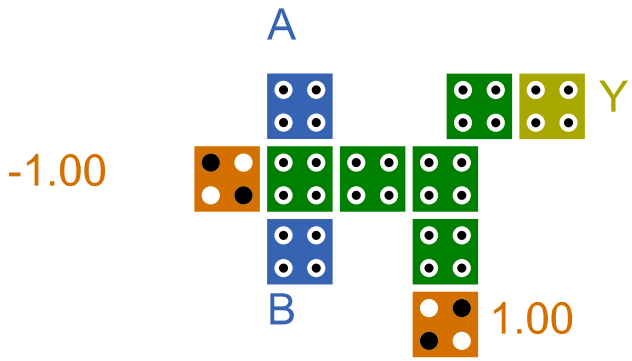
13.3 Rezultati

V tem poglavju predstavljava rezultate, pridobljene z implementacijo in simulacijo predlaganih QCA-vezij v orodju QCADesigner-E. Sledila sva metodologiji, opisani

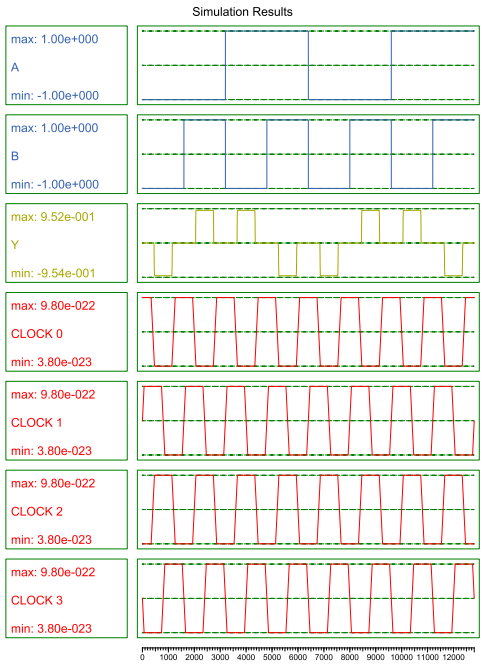
v prejšnjem poglavju, ter preverila tako logično pravilnost delovanja kot tudi ključne metrike učinkovitosti.

13.3.1 2-vhodna vrata XOR

Predlagana 2-vhodna vrata XOR temeljijo na prostorski razporeditvi QCA celic in metodi celične translacije, ne pa na klasični sestavi iz osnovnih logičnih vrat. Vhoda `A` in `B` polarizirata vhodni celici, dve dodatni celici s fiksno polarizacijo (`-1.00` in `+1.00`) pa delujeta kot konstantna vira polja. Jedro vezja sestavlja veriga prenosnih celic, pri čemer končni logični izračun omogoči polovični zamik (angl. *half-distance translation*) dveh izhodnih celic, zaradi česar se na izhodu `Y` realizira funkcija XOR. Vezje je zelo kompaktno, saj uporablja le 10 celic in eno samo urno cono (zakasnitev 0.25), njegova postavitev je prikazana na Sliki 13.1, pravilno delovanje pa potrjuje simulacija na Sliki 13.2.



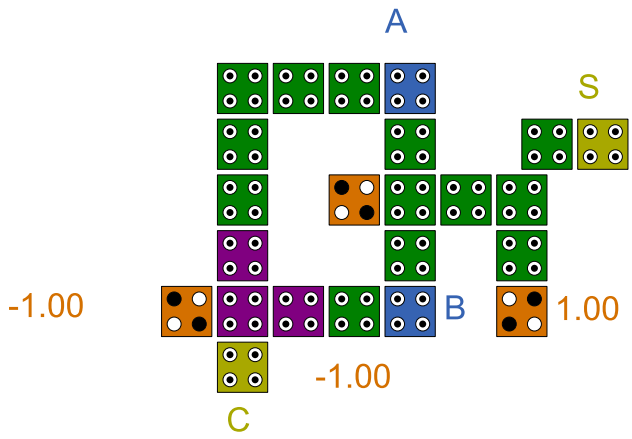
Slika 13.1: Implementacija vezja XOR v QCADesigner-E.



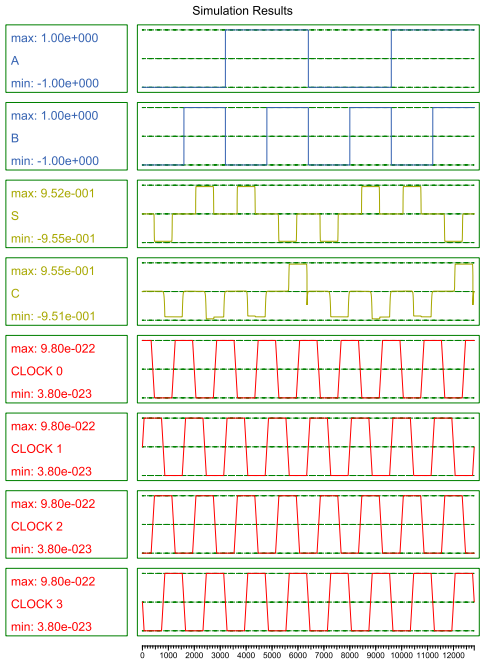
Slika 13.2: Simulacijski časovni diagram za vrata XOR.

13.3.2 Polovični seštevalnik (half-adder)

Polovični seštevalnik je kompozitno vezje z dvema vhodoma (`A`, `B`) in dvema izhodoma, vsoto `S` in prenosom `C`, pri čemer implementira funkciji $S = A \oplus B$ in $C = A \cdot B$. Za izračun vsote sva uporabila prej opisana 2-vhodna vrata XOR z metodo celične translacije, za izračun prenosa pa 3-vhodna večinska vrata z enim vhodom, fiksno nastavljenim na logično 0 (polarizacija `-1.00`), s čimer se večinska funkcija poenostavi v $M(A,B,0) = AB$. Celotno vezje je enoslojno, brez križanja signalnih poti, vsebuje 23 celic in deluje z dvema urnima conama (zakasnitev 0.50). Postavitev je prikazana na Sliki 13.3, pravilno delovanje pa potrjuje časovni diagram na Sliki 13.4.



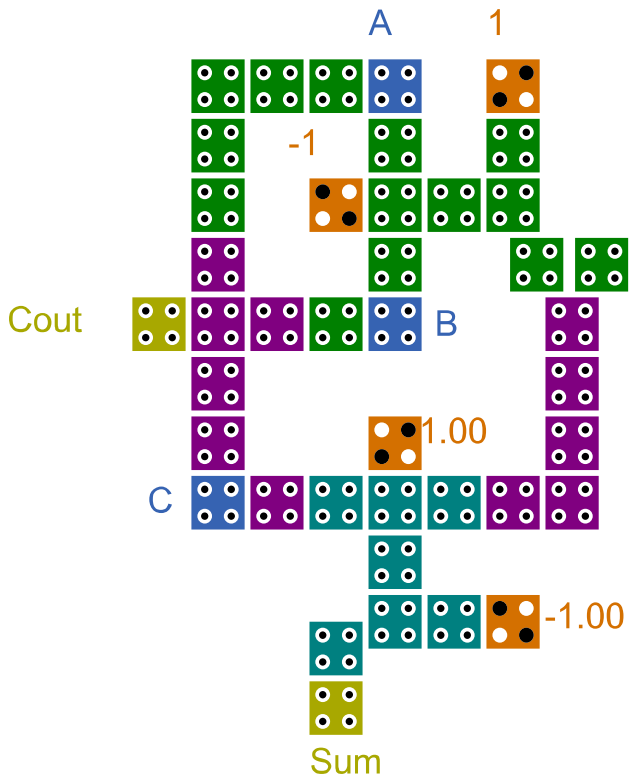
Slika 13.3: Implementacija polovičnega seštevalnika.



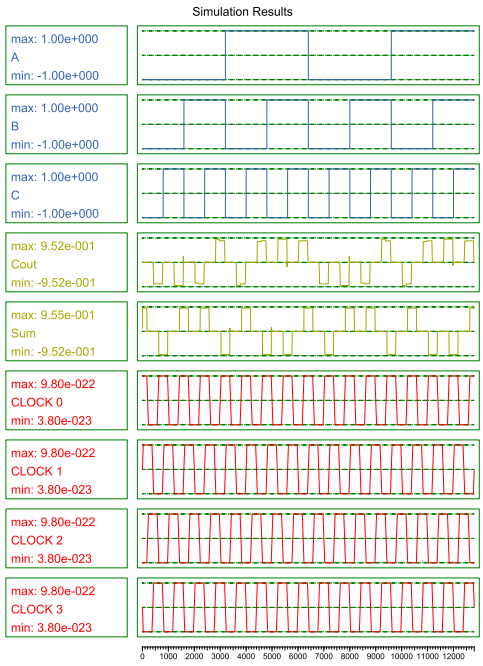
Slika 13.4: Simulacijski časovni diagram za polovični seštevalnik.

13.3.3 Polni seštevalnik (full-adder)

Polni seštevalnik je zgrajen za seštevanje treh bitov (`A`, `B`, `Cin`) in ima izhoda vsoto `Sum` in prenos `Cout`, pri čemer implementira funkciji $Sum = A \oplus B \oplus Cin$ in $Cout = M(A,B,Cin)$. Vsota je realizirana s kaskadno vezavo dveh 2-vhodnih XOR gradnikov: prva stopnja izračuna $A \oplus B$, druga pa $(A \oplus B) \oplus Cin$. Izhodni prenos `Cout` nastane v eni stopnji z uporabo 3-vhodnega večinskega vezja, na katerega so neposredno priključeni vhodi `A`, `B` in `Cin`. Celotno vezje (Slika 13.5) je enoplastno, brez križnih povezav, sestavljeno iz 41 QCA celic in za prenos signala potrebuje tri urne cone (zakasnitev 0.75). Simulacija (Slika 13.6) potrjuje pravilne vrednosti `Sum` in `Cout` za vseh osem kombinacij vhodov, skladno s pravilnostno tabelo polnega seštevalnika.



Slika 13.5: Implementacija polnega seštevalnika.



Slika 13.6: Simulacijski časovni diagram za polni seštevalnik.

13.3.4 Primerjalna analiza rezultatov

Za končno oceno uspešnosti najine implementacije sva zbrala ključne metrike učinkovitosti in jih primerjala z vrednostmi, objavljenimi v članku. Primerjava je prikazana v Tabeli 13.1.

Tabela 13.1: Primerjava rezultatov med referenčnim člankom in najino implementacijo.

Metrika	**Vezje**	**Vrednost iz članka**	**Najina vrednost**
Število celic	Vrata XOR	10	10
	Polovični seštevalnik	23	23
	Polni seštevalnik	41	41
Zakasnitev	Vrata XOR	0.25	0.25
	Polovični seštevalnik	0.50	0.50
	Polni seštevalnik	0.75	0.75
Površina [μm^2]	Vrata XOR	0.0071	0.0071
	Polovični seštevalnik	0.0165	0.0165
	Polni seštevalnik	0.0317	0.0317
Strošek zasnove	Vrata XOR	0.0178	0.0178
	Polovični seštevalnik	0.1898	0.1898
	Polni seštevalnik	0.9748	0.9748

Kot je razvidno iz Tabele 13.1, se rezultati najine implementacije v celoti ujemajo z referenčnimi vrednostmi. Uspešno sva replicirala vse tri predlagane strukture in s simulacijo potrdila njihovo logično pravilnost ter enake metrike učinkovitosti. To potrjuje, da je metoda celične translacije veljaven in ponovljiv pristop.

13.4 Zaključek

V tem članku sva uspešno replicirala in analizirala tri QCA-vezja, predlagana v delu V. K. Sharme [151]: 2-vhodna vrata XOR, polovični in polni seštevalnik. Najina implementacija v orodju QCADesigner-E je pokazala popolno ujemanje z objavljenimi rezultati glede ključnih kvantitativnih metrik, kot so število celic, zakasnitev, površina in izračunan strošek zasnove. S tem sva potrdila, da je metoda celične translacije ponovljiv in učinkovit pristop za načrtovanje kompaktnih, enoplastnih aritmetičnih vezij brez potrebe po križanju signalnih poti.

13.5 Doprinosi avtorjev

Avtorja sta pri pripravi tega dela sodelovala v vseh ključnih fazah, od načrtovanja do končnega zapisa. Vloge sta si razdelila na naslednji način:

- **Kristjan Volk** je bil odgovoren za implementacijo vseh vezij v QCADesigner-E.
- **Matevž Jecl** je bil odgovoren za analizo in primerjavo vezij z citiranim člankom.

Oba avtorja sta skupaj pregledala in uredila končno verzijo dokumenta.

Poglavje 14
Analiza algoritma za računanje delnih vsot in uteženih delnih vsot amplitud kvantnih stanj

Domen Kastelic in Jaka Kovačič

Povzetek V tem poglavju predstavimo in analiziramo kvantni algoritem za izračun delnih vsot in uteženih delnih vsot amplitud kvantnih stanj, kot sta ga predstavila Shukla in Vedula. Algoritem klasične podatke zapiše neposredno v amplitude vhodnega kvantnega stanja ter s transformacijo doseže, da se delna vsota prvih M členov pojavi v amplitudi stanja $|0\rangle$. V nasprotju s prejšnjimi pristopi podpira poljubne vrednosti M, ne le potenc števila dva, pri tem pa ne potrebuje dodatnih qubitov. Uporablja zaporedja Hadamardovih, kontroliranih Hadamardovih in kontroliranih rotacijskih vrat, njegova kompleksnost pa znaša $O(\log_2 M)$. Algoritem ohranja superpozicije, zato je izhodno stanje primerno za nadaljnjo kvantno procesiranje. V poglavju obravnavamo tudi posplošeno različico algoritma za utežene delne vsote ter prikažemo njegovo uporabo pri numerični integraciji in sorodnih računskih problemih. Predstavimo primere ter dokaze pravilnosti, ki ponazorijo delovanje in teoretično ozadje algoritma.

14.1 Uvod

Z računanjem delnih vsot in uteženih delnih vsot se srečujemo pri reševanju marsikaterega problema. Takšni problemi so na primer računanje določenih integralov in porazdelitvene funkcije, ki sta uporabna pri oceni tveganja, analizi zanesljivosti, reševanju diferencialnih enačb, in mnogih drugih problemih.

Ko za reševanje teh problemov začnemo uporabljati kvantno procesiranje, se problem izračunavanja vsote precej oteži. Podatke lahko zapišemo z amplitudami različnih kvantnih stanj. Do zapleta pride, ker podatkov, zapisanih s kvantnimi stanji, ne moremo prebrati, ne da bi jih s tem tudi uničili. Zato seštevanje števil po vrsti ni ustrezen pristop. Kljub temu obstaja več kvantnih algoritmov, ki nam takšne vsote zmorejo izračunati in hkrati ne uničijo superpozicij naših podatkov. Opisali in analizirali bomo enega izmed takšnih algoritmov, ki ga avtorja Shukla in Vedula predstavita v [152].

14.2 Zapis podatkov

Algoritem, ki ga bomo predstavili, nam omogoča izračun delne vsote amplitud kvantnih stanj. Algoritem sprejme kvantno stanje $|f\rangle$, kjer f_k označuje amplitudo kvantnega stanja $|k\rangle$. Stanje

$$|f\rangle = \sum_{k=0}^{N-1} f_k |k\rangle$$

zapisuje amplitude $N = 2^n$ osnovnih stanj zapisanih z n qubiti $|q_{n-1}q_{n-2}\ldots q_0\rangle$.

Za vrednosti f_k mora veljati $\sum_{k=0}^{N-1} |f_k|^2 = 1$, saj so to amplitude kvantnih stanj. Izračun vsote, kadar členi zaporedja niso normalizirani, je še vedno mogoč. Za poljubne vrednost f'_k lahko pripadajoče vrednosti f_k dobimo

$$f_k = \frac{f'_k}{\sum_{i=0}^{N-1} |f'_i|^2}.$$

Zapis podatkov v ustrezno začetno stanje, kot zahteva algoritem, ni trivialna naloga. Drugi algoritmi uporabljajo zapise, kjer je amplituda stanja sorazmerna korenu vrednosti namesto vrednosti sami, kar dosežejo z uporabo estimacij. Kljub temu nam takšen zapis prinaša druge prednosti, ki se kažejo pri izhodu algoritma. Algoritem zato zahteva, da so amplitude enake vrednostim, kar dosežemo s predhodno obdelavo.

Vsota prvih M členov ($2 < M \leq N$) je $S_M = \sum_{k=0}^{M-1} f_k$.

14.3 Izhod algoritma

Lastnost tega algoritma je, da ne potrebuje dodatnih qubitov za shranjevanje rezultata in ohranjanja reverzibilnosti. Algoritem nam namreč vrne novo stanje

$$\left|\widetilde{f}\right\rangle = \sum_{k=0}^{N-1} \widetilde{f}_k |k\rangle,$$

kjer vsoto prvih M členov zapiše v amplitudo stanja $|0\rangle$. Velja, da je po izvedbi algoritma to stanje enako

$$\widetilde{f}_0 = \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} f_k.$$

Delno vsoto tako lahko enostavno izračunamo iz amplitude stanja $|0\rangle$ tako, da amplitudo množimo s $\sqrt{M}$. S tem originalnih stanj nismo za vedno izgubili, saj je operacija reverzibilna, hkrati pa smo delno vsoto zapisali v amplitudo stanja.

To je tudi ena od pomembnih prednosti tega algoritma v primerjavi z ostalimi. Vendar pa imata zapis direktno z amplitudami stanj in pomanjkanje dodatnih qubitov tudi svoje pomanjkljivosti. Branje samega rezultata ni mogoče brez estimacije

amplitud, kar pa uniči superpozicije vseh stanj. Kljub temu ima takšen zapis svoje prednosti, omogoča nam namreč direktno nadaljnjo kvantno procesiranje. Podatki in vsote se še vedno nahajajo v superpozicijah in jih zato še vedno lahko uporabljamo kot take. Primeri, v katerih je uporaba algoritma smiselna, so problemi, kjer je izračun delne vsote, določenega integrala ali porazdelitvene funkcije le eden od korakov reševanja. Eden od takih primerov je v procesiranju slik s kvantnimi algoritmi, kjer intenziteto pik zapisujemo z amplitudo stanj. Kljub temu je branje delne vsote mogoče z uporabo ocenjevanja amplitud (QAE), ki pa ima višjo zahtevnost in napako kot pa sam algoritem.

14.4 Algoritem za seštevanje delne vsote

Predlagani algoritem nam pomaga izračunati vsoto amplitud prvih M stanj. Od večine drugih algoritmov se poleg načina zapisa podatkov in vračanja podatkov razlikuje tudi v tem, da lahko računa delne vsote za katerikoli M, $2 \leq M \leq N = 2^n$. Prej opisani podobni algoritmi delujejo le za specifične M, in sicer za potence števila dva.

Iskanju delne vsote, kjer so vhodi in izhodi definirani kot v prejšnjem poglavju pripada transformacija, kjer se amplituda stanja $|0\rangle$ prilega $\sqrt{M}S_M$. Naj bo unitarna matrika U matrika, ki pripada transformaciji, opisani v algoritmu.

V primeru, ko $M \neq 2^r$ so ključni koraki rotacije, ki poskrbijo, da je željena vsota enakomerna. V posplošenem algoritmu za poljubno utežene vsote so vrednosti rotacij drugačne tako, da ustrezajo željeni vsoti.

14.4.1 Pravilnost algoritma

Naj bo unitarna matrika U matrika, ki pripada algoritmu. Algoritem za iskanje delne vsote bo deloval pravilno natanko tedaj, ko je prva vrstica matrike U enaka $\frac{1}{\sqrt{M}}\left(1 \ldots 1\ 0 \ldots 0\right)$, kjer je na prvih M mestih 1 in na zadnjih $N-M$ mestih 0. Množenje vektorja f s takšno matriko nam poda vektor, v katerem je na prvem mestu (mestu, ki pripada stanju $|0\rangle$) vsota prvih M mest originalnega vektorja pomnožena s številom $\frac{1}{\sqrt{M}}$.

Preveriti moramo ali je prva vrstica matrike U res takšna. Ločeno si oglejmo oba primera, ko je M potenca števila 2 in ko ni.

Prvo si poglejmo primer, kjer je $M = 2^r$. V tem primeru v algoritmu postavimo Hadamardova vrata na qubite od 0 do $r-1$. Ker so qubiti zmnoženi v vrstnem redu $|q_{n-1}q_{n-2}\ldots q_r q_{r-1}\ldots q_0\rangle$ velja, da je ta transformacija enaka identiteti na prvih $n-r$ qubitih in Hadamardovim vratom na ostalih r qubitih. Ekvivalenten zapis bi torej bil $I^{\otimes n-r} \otimes H^{\otimes r}$. $I^{\otimes n-r}$ je identiteta velikosti 2^{n-r}, prva vrstica matrike $H^{\otimes r}$, pa je enaka 2^r vrednostim $\frac{1}{\sqrt{2^r}}$. Očitno je, da je njun tenzorski produkt primerna matrika.

Algoritem 4 Kvantni algoritem za računanje delnih vsot

Require: Normalizirano stanje $|f\rangle = \sum_{k=0}^{N-1} f_k |k\rangle$ in naravni števili M in n ($2 \leq M \leq N = 2^n$).
Ensure: $\left|\widetilde{f}\right\rangle = \sum_{k=0}^{N-1} \widetilde{f}_k |k\rangle$, kjer $\widetilde{f}_0 = \frac{1}{\sqrt{M}} \sum_{j=0}^{M-1} f_j$.

procedure AMPLITUDEPARTIALSUM($|f\rangle, M, n$)
  Vhodno stanje $|f\rangle$ je zapisano na n-qubitnem registru R, z qubiti $q_{n-1}, \ldots, q_0$.
  if $M = 2^r$ za naravno število r, $1 \leq r \leq n$ **then**
    for $i = 0$ **to** $r-1$ **do**
      Izvedi Hadamardova vrata H na $|q_i\rangle$
    end for
  else
    ▷ $l_0, \ldots, l_k$ so pozicije enic v obratnem binarnem zapisu M
    Izračunaj $l_0, \ldots, l_k$ tako, da $M = \sum_{j=0}^{k} 2^{l_j}$ in $0 \leq l_0 < \cdots < l_k \leq n-1$.
    $M_{k-1} \leftarrow M - 2^{l_k}$
    for $m = k-1$ **downto** 1 **do**
      for $i = l_{m+1} - 1$ **downto** l_m **do**
        Izvedi kontrolirana Hadamardova vrata CH na $|q_i\rangle$ s pogojem $q_{l_{m+1}} = 0$
      end for
      $M_{m-1} \leftarrow M_m - 2^{l_m}$
      Izvedi kontrolirana vrata $CR_y(\theta_m)$ na $|q_{l_m}\rangle$ s pogojem $q_m = 0$, kjer $\theta_m = 2\arccos\left(\sqrt{\frac{2^{l_m}}{M_m - M_{m-1}}}\right)$
    end for
    for $i = l_1 - 1$ **downto** l_0 **do**
      Izvedi kontrolirana vrata CH na $|q_i\rangle$ s pogojem $q_{l_1} = 0$
    end for
    Izvedi $R_y(\theta_0)$ na $\left|q_{l_0}\right\rangle$, kjer $\theta_0 = 2\arccos\left(\sqrt{\frac{M_0}{M}}\right)$
    if $l_0 > 0$ **then**
      for $i = 0$ **to** $l_0 - 1$ **do**
        Izvedi H na $|q_i\rangle$
      end for
    end if
    for $j = 1$ **to** k **do**
      Izvedi X na $\left|q_{l_j}\right\rangle$
    end for
  end if
  ▷ Register R vsebuje stanje $\left|\widetilde{f}\right\rangle$
  return $\left|\widetilde{f}\right\rangle$
end procedure

Bolj zanimiv je drugi primer, kjer $M \neq 2^r$. Uporabno je, da je matrika $U^\dagger$, enaka matriki, ki pripada algoritmu 1 iz [153], za učinkovito pripravo enakomernih superpozicijskih stanj. Tu nam prav pride reverzibilnost kvantnega procesiranja, saj algoritma namreč izvedeta enake operacije v obratnem vrstnem redu in za nasprotno vrednost kotov. Algoritma sta si zato inverzna.

Algoritem za pripravo enakomernih superpozicijskih stanj nam vrne qubite, kjer ima prvih M qubitov enako amplitudo. Vrne nam torej stanje

$$|f\rangle = \sum_{k=0}^{M-1} \frac{1}{\sqrt{M}} |k\rangle$$

Ta algoritem ni transformacija obstoječega stanja, vendar takšno stanje inicializira, in je ekvivalenten zapisu $U^\dagger |0\rangle^{\otimes N}$. Algoritem torej inicializira N qubitov z vrednostjo $|0\rangle$ in na njih izvede operacije, katerim pripada matrika $U^\dagger$. Takšni transformaciji potem pripada katerakoli unitarna matrika, kjer ima prvi stolpec matrike na prvih M vrsticah vrednost $\frac{1}{\sqrt{M}}$, na ostalih pa 0. To velja za matriko $U^\dagger$, saj algoritem iz [153] deluje (dokaz za delovanje je v originalnem članku). Potem pa vemo, da je matrika $U^\dagger$ unitarna matrika, enaka

$$U^\dagger = \frac{1}{\sqrt{M}} \begin{bmatrix} 1 & a_{01} & a_{02} & \cdots & a_{0(N-1)} \\ 1 & a_{11} & a_{12} & \cdots & a_{1(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & a_{(M-1)1} & a_{(M-1)2} & \cdots & a_{(M-1)(N-1)} \\ 0 & a_{M1} & a_{M2} & \cdots & a_{M(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a_{(N-1)1} & a_{(N-1)2} & \cdots & a_{(N-1)(N-1)} \end{bmatrix}$$

za neke vrednosti a_{ij} izven prvega stolpca. Za delovanje algoritma te vrednosti niso pomembne, vendar so določene v samem algoritmu. Ker je naš algoritem enak temu iz [153], z obratnim zaporedjem vrat in nasprotnimi vrednostmi kotov in predstavlja inverzno transformacijo je matrika $U = (U^\dagger)^{-1}$. Matrika U je potem naslednja:

$$U = (U^\dagger)^\dagger = \frac{1}{\sqrt{M}} \begin{bmatrix} 1 & 1 & \cdots & 1 & 0 & \cdots & 0 \\ \overline{a_{01}} & \overline{a_{11}} & \cdots & \overline{a_{(M-1)1}} & \overline{a_{M1}} & \cdots & \overline{a_{(N-1)1}} \\ \overline{a_{02}} & \overline{a_{12}} & \cdots & \overline{a_{(M-1)2}} & \overline{a_{M2}} & \cdots & \overline{a_{(N-1)2}} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \overline{a_{0(N-1)}} & \overline{a_{1(N-1)}} & \cdots & \overline{a_{(M-1)(N-1)}} & \overline{a_{M(N-1)}} & \cdots & \overline{a_{(N-1)(N-1)}} \end{bmatrix}.$$

Če pa je matrika, ki pripada algoritmu enaka zgoraj opisani unitarni matriki, vidimo, da algoritem deluje.

Algoritem za utežene delne vsote je enak drugemu delu algoritma za delne vsote, vendar je kot rotacije poljuben. Velja, da je algoritem za delne vsote poseben primer tistega za utežene, kjer v prvi vrstici poskušamo dobiti vse vrednosti $\frac{1}{\sqrt{M}}$. Tu imajo ključen pomen vrednosti rotacij. Da te vrednosti ustrezajo, je dokazano za algoritem 1 v [153], kjer je tudi algoritem dobro razložen. Razumevanje inverznega algoritma je tudi intuitivno lažje kot razumevanje algoritma za izračun delne vsote.

14.4.2 Primer delovanja

Oglejmo si sedaj algoritem na nekem primeru. Vzemimo splošen primer za $N = 8$ ($n = 3$) in $M = 3$. Vhodno stanje naj bo

$$|f\rangle = \sum_{i=0}^{N} f_k |k\rangle ,$$

kjer $|k\rangle$ pripadajo osnovnim stanjem tenzorskega produkta qubitov $|q_2 q_1 q_0\rangle$ s pripadajočim binarnim zapisom ($|011_2\rangle = |3\rangle$).

V tem primeru je $M = 2^0 + 2^1$ in sta $l_0 = 0$ in $l_1 = 1$. Ker je k enak ena ne bomo izvajali for zanke, velja pa $M_0 = 1$. To pomeni, da je naslednji korak izvesti *CH* vrata na $|q_0\rangle$ pod pogojem da je $q_1 = 0$. Dobili bomo novo stanje $|f'\rangle$ enako:

$$\begin{aligned} |f'\rangle = &\frac{f_0 + f_1}{\sqrt{2}} |0\rangle + \frac{f_0 - f_1}{\sqrt{2}} |1\rangle + f_2 |2\rangle + f_3 |3\rangle \\ &+ \frac{f_4 + f_5}{\sqrt{2}} |4\rangle + \frac{f_4 - f_5}{\sqrt{2}} |5\rangle + f_6 |6\rangle + f_7 |7\rangle \end{aligned} .$$

Sedaj moramo izvesti vrata $R_Y(\theta_0)$ na $|q_1\rangle$. Vrata $R_Y(\theta)$ so:

$$R_Y(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix} .$$

V našem primeru je $\theta_0 = 2 \arcsin \sqrt{\frac{1}{3}}$, torej sta $\sin\left(\frac{\theta_0}{2}\right) = \frac{\sqrt{2}}{\sqrt{3}}$ in $\cos\left(\frac{\theta_0}{2}\right) = \frac{1}{\sqrt{3}}$. Po izvedenih vratih $R_y(\theta_0)$ na qubitu $|q1\rangle$ dobimo stanje $|f''\rangle$, ki je enako:

$$\begin{aligned} |f''\rangle = &\left(\tfrac{1}{\sqrt{3}} \tfrac{f_0+f_1}{\sqrt{2}} - \tfrac{\sqrt{2}}{\sqrt{3}} f_2\right) |0\rangle + \left(\tfrac{1}{\sqrt{3}} \tfrac{f_0-f_1}{\sqrt{2}} - \tfrac{\sqrt{2}}{\sqrt{3}} f_3\right) |1\rangle \\ &+ \left(\tfrac{\sqrt{2}}{\sqrt{3}} \tfrac{f_0+f_1}{\sqrt{2}} + \tfrac{1}{\sqrt{3}} f_2\right) |2\rangle + \left(\tfrac{\sqrt{2}}{\sqrt{3}} \tfrac{f_0-f_1}{\sqrt{2}} + \tfrac{1}{\sqrt{3}} f_3\right) |3\rangle \\ &+ \left(\tfrac{1}{\sqrt{3}} \tfrac{f_4+f_5}{\sqrt{2}} - \tfrac{\sqrt{2}}{\sqrt{3}} f_6\right) |4\rangle + \left(\tfrac{1}{\sqrt{3}} \tfrac{f_4-f_5}{\sqrt{2}} |5\rangle - \tfrac{\sqrt{2}}{\sqrt{3}} f_7\right) |5\rangle \\ &+ \left(\tfrac{\sqrt{2}}{\sqrt{3}} \tfrac{f_4+f_5}{\sqrt{2}} + \tfrac{1}{\sqrt{3}} f_6\right) |6\rangle + \left(\tfrac{\sqrt{2}}{\sqrt{3}} \tfrac{f_4-f_5}{\sqrt{2}} |5\rangle + \tfrac{1}{\sqrt{3}} f_7\right) |7\rangle \end{aligned} .$$

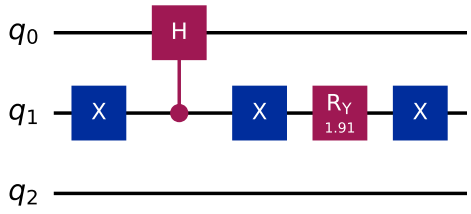
Ker je $l_0 = 0$ nam preostanejo le še X vrata na qubitu $|q_1\rangle$. Ta vrata pa nam zgolj predstavljajo zamenjave amplitud stanj $|0\rangle$ in $|2\rangle$, $|1\rangle$ in $|3\rangle$, $|4\rangle$ in $|6\rangle$ ter $|5\rangle$ in $|7\rangle$. Torej rezultat algoritma lahko preberemo iz amplitude stanja $|2\rangle$ v $|f''\rangle$. Po krajšanju in zamenjavi velja

$$\widetilde{f_0} = \frac{f_0 + f_1 + f_2}{\sqrt{3}},$$

kar pa je tudi iskana rešitev.

Že tu smo se ukvarjali z veliko računanja in dolgimi členi, za daljše vsote pa bi bil postopek še daljši in členi bolj kompleksni, zato ročnega preverjana na njih ne

bomo izvajali. Kljub temu pa je vezje, ki pripada temu primeru, precej enostavno, saj uporablja le 3 vrata (5 če ne moremo implementirat CH vrat s pogojem, da je bit 0). Skica vezja za ta problem je na sliki 14.1.



Slika 14.1 Vezje za računanje delne vsote za $M = 3$ in $N = 8$.

14.4.3 Uporaba za integracijo

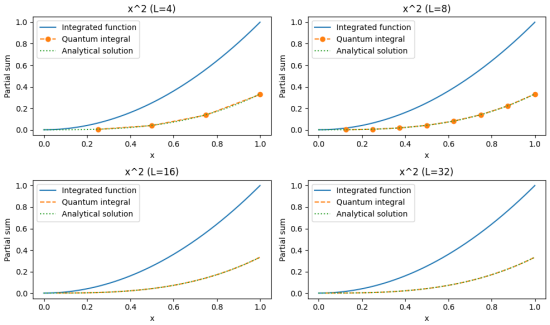
Eden od primerov uporabe algoritma je numerična integracija funkcij. Poglejmo si, kako to izvedemo v praksi. Denimo, da imamo integrabilno funkcijo $f(x)$. Želimo si čim bolj točen približek za sledečo funkcijo, definirano na intervalu (a,b):

$$F(x) = \int_a^x f(t)dt$$

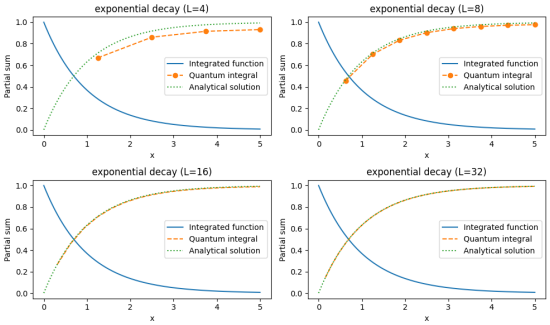
Približek dobimo tako, da interval (a,b) razdelimo na L manjših enako velikih intervalov $(x_0,x_1),(x_1,x_2),\ldots,(x_{L-1},x_L)$ in v sredini vsakega "izmerimo" f. Dobimo vrednosti $f_0,...,f_{L-1}$. Te vrednosti nam definirajo L pravokotnikov z višinami $f_0,\ldots,f_{L-1}$ in širino $\Delta x = |x_1 - x_0| = |x_2 - x_1| = \cdots = \frac{b-a}{L}$. Da dobimo približek za zgornji integral seštejemo ploščine prvih M pravokotnikov, tako, da se x_M ujema z zgornjo mejo integrala:

$$F(x_M) \approx \sum_{i=0}^{M-1} f_i \Delta x = \frac{b-a}{L} \sum_{i=0}^{M-1} f_i$$

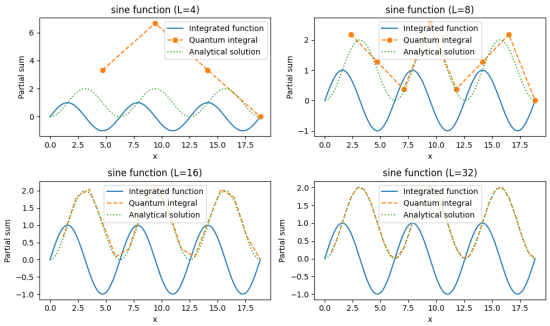
to pa je seveda točno vsota, ki jo izračuna zgoraj opisani algoritem, le pomnožena z določeno konstanto. Na slikah 14.2, 14.3, 14.4 in 14.5 lahko vidimo, kako se metoda obnese za nekaj izbranih funkcij in vrednosti L. Na prvih treh je rezultat primerjan z analitično rešitvijo, na zadnji pa s klasično izračunanim približkom dobljenim z metodo trapezov, saj analitična rešitev ne obstaja.



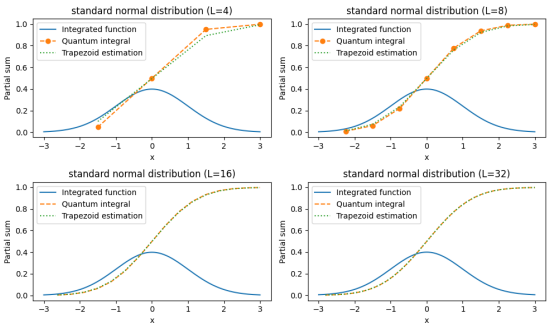
Slika 14.2 Numerična integracija funkcije $f(x) = x^2$ na intervalu $(0,1)$



Slika 14.3 Numerična integracija funkcije $f(x) = e^{-x}$ na intervalu $(0,5)$



Slika 14.4 Numerična integracija funkcije $f(x) = \sin(x)$ na intervalu $(0, 6\pi)$



Slika 14.5 Numerična integracija funkcije $f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$ na intervalu $(-3,3)$

14.5 Algoritem za računanje uteženih delnih vsot

A. Shukla in P. Vedula v [152] predstavita tudi algoritem za računanje nekaterih uteženih delnih vsot. Je posplošitev prvega algoritma, kjer vhodom dodamo še vektor $b = (b_0, ..., b_{k-1})$, pri čemer je $k+1$ število enic v binarnem zapisu M. Algoritem izračuna uteženo delno vsoto

$$|\tilde{f_0}\rangle = \sum_{r=0}^{k} \gamma_{k-r} \left(\sum_{j=S_{r-1}}^{S_r-1} f_j \right)$$

pri čemer je $S_r = \sum_{s=k-r}^{k} 2^{l_s}$ če je $0 \leq r \leq k$, sicer 0, Uteži γ pa so pridobljene na sledeč način:

$$\gamma_r = \begin{cases} \frac{b_0}{\sqrt{2^{l_0}}} & r = 0 \\ \frac{a_0 a_1 \ldots a_{r-1} b_r}{\sqrt{2^{l_r}}} & 0 < r < k \\ \frac{a_0 a_1 \ldots a_{k-1}}{\sqrt{2^{l_k}}} & r = k \end{cases}$$

Za poljubna a_r in $b_r \in \mathbb{R}$ mora veljati enačba $a_r^2 + b_r^2 = 1$.

Če algoritmu specifično podamo $b_0 = \sqrt{\frac{M_0}{M}}$ in $b_m = \sqrt{\frac{2^{l_m}}{M - M_{m-1}}}$ za ostale m, dobimo ravno prvi algoritem.

14.6 Kompleksnost

Poglejmo si kompleksnost vezij, ki pripadajo algoritmu 1. Naj bo M število členov, ki jih želimo sešteti in $N = 2^n$ število vseh členov. V primeru, ko je $M = 2^r$ za neko naravno število r, $1 \leq r \leq n$, velja, da bomo uporabili r Hadamard vrat. V primeru, kjer M ni tak, velja $M = \sum_{j=0}^{k} 2^{l_j}$, $0 \leq l_0 < \cdots < l_k \leq n-1$. V tem primeru potrebujemo l_k H ali CH vrat in $k-1$ X vrat in $k-1$ R_y ali CR_y vrat. S tem vidimo, da je število vrat navzgor omejeno z $\mathcal{O}(\log_2 M)$. V primeru, ko ne moremo implementirati kontroliranih vrat s pogojem $q = 0$, potem za vsaka takšna vrata potrebujemo še dvoja X vrata, kar pa ne vpljva na kompleksnost vezja. Vsa vrata, ki jih uporabljamo, so preprosta. Število qubitov, ki jih potrebujemo za implementacijo, je enako $\log_2 N = n$.

14.7 Zaključek

Spoznali smo algoritem za računanje delne vsote amplitud kvantnih stanj avtorjev A. Shukla in P. Vedula. Glavne lastnosti algoritma so zapis vrednosti direktno v amplitude vhodnih stanj in vračanje rezultata v amplitudi stanja qubita $|0\rangle$ na vhodnem registru. Algoritem je primeren za uporabo v bolj kompleksnih kvantnih pro-

Algoritem 5 Kvantni algoritem za računanje uteženih delnih vsot

Require: Normalizirano stanje $|f\rangle = \sum_{k=0}^{N-1} f_k |k\rangle$, naravni števili M in n ($2 \leq M \leq N = 2^n$), kjer $M \neq 2^r$ za noben $r \in \mathbb{Z}$, ter vektor $\mathbf{b} = [b_0, b_1, \ldots, b_{k-1}]$, kjer je $k+1$ enako številu enic v binarnem zapisu M.

Ensure: $\left|\widetilde{f}\right\rangle = \sum_{k=0}^{N-1} \widetilde{f}_k |k\rangle$, kjer $\widetilde{f}_0$ ustreza zahtevam.

procedure AMPLITUDEPARTIALSUM($|f\rangle, M, n$)
  Vhodno stanje $|f\rangle$ je zapisano na n-qubitnem registru R, z qubiti $q_{n-1}, \ldots, q_0$.
  ▷ Pozicije enic v obratnem binarnem zapisu števila M
  Izračunaj $l_0, l_1, \ldots, l_k$ tako, da $M = \sum_{j=0}^{k} 2^{l_j}$ in $0 \leq l_0 < l_1 < \cdots < l_k \leq n-1$.
  $M_{k-1} \leftarrow M - 2^{l_k}$
  for $m = k-1$ **downto** 1 **do**
    for $i = l_{m+1} - 1$ **downto** l_m **do**
      Izvedi kontrolirana Hadamardova vrata CH na $|q_i\rangle$ s pogojem $q_{l_{m+1}} = 0$
    end for
    $M_{m-1} \leftarrow M_m - 2^{l_m}$
    Izvedi kontrolirana vrata $CR_y(\theta_m)$ na $|q_{l_m}\rangle$ s pogojem $q_m = 0$
    kjer je $\theta_m = 2\arccos(b_m)$.
  end for
  for $i = l_1 - 1$ **downto** l_0 **do**
    Izvedi kontrolirana Hadamardova vrata CH na $|q_i\rangle$ s pogojem $q_{l_1} = 0$
  end for
  Izvedi vrata $R_y(\theta_0)$ na $\left|q_{l_0}\right\rangle$, kjer $\theta_0 = 2\arccos(b_0)$
  if $l_0 > 0$ **then**
    for $i = 0$ **to** $l_0 - 1$ **do**
      Izvedi vrata H na $|q_i\rangle$
    end for
  end if
  for $j = 1$ **to** k **do**
    Izvedi vrata X na $\left|q_{l_j}\right\rangle$
  end for
  ▷ Register R vsebuje stanje $\left|\widetilde{f}\right\rangle$
  return $\left|\widetilde{f}\right\rangle$
end procedure

cesih, nam hranjene rezultata v amplitudi kvantnega stanja omogoča nadaljnjo uporabo vrednosti v drugih kvantnih procesih, hkrati pa ne opisuje postpoka priprave začetnih stanj. Kljub temu vrednost delne vsote lahko preberemo z aproksimacijo amplitud. Algoritem je zmožen računanja delnih vsot za poljubno količino členov M, ne le potence števila 2, kot prej odkriti algoritmi. Algoritem uporablja $\mathcal{O}(\log_2 M)$ osnovnih vrat, kar je izboljšava glede na prej opisane metode. Poleg navadnega algoritma je predstavljena tudi njegova posplošitev, pri kateri gre za neenakomerno utežene vsote. Algoritem je primeren za uporabo v numeričnem računanju integralov in porazdelitvenih funkcij slučajnih spremenljivk. Poleg tega so za algoritem možne še druge posplošitve in prilagoditve za računanje bolj specifičnih vsot.

14.8 Doprinosi avtorjev

Domen Kastelic je napisal poglavja za numerično integracijo in opis posplošenega algoritma, implementiral numerično integracijo z uporabo dotičnega algoritma in jo analiziral.

Jaka Kovačič je napisal poglavja o zapisu podatkov in rezultatov, prvem algoritmu in opisal primer izvedbe in ga implementiral, Analiziral je tudi kompleksnost dobljenega vezja.

Poglavje 15
Implementacija D-latch tolerantnega na napake in s Fredkinovimi vrati v QCA

Manca Vavpotič in Blaž Grabnar

Povzetek V tej seminarski nalogi smo preučili, implementirali in testirali delovanje ter tolerantnost na napake D-latch ter drugih vezij potrebnih za izgradnjo le-tega v QCA tehnologiji z uporabo dveh različnih pristopov: z reverzibilnimi Fredkinovimi vrati ter D-latch toleranten na napake z multiplekserjem po zgledu iz dveh znanstvenih člankov.

15.1 Uvod

Kvantno-celična avtomatika (QCA) predstavlja eno najbolj obetavnih alternativ klasičnim CMOS tehnologijam, saj omogoča izjemno majhno porabo energije, visoko gostoto logičnih elementov in potencialno delovanje pri izjemno majhnih dimenzijah.

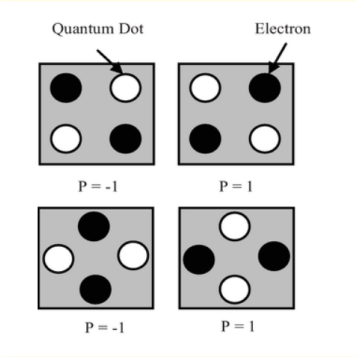
V tej seminarski nalogi smo preučili in implementirali dva različna pristopa k realizaciji osnovnega sekvenčnega elementa D-latch v QCA tehnologiji. Prvi pristop temelji na uporabi reverzibilnih Fredkinovih vrat, drugi pa na implementaciji z multiplekserjem, ki vključuje dodatne celice z namenom povečanja tolerance na napake. Oba pristopa temeljita na načrtih dveh izbranih znanstvenih člankov, ki služita kot osnovni zgled za implementacijo.

Na podlagi teh načrtov smo v okolju QCADesigner implementirali obe različici D-latcha, in elemente potrebne za izgradnjo D-latch ter izvedli simulacije njunega delovanja. V nadaljevanju predstavljamo tudi preizkus tolerance vezij na izbrane vrste napak in primerjamo dobljene rezultate z ugotovitvami iz literature.

15.2 Metode

15.2.1 QCA

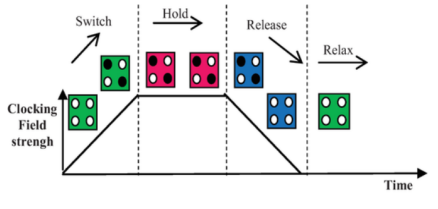
Osnovni gradnik QCA je t. i. QCA celica, ki predstavlja najmanjšo enoto v tej tehnologiji. Tipična QCA celica je sestavljena iz štirih kvantnih pik (angl. *quantum dots*) v kotih celice kvadratne oblike. V celico se vnese dva elektrona, ki se zaradi medsebojnega elektrostatskega odboja razporedita v stanje minimalne energije, tako da nista dva elektrona na istem robu kvadrata. Obstajata dve taki stabilni stanji, ki predstavljata dve polarizaciji celice, pogosto označeni kot $P = +1$ in $P = -1$. Tako dve stanji predstavljata dve logični vrednosti 1 in 0. [154]. Na sliki 15.1 so prikazane možne pozicije elektronov in polarizacija v QCA celici.



Slika 15.1: QCA celica in polarizacija glede na položaj elektronov (Vir: [154].)

Med večimi celicami je mogoč tudi prenos informacije, saj njihove polarizacije vplivajo druga na drugo. Na ta način lahko sprememba stanja ene celice povzroči spremembo stanja sosednjih celic, kar omogoča gradnjo logičnih vrat in prenos podatkov brez klasičnega toka električnega naboja. To pomeni, da QCA omogoča računske operacije z nizko porabo energije in potencialno visoko hitrostjo.

Pomemben del delovanja QCA je tudi *clocking* mehanizem. QCA ura nadzira prehod elektronov med kvantnimi pikami. S tem se celice izmenično "zaklepajo" in "odklepajo" in tako je premik elektronov v celici možen samo ob določeni urini periodi. To omogoča tudi stabilno propagacijo signala po verigi večih celic.[154] Clocking običajno poteka v štirih fazah v enem urinem ciklu: *Switch* (sprejem polarizacije), *Hold* (zaklep stanja), *Release* (sproščanje polarizacije) in *Relax* (priprava celice na novo vrednost). Grafična ponazoritev štirih časovnih faz je na sliki 15.2



Slika 15.2: Grafična ponazoritev štirih časovnih faz (Vir: [154].)

15.2.2 D-latch

D-latch je osnovno sekvenčno vezje, namenjeno shranjevanju enega bita podatka. Temelji na S-R zapahalu, vendar zaradi dodatne logike nima težave z nedovoljenim stanjem, ki nastopi, ko sta vhoda S in R hkrati enaka 1. Pri D-latchu se vhodna vrednost D prenese na izhod Q samo takrat, ko je signal (*Enable*, E) v logičnem stanju 1, ko je $E = 0$, izhod ostane nespremenjen ne glede na spremembe na vhodu D.

Obnašanje D-latcha je mogoče opisati s splošno enačbo:

$$Q_{n+1} = E \cdot D + E' \cdot Q_n,$$

kjer Q_{n+1} predstavlja naslednje stanje izhoda, Q_n prejšnje stanje, D vhodni podatek, E pa krmilni (*enable*) signal. Enačba ponazarja, da se pri aktivnem signalu $E = 1$ izhod posodobi na vrednost D, medtem ko pri $E = 0$ ohrani prejšnjo vrednost. [155]

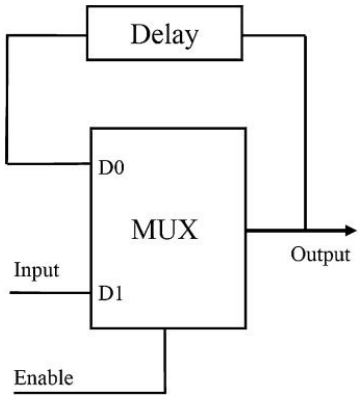
15.2.3 D-latch z multiplekserjem

Multiplekser (MUX) je vezje, ki omogoča izbiro enega izmed več vhodnih signalov in posredovanje tega na en izhod . Za izbiro, kateri vhod bo povezan na izhod, se uporabljajo kontrolne linije (*select lines*). Če ima multiplekser 2^n vhodov, potem ima n kontrolnih linij, katerih binarne vrednosti določajo izbran vhod. [156]

Ena izmed metod za realizacijo D-latcha v QCA tehnologiji temelji na uporabi 2:1 multiplekserja. Multiplekser izbere, ali naj se na izhod prenese trenutno vhodno stanje D, ali pa naj se ohrani obstoječe izhodno stanje Q. [157]

Ko je kontrolni signal *Enable* (E) v logičnem stanju 1, multiplekser izbere vhod D, zato se izhod D-latch posodobi. Ko pa je $E = 0$, multiplekser preklopi na prejšnje stanje izhoda, kar pomeni, da vezje ohrani svojo vrednost. [157]

Na sliki 15.3 je prikazan diagram D-latch z multiplekserjem.



Slika 15.3: Diagram D-latch z multiplekserskimi vrati (Vir: [157].)

15.2.4 D-latch s Fredkinovimi vrati

Fredkinova vrata (imenovana tudi controlled-SWAP ali conservative logic gate) so reverzibilna logična vrata. Vrata imajo tri vhode in tri izhode. Prvi vhod deluje kot kontrolni bit, ki se na izhod prenese nespremenjeno. Preostala dva vhodna bita se na izhodu zamenjata pod pogojem da je kontrolni bit enak 1; če je kontrolni bit 0, ostaneta bita nespremenjena. [158]

Funkcijsko obnašanje Fredkinovih vrat je mogoče izraziti z dvema nasproti postavljenima multiplekserjema 2:1. To pomeni, da se vrata obnašajo kot dva 2:1 digitalna multiplekserja, ki sta medsebojno zrcalno povezana. Uporaba Fredkinovih vrat omogoča tudi izvedbo reverzibilnega D-latch. Ker je osnovna funkcija D-latch podana kot:

$$Q_{n+1} = CLK \cdot D + CLK' \cdot Q_n,$$

je mogoče to funkcijo neposredno preslikati v vhodno–izhodno strukturo Fredkinovih vrat. V tej implementaciji kontrolni vhod predstavlja signal ure CLK, podatkovni vhod je vrednost D, medtem ko je tretji vhod povratna vrednost prejšnjega stanja Q_n. Izhod Fredkinovih vrat tako generira novo stanje Q_{n+1}, dodatna izhoda pa služita kot t. i. *garbage* izhoda, potrebna zaradi ohranjanja reverzibilnosti. [159]

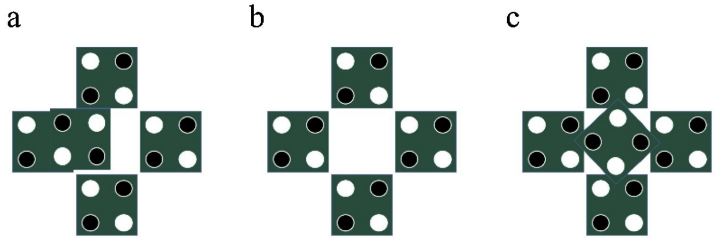
15.2.5 QCA napake

Toleranca na napake in zanesljivost so zelo pomembne pri načrtovanju robustnih QCA sistemov [157, 160]. Napake v QCA vezjih lahko nastanejo že med proizvodnjo celic ali med premikanjem celic na površino. Prva vrsta napak se pojavi med sintezo (*synthesis step*), druga pa med nanosom celic (*deposition step*). Zaradi zelo majhnih dimenzij QCA celic in zahtev po visoki natančnosti so napake med depo-

sition fazo pogoste. Tahoori et al. poročajo, da so napake pri nanosu celic, kot so manjkajoče celice ali napačno poravnane celice, bolj verjetne kot napake pri sintezi, saj je postavitev celic na natančno mesto zahtevna.

Napake QCA lahko razdelimo na tri osnovne tipe [160, 157]:

- **Cell misalignment (poravnava celic):** Celice niso pravilno poravnane glede na svoje sosede, kar lahko povzroči napake pri logičnem delovanju. Vsaka celica, ki je premaknjena za več kot polovico svoje širine, lahko povzroči okvaro. Primer je prikazan na Sliki 15.4(a).
- **Cell missing (manjkajoča celica):** Ena ali več celic manjka, kar povzroči napako manjkajoče celice. Primer je prikazan na Sliki 15.4(b).
- **Cell dislocation (premaknjena/rotirana celica):** Celica je rotirana ali premaknjena glede na sosednje celice, kar povzroči dislokacijsko napako. Primer je prikazan na Sliki 15.4(c).



Slika 15.4: Napake v QCA: (a) poravnava celic, (b) manjkajoča celica, (c) rotirana celica (Vir: [157].)

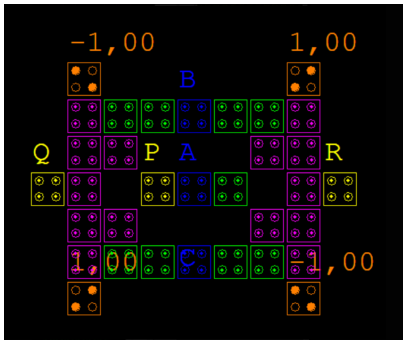
Da bi vezje še vedno delovalo pravilno z določeno toleranco do nekaga števila napak je mogoče spremeniti načrt vezja z dodajanjem redundantnih celic. [157] To je predstavljeno v implementaciji D-latch z multiplekserjem s toleranco na napake v članku Seyedi et. al. [157]

15.3 Implementacija in rezultati

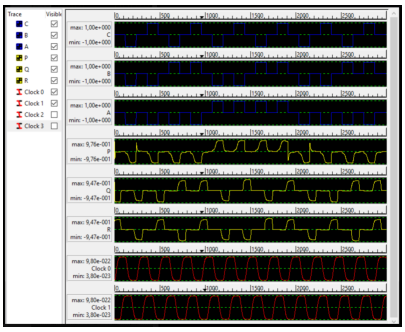
15.3.1 Implementacija D-latch z Fredkinovimi vrati

V QCADesignerju smo implementirali D-latch z uporabo Fredkinovih vrat. Pri tem smo sledili načrtu, predstavljenemu v članku Jeon et al. [159]. Naprej smo implementirali Fredkinova vrata po načrt iz članka. Implementacija in simulacija delovanja v QCADesignerju je predstavljena na sliki 15.5

Implementacija D-latch v QCADesignerju je prikazan na sliki 15.6. Vsebuje dve vhodni celici D in CLK ter tri izhodne. Dve izhodni celici G1 in G2 sta garbage



(a) Implementacija Fredkinovih vrat QCA-Designerju (Vir: lasten.)

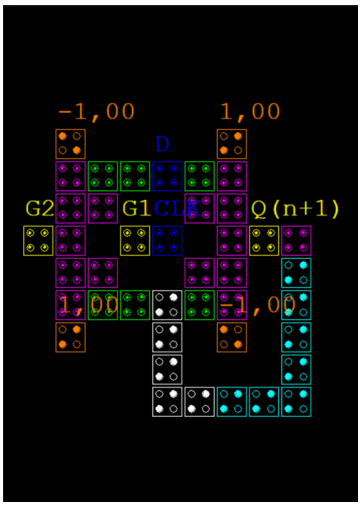


(b) Rezultat simulacije Fredkinovih vrat v QCADesignerju (Vir: lasten.)

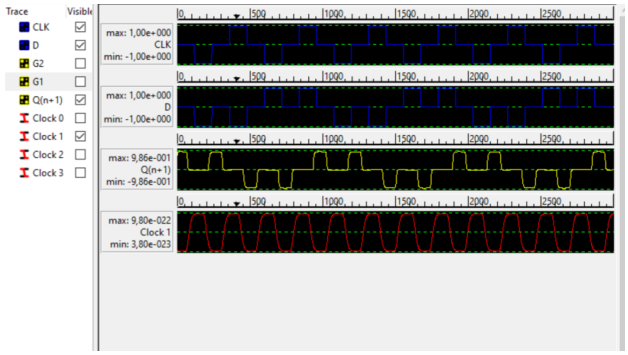
Slika 15.5: Implementacija in simulacija D-latch z Fredkinovimi vrati v QCADesignerju.

in nista pomembni razen zaradi reverzibilnosti. Glavni izhod, Q(n+1), predstavlja trenutno stanje D-latcha po obdelavi vhodnih signalov.

V QCADesignerju smo tudi izvedli simulacijo delovanja. Rezultati simulacije so prikazani na sliki 15.7. Ugotovili smo, da se rezultati ujemajo s simulacijskimi rezultati, predstavljenimi v članku Jeon et al. [159]. Pri vsakem kombinacijskem vhodu D in sinhronizacijskem signalu CLK se izhod Q(n+1) ustrezno posodobi.



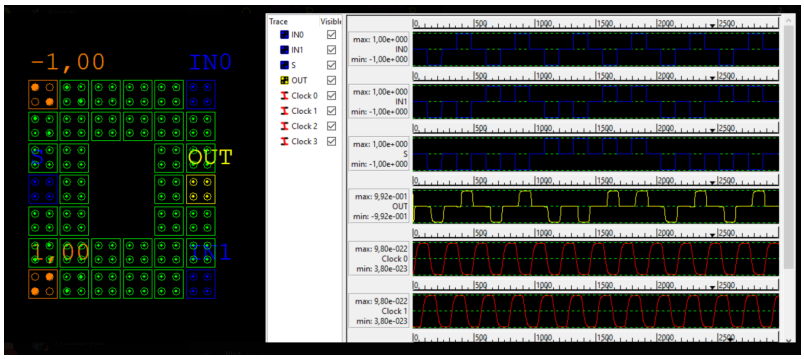
Slika 15.6: Implementacija D-latch z fredkinovimi vrati v QCADesignerju (Vir: lasten.)



Slika 15.7: Rezultat simulacije D-latch z Fredkinovimi vrati v QCADesignerju (Vir: lasten.)

15.3.2 Implementacija D-latch z multiplekserjem tolerantnega na napake

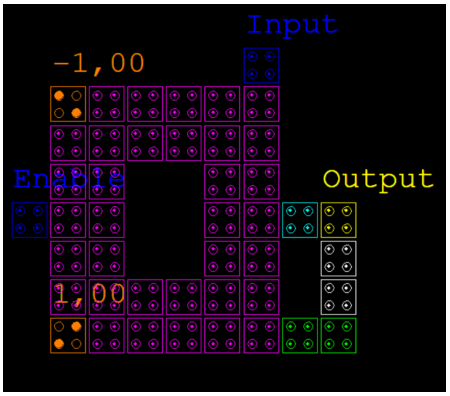
V QCADesigner smo implementirali D-latch tolerantnega na napake. Pri tem smo upoštevali načrt iz članka Saeid Seyedi et. al. [157]. Najprej smo implementirali multiplekser. Implementacija v QCADesignerju in simulacija delovanje multiplekserja je prikazana na sliki 15.8



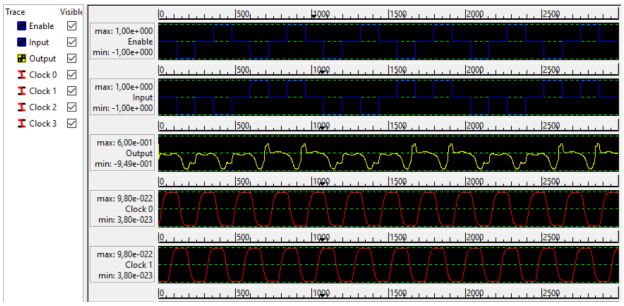
Slika 15.8: Implemantacija multiplekserja tolerantnega na napake in simulacija delovanja v QCADesignerju

Na sliki 15.9 je prikazana implementacija v QCADesignerju. Vhodni celici sta Input in Enable. Izhodna celica pa Output. V QCADesignerju smo izvedli tudi simulacijo delovanja. Rezultati simulacije so prikazani na sliki 15.10. Ugotovili smo, da se rezultati ne ujemajo s simulacijskimi rezultati, predstavljenimi v članku [157]. Na podlagi tega sklepamo, da razlog za odstopanja leži predvsem v drugačnem raz-

porejanju ure (*clocking scheme*), kot ga uporabljajo avtorji v članku Saeid Seyedi et.al. [157].



Slika 15.9: Implementacija D-latch tolerantnega na napake v QCADesignerju (Vir: lasten.)



Slika 15.10: Rezultat simulacije D-latch tolerantnega na napake v QCADesignerju (Vir: lasten.)

15.3.3 Testiranje napak

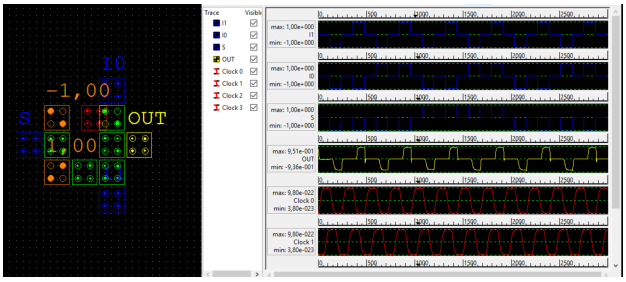
Glede na to, da so bila implementirana tako vezja brez tolerantnosti na napake kot tudi kot tudi ta z, je bilo potrebno preveriti tudi delovanje le-tega. Saeid Seyedi et. al. [157] opisuje, da je v članku predlagana verzija D-latch po kateri smo se zgledovali, toleranten na 86% napak, kjer manjka ena celica ter 67% za 2 manjkajoči celici.

Ker je že samo vezje, na napake tolerantnega D-latcha, prikazovalo napačne rezultate, še preden je bila dodana kakršna koli napaka, je bilo primerjava posvečena na napake tolerantnemu multiplekserju ter navadnem multiplekserju. Testirana so bila tudi vsa ostal vezja.

Testirani so bili vsi trije tipi napak: manjkajoča celica, premaknjena celica ter obrnjena celica.

V Saeid Seyedi et. al. [157] naj bi v članku predstavljen D-latch bil odporen na 78% napak pri premiku celic za manj kot 10 nm.

Čeprav so bili testi opravljeni na multiplekserju, pa so bili pričakovani podobni rezultati kot v članku [157] - MUX je seveda bolj enostaven kot D-latch.



Slika 15.11: Prikaz simulacije translacijske napake na navadnem MUX

Vsi testi premikanja celic so bili v rangu 1-11 nm translacije v eno izmed smeri.

Med navadnim MUX ter MUX tolerantnim na napake ni bilo razlik. Obe arhitekturi sta bili odporni na premik ene celice za 1nm, karkoli več pa je povzročilo napačno delovanje. Tudi odstranitev ali pa rotacija celice je povzročila napako na vezju.

15.3.4 Rezultati

Implementacija večine vezij je bila pravilna v primerjavi z opisom rezultatov simulacije v člankih. Le D-latch toleranten na napake ni pokazal istih rezultatov v delovanju na simulaciji kot je bilo opisano v članku.

Vsa ostala vezja so bila tudi testirana na vse tipe napak, ki se lahko pojavijo pri izdelavi. Napake tipa *misalignment*, *dislocation* ali *missing cell* so že pri minimalnih odstopanjih povzročile nepravilno delovanje. Osnoven D-latch, MUX ter FRG niso bili tolerantni na le-te, kar je bilo pričakovano. Prav tako se je izkazalo, da tudi MUX, predstavljen kot toleranten na napake, v simulacijah ni prenesel večjih odstopanj. Translacijske napake nad približno 1 nm, rotacijske napake ali odstranitev posamezne celice so v vseh primerih povzročile izgubo pravilne polarizacije in posledično nepravilno delovanje vezja.

15.4 Zaključek

V nalogi smo implementirali dve različni izvedbi D-latcha v QCA tehnologiji: izvedbo na osnovi Fredkinovih vrat ter izvedbo z multiplekserjem, ki naj bi bila odporna na napake. Implementacija vseh osnovnih vezij, vključno s Fredkinovimi vrati, standardnim multiplekserjem in osnovnim D-latchom, je delovala skladno s pričakovanji glede na članke. Tudi načrt na napake odpornega multiplekserja je bil ustrezno realiziran, medtem ko implementacija na napake odpornega D-latcha ni reproducirala rezultatov, prikazanih v članku.

Pri testiranju odpornosti na napake so vsa vezja, tako osnovna kot v članku opisana kot tolerantna, kazala podobno obnašanje: že minimalne translacijske napake nad 1 nm ter vsaka rotacija ali odstranitev celice so povzročile nepravilno delovanje. Nobeno izmed vezij ni doseglo stopnje robustnosti, ki jo poroča literatura.

Iz tega sklepava, da QCA designer, kljub ponujenim funkcionalnostim translacije ter rotacije, ni idealen za testiranje tolerantnosti na take napake ali pa je napaka v implementaciji na napake tolerantnega D-latch z multiplekserjem.

15.5 Doprinos avtorjev

Manca Vavpotič je naredila pregled obstoječega gradiva ter spisala večino poročila, Blaž Grabnar je opravil implementacijo, simulacijo ter testiranje za napake v QCA-Designerju, skupaj pa sva pripravila predstavitev ter slikovno gradivo.

Poglavje 16
Analiza in implementacija algoritma Bernstein-Vazirani

Andraž Hribernik, Timotej Kroflič, Jan Rutar

Povzetek V prispevku obravnavamo Bernstein–Vaziranijev problem in prikažemo razliko med klasičnim ter kvantnim pristopom k iskanju skritega binarnega niza. Po formalni opredelitvi problema najprej predstavimo klasični algoritem, ki za razkritje vseh bitov zahteva $O(n)$ poizvedb oraklja, nato matematično analiziramo kvantni algoritem, ki z izkoriščanjem superpozicije in interferenc skriti niz izračuna že z eno samo poizvedbo. V nadaljevanju opišemo implementacijo algoritma v Qiskitu ter izvedemo na IBM-jevem kvantnem računalniku. Rezultati potrjujejo teoretično učinkovitost algoritma za krajše nize, medtem ko se pri daljših (nad 32 kubitov) pokažejo omejitve zaradi napak in velikosti vezja. Na koncu izpostavimo praktično prednost kvantnega pristopa, obenem pa opozorimo na vpliv kvantnih napak v vezju na zanesljivost rezultatov.

16.1 Uvod

S pojavom kvantnega računalništva in kvantnega modela računanja, se je pojavila potreba po umestitvi kvantnih algoritmov v teorijo o izračunljivosti in računski kompleksnosti. Za potrebe njihove analize in umestitve obstaja kvantni Turingov stroj (QTM). Kvantni Turingov stroj je razvil in opisal Deutsch v svojem članku leta 1985 [161].

Algoritem Bernstein-Vazirani je kvantni algoritem, ki reši istoimenski problem in je opisan v poglavju 16.2.1. Je omejena verzija algoritma Deutsch–Jozsa. Algoritem je bil razvit za dokaz orakeljske razlike (angl. *oracle separation*) med računsko-kompleksnostnima razredoma **BQP** in **BPP** [162]. V tej seminarski nalogi se osredotočamo na analizo in implementacijo algoritma v orodju Qiskit in ne na pomen algoritma v teoriji računske zahtevnosti.

16.2 Metode

V tem poglavju najprej formalno opišemo obravnavani problem, nato predstavimo njegovo klasično rešitev ter, podrobno razložimo delovanje kvantnega algoritma Bernstein–Vazirani, na koncu pa še prikažemo implementacijo algoritma v jeziku Python s pomočjo knižnjice Qiskit.

16.2.1 Formalna definicija problema

Naj bo $N = 2^n$. Predstavljajmo si, da imamo vhodni vektor $x = (x_0, \ldots, x_{N-1}) \in \{0,1\}^N$. Indekse tega vektorja lahko zapišemo kot n-bitne nize $i \in \{0,1\}^n$.

Za vhodni vektor velja, da zanj obstaja neznan niz bitov $a \in \{0,1\}^n$, za katerega velja:

$$x_i = (i \cdot a) \pmod 2 \tag{16.1}$$

kjer $i \cdot a$ predstavlja skalarni produkt dveh binarnih nizov, definiran kot:

$$i \cdot a = \sum_{k=1}^{n} i_k a_k \pmod 2 \tag{16.2}$$

Tukaj i_k in a_k predstavljata k-ti bit niza i oziroma a [163].

Naš cilj je najti skriti niz a s čim manjšim številom poizvedb v črno škatlo (orakelj), ki nam za dani i vrne vrednost x_i.

16.2.2 Klasična rešitev

V klasičnem računalništvu lahko dostopamo do vrednosti x_i tako, da oraklju pošljemo niz i, ta pa nam vrne bit x_i. Ker je $x_i = i \cdot a$, vsaka poizvedba razkrije informacijo o linearni kombinaciji bitov niza a.

Če želimo natančno določiti celoten niz $a = a_1 a_2 \ldots a_n$, moramo ugotoviti vrednost vsakega posameznega bita a_k. Najbolj direktna metoda je poizvedovanje z nizi, ki imajo vrednost 1 natanko na enem mestu.

- Če pošljemo $i = 100\ldots0$, dobimo $x_i = 1 \cdot a_1 + 0 \cdot a_2 + \cdots = a_1$.
- Če pošljemo $i = 010\ldots0$, dobimo $x_i = 0 \cdot a_1 + 1 \cdot a_2 + \cdots = a_2$.
- In tako naprej do n-tega bita.

Za določitev celotnega n-bitnega niza a torej klasični algoritem nujno potrebuje n poizvedb. Vsaka posamezna poizvedba namreč vrne le en bit informacije.

16.2.3 Kvantni algoritem

Kvantni računalnik lahko ta problem reši z uporabo ene same poizvedbe, če uporabimo kvantno paralelizacijo in interferenco.

16.2.3.1 Kvantni orakelj

V kvantnem računalništvu so operacije unitarne in reverzibilne. Standardni orakelj za dostop do podatkov x je definiran kot unitarna preslikava na $n+1$ kubitih:

$$O_x : |i,b\rangle \rightarrow |i,b \oplus x_i\rangle \tag{16.3}$$

kjer je $i \in \{0,1\}^n$ naslovni register, $b \in \{0,1\}$ pa ciljni kubit [163].

Za algoritem Bernstein-Vazirani uporabljamo t.i. *fazni orakelj*. Če ciljni kubit nastavimo v stanje $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, se zgodi naslednje:

$$O_x(|i\rangle \otimes |-\rangle) = |i\rangle \otimes \frac{1}{\sqrt{2}}(|0 \oplus x_i\rangle - |1 \oplus x_i\rangle) \tag{16.4}$$

$$= |i\rangle \otimes (-1)^{x_i} |-\rangle \tag{16.5}$$

To pomeni, da se vrednost funkcije x_i preslika v fazo pred stanjem $|i\rangle$. Ta pojav imenujemo "phase kick-back". Za potrebe algoritma lahko ciljni kubit $|-\rangle$ v nadaljevanju ignoriramo in zapišemo delovanje oraklja na n kubitih kot [163]:

$$O_{x,\pm} |i\rangle = (-1)^{x_i} |i\rangle = (-1)^{i \cdot a} |i\rangle \tag{16.6}$$

16.2.3.2 Potek algoritma

Algoritem sledi standardni strukturi zgodnjih kvantnih algoritmov (kot je Deutsch-Jozsa) [163]:

1. Priprava začetnega stanja $|0\rangle^{\otimes n}$.
2. Uporaba Hadamardove transformacije na vseh kubitih.
3. Uporaba oraklja $O_{x,\pm}$.
4. Ponovna uporaba Hadamardove transformacije.
5. Meritev stanja.

16.2.3.3 Matematična analiza korakov

Začetno stanje in prva Hadamardova transformacija:

Začnemo v stanju $|0\rangle^{\otimes n}$ (vseh n kubitov je v stanju 0). Uporabimo Hadamardova vrata (H) na vsakem kubitu.

$$H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{i\in\{0,1\}^n} |i\rangle \tag{16.7}$$

To je enakomerna superpozicija vseh možnih n-bitnih nizov.

Uporaba oraklja:

Sedaj na tej superpoziciji uporabimo orakelj $O_{x,\pm}$. Kot smo izpeljali v razdelku o orakljih, ta operacija doda fazo $(-1)^{i\cdot a}$ vsakemu stanju $|i\rangle$:

$$|\psi_1\rangle = O_{x,\pm}\left(\frac{1}{\sqrt{2^n}}\sum_i |i\rangle\right) = \frac{1}{\sqrt{2^n}} \sum_{i\in\{0,1\}^n} (-1)^{x_i}|i\rangle \tag{16.8}$$

Upoštevajoč definicijo problema ($x_i = i\cdot a$), stanje zapišemo kot:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{i\in\{0,1\}^n} (-1)^{i\cdot a}|i\rangle \tag{16.9}$$

Druga Hadamardova transformacija:

To je ključni korak. Ponovno uporabimo $H^{\otimes n}$ na vseh kubitih. Splošna formula za delovanje Hadamardove transformacije na osnovnem stanju $|i\rangle$ je:

$$H^{\otimes n}|i\rangle = \frac{1}{\sqrt{2^n}} \sum_{j\in\{0,1\}^n} (-1)^{i\cdot j}|j\rangle \tag{16.10}$$

Če to uporabimo na našem stanju $|\psi_1\rangle$, dobimo:

$$|\psi_2\rangle = H^{\otimes n}|\psi_1\rangle = H^{\otimes n}\left(\frac{1}{\sqrt{2^n}}\sum_i (-1)^{i\cdot a}|i\rangle\right) \tag{16.11}$$

$$= \frac{1}{\sqrt{2^n}}\sum_i (-1)^{i\cdot a}\left(\frac{1}{\sqrt{2^n}}\sum_j (-1)^{i\cdot j}|j\rangle\right) \tag{16.12}$$

Sedaj zamenjamo vrstni red seštevanja in združimo eksponente (kjer je seštevanje v eksponentu modulo 2):

$$|\psi_2\rangle = \frac{1}{2^n}\sum_j |j\rangle\left(\sum_i (-1)^{i\cdot a+i\cdot j}\right) = \frac{1}{2^n}\sum_j |j\rangle\left(\sum_i (-1)^{i\cdot(a\oplus j)}\right) \tag{16.13}$$

Konstruktivna in destruktivna interferenca:

Analizirajmo vsoto v oklepaju: $\sum_{i\in\{0,1\}^n}(-1)^{i\cdot(a\oplus j)}$.

- **Primer $j=a$:** V tem primeru je $a\oplus j=0$ (ničelni vektor). Skalarni produkt $i\cdot 0$ je vedno 0, zato je $(-1)^0=1$. Vsota postane $\sum_i 1=2^n$.
- **Primer $j\neq a$:** V tem primeru je vektor $c=a\oplus j$ neničeln (ima vsaj en bit enak 1). Znano dejstvo iz linearne algebre nad $\mathbb{Z}_2$ je, da je za neničeln c točno polovica vektorjev i takih, da je $i\cdot c=0$, in polovica takih, da je $i\cdot c=1$. Prispevki $+1$ in -1 se v vsoti med seboj popolnoma izničijo. Vsota je 0.

  To lahko zapišemo s pomočjo Kroneckerjeve delte $\delta_{j,a}$:

$$\sum_i(-1)^{i\cdot(a\oplus j)}=2^n\delta_{j,a} \tag{16.14}$$

Če to vstavimo nazaj v enačbo za $|\psi_2\rangle$:

$$|\psi_2\rangle=\frac{1}{2^n}\sum_j|j\rangle\,(2^n\delta_{j,a})=|a\rangle \tag{16.15}$$

Končno stanje kvantnega registra je točno stanje $|a\rangle$.

16.2.3.4 Sklep

Po izvedbi druge Hadamardove transformacije se kvantni sistem nahaja v stanju $|a\rangle$. Ko izvedemo meritev v komputacijski bazi (standardni bazi), z verjetnostjo 1 (100%) izmerimo niz bitov, ki ustreza skritemu nizu a.

Povzetek učinkovitosti:

- **Klasično:** Za določitev n-bitnega niza a potrebujemo n poizvedb.
- **Kvantno:** Z algoritmom Bernstein-Vazirani skriti niz a določimo z **eno samo poizvedbo** oraklja.

To predstavlja enega najbolj čistih primerov, kjer kvantni računalnik omogoča natančno rekonstrukcijo globalne lastnosti funkcije (niz a) bistveno hitreje kot klasični računalnik, zahvaljujoč destruktivni interferenci vseh napačnih odgovorov in konstruktivni interferenci pravilnega odgovora.

16.2.4 Implementacija Bernstein-Vazirani algoritma

Algoritem Bernstein-Vazirani smo implementirali z uporabo knjižnice Qiskit v programskem jeziku Python. Za simulacijo kvantnega računalnika smo uporabili enega izmed kvantnih računalnikov, ki jih ponuja IBM. Izbrali smo računalnik z najkrajšo čakalno vrsto, v našem primeru je to bil IBM Fez, ki uporablja arhitekturo Heron r2.

```
from qiskit import QuantumCircuit
from qiskit.visualization import plot_histogram
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import SamplerV2 as Sampler

import random
import matplotlib.pyplot as plt
from collections import Counter

#dolzina skritega niza
stringLength = 10

#nastavitev kvantnega racunalnika
service = QiskitRuntimeService()
backend = service.least_busy(simulator=False, operational=True,
    min_num_qubits=stringLength+1)
print(backend)
sampler = Sampler(mode=backend)


#funkcija za definicijo bernstein-vazirani algoritma
def bernvase_circuit(s):
   #inicializacija vezja
   n = len(s)
   qc = QuantumCircuit(n + 1,n)


   qc.x(n)
   #uporaba Hadamardove transformacije
   qc.h(range(n+1))
   qc.barrier()

   #nastavitev vezja za oraklja
   for i,symbol in enumerate(reversed(s)):
      if symbol == '1':
         qc.cx(i,n)
   qc.barrier()

   #ponovna uporaba Hadamardove transformacije
   qc.h(range(n+1))
   qc.barrier()

   #Meritev stanja
   qc.measure(range(n),range(n))
```

```
    return qc

#funkcija za generiranje nakljucnega niza znakov '0' in '1'
def random01String(length):
    return ''.join(random.choice('01') for _ in range(length))

#generiranje niza
secret_string = random01String(stringLength)
print(f"The real secret string is: {secret_string}")

#izdelava vezja
qc = bernvase_circuit(secret_string)
fig = qc.draw("mpl")
fig.savefig('circuit.pdf')

#optimiziranje vezja za uporabo na kvantnem ra;unalniku
pm = generate_preset_pass_manager(backend=backend,
    optimization_level=1)
isa_circuit = pm.run(qc)
fig2 = isa_circuit.draw("mpl", idle_wires=False)
fig2.savefig('circuit2.pdf')

#izvajanje algoritma in pridobivanje podatkov
job = sampler.run([isa_circuit])
result = job.result()[0]
counts = result.join_data().get_counts()

#omejitev na 20 najbolj pogostih resitev
top_n = 20
top_counts = dict(Counter(counts).most_common(top_n))
fig3 = plot_histogram([top_counts])
plt.xticks(rotation=90)
plt.tight_layout()
fig3.savefig('result.pdf')

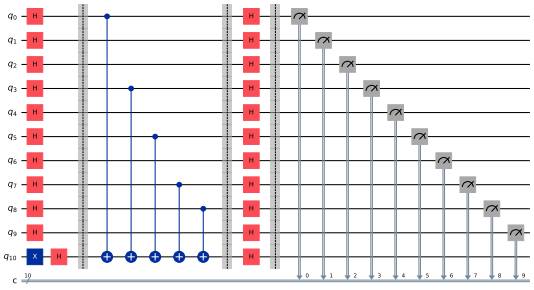
#izpis najbolj pogostega niza
counts_counter = Counter(counts)
most_common_result,_ = counts_counter.most_common(1)[0]
print(f"The found secret string is: {most_common_result}")
```

Najprej definiramo funkcija `bernvase_circuit(s)`, ki za podani skriti niz s ustvari ustrezno kvantno vezje. Dolžina niza določa število kubitov n, dodan pa je še ciljni kubit. Ta je inicializiran v stanju $|1\rangle$ z vrati X, nato se na vse kubite dodajo Hadamardova vrata, s čimer pripravimo superpozicijo vhodov.

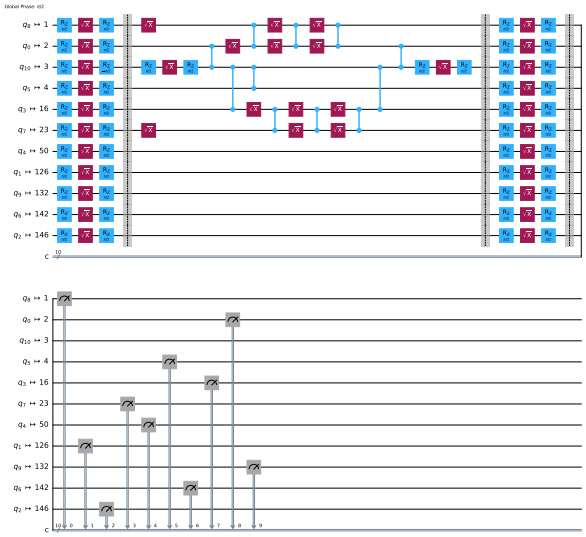
V drugem koraku funkcija zgradi orakel: za vsak bit v skritem nizu s preverimo, ali je enak 1. Če je, se na ustrezen podatkovni kubit in pomožni kubit uporabi operacija CNOT. S tem se ustvarijo fazni obrati, ki nosijo informacijo o skritem nizu.

Po oraklju se ponovno uporabijo Hadamardova vrata na prvih n kubitih, kar iz superpozicije izlušči skrivni niz. Nato se podatkovni kubiti izmerijo klasično. Rezultat meritve neposredno vrne binarni niz s.

Funkcija `random01String(length)` generira naključni niz znakov 0 in 1, podane dolžine `length`



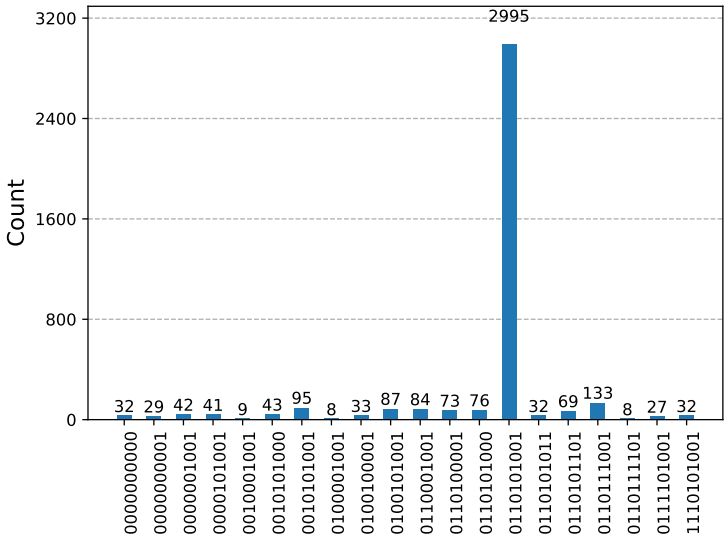
Slika 16.1: Shema vezja za neznan niz ’0110101001’



Slika 16.2: Shema vezja pripravljenega za izvajanje na kvantnem računalniku IBM Fez za neznan niz ’0110101001’

16.3 Rezultati

Za pridobitev rezultatov, se naše vezje izvede večkrat, saj lahko pride do napak pri delovanju kvantnega računalnika. Zato smo naš algoritem izvedli 4000-krat, naš rezultat pa je potem bil niz, ki smo ga največkrat našli. Za krajše dolžine niza to deluje pravilno, vendar pa se je pojavil problem pri izvajanju nizov daljših kot 32 bitov, oziroma uporabe 32 kubitov. Tukaj pride do napak zaradi velikosti vezja, in je tudi najbolj pogosto najden niz lahko nepravilen.



Slika 16.3: 20 najbolj pogostih najdenih nizov z njihovimi frekvencami pri neznanem nizu ’0110101001’

16.4 Zaključek

Najprej smo se spoznali s problemom Bernstein-Vazirani, njegovo definicijo in klasičnim ter kvantnim algoritmom, ki ga rešita. Nato smo implementirali in predstavili algoritem v orodju Quiskit. Na koncu smo prikazali in obrazložili rezultate o simulaciji izvedbi algoritma na kvantnem računalniku.

Iz napisanega opazimo, da je kvantna verzija veliko hitrejša za izvedbo, saj lahko v eni poizvedbi na orakelj izračuna skrivni niz s, kljub večkratni izvedbi algoritma na kvantnem računalniku zaradi napak, ki jih kvantni računalniki imajo med izvajanjem. Klasičen algoritem bi za to potreboval 2^n poizvedb na orakelj. Iz tega lahko sklepamo, da so kvantni algoritmi na kvantnih računalnikih časovno bolj učinkoviti

kot klasični algoritmi na klasičnih računalnikih, čeprav moramo ta algoritem zaradi napak na kvantnem računalniku večkrat pognati.

16.5 Doprinosi avtorjev

Timotej Kroflič je ustvaril Uvod, Zaključek, Povzetek in končno predstavitev, Andraž Hribernik je zapisal teoretični del poročila, Jan Rutar pa je ustvaril kodo in pripadajoči del poročila.

Poglavje 17
Zasnova in analiza pretvornika iz Grayeve v binarno kodo (G2B) v tehnologiji QCA

Javornik Anže in Matej Medved

Povzetek V seminarski nalogi obravnavamo zasnovo in implementacijo kombinacijskega vezja za pretvorbo Grayeve kode v binarni zapis (G2B). Za razliko od pretvorbe B2G, ki omogoča paralelno procesiranje, G2B zahteva serijsko obdelavo, kar v QCA tehnologiji predstavlja izziv sinhronizacije signalov in upravljanja latence. Uspešno smo implementirali 2-bitni in 3-bitni pretvornik v simulacijskem okolju QCADesigner. Pravilnost delovanja smo potrdili s simulacijo vseh vhodnih vektorjev in analizo časovnih diagramov. Prav tako smo analizirali delovanje že obstoječega obojestranskega pretvornika, kjer imamo možnost izbire naboja na celici, ki določa smer pretvarjanja (G2B ali B2G).

17.1 Uvod

Zmanjševanje velikosti tranzistorjev v uveljavljeni tehnologiji CMOS (Complementary Metal-Oxide-Semiconductor) se nezadržno bliža fizikalnim mejam, ki jih narekujejo kvantni pojavi in težave z odvajanjem toplote. Za nadaljevanje trenda rasti procesne moči je nujen prehod na nove računalniške paradigme. Ena izmed vodilnih nanotehnologij so kvantni celični avtomati (QCA), ki jih je leta 1993 predlagal Lent s sodelavci [144]. Za razliko od klasičnih vezij, ki za prenos informacije uporabljajo tok elektronov, QCA uporablja konfiguracijo naboja v stacionarnih celicah.

Pretvorniki kod so nepogrešljiv del digitalnih sistemov. Grayeva koda, kjer se sosednja števila razlikujejo le v enem bitu, se pogosto uporablja v senzorjih položaja (npr. rotacijski kodirniki) in za zmanjšanje napak v asinhronih sistemih. Vendar pa je za aritmetične operacije (seštevanje, množenje) podatke potrebno pretvoriti nazaj v standardni binarni zapis.

Cilj te naloge je zasnovati, implementirati in analizirati delovanje pretvornika iz Grayeve v binarno kodo (G2B) v tehnologiji QCA. Osredotočili se bomo na problem serijske odvisnosti bitov (ripple effect) pri 3-bitni izvedbi, ki jo bomo realizirali in preverili v orodju QCADesigner.

17.2 Teoretična izhodišča in metode

V tem poglavju predstavljamo osnovne gradnike tehnologije QCA, uporabljen pristop k načrtovanju ter logično osnovo pretvorbe.

17.2.1 QCA celica in vodilo

Osnovni gradnik je QCA celica, kvadratna nanostruktura s štirimi kvantnimi pikami na ogliščih in dvema prostima elektronoma. Zaradi Coulombovega odboja se elektrona vedno postavita diagonalno, kar vodi v dve stabilni stanji z minimalno energijo. Ti stanji definiramo kot polarizacijo $P = -1$ (logična 0) in $P = +1$ (logična 1). Informacija se med celicami prenaša s pomočjo elektrostatičnih interakcij, kar omogoča gradnjo QCA vodil.

17.2.2 Logična vrata

V tehnologiji QCA ne uporabljamo tranzistorjev. Osnovna logična vrata so:

1. **Večinska vrata (Majority Gate):** Imajo tri vhode (A, B, C) in en izhod. Izhod prevzame vrednost, ki jo ima večina vhodov. Logična funkcija je:

$$M(A,B,C) = AB + BC + AC \tag{17.1}$$

2. **Inverter (Negator):** Izkoristi lastnost diagonalnega odboja in razcepa žice, da obrne vrednost vhodnega signala.

S kombinacijo večinskih vrat in inverterjev gradimo kompleksnejša vrata, kot je XOR (izključni ALI), ki je temeljna komponenta G2B pretvornika.

17.2.3 Ura in sinhronizacija

Za usmerjanje pretoka informacij in zagotavljanje preklapljanja brez nenadnih skokov energije uporabljamo štirifazno uro (Switch, Hold, Release, Relax). Vezje razdelimo na cone (Clock 0, 1, 2, 3). Informacija potuje iz cone i v cono $(i+1) \pmod 4$. Pravilna razporeditev con preprečuje povratno potovanje signalov.

17.2.4 Metodologija načrtovanja

Pri snovanju vezja smo uporabili ad hoc metodo. To pomeni, da smo logična vrata postavljali in povezovali ročno, na podlagi logične sheme in intuicije, z namenom optimizacije prostora in zmanjšanja križanj vodil. Nismo uporabili avtomatiziranih orodij za sintezo. Glede na urno krmiljenje smo uporabili sistem strukturiranih pravokotnih urnih con. Celice so razporejene v vnaprej določene pravokotne regije, kjer vsaka regija pripada določeni fazi ure, kar zagotavlja robustnost signala.

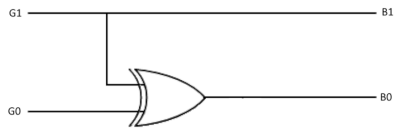
17.2.5 Logika G2B pretvorbe in serijska odvisnost

Pretvorba iz Grayeve (G) v binarno (B) kodo je rekurziven proces. Za N-bitno število velja:

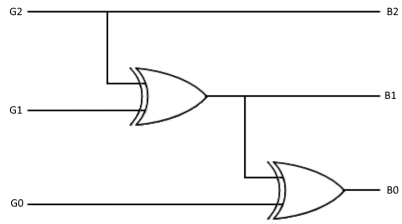
$$B_{N-1} = G_{N-1} \tag{17.2}$$

$$B_i = B_{i+1} \oplus G_i \quad \text{za} \quad i = N-2, \ldots, 0 \tag{17.3}$$

To implicira serijsko odvisnost (dependency chain) vidno na logični shemi 3-bitnega G2B (Slika 17.2) pretvornika. Za izračun B_i nujno potrebujemo že izračunan bit B_{i+1}. To v QCA vezjih povzroča t.i. "ripple effect", kjer se mora signal propagirati od najbolj pomembnega bita (MSB) do najmanj pomembnega (LSB). Posledično morajo biti vhodni signali G_i ustrezno časovno zakasnjeni (delay lines), da prispejo v logična vrata v istem urnem ciklu kot propagirani binarni bit.



Slika 17.1: Logična shema 2-bitnega G2B pretvornika



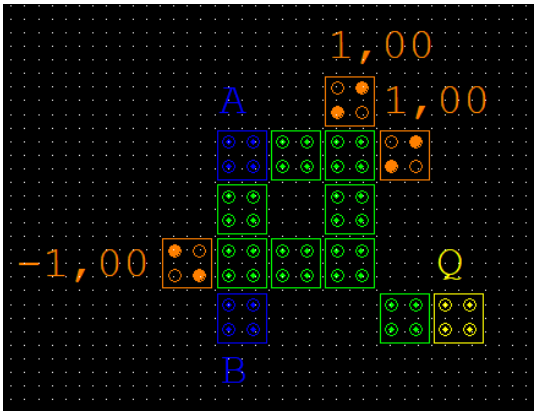
Slika 17.2: Logična shema 3-bitnega G2B pretvornika

17.3 Načrtovanje in implementacija

V okolju QCADesigner (verzija 2.0.3) smo uporabili standardne parametre: velikost celice 18 nm, razmik 2 nm, premer pik 5 nm.

17.3.1 Implementacija XOR vrat

Za operacijo A $\oplus$ B smo uporabili optimizirano kompaktno strukturo XOR vrat [164], vidno na sliki (Slika 17.3). Ta postavitev izkorišča geometrijsko interferenco in celice s fiksno polarizacijo ($P = -1$ in $P = 1$) za neposredno realizacijo logične funkcije.



Slika 17.3: Implementacija XOR vrat z QCA

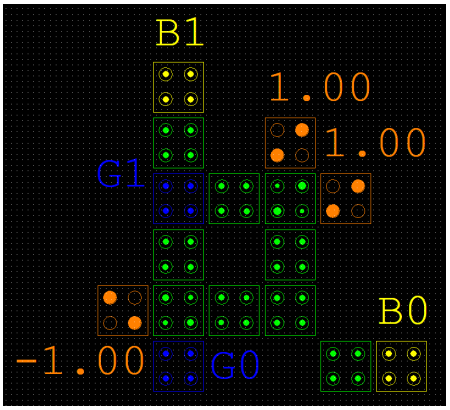
17.3.2 2-bitni G2B pretvornik

Za potrditev koncepta smo najprej realizirali 2-bitni pretvornik (Vhoda: G_1, G_0, Izhoda: B_1, B_0).

- $B_1 = G_1$ (neposredna povezava preko žice)
- $B_0 = B_1 \oplus G_0 = G_1 \oplus G_0$

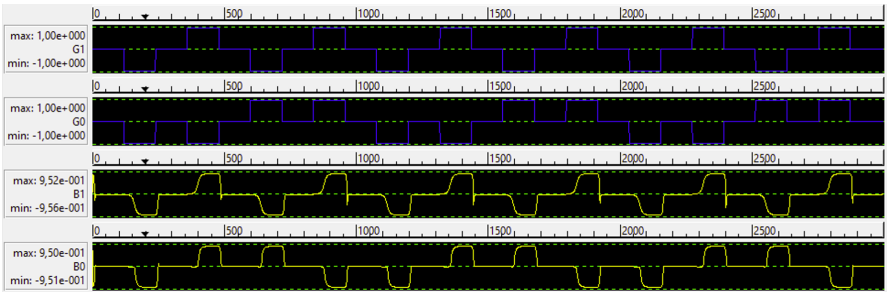
Tu sinhronizacija še ni kritična, saj še nimamo serijskih odvisnosti (vidno na Sliki 17.1) in je zamik minimalen.

Na Sliki 17.4 lahko vidimo direktno povezavo med G_1 in B_1 ter ena XOR vrata, ki vhoda G_1 in G_0 preslikajo v B_0.



Slika 17.4: Implementacija 2-bitnega G2B pretvornika.

Na grafu simulacije (Slika 17.5) lahko preberemo vse kombinacije vhodov in s pravilnostno tabelo 17.1 potrdimo pravilno delovanje pretvornika. Viden je tudi majhen zamik vrednosti na izhodu B0, ker vezje potrebuje več časa za prehod čez XOR vrata kot pri B_1, kjer je povezava skoraj neposredna.



Slika 17.5: Rezultati simulacije 2-bitnega G2B pretvornika

Tabela 17.1: Resničnostna tabela za 2-bitno G2B pretvorbo

Vhod (Gray)		Izhod (Binarno)		Decimalna
G_1	G_0	B_1	B_0	vrednost
0	0	0	0	0
0	1	0	1	1
1	1	1	0	2
1	0	1	1	3

17.3.3 3-bitni G2B pretvornik

Glavni del naloge je 3-bitni pretvornik z vhodi G_2, G_1, G_0 in izhodi B_2, B_1, B_0. Logika se razširi:

- $B_2 = G_2$
- $B_1 = B_2 \oplus G_1$
- $B_0 = B_1 \oplus G_0$

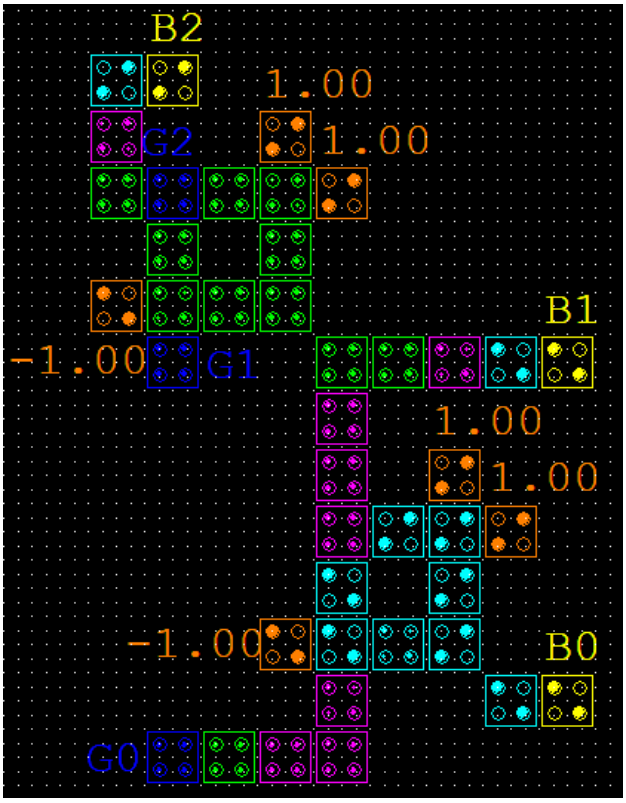
Za lažje razumevanje strukture vezja in poteka signalov prikazujemo logično shemo 3-bitnega pretvornika na sliki 17.2. Shema prikazuje kaskadno vezavo XOR vrat in nakazuje potrebno zakasnitev signala G_0.

Pri implementaciji smo naleteli na izziv sinhronizacije. Signal B_1, ki je potreben za izračun B_0, je na voljo šele po določeni zakasnitvi (približno 1 urna cona po prehodu prvih XOR vrat). Če bi signal G_0 pripeljali neposredno, bi prehitel signal B_1 in izhod bi bil napačen. Rešitev je bila uvedba zakasnilne linije za signal G_0. Vodilo za G_0 smo namenoma podaljšali skozi več urinih con, da je prispela do drugih XOR vrat sočasno z izračunanim B_1.

Na izhoda B_2 in B_1 smo prav tako dodali dodatne urine cone, da lahko izhode beremo hkrati in ni večjega zamika med njimi. To se vidi na Sliki 17.6, ko sta pred izhodoma B_2 in B_1 dodani roza in modra celica. Pravilnost delovanja za vse kombinacije prikazuje tabela 17.2.

Tabela 17.2: Resničnostna tabela za 3-bitno G2B pretvorbo

Vhod (Gray)			Izhod (Binarno)			Decimalna
G_2	G_1	G_0	B_2	B_1	B_0	vrednost
0	0	0	0	0	0	0
0	0	1	0	0	1	1
0	1	1	0	1	0	2
0	1	0	0	1	1	3
1	1	0	1	0	0	4
1	1	1	1	0	1	5
1	0	1	1	1	0	6
1	0	0	1	1	1	7



Slika 17.6: Končna postavitev celic 3-bitnega G2B pretvornika. Različne barve predstavljajo različne urine cone. Opazna je daljša pot signala G_0 za sinhronizacijo.

17.4 Dvosmerni pretvornik z izbiro SEL (B2G/G2B)

Za primerjavo z lastno zasnovo 3-bitnega G2B pretvornika sva dodatno analizirala še dvosmerni pretvornik iz članka "QCA-based 3-bit binary to gray and vice versa code converter" [165], ki v enem vezju omogoča pretvorbo iz binarne v Grayevo kodo (B2G) in obratno (G2B). Vhodni signali so označeni z A, B, C, izhodni z P, Q, R, signal *SEL* pa določa smer pretvorbe.

Logične funkcije izhodov lahko zapišemo ločeno za oba načina delovanja:

$$\text{če SEL} = 0 \quad (\text{B2G}): \quad \begin{cases} P = A \\ Q = A \oplus B \\ R = B \oplus C \end{cases} \tag{17.4}$$

$$\text{če SEL} = 1 \quad (\text{G2B}): \quad \begin{cases} P = A \\ Q = A \oplus B \\ R = A \oplus B \oplus C \end{cases} \tag{17.5}$$

V obeh primerih se prvi bit samo prenese na izhod ($P = A$), drugi bit nastane z XOR operacijo $A \oplus B$, tretji bit pa je odvisen od izbrane smeri pretvorbe. Prednost takega pristopa je, da isti blok XOR vrat realizira večino logike, izbira pretvorbe pa se določi zgolj z enobitnim signalom *SEL*.

Implementacija in metrika vezja

V QCADesignerju sva implementirala postavitev, ki sledi objavljeni shemi nereverzibilnega vezja. Vezje uporablja kaskadno vezavo XOR vrat in eno dodatno strukturo za vnos signala *SEL*, ki vpliva na izhod *R*, medtem ko izhoda *P* in *Q* ostaneta enaka za oba načina delovanja. Postavitev je zasnovana znotraj štirifaznih urnih con, podobno kot pri najinem 3-bitnem G2B pretvorniku, tako da se signal od vhoda do izhoda propagira v enem urnem ciklu.

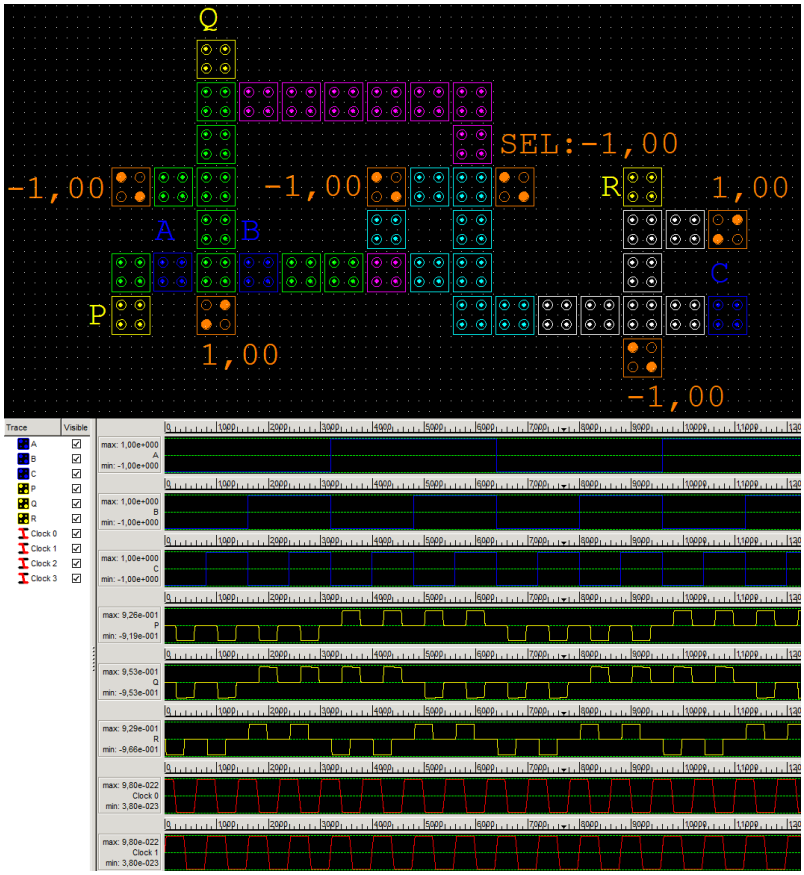
Število celic, površina in latenca so primerljive z objavljenimi rezultati:

- število celic: 44,
- površina: $\approx 0{,}048\,\mu\text{m}^2$,
- latenca: 1 urni cikel.

V primerjavi z najino samostojno G2B implementacijo je vezje nekoliko večje, vendar v zameno omogoča pretvorbo v obe smeri.

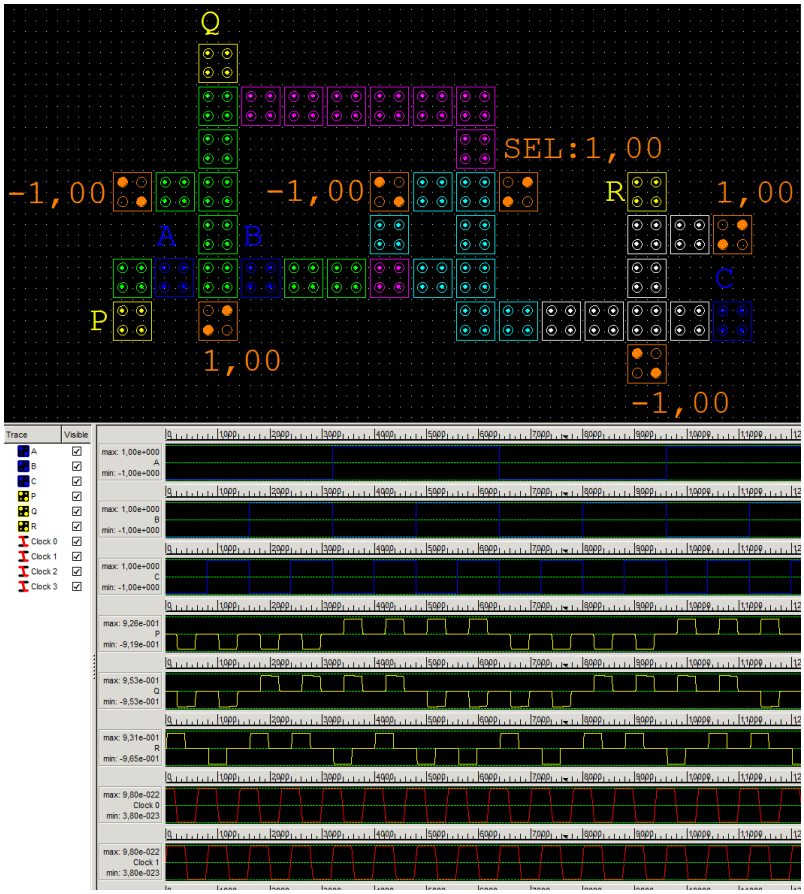
Rezultati simulacije za SEL = 0 in SEL = 1

Na Sliki 17.7 je prikazana postavitev vezja z rezultati simulacije za primer, ko je *SEL* nastavljen na 0 (B2G pretvorba). Na časovnem diagramu se vidi, da zaporedne kombinacije vhodov A,B,C ustrezno preslikajo v izhode P,Q,R v skladu z enačbo (17.4). Majhen zamik med vhodom in izhodom odgovarja enemu urnemu ciklu, ki ga vezje potrebuje, da se signal stabilizira na izhodnih celicah.



Slika 17.7: Postavitev in časovni diagram dvosmernega pretvornika pri $SEL = 0$ (delovanje kot 3-bitni B2G pretvornik).

Na Sliki 17.8 je prikazan enak preizkus pri $SEL = 1$ (G2B pretvorba). Tokrat izhod R sledi enačbi $R = A \oplus B \oplus C$, kar je lepo razvidno iz primerjave s sledi signalov A, B, C in Q. Preverili sva vse kombinacije vhodov ter jih primerjala z resničnostno tabelo iz članka; v vseh primerih je simulacija dala pravilne izhode.



Slika 17.8: Postavitev in časovni diagram dvosmernega pretvornika pri $SEL = 1$ (delovanje kot 3-bitni G2B pretvornik).

Dvosmerni pretvornik z izbiro *SEL* tako lepo dopolni najino analizo: prikazuje kompromis med kompaktnim, namensko optimiziranim G2B pretvornikom in splošnejšim vezjem, ki omogoča pretvorbo v obe smeri, vendar zahteva nekoliko večjo površino in dodatno logiko za izbiro smeri.

17.5 Rezultati in diskusija

17.5.1 Simulacija delovanja

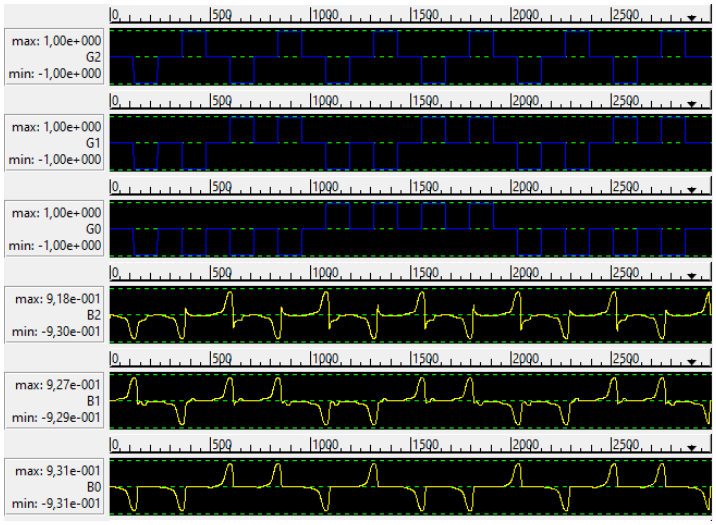
Rezultati simulacije za 3-bitni pretvornik so prikazani na Sliki 17.9. Diagram potrjuje pravilnost logike. Zamik med vhodom in stabilnim izhodom znaša približno 1 urin cikel.

Primer: Ob času t = 650 so vhodi nastavljeni na 010 (Gray), izhod po preteku latence pokaže stanje 011 (Binarno).

17.5.2 Analiza karakteristik lastnega vezja

V tabeli 17.3 smo zbrali ključne parametre implementiranega vezja. Primerjamo ga z osnovnim 2-bitnim modelom.

Opazimo, da se s številom bitov povečuje latenca. To je inherentna lastnost G2B algoritma. V CMOS tehnologiji bi to reševali z manjšimi tranzistorji, v QCA pa je gostota že tako visoka, da je glavni izziv časovna usklajenost in ne sam prostor.



Slika 17.9: Časovni diagram simulacije. Prikazani so vhodi G in ustrezni zakasnjeni izhodi B.

Tabela 17.3: Primerjava karakteristik implementiranih pretvornikov

Metrika	2-bitni G2B	3-bitni G2B
Število celic	16	41
Površina (μm^2)	0.0144	0.0468
Latenca (št. urnih ciklov)	0.5	1
Metoda snovanja	Ad hoc	Ad hoc

Primerjava 3-bit G2B pretvornika in dvosmernega B2G/G2B vezja

Tabela 17.4: Primerjava med našim 3-bitnim G2B pretvornikom in dvosmernim B2G/G2B vezjem iz članka

Metrika	3-bitni G2B	B2G/G2B (iz članka)
Število celic	41	44
Površina [μm^2]	0.0468	0.048
Latenca [urni cikli]	1	1
Smer pretvorbe	samo G2B	B2G in G2B (SEL)
Dodatna logika	/	selektor SEL

Površini obeh vezij sta primerljivi, ključna razlika je v funkcionalnosti: naš G2B pretvornik je primeren, kadar potrebujemo samo pretvorbo iz Grayeve v binarno kodo, medtem ko dvosmerni pretvornik omogoča obojestransko delovanje (B2G in G2B) brez znatnega povečanja kompleksnosti. Zaradi tega je razmerje funkcionalnost/kompaktnost pri dvosmernem pretvorniku objektivno boljše, a za ceno manj stabilnega delovanja. Vezje namreč ni pri vsaki simulaciji proizvedlo želenih rezultatov, ampak se je včasih obnašalo nestabilno. Moteč je bil prevsem R izhod, ki ni proizvajal konsistentnih rezultatov.

17.6 Zaključek

V seminarski nalogi smo predstavili zasnovo pretvornika iz Grayeve v binarno kodo v nanotehnologiji QCA. Uspešno smo implementirali in verificirali 2-bitni in 3-bitni model. Ključna ugotovitev je, da čeprav QCA omogoča izjemno zmanjšanje vezij, rekurzivni algoritmi (kot je G2B pretvorba) zahtevajo pazljivo načrtovanje urinih con za sinhronizacijo signalov, saj se napaka v časovnem zamiku propagira na vse nižje bite.

Pomembno je poudariti, da je naša implementacija ireverzibilna. Uporabljena XOR vrata in večinska vrata (z določenimi fiksnimi vhodi) ne ohranjajo informacijske entropije, kar pomeni, da iz izhodov ne moremo enolično rekonstruirati vhodov brez poznavanja notranjih stanj. To vodi v disipacijo energije (po Landauerjevem

principu). Za prihodnje delo in uporabo v pravih kvantnih sistemih bi bilo smiselno vezje predelati v reverzibilno obliko z uporabo Feynmanovih (CNOT) ali Toffolijevih vrat, ki bi omogočala pretvorbo brez izgube energije.

Doprinosi avtorjev

- **Matej Medved:** Zasnova in analiza lastne verzije 2 in 3-bitnega G2B pretvornika, pisanje poročila in izdelava predstavitve.
- **Javornik Anže:** Analiza obstoječega dvosmernega pretvornika, z možnostjo izbire smeri pretvorbe s celico select, pisanje poročila in izdelava predstavitve.

Poglavje 18
Implementacija in analiza Shorovega algoritma za faktorizacijo števil

Blaž Grilj, Ana Poklukar

Povzetek V seminarski nalogi obravnavamo Shorov algoritem kot kvantni pristop k faktorizaciji celih števil in njegove posledice za kriptografske sisteme, kot je RSA. Na kratko predstavimo problem faktorizacije in teoretično ozadje algoritma, zlasti vlogo kvantne Fourierjeve transformacije pri iskanju periode. Nato opišemo implementacijo v Pythonu z uporabo Qiskita ter eksperimentalno preizkusimo algoritem na idealnem simulatorju in IBM kvantnem procesorju. Analiziramo vpliv parametra a, števila ponovitev in velikosti N ter rezultate primerjamo z brute-force pristopom in Pollardovim ρ-algoritmom. Pokazali smo, da kvantna implementacija kljub teoretični učinkovitosti uspešno zazna periodičnost za večja števila, vendar njeno delovanje v praksi omejuje šum in majhno število stabilnih kubitov.

18.1 Uvod

Kvantno računalništvo predstavlja eno najperspektivnejših smeri nadaljnjega razvoja procesnih tehnologij, saj obljublja reševanje problemov, ki so za klasične računalnike izračunljivo zelo zahtevni ali praktično nerešljivi. Med najpomembnejšimi kvantnimi algoritmi izstopa Shorov algoritem, ki omogoča eksponentno hitrejšo faktorizacijo velikih kompozitnih števil. Ker varnost sodobnih kriptografskih sistemov, kot je RSA [166], neposredno temelji na težavnosti faktorizacije, ima Shorov algoritem pomembne posledice za področje informacijske varnosti.

V tej seminarski nalogi raziskujemo Shorov algoritem z vidika teoretičnih temeljev, implementacije in praktične uporabnosti v trenutnem stanju kvantne tehnologije. Najprej predstavimo problem faktorizacije celih števil in teoretično ozadje Shorovega algoritma, nato opišemo našo implementacijo v okolju Qiskit [137] ter zasnovo eksperimentalnih preizkusov. V nadaljevanju analiziramo rezultate, pridobljene na idealnem simulatorju in na pravem IBM kvantnem procesorju [167], ter jih primerjamo z izbranimi klasičnimi metodami faktorizacije. Na koncu povzamemo ključne ugotovitve in izpostavimo omejitve trenutnega pristopa.

18.2 Metode

To poglavje podrobno predstavlja matematične osnove, uporabljene algoritme ter praktične postopke, ki so bili uporabljeni pri analizi, implementaciji in eksperimentalnem preizkušanju Shorovega algoritma.

18.2.1 Problem faktorizacije celih števil

Problem faktorizacije celih števil obravnava razcep sestavljenega števila N na njegova praštevilska faktorja. Čeprav je naloga na prvi pogled preprosta, postane izračunsko zahtevna, ko je N zelo veliko (npr. 1024-bitno ali 2048-bitno število). Klasični računalniki ne poznajo polinomsko hitre metode za faktorizacijo splošnih celih števil, zato je ta problem temelj številnih kriptografskih sistemov, med katerimi je najpomembnejši RSA.

Velika sestavljena števila se v kriptografiji uporabljajo kot del javnega ključa. V RSA-ju je javni ključ sestavljen iz para (N, e), kjer je $N = pq$ produkt dveh velikih praštevil, medtem ko sta p in q skrita. Varnost sistema temelji na tem, da je iz javnega ključa enostavno izvajati šifriranje, medtem ko je dešifriranje možno le, če poznamo zasebni ključ, ki je odvisen od faktorjev p in q. Ker je faktorizacija velikega N izjemno težavna, lahko sistemi varno uporabljajo tudi 2048-bitne ali 4096-bitne ključe, pri čemer bi klasični računalniki za razcep takih števil potrebovali veliko časa.

V praksi se danes uporabljajo različni klasični algoritmi za faktorizacijo, kot so preizkušanje deliteljev (brute-force), Pollardov ρ-algoritem [168], kvadratna sita [169] in GNFS [170]. Ti algoritmi imajo različne prednosti, vendar vsi za dovolj velike vrednosti N postanejo časovno neučinkoviti. Shorov algoritem predstavlja radikalno drugačen, kvantni pristop k faktorizaciji, saj problem reducira na iskanje periode in ga reši v polinomskem času. Zaradi njegove učinkovitosti bi dovolj velik kvantni računalnik, zmožen poganjati Shorov algoritem, ogrozil večino danes uporabljenih kriptografskih sistemov, kar je glavni razlog za intenzivno raziskovanje in razumevanje tega algoritma.

18.2.2 Teoretično ozadje Shorovega algoritma

Shorov algoritem, ki ga je leta 1994 predstavil Peter W. Shor [23], predstavlja enega najpomembnejših mejnikov v razvoju kvantnega računalništva, saj omogoča faktorizacijo velikih sestavljenih celih števil v polinomskem času. Ker varnost kriptografskih shem, kot je RSA, temelji na težavnosti faktorizacije, ima Shorov algoritem izjemen teoretični in praktični pomen.

Shorov algoritem temelji na preoblikovanju problema faktorizacije v problem iskanja periodičnosti. Naj bo N sestavljeno število, ki ga želimo razstaviti na prafak-

torje. Izberemo naključno število a, za katerega velja $1 < a < N$ ter $\gcd(a,N) = 1$. Definiramo funkcijo

$$f(x) = a^x \bmod N,$$

ki je periodična z najmanjšo pozitivno periodo r, tako da velja $a^r \equiv 1 \pmod{N}$. Če je r soda, lahko iz periodičnosti izpeljemo prafaktorja števila N z izračunom

$$p_1 = \gcd(a^{r/2} - 1, N), \qquad p_2 = \gcd(a^{r/2} + 1, N),$$

pri čemer bo z visoko verjetnostjo vsaj eden od teh izrazov dal netrivialen prafaktor. Če pa je r liho število, metoda ne deluje, saj $r/2$ ni celo število, zato je treba izbrati novo vrednost a in postopek ponoviti.

Ključni prispevek Shorovega algoritma je uporaba kvantne Fourierjeve transformacije (*ang.* Quantum Fourier Transform ali QFT), ki omogoča iskanje periode r v polinomskem času. Kvantna Fourierjeva transformacija v prostoru dimenzije 2^n, kjer je n število uporabljenih kvantnih bitov, je definirana kot linearna transformacija

$$\mathrm{QFT}_{2^n} : |x\rangle \mapsto \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i x k / 2^n} |k\rangle .$$

QFT preslika stanje $|x\rangle$ v valovno obliko, kjer superpozicija vseh baznih stanj $|k\rangle$ vsebuje faze, odvisne od produkta xk. Ti fazni premiki ustvarijo interferenčni vzorec, iz katerega lahko z veliko verjetnostjo razberemo periodo funkcije $f(x)$. Ključni razlog, da to deluje, je dejstvo, da se periodičnost funkcije $f(x)$ odraža v periodičnih amplitudah prvega registra, na katere QFT zelo učinkovito reagira.

Ko enkrat z uporabo kvantne Fourierjeve transformacije dobimo oceno periode r, lahko preostanek postopka izvedemo klasično. Če je r sodo in velja $a^{r/2} \not\equiv -1 \pmod{N}$, izračunamo vrednosti $\gcd(a^{r/2} - 1, N)$ in $\gcd(a^{r/2} + 1, N)$, ki z visoko verjetnostjo vrneta netrivialna prafaktorja števila N. Če dobimo liho periodo ali neuporaben rezultat, izberemo nov a in postopek ponovimo, dokler ne najdemo primerne periode.

Časovna zahtevnost celotnega algoritma izhaja iz kvantnega dela, ki vključuje modularno potenciranje in kvantno Fourierjevo transformacijo. Ti operaciji zahtevata največ $\mathcal{O}(n^3)$ časa, kjer je $n = \log_2 N$. To pomeni, da Shorov algoritem faktorizira števila v polinomskem času, kar je izjemno izboljšanje v primerjavi s klasičnimi metodami, ki imajo lahko eksponentno zahtevnost.

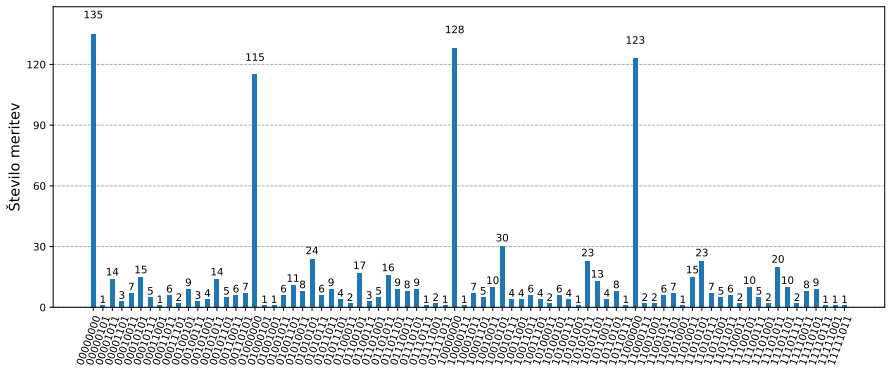
18.2.3 Opis implementacije

V tem podpoglavju je predstavljena implementacija Shorovega algoritma za faktorizacijo števila N.

Program je napisan v Pythonu in uporablja ogrodje Qiskit, ki omogoča gradnjo in izvajanje kvantnih vezij tako na simulatorjih kot tudi na dejanskih IBM kvantnih procesorjih. Implementacija združuje kvantni del in klasično obdelavo rezultatov.

Koraki implementacije so naslednji: najprej naključno izberemo število a, ki je med 2 in $N-1$. Takoj preverimo največji skupni delitelj d. Če je $d \neq 1$, smo že našli netrivialni faktor števila N in ga vrnemo kot rešitev. Če $d = 1$, nadaljujemo z gradnjo kvantnega vezja za ocenjevanje faze (*ang.* Quantum Phase Estimation ali QPE) z modularnim množenjem izbranega števila a.

Vezje nato izvedemo na simulatorju ali kvantnem računalniku in pridobimo histogram izmerjenih rezultatov. V simulaciji je ta histogram dostopen v celoti, saj simulator izvede veliko neodvisnih meritev in nam vrne celotno porazdelitev verjetnosti. V izvirnem Shorovem algoritmu pa QPE vrne le eno samo meritev, saj je kvantno vezje mogoče izvesti le enkrat na dejanski napravi, rezultat pa je posamezna bitna vrednost. Dostop do celotne porazdelitve (histograma) je zato umetna prednost simulacije in omogoča podrobnejšo analizo statistike meritev. Primer porazdelitve meritev za naključno izbran a in $N = 15$ pri `shots` = 1024 je prikazan na sliki 18.1, kjer so jasno vidni vrhovi, ki ustrezajo približkom periode.



Slika 18.1: Histogram izmerjenih vrednosti iz simulacije QPE.

V naši implementaciji zato iz histograma izberemo meritev z največjo verjetnostjo, ki predstavlja najbolj stabilen približek izmerjene faze. Iz te meritve pretvorimo bitni niz v decimalno vrednost in na podlagi nje s pomočjo nadaljevalnih ulomkov izračunamo kandidat za periodo zaporedja. Če je dobljena perioda smiselna, izračunamo dva možna delitelja števila N na osnovi polovice periode. Če sta oba delitelja netrivialna, smo uspešno razcepili število N. Če kandidat za periodo ne vodi do uporabnih deliteljev, postopek označimo kot neuspešen. Psevdokoda korakov implementacije je prikazana v algoritmu 6.

18.2.3.1 Konstrukcija kvantnega vezja

V nadaljevanju na kratko, a nekoliko podrobneje razložimo kvantni del implementacije Shorovega algoritma.

Algoritem 6 Shorov algoritem

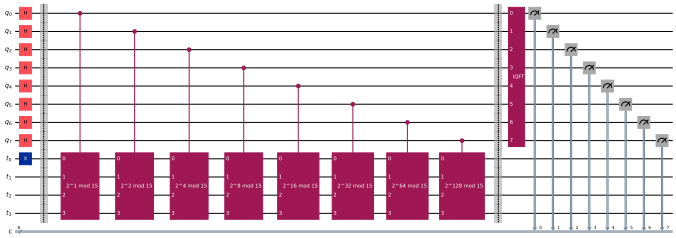
izberi naključno število $2 \leq a < N$, d = gcd(a, N)
if $d \neq 1$ **then**
    return d
end if
zgradi kvantno vezje za fazno ocenjevanje z modularnim množenjem števila a
zaženi vezje in pridobi histogram meritev
izberi meritev z največjo verjetnostjo
pretvori meritev v decimalno vrednost
izračunaj periodo
if je dobljena perioda ustrezna **then**
    preveri delitelja, dobljena iz polovice periode
    if sta delitelja netrivialna **then**
        return faktorja števila N
    end if
end if
return neuspeh

Konstrukcija vezja temelji na algoritmu za ocenjevanje faze QPE, ki predstavlja osrednji kvantni del Shorovega algoritma. Postopek se začne z določitvijo velikosti registrov glede na bitno dolžino števila N, ki ga želimo faktorizirati. V ta namen vzpostavimo dva kvantna registra: merilni (oziroma števni) register, ki obsega dvakratno število kubitov, in ciljni register za shranjevanje stanja. Za beleženje končnih rezultatov dodamo še klasični register ustrezne velikosti.

V začetni fazi ciljni register inicializiramo v stanje $|1\rangle$, kar predstavlja lastno stanje za operacijo modularnega množenja. Hkrati vse kubite v merilnem registru s Hadamardovimi vrati postavimo v superpozicijo, s čimer omogočimo interferenco in zajem fazne informacije. Jedro vezja tvori zaporedje kontroliranih operacij modularnega množenja. Za vsak kubit v merilnem registru izvedemo specifična kontrolirana vrata, ki na ciljnem registru aplicirajo operacijo $a^{2^k} \pmod N$. Ker je ta logika vezana na potence števila 2, s tem efektivno zgradimo operator modularnega potenciranja.

Proces zaključimo z inverzno kvantno Fourierjevo transformacijo na merilnem registru. Ta korak prevede skrito fazno informacijo v verjetnostno porazdelitev stanj, ki jo z meritvijo preslikamo v klasični register.

Na sliki 18.2 lahko vidimo vizualizacijo kvantnega vezja za primer $N = 15$. Kubiti so najprej poslani skozi Hadamardova vrata, kar jih postavi v superpozicijo vseh možnih vrednosti x. Veliki kontrolirani bloki z oznakami $13^{(2^k)} \bmod 15$ predstavljajo korake modularnega potenciranja, ki izračunajo $13^x \bmod 15$. Spodnji kubiti služijo kot register, v katerega se zapisuje rezultat teh izračunov. Blok inverzne kvantne Fourierjeve transformacije (*ang.* Inverse Quantum Fourier Transform ali IQFT) na desni pretvori stanje zgornjega registra tako, da se v njem razkrije perioda funkcije. Merilniki na koncu izmerijo zgornje kubite in tako podajo vrednost, iz katere se lahko izračuna perioda.



Slika 18.2: Kvantno vezje, zgrajeno za fazno ocenjevanje v Shorovem algoritmu.

18.3 Rezultati

V tem poglavju predstavimo rezultate eksperimentalnih preizkusov Shorovega algoritma, izvedenih tako na lokalnem simulatorju kot na pravem IBM kvantnem procesorju. Vsi lokalni izračuni so bili izvedeni na osebnem računalniku z Intel Core i7–9700 procesorjem. Rezultati vključujejo analizo vpliva izbire števila a, vpliv števila ponovitev, primerjavo med simulatorjem in fizično napravo ter primerjavo z izbranimi klasičnimi faktorizacijskimi metodami.

18.3.1 Vpliv izbire števila a

Najprej smo se lotili preverjanja, kako izbira parametra a vpliva na uspešnost faktorizacije v Shorovem algoritmu. Parameter a je osrednji del postopka, saj določa funkcijo $f(x) = a^x \bmod N$, katere periodo poskuša algoritem najti. V izvirni različici algoritma je a izbran naključno, v našem poskusu pa smo ga za potrebe analize nastavili na fiksno vrednost in sistematično preizkusili vse dovoljene vrednosti a, za katere velja $\gcd(a, 15) = 1$. Število N smo postavili na 15, parameter `shots` nastavili na 1024, nato pa smo za vsak izbran a izvedli 100 ponovitev algoritma in izračunali delež uspešno izvedenih faktorizacij. V tabeli 18.1 so prikazane povprečne uspešnosti za posamezne vrednosti a.

Iz dobljenih rezultatov lahko vidimo, da se delež uspešnih faktorizacij med različnimi vrednostmi a nekoliko razlikuje, vendar so te razlike razmeroma majhne. To je pričakovano, saj ima večina dovoljenih vrednosti a za $N = 15$ podobno obnašanje pri modularnem potenciranju, kar vodi do enakomernih meritev periode. Podobne rezultate bi dobili tudi pri večini drugih izbranih vrednosti N, kjer bi bila uspešnost za različne vrednosti a precej enakomerna. Slabša uspešnost bi se pojavila predvsem v primerih, ko ima izbrani a neustrezno periodo (npr. liho periodo ali zelo majhno periodo), zaradi česar algoritem ne bi mogel pravilno izračunati faktorizacije.

a	**Uspešnost [%]**
2	77
4	45
7	82
8	77
11	50
13	73
14	76

Tabela 18.1: Vpliv izbire števila a na uspešnost faktorizacije za fiksno $N = 15$.

18.3.2 Vpliv števila ponovitev

Nato smo raziskovali, kako število ponovitev kvantnega vezja (parameter `shots`) vpliva na uspešnost faktorizacije. Parameter `shots` določa, kolikokrat zapored se izvede isto kvantno vezje in izvede meritev, kar neposredno vpliva na kakovost statistične porazdelitve izidov. Večje število ponovitev praviloma pomeni natančnejši približek dejanske verjetnostne porazdelitve po QFT. Tudi v tem poskusu smo uporabili fiksno vrednost $N = 15$, parameter a pa je bil izbran naključno. Za vsako izbrano število `shots` smo algoritem izvedli 100-krat in izračunali povprečen delež uspešno izvedenih faktorizacij, kot je prikazano v tabeli 18.2.

Št. ponovitev (`shots`)	**Uspešnost [%]**
1	65
128	79
512	73
1024	80
4096	83

Tabela 18.2: Vpliv števila ponovitev (`shots`) na uspešnost faktorizacije za fiksno vrednost $N = 15$.

Rezultati v tabeli kažejo, da se uspešnost faktorizacije na splošno izboljšuje z naraščajočim številom ponovitev. Razlog za to je, da večje število `shots` ustvari natančnejšo statistično porazdelitev meritev, iz katere lahko algoritem bolj zanesljivo razbere približek periode. Pri zelo majhnem številu meritev (npr. `shots` = 1) je rezultat praktično naključen, saj posamezna meritev ne odraža dejanskih amplitud po kvantni Fourierjevi transformaciji. Pri največjih vrednostih `shots` se uspešnost stabilizira, saj nadaljnje povečanje ne prinese bistvenega dodatnega statističnega izboljšanja za tako majhno število N.

18.3.3 Primerjava idealnega simulatorja in pravega kvantnega procesorja

V tem delu smo opravili primerjavo med idealnim kvantnim simulatorjem, v katerem smo izvajali našo implementacijo Shorovega algoritma, ter pravim kvantnim procesorjem IBM.

Idealni simulator v okolju Qiskit predstavlja model kvantnega računalnika brez šuma, napak pri meritvah ali netočnosti kvantnih vrat, medtem ko IBM-jev kvantni procesor predstavlja resnično strojno napravo, kjer vezje teče na kvantnih bitih, ki so občutljivi na šum in generirajo meritve z napakami. Za primerjavo smo izbrali več različnih vrednosti N, parameter `shots` nastavili na 1024 in za vsak primer izmerili čas izvajanja ter uspešnost faktorizacije. Lokalni simulator smo pognali 100-krat in izračunali povprečje uspešnosti, medtem ko smo na IBM procesorju izvedli le po 10 ponovitev za vsako vrednost N, saj je dostop do pravega kvantnega računalnika omejen in drag.

N	**Čas [s]**	**Uspešnost [%]**
15	0.068	80
21	0.080	63
33	0.085	53
143	0.804	32
314191	/	/

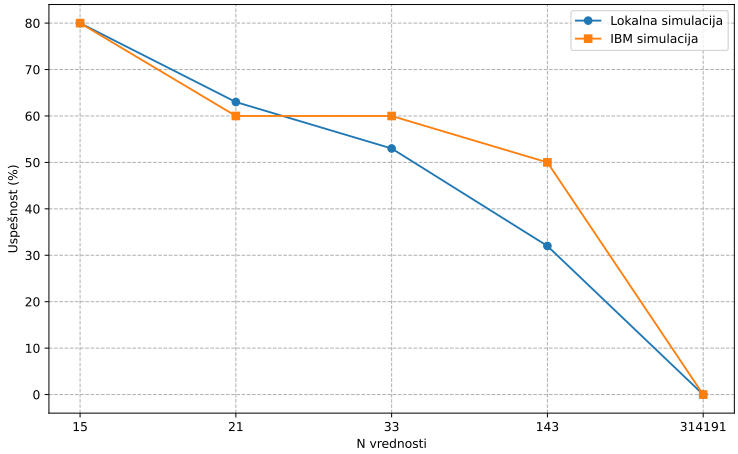
Tabela 18.3: Rezultati faktorizacije za različne vrednosti N, izvedeni na lokalnem idealnem simulatorju (100 ponovitev).

N	**Čas [s]**	**Uspešnost [%]**
15	2	80
21	2	60
33	2	60
143	2	50
314191	3	0

Tabela 18.4: Rezultati faktorizacije za različne vrednosti N, izvedeni na IBM kvantnem procesorju (10 ponovitev).

Rezultati v tabelah 18.3 in 18.4 kažejo, da čas izvajanja na lokalnem simulatorju z velikostjo N narašča, a je še vseeno bistveno krajši kot na IBM kvantnem procesorju. To je pričakovano, saj simulator izvede izračun klasično in brez dodatnih režijskih časov, ki jih na IBM-ju predstavljajo čakalne vrste, prenosi podatkov, inicializacija vezja ter fizične omejitve naprave. Uspešnost faktorizacije je razmeroma podobna pri obeh pristopih, IBM procesor nekoliko bolje deluje pri večjih vrednostih N, kar lahko opazimo tudi na sliki 18.3. Na največjem testiranem številu lokalni simulator

sploh ni mogel izvesti algoritma, saj smo bili omejeni na največ 30 kvantnih bitov, medtem ko je IBM računalnik kljub šumu uspel vezje izvesti in neuspeh vrniti v nekaj sekundah. Upoštevati moramo tudi, da je bila statistika IBM-jevih rezultatov manj zanesljiva, ker smo lahko vsako vrednost N pognali le desetkrat.



Slika 18.3: Primerjava uspešnosti faktorizacije med lokalnim simulatorjem in IBM kvantnim procesorjem za različne vrednosti N.

18.3.4 Primerjava s klasičnimi metodami faktorizacije

Na koncu smo primerjali in analizirali klasične pristope za faktorizacijo s kvantno implementacijo Shorovega algoritma. Primerjali smo tri metode: naivni brute-force pristop, ki preizkusi vse možne delitelje do $\sqrt{N}$, Pollardov ρ-algoritem, ki predstavlja eno izmed najhitrejših klasičnih naključnostnih metod za faktorizacijo srednje velikih števil, ter našo implementacijo Shorovega algoritma, izvedeno na IBM kvantnem procesorju. Za vsako metodo smo merili čas izvajanja in izbrali tri vrednosti N, ki pokrivajo majhno, srednjo in večjo velikost problema. Parameter `shots` nastavili na 1024, izbira a je bila naključna. Izmerjeni časi predstavljajo povprečen čas, potreben za uspešno faktorizacijo.

Rezultati v tabeli 18.5 kažejo, da je pri majhnih vrednostih N kvantna implementacija daleč najpočasnejša, kar je pričakovano, saj IBM kvantni procesorji vključujejo dodatno zakasnitev zaradi inicializacije sistema in samega izvajanja kvantnega vezja. Klasični metodi v tem primeru porabita zanemarljivo malo časa.

N	**Brute-force čas [s]**	**Pollard ρ čas [s]**	**Kvantni čas [s]**	**Št. kubitov**
15	0	0	2	12
314191	0.001	0	2	57
384210388873	/	0.002	10	117

Tabela 18.5: Primerjava časa izvajanja brute-force metode, Pollardovega ρ algoritma in kvantne implementacije Shorovega algoritma za različne vrednosti N.

Tudi pri srednjih vrednostih N se razmerje bistveno ne spremeni. Pri največjem izbranem številu brute-force pristop odpove zaradi prevelikega iskalnega prostora, Pollardov ρ pa še vedno uspe hitro najti faktorje. Kvantna implementacija v tem primeru potrebuje bistveno več časa, ker število kubitov hitro narašča z velikostjo N, globina kvantnega vezja postane zelo velika, posledično pa se povečajo tudi napake, zato je potrebno več ponovitev. Za občutek zahtevnosti naloge smo v tabelo vključili tudi število uporabljenih kubitov, ki narašča z velikostjo N. Za dodatno orientacijo naj omenimo, da ima uporabljen IBM-jev procesor največ 156 kubitov, medtem ko njihov trenutno največji procesor Condor vsebuje 1121 kubitov [171], ampak ta ni na voljo širši javnosti.

18.4 Zaključek

V seminarski nalogi smo preučili Shorov algoritem z vidika teoretičnih temeljev, praktične izvedbe in primerjave z izbranimi klasičnimi metodami faktorizacije. Implementacija algoritma v okolju Qiskit nam je omogočila podrobno analizo vpliva parametra a, števila ponovitev ter obnašanja algoritma na idealnem simulatorju in pravem IBM kvantnem procesorju. Rezultati so potrdili znano dejstvo, da idealni simulator dosega visoko stabilnost in zanesljivost, a nas omeji s številom kubitov, medtem ko je uspešnost na pravih kvantnih napravah omejena predvsem zaradi šuma in napak zaradi globine vezja. Primerjava s klasičnimi metodami je pokazala, da kvantni pristop pri trenutni tehnologiji še ni konkurenčen nobeni zmogljivi klasični metodi, saj zahteva bistveno več časa in je manj stabilen.

Kljub temu pa Shorov algoritem ostaja izjemno pomemben dokaz koncepta, ki kaže na velik potencial kvantnega računalništva. Njegova teoretična časovna zahtevnost je namreč polinomska, kar pomeni, da bi lahko ob zadostnem številu stabilnih kubitov in znatno manjšem šumu kvantni računalniki v prihodnosti dejansko prehiteli tudi najučinkovitejše klasične faktorizacijske algoritme. Pričakovati je, da se bo z napredkom kvantnih naprav ter boljšimi tehnikami zmanjševanja šuma njegova praktična uporabnost postopoma povečevala. Naše delo tako predstavlja majhen prispevek k razumevanju trenutnega stanja tehnologije in smernicam za nadaljnji razvoj kvantnih algoritmov.

18.5 Doprinosi avtorjev

Avtor Blaž Grilj je implementiral Shorov algoritem in vse uporabljene klasične metode za primerjavo, pripravil podpoglavje o implementaciji in vizualizacije, izvedel vse eksperimentalne poskuse in naredil prosojnice za predstavitev. Avtorica Ana Poklukar je zasnovala strukturo seminarske naloge, napisala uvod, teoretični del poglavja Metode (razen implementacijskega dela), pripravila analize rezultatov ter oblikovala zaključek naloge.

Poglavje 19
Analiza enobitnih primerjalnikov v QCA

Maks Popović

Povzetek V seminarski nalogi sem analiziral tri članke, ki opisujejo zasnovo enobitnega primerjalnika v QCA. Predlagana vezja sem implementiral v QCADesignerEju. Njihovo delovanje sem potrdil s simulacijo, nato pa zagnal simulacijo porabe energije. Vsa tri vezja so delovala pravilno in imela enako latenco. Eno izmed njih je bilo veliko bolj kompaktno od ostalih, toda se kljub temu ni izkazalo za najbolj energetsko učinkovito.

19.1 Uvod

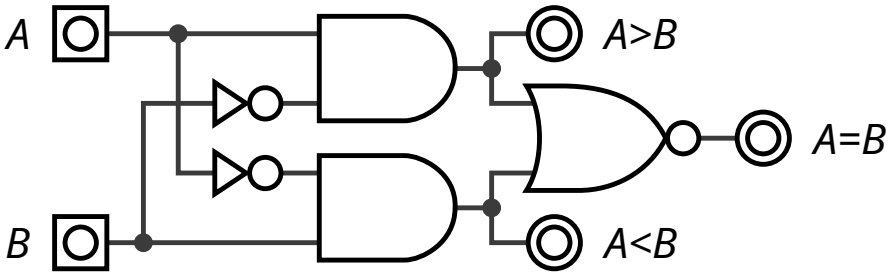
Zaradi težav pri miniaturizaciji tradicionalnih vezij CMOS postaja razvoj alternativnih tehnologij za procesiranje podatkov bolj in bolj pomemben. Ena izmed teh alternativ je računalništvo, ki temelji na kvantnih celičnih avtomatih (QCA). V teoriji nam obljublja veliko manjšo porabo energije in večjo gostoto, toda potrebuje drugačno zasnovo osnovnih gradnikov logičnih vezij. V tej seminarski nalogi bom predstavil in analiziral tri različne zasnove enobitnega primerjalnika s tehnologijami QCA.

19.1.1 Enobitni primerjalnik

Enobitni primerjalnik je vezje, ki primerja dva vhodna bita A in B in vrne tri izhodne bite: $A{>}B$ nam pove, če je vhod A strogo večji od vhoda B, $A{<}B$ nam pove, če je vhod A strogo manjši od vhoda B, in $A{=}B$ nam pove, če sta vhoda enaka. Njegova pravilnostna tabela je navedena v tabeli 19.1. Primer implementacije z logičnimi vrati je prikazan v sliki 19.1.

Tabela 19.1: Pravilnostna tabela enobitnega primerjalnika.

A	B	A>B	A<B	A=B
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1



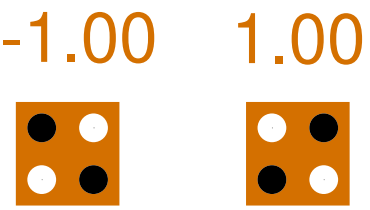
Slika 19.1: Implementacija enobitnega primerjalnika z logičnimi vrati.

19.1.2 QCA

QCA temelji na celicah, razporejenih v mreži. Vsaka celica vsebuje dva elektrona. Elektroni se znotraj celice in med soležnimi celicami odbijajo in se sčasoma razporedijo v stabilno konfiguracijo.

19.1.2.1 Celice

Vsaka celica je lahko polarizirana v eni izmed dveh stabilnih konfiguracij, kot je prikazano v sliki 19.2. Polarizacijo -1 interpretiramo kot logično 0, polarizacijo $+1$ pa interpretiramo kot logično 1.



Slika 19.2 Polarizacija QCA celic. Črne pike predstavljajo elektrone.

Procesiranje se dogaja tako, da elektroni v soležnih celicah vplivajo drug na drugega in tako spremenijo polarizacijo njihovih celic. Določene celice lahko imajo fiksno polarizacijo, kar pomeni, da lahko vplivajo na ostale celice, ostale celice pa na njih ne vplivajo. Če takšne celice krmili zunanji sistem, jim rečemo, da so vhodne celice. Določene celice lahko tudi označimo kot izhodne celice, kar ne spremeni njihovega vedenja, ampak nam dovoli branje njihovih vrednosti.

19.1.2.2 Ura

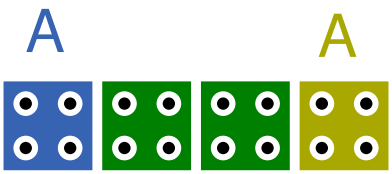
Za nadzor toka podatkov uvedemo uro, ki nadzoruje zapore med eletroni znotraj vsake celice. Vsaka celica, ki ni fiksno polarizirana ali vhodna celica, pripada eni izmed štirih urinih con. Urine cone so ciklične, in vsaka cona je za 0.25 urine periode zakasnjena od prejšnje.

Vsaka urina perioda je sestavljena iz štirih ločenih stanj:

1. *Switch*: Zapore med elektroni se začnejo vklapljati. Tukaj lahko celica spremeni polarizacijo glede na stanja bližnjih celic.
2. *Hold*: Zapore med elektroni so aktivne. Celice so polarizirane in stanja ne morejo spremeniti.
3. *Release*: Zapore med elektroni se začnejo izklapljati. Celice izgubijo polarizacijo.
4. *Relax*: Zapore med elektroni so neaktivne. Celice so nepolarizirane in so pripravljene na naslednjo urino periodo.

19.1.2.3 Žica

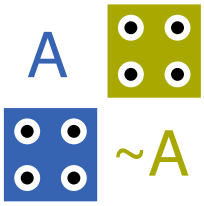
Eden izmed osnovnih gradnikov v QCA zasnovah je žica (primer prikazan v sliki 19.3). Če celice postavimo v vodoravno ali navpično črto, se bodo vse celice v črti polarizirale enako. To nam dovoli prenos signala med različnimi deli vezja.



Slika 19.3 QCA žica. Vhodna celica je pobarvna modro, izhodna rumeno.

19.1.2.4 Negator

Če celice postavimo v diagonalo, bo imela vsaka celica nasprotno polarizacijo od prejšnje. To lahko uporabimo kot negator (primer prikazan v sliki 19.4). Iz negatorjev lahko zgradimo tudi žico, ampak moramo paziti, če imamo sodo ali liho število celic.



Slika 19.4 QCA negator.

19.1.2.5 Majoritetna vrata

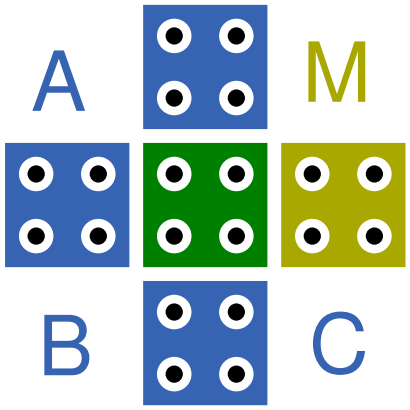
Majoritetna vrata so zelo pogost gradnik v QCA vezjih. So logična struktura z vhodi A, B, C in izhodom $M(A,B,C) = AB \vee AC \vee BC$. Pravilnostna tabela je prikazana v tabeli 19.2.

Tabela 19.2: Pravilnostna tabela majoritetne funkcije.

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Z njimi lahko implementiramo logični AND med A in B tako, da nastavimo $C = 0$. Logični OR med A in B lahko implementiramo tako, da nastavimo $C = 1$.

V sliki 19.5 so prikazana 3-vhodna majoritetna vrata. Lahko naredimo tudi majoritetna vrata z večjim številom vhodov, na primer 5-vhodna in 7-vhodna majoritetna vrata.



Slika 19.5 QCA majoritetna vrata.

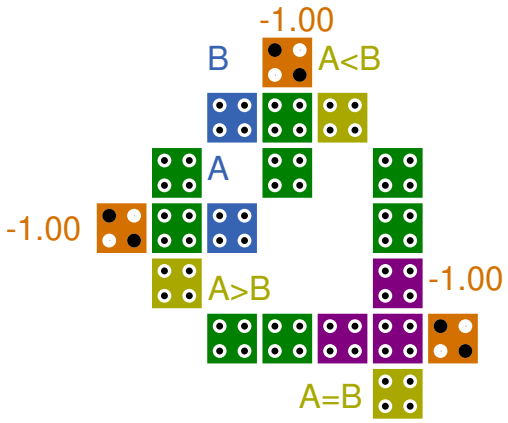
19.2 Metode

V orodju QCADesigner-E sem implementiral tri različne zasnove enobitnega primerjalnika. Za verifikacijo pravilnosti delovanja vsake zasnove sem izvedel simulacijo z metodo koherenčnega vektorja. Pognal sem tudi simulacijo, pri kateri QCADesigner-E izračuna oceno porabe energije s privzetimi nastavitvami.

Zasnove sem ocenil glede na velikost, število porabljenih celic in povprečno porabo energije.

19.2.1 Vezje 1

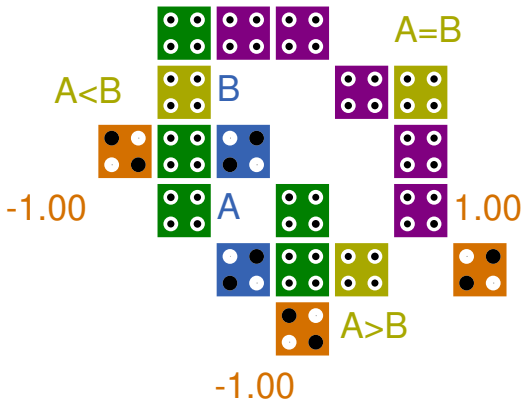
Zasnova primerjalnika iz članka [172] (prikazana na sliki 19.6) je sestavljena iz štirih negatorjev in treh 3-vhodnih majoritetnih vrat. Prve dve majoritetni vrati sta postavljeni v konfiguraciji AND (eden izmed vhodov ima fiksno polarizacijo -1). Postavljeni sta soležno, da lahko vsak izmed vhodov deluje kot vhod enim vratom, in kot negiran vhod drugim. Negirana izhoda teh vrat sta nato popeljana v tretja majoritetna vrata (tudi v konfiguraciji AND), da dobimo zadnji izhod.



Slika 19.6 Zasnova primerjalnika iz [172].

19.2.2 Vezje 2

Zasnova primerjalnika iz članka [173] (prikazana na sliki 19.7) je podobna prejšnji zasnovi, toda namesto običajnih majoritetnih vrat za $A{=}B$ uporabi strukturo, ki v tej konfiguraciji opravi isto funkcijo, ampak je porabi eno celico manj.

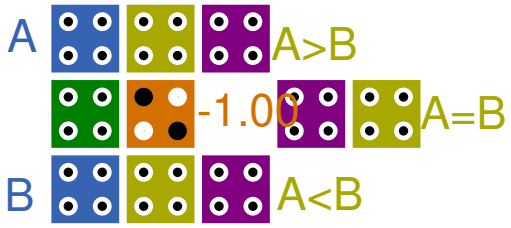


Slika 19.7 Zasnova primerjalnika iz [173].

19.2.3 Vezje 3

Zasnova primerjalnika iz članka [174] (prikazana na sliki 19.8) je popolnoma drugačno in bolj kompaktno od prejšnjih dveh zasnov. Temelji na strukturi, ki je uporabna za več od le primerjalnika. Avtorji so z isto strukturo implementirali tudi

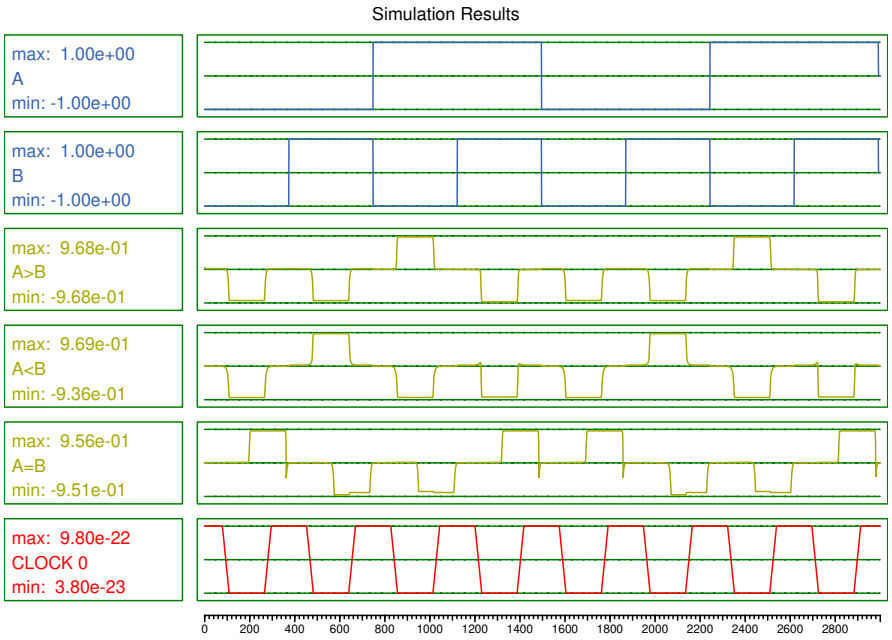
vrata XOR, XNOR, polovični seštevalnik, polni seštevalnik, polovični odštevalnik in polni odštevalnik.



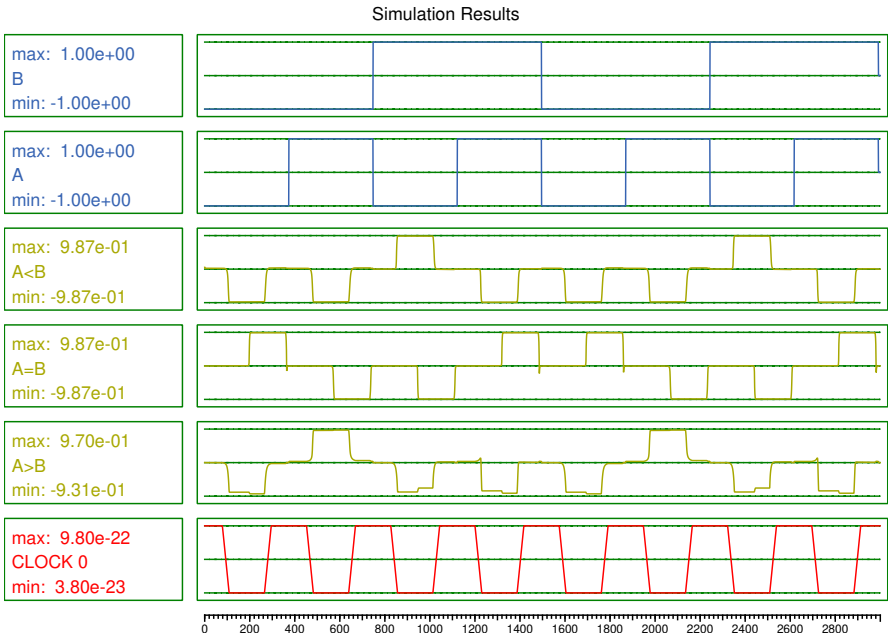
Slika 19.8 Zasnova primerjalnika iz [174].

19.3 Rezultati

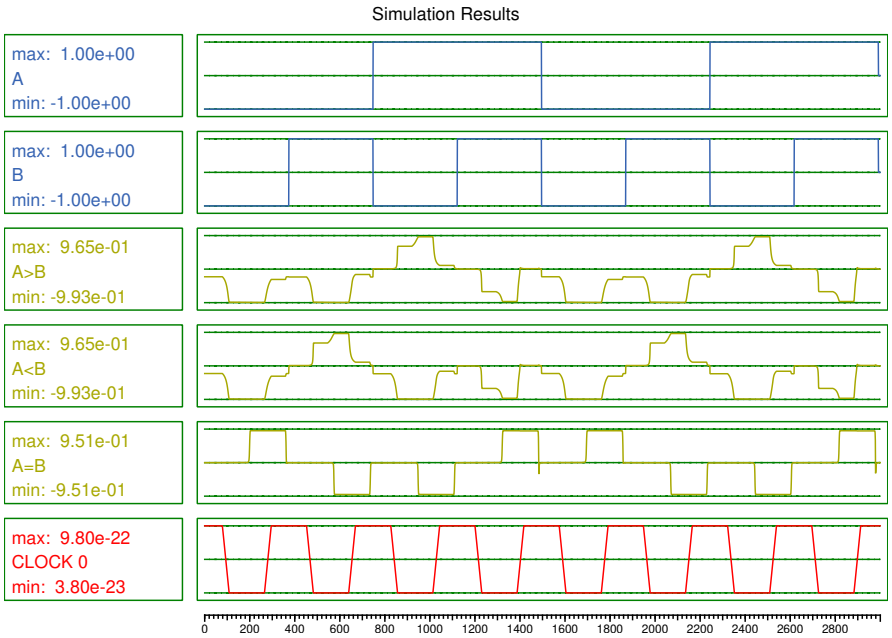
Rezultati simulacij so vidni v slikah 19.9, 19.10 in 19.11. Razvidno je, da vsa tri vezja delujejo pravilno.



Slika 19.9: Rezultati simulacije primerjalnika iz [172] s koherenčnim vektorjem.



Slika 19.10: Rezultati simulacije primerjalnika iz [173] s koherenčnim vektorjem.



Slika 19.11: Rezultati simulacije primerjalnika iz [174] s koherenčnim vektorjem.

V tabeli 19.3 je prikazana primerjava ključnih metrik med vsemi tremi vezji.

Tabela 19.3: Primerjava vezij.

Vezje	Št. celic	Velikost [μm^2]	Zakasnitev [urine periode]	Povp. poraba energije na cikel [meV]
[172]	19	0.03	0.5	1.230 ± 0.128
[173]	18	0.03	0.5	0.573 ± 0.055
[174]	10	0.01	0.5	0.859 ± 0.092

19.4 Zaključek

V seminarski nalogi sem v QCADesigner-ju implementiral in primerjal tri QCA vezja za enobitni primerjalnik. Vsa tri vezja pravilno delujejo in imajo isto latenco, razlikujejo se pa po velikosti in porabi energije.

Prvo vezje (vzeto iz [172]) se je izkazalo za najslabje pri vseh metrikah. Drugo vezje [173] je bilo najbolj energetsko učinkovito, čeprav je približno trikrat večje od tretjega vezja [174] in vsebuje skoraj enkrat več vrat.

Kljub temu, da je struktura iz tretjega vezja daleč najbolj kompaktna in se lahko uporablja za realizacijo drugih funkcij, ni nujno najboljša izvedba enobitnega primerjalnika. Med drugim in tretjim vezjim moramo izbirati glede na potrebe (manjša poraba energije proti manjši velikosti).

19.5 Doprinosi avtorjev

Avtor je en sam.

Poglavje 20
Analiza in implementacija reverzibilnega mrežnega stikala na podlagi kvantnih celičnih avtomatov

Jan Novak Domijan

20.1 Uvod

Kvantni celični avtomati predstavljajo eno izmed možnih alternativnih tehnologij za realizacijo digitalnih vezij. Osnovana je na podlagi posameznih celic, ki so sestavljene iz štirih kvantnih pik in dveh elektronov. Elektrona bosta zaradi odbojnih sil zavzela nasprotni mesti kar nam omogoča razločevati med dvema stanjema. Vsaka celica je dodeljena enemu izmed štirih urinih ciklov, zamaknjenih za 90 stopinj, ki določa mero prehodnosti elektronov med stanji celice. S postavitvijo celic nato dosežemo zaželjeno logično vezje.

Za širok nabor vezji poznamo ekvivalentno vezje iz QCA celic vendar ostaja odprto vprašanje ali so ta vezja najboljša možna. Avtorji članka [175] so zasnovali reverzibilno mrežno stikalo na podlagi Fredkinovih vrat, ki vsebuje manj celic kot prej predlagana mrežna stikala.

20.2 Vezja

Predlagano mrežno stikalo je sestavljeno iz petih ločenih vezji zasnovanih na podlagi Fredkinovih vrat: dveh 2:1 multiplekserjev, križnega stikala in dveh 1:2 demultiplekserjev. Multiplekser v vezju zavzame vlogo oddajnika, ki glede na vrednost kontrolnega bita vrne enega izmed dveh vhodnih signalov. Križno stikalo vzame dva vhoda in jih glede na vrednost kontrolnega bita ali pošlje naprej nespremenjene ali jih pa zamenja. Demultiplekser v vezju zavzame vlogo oddajnika in pošlje vhod na izbran izhod ter zamaskira drugi izhod z vrednostjo 0. Celotno vezje torej prejme štiri vhodne signale in pet kontrolnih bitov in glede na vrednosti kontrolnih bitov izbere dva poljubna vhodna signala in jih pošlje na dva poljubna izhoda kjer preostala dva izhoda vedno dobita vrednost 0.

20.2.1 Fredkinova vrata

Sestavni deli mrežnega stikala so implementirani s pomočjo Fredkinovih vrat prikazanih na sliki 20.1. Vrata so reverzibilna s tremi vhodi in izhodi, ter implementirajo CSWAP funckijo definirano kot

$$f(A,B,C) = \begin{cases} (A,B,C) & A = 0 \\ (A,C,B) & A = 1 \end{cases}$$

s pravilnostno tabelo 20.1.

Vezje je sestavljeno iz 63 celic in deluje na osnovi 2 majoritetnih vrat. Ima zakasnitev 1 kar pomeni, da mora signal potovati skozi 4 različne urine cone preden lahko preberemo vrednosti iz izhodov. V splošnem lahko definiramo zakasnitev vezja kot največje število zaporednih urinih ciklov celic na poteh od vhoda do izhoda deljeno s 4. Prednost zasnove je, da ne uporablja niti rotiranih QCA celic niti več plasti in da so vsi vhodi in izhodi lahko dosegljivi. Ne uporablja pa pravokotnih urinih con.

Tabela 20.1: Pravilnostna tabela Fredkinovih vrat

A	**B**	**C**	**P**	**Q**	**R**
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	0	1
1	1	1	1	1	1

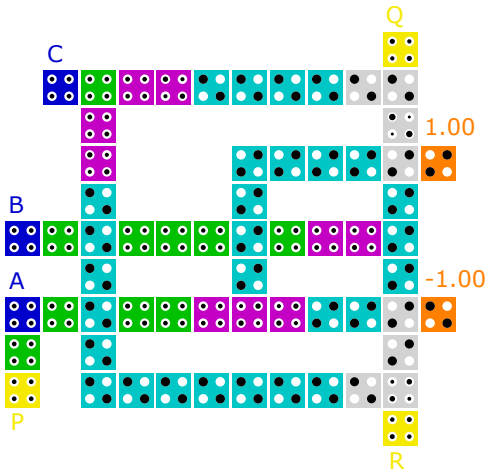
Naredil sem še pregled literature za Fredkinova vrata, ki bi bila v nekem aspektu boljša od izbranih Fredkinovih vrat. Našel sem tri vezja z manjšim številom celic.

Na sliki 20.2a so prikazana Fredkinova vrata sestavljena iz 48 celic in zakasnitvijo 1. Vezje se zdi idealno za implementacijo mrežnega stikala saj so vsi vhodi in izhodi relativno enostavno dostopni v primerjavi z drugima dvema vezjema. Problem je le v tem, da vrata v moji simulaciji niso delovala pravilno.

Na sliki 20.2b je vezje, ki je sestavljeno iz 30 celic a vhod A ni dostopen. S pomočjo večih nivojev bi lahko naredili vhod A dostopen vendar je vsak poskus bil neuspešen.

Zadnje vezje na sliki 20.2c je sestavljeno iz 32 celic a ima enak problem kot pri prejšnjem vezju. Tudi tukaj je vsak poskus premika vhoda A bil neuspešen.

Slika 20.1 Fredkinova vrata



20.2.2 Križno stikalo, 2:1 multiplekser in 1:2 demultiplekser

Če vzamemo Fredkinova vrata in primerno dodelimo naloge vhodnim ter izhodnim signalom, dobimo vsa tri vezja, ki sestavljajo mrežno stikalo.

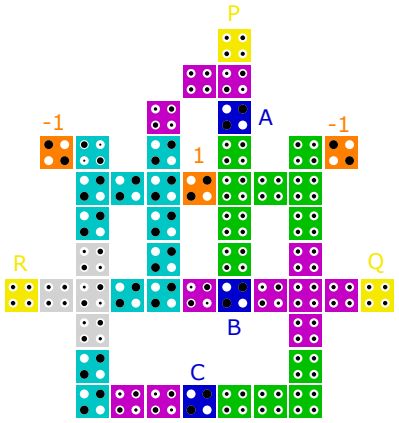
Na sliki 20.3c je prikazano križno stikalo implementirano s Fredkinovimi vrati. Signala B in C iz slike 20.1 smatramo za vhodna signala medtem, ko signal A uporabimo kot selektor. Oba izhodna signala sta uporabljena za nadaljnje računanje skupaj s pomožnim izhodom za reverzibilnost. Rezultat je vezje, ki glede na izbiro vrednosti A ali zamenja vhoda ali jih pusti nespremenjena.

Na sliki 20.3b je prikazan 2:1 multiplekser. Signal B iz slike 20.1 smatramo za selektor in signala A in C kot vhodna. Izhodni signal Y_1 je uporabljen za nadaljnje računanje medtem, ko je signal Y_2 uporabljen kot pomožni bit za izpolnjevanje reverzibilnosti. Dobimo vezje, ki prejme dva vhodna signala in glede na vrednost signala B pošlje izbran vhodni signal naprej ter preostali vhodni signal "zavrže".

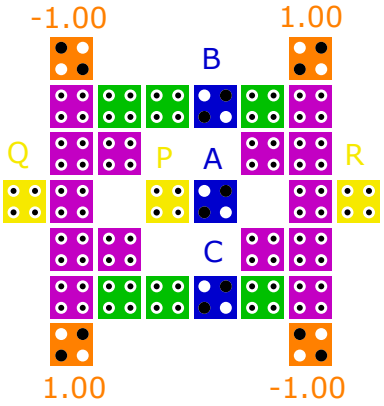
Na sliki 20.3a je prikazan 1:2 demultiplekser. Vezje je podobno tistemu za križno stikalo le, da en od vhodnih signalov vedno zavzame vrednost 0. Posledično to stikalo en vhodni signal pošlje na izbran izhod in drugi izhod zamaskira z vrednostjo 0.

20.2.3 Mrežno stikalo

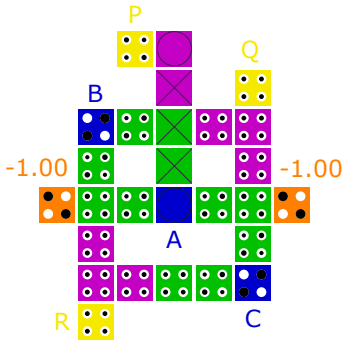
Celotno stikalo, ki je kombinacija vseh prej naštetih vezij, je prikazano na sliki 20.4. Skupaj ima 455 celic in zakasnitev 3. Vezje ima 9 vhodov, 4 signale in 5 kontrolnih bitov, ter 11 izhodov, 4 signale in 7 pomožnih bitov. Za potrebe reverzibilnosti



(a) Enonivojska implementacija s 48 celicami [176].



(b) Enonivojska implementacija s 30 celicami [159].



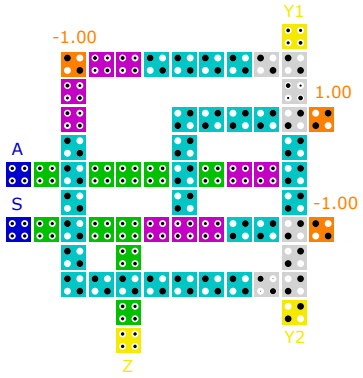
(c) Večnivojska implementacija s 30 celicami[177].

Slika 20.2: Alternativne implementacije Fredkinovih vrat.

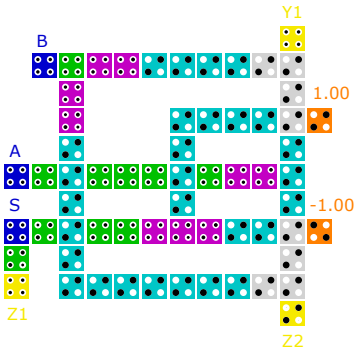
smatramo statično celico v zgornjem levem delu 1:2 demultiplekserja kot vhod s konstantno vrednostjo 0. Reverzibilnost vezja nato izhaja iz reverzibilnosti vsakega posameznega vezja.

20.3 Metodologija

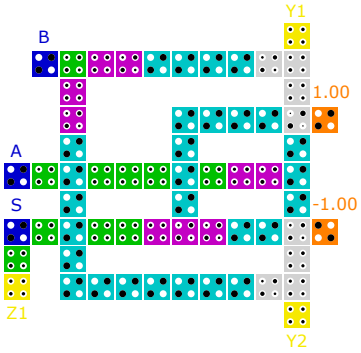
Vezje mrežnega stikala in vezja vseh individualnih komponent sem iz izvornega članka [175] prenesel v program QCADesigner-E [178]. Nato sem za vsako vezje opravil simulacijo s koherenčnimi vektorji s parametri v tabeli 20.3. Vsi parametri so



(a) 1:2 Demultiplekser.



(b) 2:1 Multiplekser.



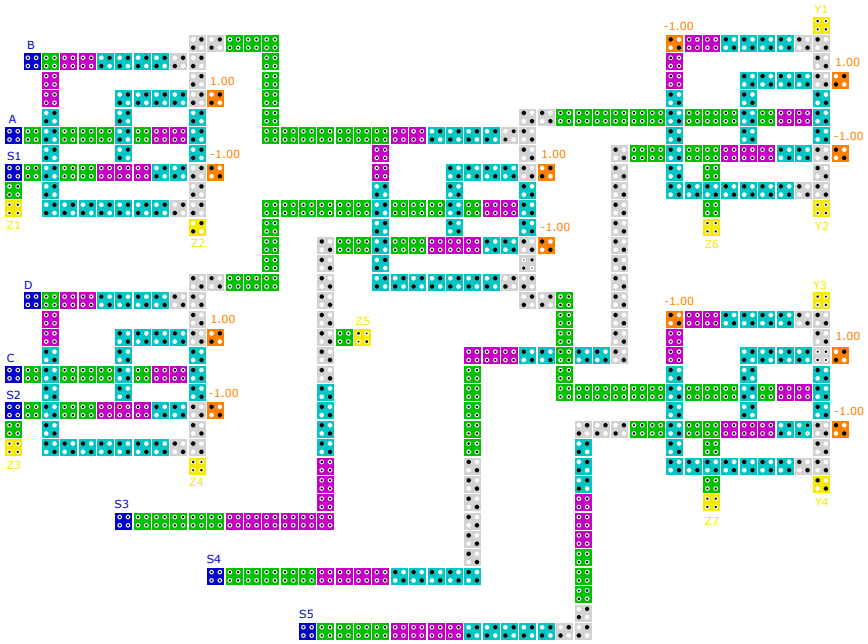
(c) Križno stikalo.

Slika 20.3: Sestavni deli reverzibilnega mrežnega stikala.
A in B označujeta vhodne signale, S označuje kontrolni vhod, Y_1 in Y_2 označujeta izhodna signala, Z_1 in Z_2 označujeta pomožne izhode.

enaki privzetim vrednostim razen ”Radius of effect” kjer so avtorji izbrali vrednost 65nm, ki ni privzeta.

Po opravljeni simulaciji sem opravil analizo vhodnih in izhodnih signalov po naslednjih korakih:

- Zavrgel sem prvo polovico vzorcev. Preostali vzorci so zadostni za analizo saj zajamejo vse možne kombinacije vhodnih vrednosti.
- Nato sem preostale vzorce razdelil na 2^n enakomernih intervalov, kjer je n število vhodnih signalov. Izbira intervalov je primerna saj vsebujejo hold faze vseh urinih ciklov razen drugega in noben od izhodov ne spada pod drugi urin cikel.
- Za vsak interval sem izračunal povprečno vrednost naboja posameznih izhodov. Če je povprečni naboj pozitiven smatramo, da je na izhodu vrednost 1, in če je naboj negativen smatramo, da je na izhodu vrednost 0.



Slika 20.4: Reverzibilno stikalo sestavljeno iz dveh 2:1 multiplekser enot na levem, dveh 1:2 demultiplekser enot na desnem in križnim stikalom na sredini.

- Izhodne vrednosti sem nato zamaknil glede na urine cikle izhodov.

Dobljen rezultat je pravilnostna tabela za izbrano vezje. Pravilnost alternativnih vezji na sliki 20.2 sem preveril ročno.

20.4 Rezultati

Rezultati simulacij in analize za vezja na sliki 20.3 so skladni z rezultati v izvirnem članku. Izračunane pravilnostne tabele posledično tudi ne odstopajo od pričakovanj. Izračun pravilnostne tabele za celotno vezje ni bil mogoč saj program QCADesigner-E omeji izhodno tabelo na 3000 vzorcev kar ne ponuja zadostne granularnosti s 6 vzorcev na vsako kombinacijo vhodnih vrednosti. Druga možnost bi bila uporaba bistabilne aproksimacije vendar ta tudi za manjša vezja ni bila konsistentna.

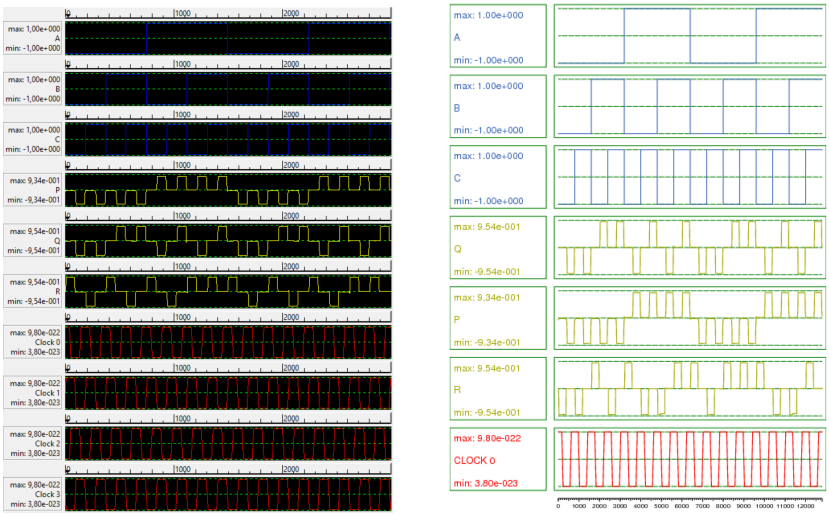
Od vezij na sliki 20.2 so rezultati skladni z rezultati v izvornih člankih z izjemo vezja 20.2a. Kot vidimo na sliki 20.5 je prva vrednost izhoda R 0 namesto 1. Dopuščam možnost, da je do odstopanja prišlo zaradi drugačnih parametrov simulacije saj avtorji v izvornem članku ne navedejo parametrov simulacije.

Tabela 20.2: Pravilnostna tabela mrežnega stikala.

S_1	S_2	S_3	S_4	S_5	Y_1	Y_2	Y_3	Y_4
0	0	0	0	0	A	0	C	0
0	0	0	0	1	A	0	0	C
0	0	0	1	0	0	A	C	0
0	0	0	1	1	0	A	0	C
0	0	1	0	0	C	0	A	0
0	0	1	0	1	C	0	0	A
0	0	1	1	0	0	C	A	0
0	0	1	1	1	0	C	0	A
0	1	0	0	0	A	0	D	0
0	1	0	0	1	A	0	0	D
0	1	0	1	0	0	A	D	0
0	1	0	1	1	0	A	0	D
0	1	1	0	0	D	0	A	0
0	1	1	0	1	D	0	0	A
0	1	1	1	0	0	D	A	0
0	1	1	1	1	0	D	0	A
1	0	0	0	0	B	0	C	0
1	0	0	0	1	B	0	0	C
1	0	0	1	0	0	B	C	0
1	0	0	1	1	0	B	0	C
1	0	1	0	0	C	0	B	0
1	0	1	0	1	C	0	0	B
1	0	1	1	0	0	C	B	0
1	0	1	1	1	0	C	0	B
1	1	0	0	0	B	0	D	0
1	1	0	0	1	B	0	0	D
1	1	0	1	0	0	B	D	0
1	1	0	1	1	0	B	0	D
1	1	1	0	0	D	0	B	0
1	1	1	0	1	D	0	0	B
1	1	1	1	0	0	D	B	0
1	1	1	1	1	0	D	0	B

Tabela 20.3: Parametri simulacije s koherenčnimi vektorji.

Parameter	**Vrednost**
Temperature	1,000000 K
Relaxation Time	1,000000e-015 s
Time Step	1,000000e-016 s
Total Simulation Time	7,000000e-011 s
Clock High	9,800000e-022 J
Clock Low	3,800000e-023 J
Clock Shift	0,000000e+000
Clock Amplitude Factor	2,000000
Radius of Effect	65,000000 nm
Relative Permittivity	12,900000
Layer Separation	11,500000 nm



Slika 20.5: Rezultati simulacij za vezje 20.2b, na levi so rezultati iz programa QCADesigner-E s privzetimi parametri in na desni rezultati iz izvornega članka [176].

20.5 Zaključek

Glede na rezultate analize izbranih Fredkinovih vrat in alternativ se zdi, da je izbrano vezje dokaj robustno z relativno majhno količino celic in solidno zakasnitvijo. Posledično bi lahko sklepali, da je efektivno tudi predlagano mrežno stikalo, vendar zaradi tehničnih omejitev tega nisem mogel preveriti. Presenetljivo je bilo odstopanje rezultatov na sliki 20.5 saj že v osnovni obliki vezje ni delovalo pravilno.

Poglavje 21
Primerjava kvantnih pristopov za problem N-kraljic

Rudi Val Arčon, Lara Anžur

Povzetek V seminarski nalogi obravnavamo problem N-kraljic kot predstavnika NP-polnih omejitvenih problemov (CSP) ter raziskujemo možnosti kvantnega pospeška pri njegovem reševanju. Za primer vzamemo $N = 4$ in primerjamo dva konceptualno različna kvantna pristopa. Prvi pristop temelji na prevedbi CSP v SAT in uporabi Groverjevega algoritma, pri čemer formulo SAT v konjunktivni normalni obliki pretvorimo v kvantni orakelj s pomočjo knjižnice `PhaseOracle`. Drugi pristop izkoristi strukturo problema in uporablja neposredno kvantno kodiranje CSP: vrstice šahovnice predstavimo z W-stanji, stolpčne in diagonalne omejitve pa preverjamo s sistemom pomožnih kubitov. Rezultati pokažejo, da je prvi pristop kubitno varčen in enostaven za uporabo, a generira globlja vezja z zahtevnimi večkrat kontroliranimi vrati, medtem ko drugi potrebuje bistveno več kubitov, a doseže zelo plitvo vezje, ki je z vidika fizičnih kvantnih naprav potencialno bolje prilagojeno omejeni koherenci. Kvantna rešitev CSP tako ni nujno le vprašanje izbire časovno najučinkovitejšega algoritma, temveč tudi iskanje pravega kompromisa med splošnostjo, izrabo strukture problema in omejitvami konkretne kvantne arhitekture.

21.1 Uvod

Omejitvene probleme (angl. *constraint satisfaction problems*, CSPs) [180] obravnavamo kot model, v katerem iščemo takšne vrednosti spremenljivk, ki zadoščajo vnaprej predpisanim omejitvam. S temi problemi se pogosto srečujemo v praksi, največkrat pri razporejanju, načrtovanju in reševanju ugank (npr. sudoku). Kljub temu da je probleme CSP enostavno formulirati, so ti prvi primer, za katerega je bilo dokazano, da je NP-poln problem [181]. To nam pove, da ne morejo biti rešeni v polinomskem času, hkrati pa vsaka spremenljivka eksponentno poveča iskalni prostor.

Eden izmed tipičnih primerov CSP je problem N-kraljic [182], kjer želimo na šahovnico velikost $N \times N$ postaviti N kraljic na način, da se noben par kraljic med seboj ne napada. Primer je zaradi enostavnosti formulacije, a hkrati svoje kombinatorične zahtevnosti (časovna zahtevnost $O(n!)$, pogosto uporabljen kot testni primer pri razvoju novih metod reševanja.

Z razvojem kvantnega računalništva se odpirajo možnosti po optimizaciji reševanja NP-polnih problemov z uporabo kvantnih algoritmov. Groverjev algoritem nudi kvadratno pospešitev pri iskanju na neurejenih množicah, pogosto pa se uporablja v kombinaciji s formalizacijami SAT, kjer gre za prevedbo CSP na iskanje zadovoljivega logičnega stanja. Tak pristop pa je splošen in ne izkorišča nobene posebne strukture CSP, s čimer se postavi vprašanje, ali bi lahko optimizacijo izvedli še na kak drug način.

V nalogi smo primerjali dva kvantna pristopa za reševanje problema N kraljic, kjer prvi pristop vključuje kombinacijo prevedbo CSP na SAT in Groverjev algoritem, drugi pa temelji na neposrednem kvantnem kodiranju CSP, ki izkoristi strukturirane superpozicije in omogoča bistveno manjši iskalni prostor.

21.2 Teoretično ozadje

21.2.1 CSP in SAT

Problem N kraljic torej spada med NP-polne probleme, natančneje v skupino omejitvenih problemov (CSP). Te lahko matematično definiramo kot trojček (X, D, C), kjer:

- X predstavlja množico spremenljivk $x_1, x_2, ..., x_n$,
- D predstavlja množico končnih domen $D_1, D_2, ...D_n$ vsake izmed spremenljivk x_i (domena D_i pripada spremenljivki x_i) ter
- C množico omejitev $C_1, C_2, ..., C_n$, kjer vsaka C_i omejujejo zavzem določenih vrednosti na neki podmnožici spremenljivk $X' \subseteq X$

V sklopu družine problemov CSP ločimo dve kategoriji. Pravimo, da je problem je izpolnjiv (angl. *satisfiable*, SAT), kadar obstaja vrednost za vsako spremenljivko na način tako, da s tem zadovoljimo vse omejitve. V nasprotnem primeru problem ni izpolnjiv (angl. *unsatisfiable*, UNSAT).

Ker želimo v seminarski nalogi ponazoriti delovanje problemov CSP z uporabo kvantnega računalnika, je smiselno, da uporabimo binarno domeno, saj kvantni računalniki operirajo s kubiti, s čimer se izognemo kompleksnejšemu kodiranju več stanj. Poleg tega je uporaba binarnega kodiranja smiselna tudi zaradi omejitev strojne opreme, ki predvideva določena vrata, ta pa delujejo na podlagi preprostih logičnih operacij - opisani podmnožici problemov rečemo problemi SAT (angl. *boolean satisfiability problem*) [183, 184]. Ti torej predstavljajo poseben primer CSP, kjer vse spremenljivke zavzamejo vrednosti logičnega tipa (torej 0 ali 1). Definiramo jih kot pare (X, F), kjer je:

- X množica spremenljivk, pri čemer vsaki pripada domena $D_i = 0,1$,
- F pa formula (oz. izjavni izraz), ki jo sestavljajo spremenljivke iz X in so med seboj povezane z logičnimi operatorji in, ali oz. ne (negacija).

Formula F je navadno predstavljena s konjunktivno normalno obliko (KNO, angl. *conjunctive normal form*, CNF). To pomeni, da je sestavljena iz stavkov, stavki pa so med seboj povezani z disjunktivnimi literali (z literali označujemo logično spremenljivko x_i in njeno negativno vrednost $\overline{x_i}$), sami stavki pa so med seboj povezani s konjunkcijo. Da dobimo rešitev problema, moramo poiskati take vrednosti $x_1, x_2, ..., x_n$, da velja $F(X) = 1$. Takih rešitev je seveda lahko več - v tem primeru pravimo, da rešujemo problem #SAT. Hkrati pa obstaja tudi možnost, da rešitev za primer ne obstaja - v tem primeru reševalnik problema SAT kot izhod vrne, da je problem neizpolnljiv.

Pri kvantnih pristopih je še posebej pomembno, da je formula F podana v KNO, saj jo lahko tako neposredno prevedemo v kvantno vezje. S tem omogočimo konstrukcijo kvantnega oraklja, ki izvaja fazni obrat

$$|x\rangle \mapsto (-1)^{F(x)} |x\rangle ,$$

kadar dodelitev x zadovolji vse stavke v F.
Vsak stavek se lahko implementira s kombinacijo negacij (X-vrat), konjunkcij (Toffoli oziroma večkrat kontroliranih vrat) in faznega Z-obrata. V naši implementaciji to funkcionalnost realiziramo s knjižnico Qiskit (metoda `PhaseOracle`), ki iz logičnega izraza samodejno sintetizira pripadajoče kvantno vezje.

21.2.2 Groverjev algoritem

Groverjev algoritem [185, 186] je kvantni iskalni algoritem, ki rešuje problem nestrukturiranega iskanja - matematično rešuje problem iskanja X, da velja $F(X) = 1$, pri čemer je lahko F primer formulacije nekega CSP. Medtem ko klasični algoritmi za takšno iskanje potrebujejo linearni čas $O(n)$, Groverjev algoritem doseže kvadratno pospešitev na $O(\sqrt{n})$. Temelji na ideji ojačevanja amplitude ciljnega kvantnega stanja - z ustreznim faznim označevanjem in difuzijo se amplituda pravilne rešitve postopno poveča, verjetnost njenega opazovanja pa postane znatno večja. Posledično pa je algoritem probabilističen, kar pomeni, da uspešnost ni zagotovljena v vsaki izvedbi. Za največjo verjetnost uspeha moramo izvesti približno $\frac{\pi}{4}\sqrt{N}$ iteracij, kjer je N velikost prostora iskanja. Število iteracij se spremeni, če predpostavimo, da iščemo več kot eno rešitev [187].

Groverjev algoritem lahko v grobem razdelimo v tri ključne korake. Prvi je inicializacija, kjer pripravimo enakomerno superpozicijo vseh možnih vhodov z uporabo Hadamardovih vrat. Drugi korak je kvantni orakelj, ki ciljni konfiguraciji spremeni fazo (navadno jo invertira). Tretji korak je difuzijski operator, v katerem se najprej izvede povprečenje amplitud - to se zmanjša zaradi negativne amplitude ciljnega stanja - nato pa se ta ponovno prezrcali na pozitivno stran, s čimer ima v primerjavi

s preostalimi stanji večjo amplitudo in posledično večjo verjetnost za pravilnost. Z zaporednim ponavljanjem teh dveh operacij se trenutno stanje vedno bolj geometrijsko približuje ciljnemu stanju, ki tako postopno pridobiva največjo amplitudo.

21.2.3 W-stanja

Glede na porazdeljenost vrednosti amplitud ločimo enakomerno superpozicijo, s katero na primer inicializiramo stanje v Groverjevem algoritmu, ter neenakomerno kvantno superpozicijo, kjer vrednosti amplitud stanj niso enako porazdeljene [188]. Primer iz omenjene skupine so W-stanja, ki jih umeščamo med simetrična stanja superpozicije. Gre za poseben razred, znotraj katerega se uporablja permutacijska simetrija, zato se W-stanja ne spreminjajo, ali pa se spreminjajo na predvidljiv način, kadar se kubiti v njih zamenjajo.

Za W-stanja [189] je značilna enakomerna superpozicija enega samega vzburjenega kubita med več nosilci. Za n-kubitni sistem je W-stanje oblike

$$|W_n\rangle = \frac{1}{\sqrt{n}}(|1,0,\ldots,0\rangle + |0,1,\ldots,0\rangle + \cdots + |0,0,\ldots,1\rangle)$$

Specifično zanje je, da ostanejo v stanju prepletenosti, tudi če je en kubit izmerjen. Če iz stanja odstranimo en kubit (npr. zaradi izgube), preostanek sistema z visoko verjetnostjo ostane W-stanje z enim kubitom manj. W-stanja ob napaki ohranijo informacije o globalni superpoziciji, zaradi česar so primerna na področju robustnega kvantnega kodiranja [190].

W-stanja so uporabna pri N-kraljicah, saj jih lahko uporabimo na način eničnega (angl. *one-hot*) kodiranja - v superpoziciji imajo natanko en kubit v stanju $|1\rangle$, kar ustreza pogojema, da je lahko v vsaki vrstici le ena kraljica in da je lahko na enem polju največ ena kraljica. S tem del problema predhodno vgradimo v kvantno stanje, s čimer precej omejimo iskalni prostor. Poleg tega je preverjanje stolpčnih in diagonalnih pogojev bistveno manj zahtevno kot pri pristopu SAT, kar je razvidno tudi iz razlike v številu uporabljenih kvantnih vrat [191].

21.3 Metode

21.3.1 Problem N-kraljic kot CSP

Ker v nadaljevanju obravnavamo problem N-kraljic v obliki CSP, moramo najprej ga definirati. V poglavju 21.2.1 smo predstavili splošno definicijo problema CSP. Če to prevedemo na naš konkretni primer, dobimo, da so:

- spremenljivke iz množice X enake kraljicam $x_1, x_2, \ldots, x_n$,

- domene za vsako izmed kraljic določene tako, da vsaka kraljica zavzame eno (fiksno) vrsto na šahovnici, medtem ko lahko katerakoli kraljica zavzame katerikoli stolpec v svoji vrstici, pri čemer
- mora upoštevati omejitve problema, in sicer: ena kraljica na vrstico, ena kraljica na stolpec in ena kraljica na diagonalo.

21.3.2 Metoda 1: Uporaba SAT in Groverjevega algoritma

21.3.2.1 Prevedba na SAT

V tem odseku bomo prevedli zastavljene omejitve iz predhodnega podpoglavja na problem SAT, katerega glavni značilnosti sta uporaba binarnih domen in preoblikovanje funkcije v KNO zaradi večje učinkovitosti. Za demonstracijo uporabimo N = 4.

Sprva smo določili način kodiranja, kjer je bilo mogoče uporabiti unarno in binarno kodiranje. Unarno kodiranje nam pomeni, da bi vsako vrstico na šahovnici kodirali s 4 biti (enično kodiranje). Skupno je tako na šahovnici z velikostjo 4x4 potrebnih 16 bitov, da zakodiramo pozicije vseh štirih kraljic. Ker ta način ni najbolj učinkovit, smo se odločili za uporabo binarnega kodiranja. Tukaj za vsako vrstico kodiramo le pripadajoče indekse stolpcev. Ker imamo v našem primeru 4 vrstice, indekse 4 stolpcev pa je mogoče kodirati z dvema bitoma, to pomeni, da v tem primeru za celotno kodiranje porabimo le 8 bitov, kar je polovica manj kot pri unarnem kodiranju.

V obeh primerih se privzame vrstice za implicitno vključene v kodiranje, s čimer se izognemo omejitvi, vezani na vrstice - v sklopu osmih bitov že enolično kodiramo vrstice in tako lahko predpostavimo, da je ta omejitev že izpolnjena.

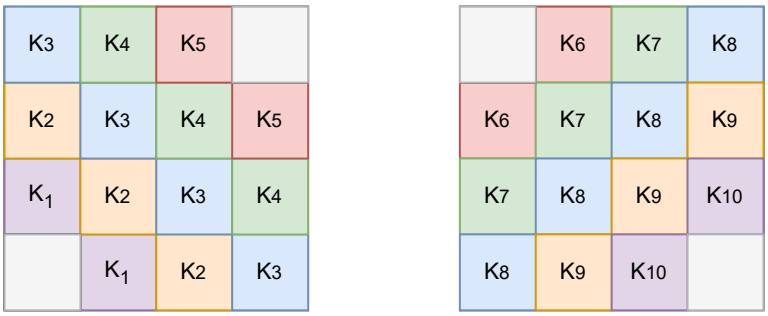
Preostaneta nam tako še problem istih stolpcev in diagonal. Najprej se lotimo problema s stolpci - želimo zagotoviti, da se kodiranje stolpcev razlikuje vsaj v enem bitu, lahko pa tudi v obeh. Privzamemo, da vrstice šahovnice od zgoraj navzdol označujemo s črkami a,b,c,d, vsako od teh pa lahko zakodiramo z dvema bitoma - če za primer vzamemo a, potem je ta kombinacija vrednosti a_1 in a_0. Če to zapišemo s formulo, dobimo:

$$((a_1 \oplus b_1) \vee (a_0 \oplus b_0))$$

Podobno mora veljati za vse celice na šahovnici, celotna omejitev pa je v končnem koraku:

$$\begin{aligned} K_0 = {} & ((a_1 \oplus b_1) \vee (a_0 \oplus b_0)) \wedge ((a_1 \oplus c_1) \vee (a_0 \oplus c_0)) \wedge \\ & ((a_1 \oplus d_1) \vee (a_0 \oplus d_0)) \wedge ((b_1 \oplus c_1) \vee (b_0 \oplus c_0)) \wedge \\ & ((b_1 \oplus d_1) \vee (b_0 \oplus c_0)) \wedge ((c_1 \oplus d_1) \vee (c_0 \oplus d_0)) \end{aligned}$$

Nekoliko težje pa je določiti omejitve pri diagonalah. Ob kratkem premisleku najdemo 10 spornih diagonal (označene na spodnjih dveh slikah):



Za primer modelirajmo diagonalo K_2. Celice so v vrsticah b, c in d. Pozicijo ene celice zapišemo kot:

$$(\overline{a_1} \wedge \overline{a_0})$$

kar pomeni celico s koordinatami (0,0). Če bi želeli, da so označene vse celice na diagonali, bi za izbrano diagonalo zapisali

$$K_2 = (\overline{b_1} \wedge \overline{b_0}) \wedge (\overline{c_1} \wedge c_0) \wedge (d_1 \wedge \overline{d_0})$$

Ker pa tega ne želimo, celoten izraz še negiramo. Postopek nato ponovimo še za ostalih 9 diagonal, njihove izraze pa združimo s konjunkcijo. Skupna omejitev K je torej konjunkcija vseh omejitev:

$$K = \bigwedge_{i=0}^{10} K_i$$

21.3.2.2 Uporaba Groverjevega algoritma

Pri praktični uporabi Groverjevega algoritma je potrebno koncepte iz poglavja 21.2.1 implementirati na kvantnem vezju.

Začnemo pri inicializaciji, kjer vsak vhodni kubit pošljemo čez Hadamardova (H) vrata in tako pripravimo uniformno superpozicijo $|s\rangle$.

Temu sledi gradnja kvantnega oraklja. V našem primeru orakelj ne temelji na ročni uporabi Toffolijevih vrat, temveč ga sintetiziramo neposredno iz formule KNO s pomočjo modula `PhaseOracle`. Tak orakelj implementira fazni obrat za vse dodelitve, ki zadovoljijo formulo, in je sestavljen iz večkrat kontroliranih Z-operacij ter enokubitnih negacij, ki odražajo logično strukturo posameznih stavkov.

Namen oraklja je, da prepozna ciljno konfiguracijo. X-vrata najprej obrnejo določene kubite, da pripravijo takšen vzorec, da ga Toffolijeva vrata zaznajo - to

se zgodi, kadar so vsi relevantni kubiti v stanju $|1\rangle$. Ko orakelj zazna iskani vzorec, invertira fazo ciljnega stanja, tj. spremeni amplitudo končnega stanja v negativno.

Difuzijski operator najprej uporabi Hadamardova (H) vrata, da sistem transformira iz superpozicijske baze v neko preprostejšo računsko bazo. Pri tem se os refleksije preslika v stanje $|00\ldots0\rangle$. V naslednjem koraku se s kombinacijo X in kontroliranih faznih vrat izvede fazni obrat za vsa stanja, razen za $|00\ldots0\rangle$. Nazadnje se z uporabo H vrat sistem prevede v prvotno bazo, v kateri izvedeni fazni obrat ustreza refleksiji okrog povprečja, kar povzroči ojačitev amplitude ciljnega stanja.

Postopek oraklja in difuzorja iteriramo, dokler amplituda ciljnega stanja ni dovolj povečana. Ker imajo vsa stanja na začetku enako amplitudo, orakelj najprej enkratno spremeni fazo ciljnega stanja, difuzor pa nato zrcali vse amplitude okoli njihovih povprečnih vrednosti. Posledično se amplituda pravilne rešitve v vsaki iteraciji poveča, relativna amplituda ostalih pa zmanjša. Po ustreznem številu iteracij je verjetnost opazovanja pravilne rešitve največja.

21.3.3 Metoda 2: Direktno kvantno CSP kodiranje

V drugi metodi izkoristimo strukturo problema *N* kraljic tako, da omejitve vgradimo neposredno v kvantno stanje in kvantne operacije, brez vmesnega prevajanja v SAT. Ključna ideja je, da začetno stanje pripravimo kot superpozicijo vseh razporeditev, ki že izpolnjujejo najosnovnejši pogoj: v vsaki vrstici je lahko natanko ena kraljica. Preostali stolpčni in diagonalni pogoji pa se preverjajo kvantno z uporabo dodatnih pomožnih (ancilla) kubitov.

21.3.3.1 Kodiranje z W-stanji

Za šahovnico velikosti 4×4 uporabimo štirikubitno predstavitev vsake vrstice, kjer vsak kubit predstavlja eno polje v vrstici. Za vsako vrstico pripravimo W-stanje

$$|W_4\rangle = \frac{1}{2}(|1000\rangle + |0100\rangle + |0010\rangle + |0001\rangle),$$

ki predstavlja enakomerno superpozicijo vseh veljavnih položajev ene kraljice v vrstici. Ker W-stanje vsebuje natanko en vzbujen kubit, s tem pogoj ”ena kraljica na vrstico”vgradimo že v začetno kvantno stanje. Za celotno šahovnico pripravimo tenzorski produkt štirih takšnih W-stanj, kar ustvari 16-kubitni register, ki predstavlja vse razporeditve, pri katerih je ena kraljica v vsaki vrstici [192].

21.3.3.2 Preverjanje stolpčnih omejitev

Da prihranimo na številu uporabljenih kubitov, namesto, da bi uporabili štiri ancilla kubite, da preverimo pogoj ”ena kraljica na stolpec”, lahko uporabimo le tri. Prvi

ancilla kubit namreš preverja prvi stolpec, drugi drugega, tretji tretjega; četrti stolpec pa je s tem določen implicitno. Vsak ancilla kubit inicializiramo v stanje $|0\rangle$, nato nanj izvedemo zaporedje kontroliranih Z operacij, kontroliranih s kubiti, ki pripadajo istemu stolpcu v različnih vrsticah. Na ta način ancilla kubit zabeleži *parnost* števila kraljic v stolpcu: ostane v stanju $|1\rangle$ natanko takrat, ko stolpec vsebuje liho število vzbujenih kubitov (tj. eno kraljico), in preide v stanje $|0\rangle$, kadar je teh nič ali več (tj. 0 ali 2 kraljici). Po transformaciji Hadamard–kontrolirani Z–Hadamard tako ancilla kubit preide v stanje $|1\rangle$ natanko takrat, ko stolpec vsebuje veljavno (tj. natanko eno) kraljico.

21.3.3.3 Preverjanje diagonalnih omejitev

Podobno kot pri preverjanju stolpčnih omejitev tudi tu pazimo na število kubitov. Sledimo raziskovalni nalogi, ki definira, naj se za preverjanje diagonalnih omejitev uporabi $n(n-1)/2$ pomožnih kubitov [192]. Za našo nalogo omejeno z $n = 4$ smo torej uporabili šest ancilla kubitov, pri čemer vsak predstavlja en par vrstic (i, j) z $i < j$. Ker W-stanja zagotavljajo natanko eno kraljico na vrstico, je diagonalne konflikte treba preverjati le parno, tj. med dvema vrsticama hkrati. Za vsako takšno kombinacijo inicializiramo ancilla kubit v stanje $|1\rangle$. Nato za vsak par polj v vrsticah i in j, ki ležita na isti diagonali, izvedemo Toffolijeva (CCX) vrata, kjer sta oba kubita iz polj kontrolna, ancilla kubit pa cilj. Če sta v enem izmed konfliktnih parov prisotni dve kraljici, se ancilla preklopi iz $|1\rangle$ v $|0\rangle$.

Po izvedbi vseh kontroliranih operacij stanje pomožnih kubitov zabeleži informacijo o veljavnosti diagonalnih pravil: ancilla ostane v stanju $|1\rangle$, če med izbranima vrsticama ni diagonalnega konflikta, sicer preide v stanje $|0\rangle$.

21.4 Implementacija metod in primerjava rezultatov

21.4.1 Knjižnice in izvorna koda

Implementacija je bila izvedena v programskem jeziku Python z uporabo knjižnice Qiskit, kjer smo s `AerSimulator`-jem simulirali obnašanje kvantnih vezij. Uporabili smo idealni model, tako da šumi, ki jih lahko pričakujemo v praktičnih izvedbah, niso upoštevani.

Pri gradnji in izvajanju kvantnih vezij smo uporabili več ključnih komponent okolja Qiskit:

- **QuantumCircuit**: osnovni gradnik, ki omogoča sestavljanje kvantnih vrat in celotne logike algoritma.
- **QuantumRegister** in **ClassicalRegister**: ločeno definirata kvantne kubite (podatkovne in pomožne) ter klasične bite, v katere so shranjene meritve.

- **transpile**: optimizacijski postopek, ki kvantno vezje prevede v niz osnovnih vrat, združi redundantne operacije in zmanjša globino vezja. Transpilacija je ključna pri primerjavi metod, saj razkrije dejansko kompleksnost oraklja in preostalih operacij.

Izvorna koda je dostopna na `https://github.com/ruvaar/Quantum-n queens`.

21.4.2 Omejitev problema

V nalogi se osredotočimo na primer $N = 4$. Čeprav so obravnavane metode splošne in bi jih bilo mogoče uporabiti tudi za večje primere, postane simulacija kvantnih vezij na klasičnem računalniku hitro računsko nedosegljiva. Število kubitov, zahtevana globina vezja in kompleksnost večkrat kontroliranih vrat naraščajo eksponentno z N, kar povzroči drastično povečanje časa simulacije.

Pomembno je tudi poudariti, da ima problem štirih kraljic natanko dve pravilni postavitvi. Ti rešitvi predstavljata referenčno osnovo, s katero lahko preverimo pravilnost delovanja obeh kvantnih pristopov. Rešitvi sta:

$$\begin{array}{cccc} . & . & Q & . \\ Q & . & . & . \\ . & . & . & Q \\ . & Q & . & . \end{array} \qquad \begin{array}{cccc} . & Q & . & . \\ . & . & . & Q \\ Q & . & . & . \\ . & . & Q & . \end{array}$$

V nadaljevanju bosta rešitvi zapisani v obliki $(2,0,3,1)$ ali $(1,3,0,2)$. Tak zapis pomeni, da vsako število označuje stolpec, v katerem stoji kraljica v posamezni vrstici: prvi element za vrstico 0, drugi za vrstico 1 itd. Na primer $(2,0,3,1)$ pomeni, da so kraljice v stolpcih 2, 0, 3 in 1 (vrstice od zgoraj navzdol).

21.4.3 Rezultati metode 1: SAT–Grover

Za potrebe rezultatov smo izvedli simulacijo, kjer se vezje najprej transpilira in nato izvede na `AerSimulator`-ju. Transpilacija odstrani redundantne operacije in kompleksna vrata razbije na zaporedje enostavnejših, kar je razvidno iz dobljenih rezultatov:

```
Čas simulacije: 0.020 s
Število kubitov v vezju: 8

Globina (pred transpilanjem): 101
Število vrat (pred transpilanjem):
  {'h': 170, 'x': 162, 'c7z_o114': 9,
```

```
    'c7z_o13': 9, 'mcx': 9, 'measure': 8}

  Globina (po transpilanju): 56
  Število vrat (po transpilanju):
    {'u2': 146, 'x': 63, 'mcx': 27,
    'u1': 11, 'measure': 8, 'h': 4}
```

Pomembno je, da nas izjemno hiter čas simulacije na klasičnem računalniku ne zavede, saj je ta v glavnem odvisen od števila kubitov, medtem ko je kvantni računalnik veliko bolj odvisen od globine. Globina vezja namreč označuje največje število zaporednih kvantnih operacij na poti od začetnega do končnega stanja. Vrata `h` in `x` predstavljajo enokubitna Hadamardova in Pauli-X vrata, oznaki `u1` in `u2` pa generična enokubitna rotacijska vrata, v katera transpilator razbije bolj kompleksne operacije. Oznaka `mcx` pomeni večkrat kontrolirana X-vrata, ki nastanejo pri dekompoziciji večkrat kontroliranih Z-vrat, ki jih generira `PhaseOracle`. Opazimo, da se po transpilaciji globina vezja prepolovi, število Hadamardov drastično pade (iz 170 na 4), število `mcx` pa zraste, kar potrjuje, da je SAT-orakelj najkompleksnejši del metode.

Po devetih Groverjevih iteracijah algoritem pravilno najde obe rešitvi problema. Pri 4096 meritvah sta prevladovali rešitvi z 51.245 %: $(2,0,3,1)$ in z 47.485 %: $(1,3,0,2)$. Skupna verjetnost pravilnega izida tako znaša 98.73 %. Visoka stopnja uspeha potrjuje, da kvantni orakelj pravilno kodira SAT-formulo in da Groverjev algoritem učinkovito ojača amplitudi obeh pravilnih rešitev.

21.4.4 Rezultati metode 2: Direktno kvantno CSP kodiranje

V tem pristopu izkoristimo dejstvo, da je mogoče velik del omejitev problema štirih kraljic vgraditi neposredno v kvantno stanje. Omejitve za stolpce in diagonale se preverjajo kvantno, s pomočjo dodatnih pomožnih kubitov (ancill). V implementaciji so uporabljeni trije ancilla kubiti, ki s postopkom Hadamard–kontrolirani Z–Hadamard zabeležijo, ali se v posameznem stolpcu pojavi natanko ena kraljica. Ker četrti stolpec sledi iz prvih treh, dodatnega ancilla kubita ne potrebujemo.

Za diagonalne omejitve uporabimo šest ancill, kolikor je parov vrstic pri $n = 4$. Vsak diagonalni ancilla je inicializiran v stanje $|1\rangle$ in se prek točkasto usmerjenih Tofolijevih (`ccx`) vrat preklopi v stanje $|0\rangle$, če v ustreznem paru vrstic nastopi diagonalni konflikt. Ker W-stanja zagotavljajo, da je v vsaki vrstici natanko ena kraljica, zadostuje preverjanje konfliktov samo paroma.

Po končanih preverjanjih vsebinskih omejitev imamo tako 16 kubitov za šahovnico ter 9 ancill (3 za stolpce, 6 za diagonale), skupaj torej 25 kubitov. Končni merjeni bitni niz ima zato dolžino 25 bitov: prvih 16 predstavlja položaje kraljic, preostalih 9 pa vrednosti stolpčnih in diagonalnih ancill. Izkaže se, da je za povprečen računalnik to že zelo blizu zgornje meje števila kubitov, ki jih lahko simulira.

21.4.4.1 Rezultati simulacije

Izvedli smo simulacijo metode, kjer se kvantno vezje sestavi, izvede in nato klasifikacijsko filtrira. Izpis simulatorja je bil naslednji:

```
Čas simulacije: 4316.11 s
Število kubitov: 25

Število vrat:
{'ccx': 28, 'measure': 25, 'cz': 12,
'h': 6, 'x': 6, 'initialize': 4}

Globina: 24
```

Majhna globina vezja nam potrjuje prednost metod, ki izkoriščajo strukturo problema. Pri tem je treba poudariti, da je to dober rezultat, kljub temu da smo za rezultat trivialne naloge čakali več kot eno uro, namreč v resničnem kvantnem procesorju bi bil čas izvedbe določen z globino vezja, ne s številom kubitov, in bi bil zato bistveno krajši.

Simulator je izvedel 2048 meritev celotnega 25-bitnega registra. Veljavne rešitve prepoznamo tako, da morajo vsi stolpčni ancilli imeti vrednost $|1\rangle$, prav tako vseh šest diagonalnih ancill. Takih meritev je bilo:

$$17 \text{ od } 2048 \quad \Rightarrow \quad p \approx 0.83\,\%.$$

To nizko razmerje je pričakovano: metoda 2 ne uporablja Groverjevega ojačevanja amplitude. Kvantno vezje tako zgolj filtrira superpozicijo, ne pa tudi povečuje verjetnosti pravilnih stanj. Rezultati odražajo točno tisto, kar implementacija dejansko izvede.

Med veljavnimi merjenimi bitnimi nizi se pojavita natanko dve različni konfiguraciji, ki ustrezata obema rešitvama problema štirih kraljic:

- bitni niz `111111111001010000010100` (9-krat),
- bitni niz `111111111010000011000010` (8-krat).

V merjenem 25-bitnem stanju prvih 9 bitov pripada ancilla kubitom (3 za stolpce, 6 za diagonale), preostalih 16 bitov pa predstavlja štiri W-vrstice, vsaka s štirimi kubiti. Dekodiranje teh bitnih nizov pokaže postavitvi $(2,0,3,1)$ in $(1,3,0,2)$, kar se ujema z znanima rešitvama problema štirih kraljic. To nam potrjuje pravilno delovanje vseh delov metode — priprave W-stanj, preverjanja stolpčnih omejitev in preverjanja diagonalnih omejitev.

21.4.5 Primerjava metod

Oba pristopa v nalogi rešujeta isti problem, vendar se razlikujeta tako v zasnovi kot v praktičnih lastnostih. V nadaljevanju podamo primerjavo, ki temelji na dejanskih rezultatih simulacij iz poglavja 21.4.

21.4.5.1 Kvantitativna primerjava

Za lažjo preglednost podamo povzetek ključnih metrik obeh pristopov v Tabeli 21.1.

Lastnost	**SAT–Grover**	**Direct–CSP**
Število podatkovnih kubitov	8	16
Število pomožnih kubitov	0	9
Skupno število kubitov	8	25
Dolžina izhodnega bitnega niza	8	25
Globina vezja (po transp.)	56	24
Število meritev	4096	2048
Uspešnost pravilnih izidov	98.6 %	0.83 %
Št. enokubitnih vrat (H, X, U1, U2, U3)	220	16
Št. dvokubitnih vrat (CZ)	0	12
Št. trokubitnih vrat (CCX)	27	28
Št. kompleksnih več-kontrolnih (C7Z) vrat	18	0
Čas simulacije (klasični PC)	0.02 s	4316 s

Tabela 21.1: Primerjava SAT–Grover in Direct–CSP pristopa pri reševanju problema štirih kraljic.

21.4.5.2 Interpretacija rezultatov

Kubitna zahtevnost

Pristop SAT Grover uporablja binarno kodiranje in zato potrebuje le 8 podatkovnih kubitov, metoda Direct–CSP pa uporablja štirikubitna W-stanja za vsako vrstico, kar pomeni 16 podatkovnih kubitov, ter dodatnih 9 pomožnih kubitov za preverjanje stolpčnih in diagonalnih omejitev. Skupno število bitov znaša 25, kar je več kot trikratno povečanje v primerjavi s prvo metodo. Ta razlika je ključna za razumevanje simulacijskih časov - klasični simulatorji so občutljivi predvsem na število kubitov, saj morajo upravljati prostor velikosti 2^n, in tako postane simulacija Direct–CSP metode izjemno počasna.

Globina vezja in zapletenost vrat

Metoda z direktnim kodiranjem ima presenetljivo majhno globino, kar kaže na to, da so stolpčne in diagonalne omejitve izvedene z lokalnimi in dobro strukturiranimi operacijami. Čeprav uporablja 28 Tofolijevih (`ccx`) vrat in 12 dvokubitnih `cz` vrat, se te operacije razporedijo v plitve plasti, kar ohranja nizko globino. Metoda SAT–Grover pa uporablja 18 kompleksnih večkrat kontroliranih Z-vrat (`c7z`), ki jih transpilator razbije v več plasti `mcx` vrat. Zato je končna globina po transpilaciji večja, kar bi bilo v resničnem kvantnem računalniku veliko bolj občutljivo na šum.

Uspešnost meritev

SAT–Grover metoda uporablja ojačevanje amplitude, zato pravilni rešitvi predstavljata kar 98.6 % vseh meritev. Direct–CSP pristop ne uporablja Groverjevega ojačevanja, temveč deluje kot kvantni filter: preveri superpozicijo vseh razporeditev in le tiste z ustreznimi stolpčnimi ter diagonalnimi ancillami ohrani kot veljavne. Ker se amplitude ne ojačijo, pravilne rešitve predstavljajo le 0.83 % meritev, kar je povsem skladno s strukturo algoritma in kvantnih stanj.

Simulacijska in eksperimentalna perspektiva

Največja razlika med metodama je v tem, kako bi se obnašali na dejanskem kvantnem računalniku. SAT–Grover bi tu imel prednost v številu kubitov, a bi bil občutljiv na šum zaradi večje globine in kompleksnih večkrat kontroliranih vrat, izvedba direktnega kodiranja pa bi bila bistveno hitrejša na kvantni napravi zaradi nizke globine, vendar bi potrebovala več kubitov.

Kvalitativna razlika med metodama

Metodi se razlikujeta tudi konceptualno, saj rešitev z Groverjevim algoritmom transformira problem v iskalni problem in rešitev aktivno išče z ojačevanjem amplitude ter posledičnim povečevanjem verjetnosti pravilnih rešitev. Druga metoda pa, nasprotno, rešitve ne išče aktivno, ampak pripravi superpozicijo vseh skladnih konfiguracij in jih nato kvantno filtrira.

Sklep

SAT–Grover pristop je bistveno enostavnejši za implementacijo in v simulaciji deluje izjemno hitro, saj zahteva malo kubitov in uporablja splošni mehanizem kvantnega iskanja. Njegova ključna prednost je prav generičnost: vsak problem, ki ga

lahko zapišemo v obliki SAT, lahko takoj uporabimo kot orakelj v Groverjevem algoritmu, ne da bi morali izkoriščati dodatne strukture problema.

V primerjavi z njim je Direct–CSP pristop problemsko specifičen in izkoristi konkretno strukturo problema štirih kraljic, predvsem prek W-stanj in namensko zasnovanih ancilla mehanizmov za preverjanje omejitev. Zaradi tega doseže zelo plitvo vezje, kar je pomembna prednost v kontekstu dejanskih kvantnih računalnikov, kjer globina pogosto omejuje skalabilnost zaradi šuma. Vendar pa je tak pristop uporaben samo za probleme z jasno izraženo enično kodirano strukturo in zahteva veliko več kubitov.

Oba pristopa skupaj ponazarjata dva temeljna pogleda na kvantno reševanje NP-polnih omejitvenih problemov: splošne metode (npr. SAT–Grover), ki stremijo k univerzalnosti, ter problemsko specifične metode (npr. Direct–CSP), ki izkoristijo strukturo konkretnega problema v zameno za nižjo splošnost. Oba pristopa sta pomembna in se med seboj dopolnjujeta.

21.5 Zaključek

V prispevku smo obravnavali problem štirih kraljic kot konkreten primer NP-polnega omejitvenega problema in preučili dva kvantna pristopa k njegovemu reševanju. Eksperimentalni rezultati na kvantnem simulatorju Qiskit so pokazali, da prvi pristop z 8 kubiti in 9 Groverjevimi iteracijami doseže zelo visoko verjetnost pravilnih meritev (okoli 98,7 %), pri čemer je globina vezja po transpilaciji 56. Pristop z direktnim kodiranjem pa na drugi strani uporablja 25 kubitov, vendar doseže globino zgolj 24 in brez ojačevanja amplitude deluje kot kvantni filter, ki iz superpozicije izlušči zgolj veljavne konfiguracije.

Pri interpretaciji rezultatov je pomembno ločiti med simulacijskim in eksperimentalnim vidikom, saj je na klasičnem računalniku glavno ozko grlo število kubitov, zaradi česar je prvi pristop precej hitrejši od drugega. Na dejanskem kvantnem procesorju pa postane odločilna globina vezja in število zaporednih večkubitnih operacij; v tem kontekstu ima pristop direktnega kodiranja CSP s plitvim vezjem in lokalnimi operacijami potencialno pomembno prednost, čeprav zahteva več kubitov.

Zaradi eksponentne rasti prostora stanj smo se omejili na primer $N = 4$, pri čemer nismo upoštevali šuma in napak, ki so prisotni na realnih kvantnih napravah. V nadaljnjem raziskovanju bi bilo smiselno preučiti skaliranje obeh pristopov na nekoliko večje N, vključiti modele šuma ter raziskati hibridne pristope, na primer kombinacijo W-stanj z Groverjevim ojačevanjem ali izboljšano sintezo orakljev za SAT.

Doprinosi avtorjev

- Rudi: poglavja 3.3, 4, 5 in implementacija kode
- Lara: poglavja 1, 2, 3.1, 3.2

Poglavje 22
Zasnova in analiza kvantno-celičnega reverzibilnega komparatorja

Žane Bučan, Stjepan Sazonov, Gregor Bučar, Lan Pavletič

Povzetek V seminarski nalogi predstavimo zasnovo in analizo kvantno-celičnega reverzibilnega komparatorja. Za implementacijo vezja smo uporabili orodje QCA-Designer. V seminarski nalogi najprej predstavimo osnove QCA, nato pregledamo Boolovo specifikacijo 1-bitnega komparatorja in njegove osnovne logične funkcije. Potem razložimo postopek njegove pretvorbe v reverzibilno obliko, kjer izpeljemo novo resničnostno tabelo in definiramo dodatne vhode ter izhode, potrebne za reverzibilno delovanje. Sledi opis uporabljenih Feynmanovih in Fredkinovih vrat ter predstavitev njihove implementacije v orodju QCADesigner, s katerim izvedemo tudi simulacijo in analiziramo pravilnost delovanja zasnovanega reverzibilnega komparatorja.

22.1 Uvod

Komparator je eno izmed osnovnih kombinacijskih logičnih vezij, ki omogoča primerjanje dveh binarnih vrednosti in določa njihovo medsebojno relacijo. Na izhodu dobimo vrednosti s katerimi lahko preprosto ugotovimo, ali je prva vrednost večja, manjša ali enaka drugi. Takšna vezja so ključna v številnih digitalnih sistemih, kot so aritmetično-logične enote, procesorji, pomnilniški moduli in sistemi za obdelavo signalov, kjer primerjanje predstavlja osnovno operacijo pri odločanju in krmiljenju.

Pri klasični realizaciji logičnih vezij se informacija med procesiranjem delno izgublja. Z razvojem reverzibilne logike se odpira možnost načrtovanja vezij, pri katerih so vse vhodne spremenljivke enolično določene iz izhodnih stanj. Reverzibilna vezja tako ohranjajo informacijo in zmanjšujejo izgube energije, kar je ključno za nadaljnji razvoj učinkovitih digitalnih in kvantnih sistemov.

V tej seminarski nalogi obravnavamo zasnovo in analizo reverzibilnega komparatorja, pri katerem zagotovimo, da ima vsaka kombinacija vhodov enoličen izhod in obratno. Osnovna ideja temelji na razširitvi klasičnega 1-bitnega komparatorja v

reverzibilno obliko z dodajanjem dodatnih vhodov in izhodov, ki ohranjajo informacijsko celovitost sistema.

Pri tem smo uporabili dve osnovni vrsti reverzibilnih logičnih vrat. Prva so Feynmanova vrata, ki omogočajo kopiranje in negacijo signalov, ter Fredkinova vrata, ki izvajajo kontrolirano zamenjavo vhodnih signalov in omogočajo realizacijo logičnih funkcij, kot je konjunkcija. S kombinacijo teh dveh vrst vrat je mogoče zgraditi popolnoma reverzibilen 1-bitni komparator, ki ohranja enako funkcionalnost kot klasični, hkrati pa zadosti pogojem reverzibilnosti.

22.2 QCA

Quantum cellular automaton (QCA) predstavlja eno izmed možnih alternativ klasičnih CMOS tehnologij. QCA ne temelji na toku električnega naboja, temveč na razporeditvi elektronov znotraj kvantnih pik, kar omogoča bistveno manjšo porabo energije ter potencialno veliko višjo gostoto integracije.

22.2.1 Osnove

Osnovni gradniki vseh QCA vezji je kvantna celica, ki je običajno sestavljena iz štirih kvantnih pik, razporejenih na ogliščih kvadrata. V celico sta ujeta dva elektrona, ki zaradi medsebojnega odboja zasedeta diagnoalno nasprotni kvantni piki. Tako lahko celica zavzame eno izmed dveh stabilnih kofiguracij, ki jih interpretiramo kot logični stanji 0 in 1 (polariteta -1 ali +1).

Informacija se torej prenaša z interakcijo med sosednjimi celicami, kjer se polariteta ene celice prenese na naslednjo, brez dejanskega toka elektronov. Zaradi tega je QCA koncept primeren za nizkoenergijska logična vezja.

22.2.2 Vodila

QCA vodila so zaporedja celic, ki omogočajo prenos logične informacije od vhoda do izhoda. Obstajata dve osnovni vrsti vodil:

- **Ravno vodilo:** kjer celice nizamo v linijo. Polariteta se stabilno prenese vzdolž niza.
- **Diagonalno vodilo:** ki omogoča spremembo smeri signala ali kompakten prenos v dvodimenzionalnih postavitvah.

Pri prenosu signala je pomembna raba takterizacije, ki omogoča nadzor nad propagacijo in je ključna za delovanje kompleksnejših struktur. Take cone določajo, ali celica trenutno “zadržuje”, “sprošča” ali “sprejema” informacijo.

22.2.3 Vrata

QCA tehnologija omogoča realizacijo logičnih funkcij preko interakcij med celicami, zato so logična vrata strukturirana drugače kot v klasičnih CMOS tehnologij. Ključni gradnik skoraj vseh logičnih funkcij v QCA je majoritetno (večinsko) stikalo, iz katerega lahko tvorimo AND, OR in kompleksnejše logične operacije. Poleg tega pomembno vlogo igra tudi invertot, ki spreminja polaritetno signala.

Majoritetna vrata imajo tri vhodne celice in eno izhodno celico. Izhod prevzame polariteto, ki jo ima večina vhodov. Matematično lahko delovanje predstavimo kot:

$$M(A,B,C) = AB + AC + BC \tag{22.1}$$

kjer je M izhod majoritetnih vrat A, B, C so pa vhodi. To pomeni:

- če sta vsaj dva vhoda 1, bo izhod 1,
- če sta vsaj dva vhoda 0, bo izhod 0.

Majoritetna vrata so osnova za vse logične funkcije v QCA saj lahko z ustreznimi vezavami vhodov ustvarimo običajna logična vrata:

- **AND funkcija:** en vhod nastavljen na $0 \rightarrow M(A,B,0) = AB$
- **OR funkcija:** en vhod nastavljen na $1 \rightarrow M(A,B,1) = A + B$

Ta preprost in kopakten pristop omogoča izjemno učinkovito implementacijo logike na nanometrski skali.

22.2.4 Ura

Delovanje QCA vezji je tesno povezano s t.i. taktiranjem (clocking). Za razlio od klasičnih vezij, kjer ura zgolj določa časovno usklajenost med elementi, v QCA ura aktivno vpliva na energijsko porabo celic, s čimer nadzoruje prenos informacij med njimi.

QCA uporablja štiri faze ure, ki se neprekinjeno ponavljajo:

- **Switch (preklop):** celica postopoma prevzame polariteto vhodnih celic.
- **Hold (zadrževanje):** energijska bariera je visoka, celica stabilno ohranja svojo polariteto.
- **Release (sproščanje):** bariera se znižuje, celica izgublja svojo informacijo.
- **Realx (relaksacija):** celica se vrne v nevtralno stanje brez polaritete.

Celice so razdeljene v zamikane ure (clock zones), kar omogoča kontroliran prenos podatkov. Signali se tako premikajo od cone ute 0 do cone ure 3 in se nato vračajo na začetek. To omogoča delovanje vodil, logičnih vrat in sekvenčnih struktur (npr. registrov).

Taktiranje je ključno za stabilnost vezij, saj zagotavlja pravilno propagacijo signala in preprečuje nenadzorovano spreminjanje polaritet.

22.2.5 Pomen

QCA predstavlja zanimivo raziskovalno področje zaradi svojih potencialnih prednosti pred CMOS tehnologijo:

- **Izjemno majhna poraba energije:** ker ni električnega toka, temveč zgolj preurejanje nabojev.
- **Visoka gostota integracije:** celice so lahko nanometrske velikosti.
- **Hitro delovanje:** zaradi majhnih razdalj in narave interakcij.

Čeprav je tehnologija še vedno večinoma v raziskovalni fazi, se QCA smatra kot ena izmed možnih platform za nadaljnje miniaturiziranje elektronike ter kot obetavna alternativa klasičnim silicijevim vezjem.

22.3 Boolova specifikacija 1 bitnega komparatorja

Binarni komparator je kombinatorno logično vezje, ki pokaže, ali je binarni bit večji, manjši ali enak drugemu binarnemu bitu. Za realizacijo 1-bitnega komparatorja potrebujemo dva vhoda A in B ter 3 možne izhode L, G in E, katere definiramo kot:

- G = ($A >$B)
- E = ($A =$B)
- L = ($A <$B)

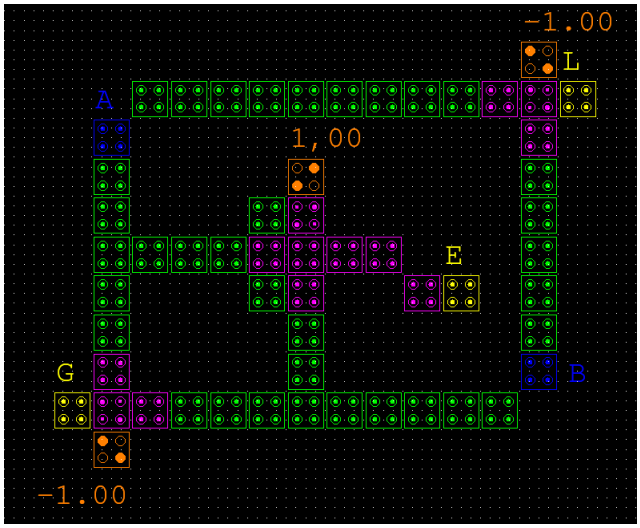
A	**B**	**A > B**	**A = B**	**A < B**
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Tabela 22.1: Resničnostna tabla za 1-bitni komparator.

Izodi G, E in L pokažejo v kakšnem razmerju sta si A in B, te lahko izračunamo po sledečih formulah:

$$G(A > B) = A \cdot \overline{B}$$
$$E(A = B) = A \odot B$$
$$L(A < B) = \overline{A} \cdot B$$

Na podlagi zgornjih enačb smo zasnovali in realizirali prvo različico 1-bitnega komparatorja (Slika 22.1). S tem korakom smo postavili temelje za nadaljnje nadgradnje in optimizacije komparatorja,



Slika 22.1: 1-bitni komparator

Resničnostna tabela predlaganega vezja jasno pokaže, da prvotno definirana funkcija ni reverzibilna, saj se število vhodov in izhodov razlikuje [124]. Da bi zagotovili reverzibilnost, smo vhodnim spremenljivkam dodali dve konstanti z vrednostjo 0. Na Izhod pa smo dodali kar preslikano vrednost A.

A	**B**	**0**	**0**	**izodni A**	**A > B**	**A = B**	**A < B**
0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	1
1	0	0	0	1	1	0	0
1	1	0	0	1	0	1	0

Tabela 22.2: Resničnostna tabla za 1-bitni reverzibilni komparator.

22.4 Prvi postkus implementacije 1-bitnega komparatorja

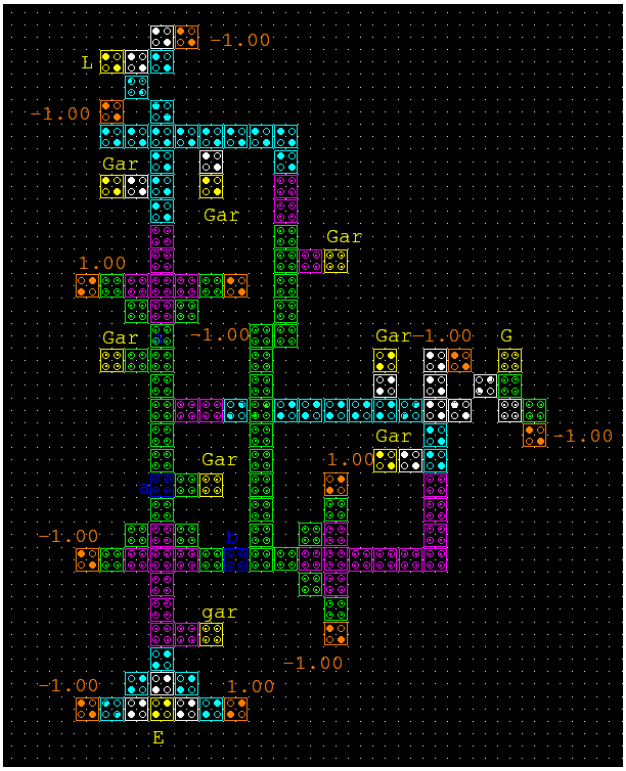
Naša implementacija 1 bitnega reverzibilnega komparatorja je sestavljena iz štirih Feynmanovih vrat in iz dveh Toffolijevih vrat. Vzor tega vezja je bil iz članka [193].

Sestava vrat:

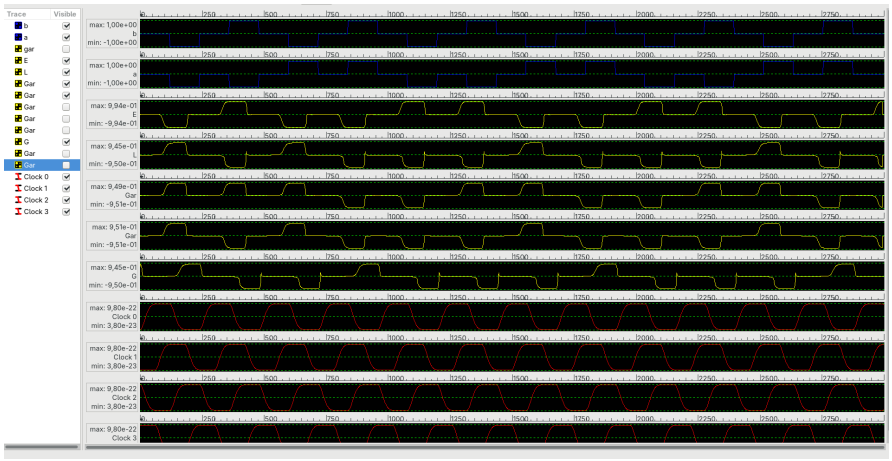
- 4 × Feynmanova (CNOT) vrata, ki se uporabljajo predvsem za negacijo signalov na podlagi XOR vrat.
- 2 × Toffolijeva (CCNOT) vrata, ki pa nam omogočajo uporabo AND vrat za primerjanje neenakosti

Vezje ima 8 neuporabnih (angl. garbage) izhodov, ki so potrebni za ohranjanje reverzibilnosti, vendar ne prispevajo k primarni funkciji komparatorja.

Celotno vezje ima 2 periodi zamude.



Slika 22.2: Realizacija 2-bitnega komparatorja



Slika 22.3: Realizacija 2-bitnega komparatorja

22.5 Drugi postkus implementacije 1-bitnega komparatorja

V nadaljevanju je predstavljen naš drugi poskus implementacije 1-bitnega komparatorja.

Naš načrt delovanja tega komparatorja je sledeč:

- Vhodi: $(A, B, 0, 0)$
- Izhod: (A, B, G, L)

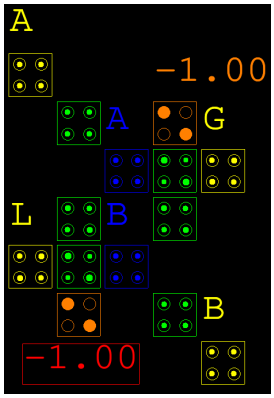
Kjer so:

- A, B binarna vhoda v vezju, ki ju primerjamo.
- G (greater), ki ima vrednost 1, če velja $A > B$, drugače 0.
- L (less), ki ima vrednost 1, če velja $A < B$, drugače 0.
- če pa velja $A = B$, potem pa sta oba G in L enaka 0.

G in L so izračunani po sledeči formuli:

$$G = A \wedge \neg B, \quad L = \neg A \wedge B$$

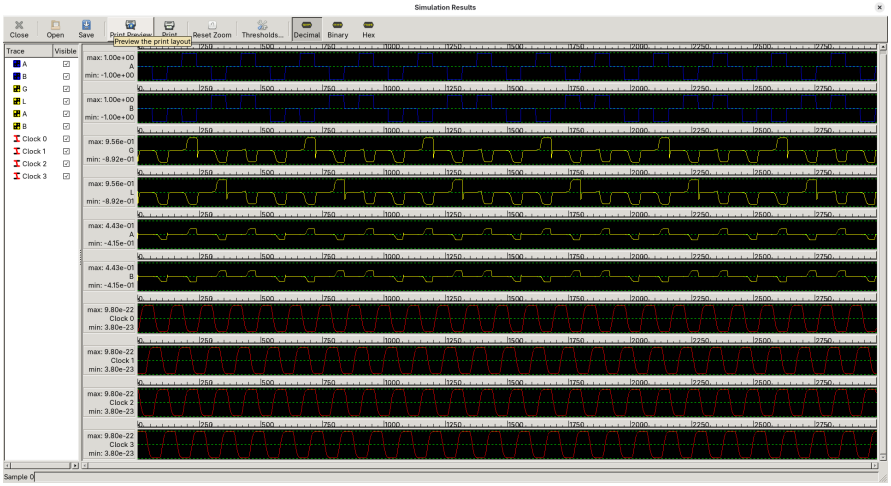
S temi načrti smo naše vezje sestavili tako:



Slika 22.4: Realizacija 1-bitnega komparatorja

Ker oba izhoda potrebujeta oba vhoda, jih postavimo na sredino vezja. Vrata `AND` implementiramo z *majoritetnimi* vrati, medtem ko `NOT` enostavno dobimo z zamikom *QCA* celic. Izhod majoritetnih vrat predstavlja logično vrednost večine vhodov. Na ta način lahko enako strukturo uporabimo tako za `AND` kot za `OR` vrata. Razlike so samo v vrednosti tretjega vhoda. V našem primeru jo nastavimo na -1, kar ustreza logični vrednosti 0.

Ker so vse povezave med celicami neposredne in izhodi niso odvisni od prejšnjih časovnih stanj, vezje deluje *asinkrono*, torej brez ure. Logične vrednosti se prenesejo po celicah takoj, ko so vhodni signali stabilni, zato ni potrebe po sinhronizaciji s pulznim signalom.



Slika 22.5: Simulacija 1-bitnega komparatorja

Skozi simulacijo na grafu lahko opazimo, da izhodi sledijo pravilni logiki. Preverimo za logiko A $<$ B:

- Želene Vhodne Vrednosti (A=0, B=1):
  - $A = 0$ ($\rightarrow$ High, +1)
  - $B = 1$ ($\rightarrow$ Low, -1)
- Pričakovani Izhod:
  - Ker je $A = 0$ in $B = 1$, velja $A < B$.
  - G (Greater) mora biti 0 (Low, -1).
  - L (Lesser) mora biti 1 (High, +1).
- Pregled Simulacije (malo po 750):
  - Vhodne sledi: A je na -1, B pa na +1. Vhod je tako $A = 0$ in $B = 1$.
  - Izhodne sledi: Signal G je na -1 (se pravi logična 0), signal L pa se dvigne na +1 (se pravi logična 1).

Simulacija torej v tem trenutku potrjuje logični pogoj A $<$ B.

22.6 Realizacija 2-bitnega komparatorja

Našo raziskavo smo nadaljevali z razširitvijo našega drugega poskusa implementacije 1-bitnega komparatorja na 2-bitni komparator.

Naš načrt delovanja dvobitnega komparatorja:

- Vhodi: $(A_1, A_2, B_1, B_2, 0, 0)$
- Izhodi: $(A_1, A_2, B_1, B_2, G, L)$

Kjer so:

- A_1, A_2 dva bita prvega števila.
- B_1, B_2 dva bita drugega števila.
- G (greater), ki ima vrednost 1, če velja $A > B$, drugače 0.
- L (less), ki ima vrednost 1, če velja $A < B$, drugače 0.
- če pa velja $A = B$, potem pa sta oba G in L enaka 0.

Za izračun uporabimo dve kopiji prej implementiranega enobitnega komparatorja:

$$(G_1, L_1) = \text{compare}(A_1, B_1), \quad (G_2, L_2) = \text{compare}(A_2, B_2)$$

G in L so izračunani v dveh korakih:

$$G_1 = A_1 \wedge \neg B_1, \quad L_1 = \neg A_1 \wedge B_1, \quad G_2 = A_2 \wedge \neg B_2, \quad L_2 = \neg A_2 \wedge B_2$$

Se pravi je naša logična tabela sledeča:

Na podlagi tabele je število A večje od B ($G = 1$), če je $G_1 = 1$, ali če so vrednosti:

G_1	L_1	G_2	L_2	**G**	**L**
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	0
0	1	0	0	0	1
0	1	0	0	0	1
0	1	1	0	0	1
1	0	0	0	1	0
1	0	0	1	1	0
1	0	1	0	1	0

Tabela 22.3: Resničnostna tabla računanja pri 2-bitnem komparatorju.

$$G_1 = 0,\ L_1 = 0,\ G_2 = 1,\ L_2 = 0$$

Zapišemo logično formulo:

$$G = G_1 \vee (\neg G_1 \wedge \neg L_1 \wedge G_2 \wedge \neg L_2)$$

Ker za vse vrstice, kjer je $G_2 = 1$, velja $L_2 = 0$, lahko poenostavimo:

$$G = G_1 \vee (\neg G_1 \wedge \neg L_1 \wedge G_2)$$

Podobno za L velja: $L = 1$, če je $L_1 = 1$, ali če so vrednosti:

$$G_1 = 0,\ L_1 = 0,\ G_2 = 0,\ L_2 = 1$$

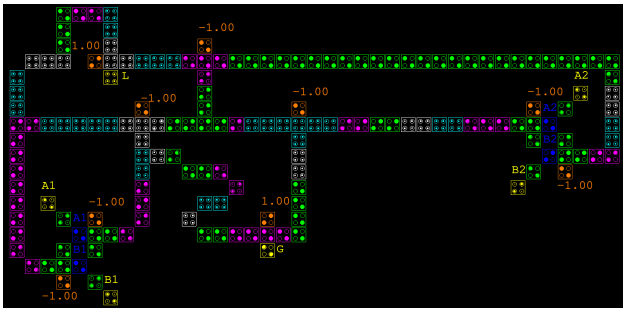
Formulo zapišemo:

$$L = L_1 \vee (\neg G_1 \wedge \neg L_1 \wedge \neg G_2 \wedge L_2)$$

Ker za vse vrstice, kjer je $L_2 = 1$, velja $G_2 = 0$, lahko poenostavimo:

$$L = L_1 \vee (\neg G_1 \wedge \neg L_1 \wedge L_2)$$

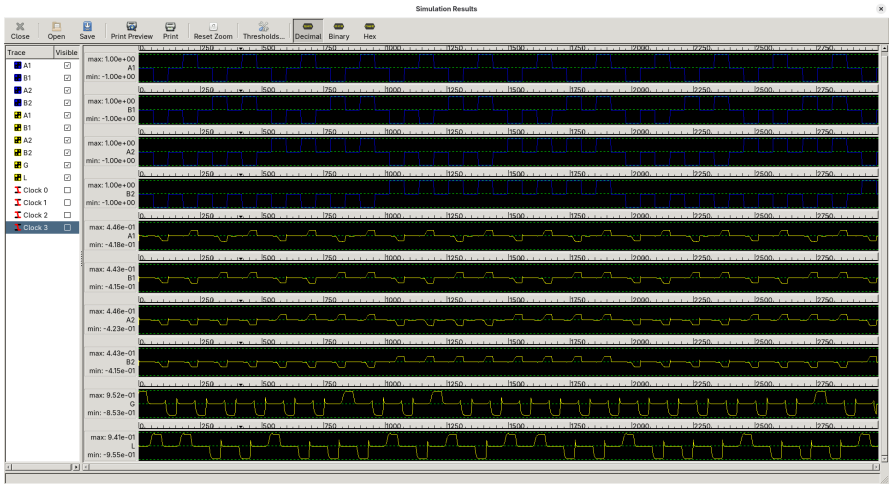
S sledenjem naslednjih načrtov, smo sestavili vezje:



Slika 22.6: Realizacija 2-bitnega komparatorja

Za implementacijo dvo-bitnega komparatorja uporabimo *UXRO* (**Universal Crossbar Routing Output**) za učinkovito razporeditev vodil med enobitnimi komparatorji. UXRO omogoča, da izhode prve in druge primerjave fleksibilno povežemo na logične formule za G in L brez potrebe po dodatnih logičnih stopnjah ali ure.

Končna implementacija poteka tako, da kopiramo obstoječi enobitni komparator, nastavimo vhode prvega na (A_1, B_1) in drugega na (A_2, B_2). Izhode (G_1, L_1, G_2, L_2) povežemo po zgornjih formulah v globalna izhoda G in L, pri čemer uporabimo negacijo, majoritetna vrata in nekaj inovativenh povezav v UXRO, da zagotovimo pravilno delovanje vezja.



Slika 22.7: Simulacija 2-bitnega komparatorja

Skozi simulacijo na grafu lahko opazimo, da izhodi sledijo pravilni logiki. Preverimo za logiko A $<$ B:

- Želene Vhodne Vrednosti (A=01, B=11):
  - $A_1 = 0$ ($\rightarrow$ Low, -1)
  - $A_2 = 1$ ($\rightarrow$ High, +1)
  - $B_1 = 1$ ($\rightarrow$ High, +1)
  - $B_2 = 1$ ($\rightarrow$ Low, +1)
- Pričakovani Izhod:
  - Ker je $A = 1$ in $B = 3$, velja $A < B$.
  - G (Greater) mora biti 0 (Low, -1).
  - L (Lesser) mora biti 1 (High, +1).
- Pregled Simulacije (okoli 1250):

- Vhodne sledi: A_1 je na -1, A_2 je na +1, B_1 je na +1, B_2 je na +1. Vhod je tako $A = 01$ in $B = 11$.
- Izhodne sledi: Signal G je na -1 (se pravi logična 0), signal L pa se dvigne na +1 (se pravi logična 1).

Simulacija torej v tem trenutku potrjuje logični pogoj A $<$ B.

22.7 Zaključek

Če povzamemo našo seminarsko nalogo, smo uspešno implementirali reverzibilen komparator. Implementirali smo dve rešitve. Prvi implementaciji je bil vzor članek [193]. Ker je pa že ta bil kompleksnejši in bolj zapleten, smo se odločili implementirati drugi pristop, katerega smo razširili še na dvo bitno rešitev.

Raziskava se lahko nadaljuje na še večjo razširitev (na 4, 8, 12 ...). Skozi čas bi lahko ustvarili univerzalni komparator, ki bi lahko primerjal katerokoli možni števili.

Appendix

Poglavje 23
Primerjava simulatorjev za implementacijo Shorovega algoritma

Katarina Perman, David Goljat

Povzetek Kvantni računalniki so že dolgo raziskovano področje, ki obljublja prihodnost, kjer bo možno počasne in zahtevne algoritme izvesti mnogo hitreje. Algoritmi, ki bi na klasičnih računalnikih trajali milijone let, bi na kvantnem računalniku potrebovali le nekaj ur. V tej seminarski nalogi se bomo spustili v raziskovanje področja kvantnega računalništva. Natančneje, ogledali si bomo proces implementacije Shorovega algoritma na simulacijskih orodjih Qiskit in Cirq - to sta knjižnici za programski jezik python, ki znata simulirati delovanje kvantnih računalnikov, Qiskit pa tudi dovoljuje njihovo zaganjanje na dejanski kvantni arhitekturi. Najprej bomo na kratko razložili Shorov algoritem in kako deluje, med drugim tudi zakaj kvantni del, torej algoritem za ocenjevanje faze, ključno prispeva k pospeševanju faktorizacije števil. Potem pa se bomo podrobno poglobili v proces implementacije, izpostavili nekaj ključnih delov tega procesa in na koncu ocenili obe orodji.

23.1 Uvod

Faktorizacija števil na prvi pogled izgleda kot nedolžen algoritem. Klasični računalniki za to potrebujejo eksponentno veliko računskih operacij, saj je trenutno najboljši klasični algoritem za faktorizacijo razširjena različica Eratostenovega sita, ki deluje v sub-eksponentnem času. Na to časovno zahtevnost pa se zanašajo šifre na osnovi RSA, ki kot podlago uporabljajo zmnožek dveh zelo velikih praštevil, ki jih je za klasične računalnike praktično nemogoče razbiti. Shorov algoritem pa ima polinomsko časovno zahtevnost, kar bi znalo ogrožati računalniško šifriranje, kot ga poznamo danes.

23.2 Kvantni računalniki

Kvantni računalniki delujejo na osnovi kubitov oz. kvantnih bitov. Ti so v fizični implementaciji najpogosteje realizirani kot npr. kvantne celice, ki s postavitvijo elektronov izrazijo stanje 0 ali 1. Ker delujejo na osnovi naboja, vplivajo tudi na sosedne kubite, in s dosežejo obnašanje podobno vezju. Ti kubiti imajo dostop do stanja, ki se imenuje superpozicija, ki kvantnim računalnikom omogoča obdelavo več različnih stanj hkrati. To stanje je izraženo kot verjetnost, da v danem trenutku v nekem qubitu izmerimo vrednost 0 ali 1. Ker kvantna vezja še vedno delujejo tudi v superpozicioniranem stanju, lahko vezje poženemo enkrat in nato le izmerimo vrednosti za več možnih rezultatov.

Leta 2001 je IBM faktorizirala število 15 v $3 * 5$ s Shorovim algoritmom [194]. Uporabili so implementacijo NMR, ki za kubite uporablja molekule. Za njihovo implementacijo so porabili sedem kubitov. Za tem je več skupin doseglo faktorizacije števila 15 s Shorovim algoritmom, z različnimi fizičnimi pristopi. Leta 2012 so faktorizirali tudi število 21 [195]. Ker pa so vezja za Shorov algoritem odvisna od konkretnih podatkov, so bile vse prejšnje implementacije precej poenostavljene. Leta 2016 pa so dosegli faktorizacijo števila 15 brez teh poenostavitev [196]. Ker pa je na kvantnem nivoju zelo veliko šuma, potrebujemo za popravljanje napak v kvantnih računalnikih sorazmeroma veliko kubitov, zato so kvantni računalniki še daleč od tega, da bi lahko ogrožali kibernetsko varnost.

23.3 Shorov algoritem

Časovna zahtevnost Shorovega algoritma za kvantne računalnike je ocenjena kot $O((\log N)^2(\log\log N)(\log\log\log N))$. Obstajajo implementacije, ki to časovno zahtevnost še izboljšajo z boljšimi algorimi za množenje, a te za nas niso relevantne. Shorov algoritem je sestavljen iz dveh delov: algoritem za klasični računalnik, ki problem prevede na iskanje reda, in algoritem za kvantni računalnik, ki iskanje reda prevede na ocenjevanje faze.

23.3.1 Klasični del

Zaradi trivialnosti preverjanja, ali je N lih ali sod, lahko predpostavimo, da je N lih. Prav tako lahko predpostavimo, da ima N le dva praštevilska prafaktorja, v obratnem primeru pa lahko rekurzivno poženemo algoritem na dobljenih faktorjih in tako v celoti faktoriziramo N. Najprej izberemo naključno število a, ki ustreza $2 \leq a \leq N$. Nato izračunamo $d = \gcd(a,N)$. Če je d več kot 1, smo našli netrivialen prafaktor, in se algoritem lahko zaključi. Če pa je d enak 1, pa to pomeni, da je a element v multiplikativni grupi celih števil po modulu N, torej lahko najdemo tak najmanjši

r, da je $a^r \mathrm{mod} N = 1$, čemur rečemo red a po modulu N. Za to uporabimo kvantni algoritem, saj je ključna pohitritev v primerjavi s klasičnim računalnikom.

Ko dobimo iskani r, vemo, da N deli $(a^r - 1)$, kar zapišemo kot $N|(a^r - 1)$. Tu predpostavimo, da je r sod - če ni, izberemo nov a in postopek ponovimo. Ker je $(a^r - 1)$ razlika kvadratov, lahko $(a^r - 1)$ razbijemo na $(a^{\frac{r}{2}} - 1) * (a^{\frac{r}{2}} - 1)$. N ne more deliti $(a^{\frac{r}{2}} - 1)$, saj bi to bilo v protislovju z dejstvom, da je r red a po modulu N. Tu pride na vrsto ključen trik Shorovega algoritma. Če N deli $(a^{\frac{r}{2}} + 1)$, iz a ne moremo dobiti netrivialnega faktorja za N. Če pa N ne deli $(a^{\frac{r}{2}} + 1)$, pomeni, da en od prafaktorjev N deli $(a^{\frac{r}{2}} - 1)$, en pa $(a^{\frac{r}{2}} - 1)$, zato nam algoritem zagotavlja, da $\gcd(N, (a^{\frac{r}{2}} + 1))$ vrne netrivialen faktor števila N.

23.3.2 Kvantni del

Kvantni del algoritma je zadolžen za iskanje reda a po modulu N. Problem prevede na iskanje lastnih vrednosti matrik oz. ocenjevanje kvantne faze. Lastne vrednosti unitarne matrike imajo vse naslednjo lastnost: $|\lambda| = 1$. To pomeni, da lahko vsako lastno vrednost zapišemo kot $e^{2\pi i\theta}$, kjer θ označuje 'fazo' vrednosti, saj je sama vrednost periodična. Zato lahko predpostavimo $a \leq \theta < 1$.

Ključni mehanizem, ki stoji za tem algoritmom, je kvantna Fourierova transformacija - za transformacijo funkcije f kubite prenese v bazo, ki opisuje frekvence periodičnosti v f. Fazo f lahko dobro ocenimo tako, da zanjo definiramo unitarni operator U in kontrolirano uporabljamo U, U^2, U^4, ... $U^{2^{m-1}}$ nad vhodnim vektorjem, kjer je m število kubitov, na katerem delujejo operatorji U.

Izkaže se, da če definiramo permutacijsko matriko za množenje z a po modulu N, ki vse neveljavne (torej večje ali enake N) vrednosti preslika same vase, dobimo unitarno matriko. Prav tako se izkaže, da je periodičnost take matrike povezana z algoritmom za iskanje faze, saj je sam ta operator tudi periodičen. Ker lahko kvantni računalnik obdela veliko količino možnih stanj hkrati, to omogoča hitro računanje reda a po modulu N.

23.4 Implementacija

V tem delu seminarske naloge ocenimo implementacijo v orodjih Qiskit in Cirq.

23.4.1 Qiskit

Qiskit je odprtokodna Python kvantna knjižnica. Uporabimo jo lahko za programiranje kvantnih algoritmov, transpilacijo (pretvorbo algoritma v izvedljivo vezje) in simulacijo vezij.

Omogoča definicijo kvantnih registrov in kvantnih operacij, uporabo standardnih kvantnih operacij (H, CX, S, T, ...) in sestavljanje bolj kompleksnih kvantnih rutin (QFT, Grover, Shor, ...), gradnjo poljubnih unitary operatorjev, kar poenostavi implementacijo, optimizacijo števila vrat, znižanje qbit globine ter prilagoditev topologiji fizičnega čipa.

Qiskit ima dva glavna dela za simuliranje: Aer (visoko-zmogljivi simulatorji) ter Primitive (Sampler, Estimator) - API, ki je bolj abstrakten ter deluje tudi na pravih napravah.

Implementacijo Shorovega algoritma opravimo z generiranjem modularnih operacij, ocenjevanjem periodičnosti preko QFT, pretvorbo meritev v približek racionalnega števila ter uporabo klasičnega dela za izračun faktorjev.

Ena zanimivejših lastnosti je generator unitarnih vrat. Qiskit omogoča, da ustvarimo poljuben unitarni operator, tudi tiste, ki niso v standardni knjižnici (X, H, CX, ipd.). To je ključno, ker modularna množenja zahtevajo permutacijsko matriko, ki je zelo specifična – Qiskit bi je sam ne znal sintetizirati.

```
def mod_mult_gate(b):
    """
    Modular multiplication gate
    from permutation matrix.
    """
    if gcd(b, N) > 1:
        print(f"Error: gcd({b},{N}) > 1")
    else:
        # print("Call to mod_mult_gate, got n: ", n)
        U = np.full((2**n, 2**n), 0)
        for x in range(N):
            U[b * x % N][x] = 1
        for x in range(N, 2**n):
            U[x][x] = 1

        G = UnitaryGate(U)
        G.name = f"M_{b}"
        return G
```

Uporabljamo kontrolne qbite ter tarčne qbite za modularno množenje, kar bomo podali skozi inverzno Fourierjevo transformacijo. Za dejanski izračun bomo uporabili približek kvantne faze.

```
def qua_order_subroutine(a):
    # Number of qubits
    print("call to qua_order_subroutine with a:", a,
              "; num_target: ", n,
               ", num_control: ", num_control)
    # List of M_b operators in order
    k_list = range(num_control)
```

```
b_list = [a2kmodN(a, k) for k in k_list]
print("calling a2kmodN for k_list: ", k_list,
          "; b_list is ", b_list)

# Initialize the circuit
control = QuantumRegister(num_control, name="C")
target = QuantumRegister(num_target, name="T")
output = ClassicalRegister(num_control, name="out")
circuit = QuantumCircuit(control, target, output)

# Initialize the target register to the state |1>
circuit.x(num_control)

# Add the Hadamard gates and controlled versions
# of the multiplication gates
for k, qubit in enumerate(control):
    circuit.h(k)
    b = b_list[k]
    if b != 1:
        circuit.compose(
            mod_mult_gate(b).control(),
            qubits=[qubit] + list(target),
            inplace=True
        )
    else:
        break  # M1 is the identity operator

# Apply the inverse QFT to the control register
circuit.compose(
       QFT(num_control, inverse=True),
       qubits=control,
       inplace=True
 )

# Measure the control register
circuit.measure(control, output)
```

Zakaj AerSimulator?

Qiskit primitivi so namenjeni abstrakciji nad pravimi napravami, ne pa nujno nad poljubnimi unitarijami z velikimi matrikami.

Primitivi imajo določene omejitve: pogosto ne sprejmejo velikih unitarnih matrik, lahko zahtevajo, da je vezje transpilirano za specifičen backend, morda ne izvajajo vseh operacij (zlasti za custom UnitaryGate), lahko imajo omejitve nad številom qubitov v spominu.

Aer simulator, v primerjavi sprejme katerakoli vrata (tudi UnitaryGate), podpira statevector, density matrix, stabilizer modele, nima omejitev ”backend compatibility-ja ter je hitrejši in bolj robusten za prototipna vezja.
Interpretacija meritev poteka na naslednji način:

- Izberemo najpogostejše bitne nize,
- Bitne nize spremenimo v njihovo decimalno vrednost,
- delimo z (2^n),
- Penostavimo ulomek,
- Poračunamo oceno reda funkcije, ter
- V končni fazi najdemo dejanske faktorje.

```
bitstring = list(counts_keep.keys())[num_attempt]
num_attempt += 1

# Find the phase from measurement
decimal = int(bitstring, 2)
# phase = k / r
phase = decimal / (2**num_control)

# Guess the order from phase
frac = Fraction(phase).limit_denominator(N)
r = frac.denominator  # order = r
print("Loop - decimal: ", decimal,
          "; phase: ", phase, "; frac: ", frac)


# Guesses for factors are gcd(a^{r / 2} ± 1, 15)
if phase != 0 and r % 2 == 0:
    x = pow(a, r // 2, N) - 1
    if x == 0:
        continue
    d = gcd(x, N)
    print("factor guesses - x: ", x, "; d: ", d)
    if d > 1:
        FACTOR_FOUND = True
        print(f"Non-trivial factor found: {d}")
```

Simulatorji imajo določene lastnosti, na katere moramo biti pozorni. UnitaryGate je občutljiv konstrukt, zato morajo vhodne dimenzije biti natančne. Že majhna napaka pomeni eunitarnost, kar pomeni, da se vezje ne bo ‘transpiliralo‘. Če pa je matrika prevelika, se program sesuje. Pozorni moramo biti tudi na to, da StatevectorSampler ni enakovreden simulatorju. Primitives API ni implementacija simulatorjev in zahteva drugačen backend, prav tako pa pogosto ne podpira poljubnih enotaričnih matrik, vse skupaj pa ni dobro dokumentirano. Posebno pozornost moramo nameniti tudi velikosti vezja: že pri N=15 uporabljamo 8 kontrolnih qubitov, 4 tarčne

qubite ter UnitaryGate dimenzije 16*16. Pri večjih N velikost eksponentno naraste, kar povzroči bistveno večjo porabo pomnilnika ter daljši čas izvedbe simulacije.

Prednosti Qiskita:

- Qiskit je odličen za raziskovanje kvantnih algoritmov, saj je zelo enostavno sestaviti kvantno vezje ter ga simulirati in testirati.
- Podpira prave IBM Q naprave, kar pomeni, da lahko simuliran prototip brez dodatnega prevajanja zaženemo na dejanski strojni opremi.
- Omogoča ustvarjanje poljubnih unitarnih operacij - je ena redkih knjižnic, ki to omogoča neposredno.
- Vsebuje dober transpiler, ki efektivno optimizira število vrat, izračuna topologijo ter minimizira napake.
- Aer simulator je zelo zmogljiv, komponente kot so Statevector, stabilizer, density-matrix, modeli šuma in ostalo, so optimalno implementirani.

Slabosti Qiskita:

- Dokumentacija ni vedno jasna. Ni očitno, kdaj se uporablja simulator, kdaj je potreben backend ter zakaj nekatere operacije niso podprte.
- Čeprav je UnitaryGate dober konstrukt, pri večjih vezjih hitro postane neuporaben, saj velikost raste eksponentno. Za večje matrike hitro presežemo tipično velikost pomnilnika.
- Qiskit je zelo zmogljiva knjižnica, kar pa prinese tudi veliko kompleksnost. Različne konfiguracije ter simulatorji nas lahko hitro zmedejo.

23.4.2 Cirq

Cirq je Google-ov kvantni framework, zasnovan za kvantna vezja, ki delujejo na linearnem nizu qbitov. Qubiti so običajno predstavljeni kot LineQubit.
Tako kot Qiskit, tudi Cirq omogoča:

```
gate = cirq.MatrixGate(myUn)
```

To je ekvivalent Qiskit-ovemu:

```
UnitaryGate(U)
```

Cirq je pri tem hitrejši in bolj stabilen, saj interne strukture niso tako kompleksne. Cirq ima ‘Simulator()‘, ki je idealna simulacija statevector konstrukta; ‘DensityMatrixSimulator()‘ za ‘decoherence‘ modele ter možnosti za simulacijo realnih Sycamore šumov. Vezja logično organizira v momente: Vsa vrata so obdelana v istem časovnem koraku. To daje lažji vpogled v časovne lastnosti algoritmov.

Razlike v implementaciji
Namesto

```
mod mult gate(b).control()
```

Cirq uporablja

```
gate = modmultgate(b).on(*target).controlledby(controlqubit)
```

V Cirq ga moramo torej implementirati sami, saj ni na voljo v knjižnici.

V Qiskitu je AerSimulator ločen objekt, v Cirq-u pa se uporablja

```
sim = cirq.Simulator()
result = sim.run(circuit, repetitions=1000)
```

Cirq je precej bolj enostaven, saj nima "samplerjev", "backendov", tokenov in podobnega. Interpretacija meritev je podobna kot v Qiskitu - Izračun faze, približek ulomka, izračun reda ter izbira kandidata faktorja.
Prednosti Cirq:

- Cirq je preprostejši API za simulacijo. V Qiskitu pogosto ni jasno, kaj je backend, sampler, runtime; v Cirq je proces simulacije poenostavljen.
- Omogoča hitrejše in bolj stabilno delo s poljubnimi unitarijami, saj zna bolje upravljati 'MatrixGate' - pogosto hitreje kot Qiskit.
- Vezja so organizirana v momente, kar pomaga pri optimizacijah časovnih korakov in globini.
- Ima integracijo z Google-ovimi Sycamore procesorji. Če želimo realno kvantno izvajanje nad Sycamore arhitekturo, je Cirq boljša izbira.

Slabosti Cirq:

- Cirq ponuja manj pripravljenih konstruktov kot Qiskit, zato moramo marsikaj implementirati sami. Po privzetem ni QFT, Groverja, Bernsteina-Vazarija.
- Slabša podpora za poljubne tipologije naprav. Qiskit ima močan transpiler, medtem ko je Cirq težje prilagoditi drugim arhitekturam.
- Manj integracije z programskimi paketi. Qiskit ima vizualizacijo, Runtime, Noise-model, AER, Ignis, Terra. Cirq je pri integraciji bolj ohlapen.
- Krajša dokumentacija, kar je delno posledica manjšega števila konstruktov. Za začetnike je lažje začeti v Qiskitu, četudi je dokumentacija občasno nejasna.

23.5 Doprinosi avtorjev

Oba avtorja sta si delo delila enakomerno, meje med doprinosi se ne da jasno začrtati.

Poglavje 24
Seštevanje v kvantnih računalnikih z uporabo QCADesignerja-E

Mitja Kocjančič

Povzetek V članku obravnavamo načrtovanje in implementacijo binarnega seštevalnika v kvantnih celičnih avtomatih (QCA). Predstavljena je kratka razlaga osnov QCA, majoritetnih vrat in uporabe teh elementov pri izvedbi aritmetičnih operacij. Implementacijo polnega seštevalnika smo izvedli z orodjem QCADesignerE ter jo eksperimentalno potrdili s simulacijami. Poleg tega smo pripravili tudi programsko implementacijo v Pythonu s pomočjo knjižnice Qiskit, ki omogoča preverjanje pravilnosti seštevanja s kvantnimi vezji. Rezultati kažejo, da je možno aritmetične operacije učinkovito izvesti v kvantnem okolju, pri čemer se izkažejo prednosti nizke porabe energije, skalabilnosti in zmanjšane kompleksnosti vezij v primerjavi s klasičnimi CMOS rešitvami.

24.1 Uvod

Tradicionalna CMOS tehnologija se v zadnjem času sooča s številnimi izzivi, povezanimi z miniaturizacijo vezij, saj kvantni učinki začenjajo vplivati na obnašanje vezij že pri izjemno majhnih dimenzijah. Kvantno-pikčasti celični avtomati (QCA) se pojavljajo kot obetavna alternativa, ki ponuja bistveno nižjo porabo energije, izboljšano skalabilnost in združljivost z uveljavljenimi procesi izdelave polprevodnikov.

QCA uporablja kvantne pike za predstavitev binarnih stanj 0 in 1. Coulombova interakcija omogoča nadzor nad stanjem polarizacije teh kvantnih pik, ista interakcija pa zagotavlja zanesljivo širjenje informacij po vezjih QCA.

Področje približnega računalništva izkorišča inherentne napake in tolerance za povečanje računske zmogljivosti, izboljšanje energetske učinkovitosti ter zmanjšanje strojne in prostorske kompleksnosti. Priznava, da številne sodobne aplikacije ne zahtevajo popolnoma natančnih rezultatov, zlasti na področjih, kot sta strojno učenje in obdelava multimedije. Aproksimacijska vezja tako proizvajajo rezultate, ki niso

točni, a zadostujejo za takšne aplikacije. Takšna vezja zadostne rezultate dosežejo z manjšo porabo virov kot točna vezja. [124]

24.2 QCA Osnove

Osnovna celica QCA je sestavljena iz štirih kvantnih pik v kvadratnem nizu, povezanih s tunelsko fazno logiko. Med štirimi kvantnimi pikami dve sestavljata elektroni. Dve kvantni piki, ki jo sestavljajo elektroni, sta prisotni diagonalno. Elektron mora začeti pri prvi piki. Elektroni ne morejo zapustiti celice, sicer lahko tunelirajo med pikami.



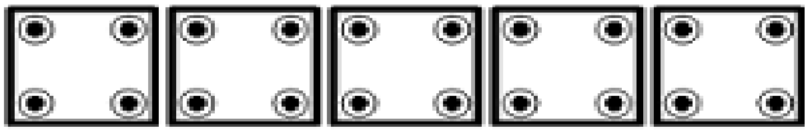
Slika 24.1: Osnovna QCA Celica

Če sta v celici več kot dva elektrona, bo prišlo do Coulombovega odbijanja in elektrona se bosta odbijala v nasprotni smeri. Torej obstajata dve energetsko ekvivalentni polarizaciji osnovnega stanja, in sicer logična 0, ko je P=-1, in logična 1, ko je P=+1.

24.2.1 Celična linija

Linija celic je najpreprostejša matrika QCA. Ker so celice povezane s svojimi sosedi, je v osnovnem stanju vsake celice linije prisotna enaka polarizacija. Zato so elektroni v teh stanjih močno ločeni. Vhod se dovaja na levi konec linije QCA in ima enako polarizacijo po celotni liniji.

V liniji celic QCA je mogoče upravljati do 10 celic z enako logiko, po 10 celicah pa se logika lahko razlikuje od 0 do 1. Ker celice komunicirajo s sosednjimi celicami, ni potrebe po dolgih medsebojnih povezavah. Vhod se dovede v celico na robu levega konca celice QCA, izhod pa se prikaže v celici na robu polja QCA.



Slika 24.2: QCA Linija

24.3 Klasično seštevanje

Klasično binarno seštevanje je temeljna operacija v digitalnih sistemih in služi kot osnova za bolj kompleksna aritmetična vezja. Delovanje binarnega seštevalnika se opira na tri bistvena logična vrata: IN, ALI in XOR. Ta vrata določajo, kako se vhodni biti kombinirajo, da ustvarijo ustrezno vsoto in izhodni prenos.

24.3.1 Vrata AND

Vrata AND oddajo logično 1 le, če so vsi njihovi vhodi enaki 1. V kontekstu binarnega seštevanja se operacija AND uporablja predvsem za generiranje delnih prenosnih signalov.

$$C = A \cdot B,$$

kjer sta A in B binarna vhoda, C pa je rezultat.

24.3.2 Vrata OR

Vrata ALI oddajo logično 1, ko je vsaj eden od njihovih vhodov enak 1. Pogosto se uporabljajo v seštevalnikih za združevanje vmesnih prenosnih signalov. Boolov izraz za vrata OR z dvema vhodoma je

$$Y = A + B.$$

24.3.3 Vrata XOR

Vrata XOR (izključni OR) izpišejo logično 1 le, če se njihovi vhodi razlikujejo. Zaradi te lastnosti ima operacija XOR ključno vlogo pri ustvarjanju bita vsote pri binarnem seštevanju. Boolov izraz za vrata XOR z dvema vhodoma je

$$S = A \oplus B.$$

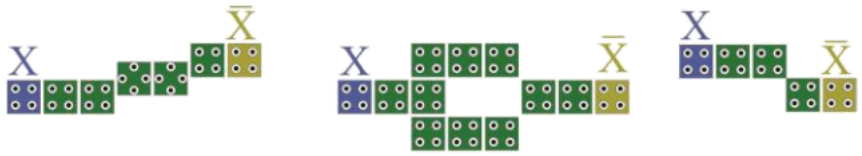
Vrata XOR učinkovito določajo, ali naj se bit vsote preklopi glede na razliko med vhodnima bitima, zaradi česar so temeljna za načrtovanje polseštevalnikov in polnih seštevalnikov.

24.4 Kvantno seštevanje

Kvantni celični avtomati (QCA) omogočajo izvedbo aritmetičnih operacij z večjo hitrostjo in z bistveno manjšo porabo energije v primerjavi s klasičnimi CMOS vezji. Namesto tokov elektronov QCA izkorišča polarizacijo celic, ki se prenaša s kvantnim tuneliranjem. To omogoča realizacijo vseh osnovnih logičnih vrat, potrebnih za seštevanje: negatorja, AND, OR in XOR.

24.4.1 QCA Negator

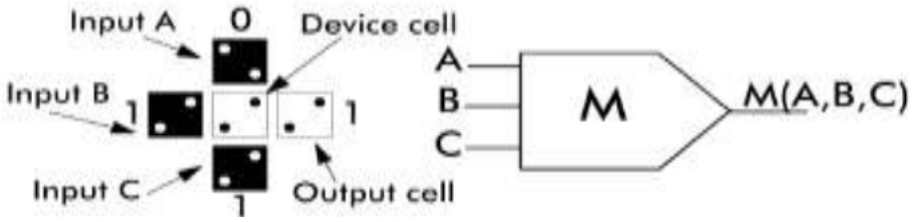
Negator je gradnik logičnega vezja, ki obrne svojo vhodno vrednost; njegova preklopna funkcija ustreza logični negaciji. V okviru kvantnih celičnih avtomatov ga lahko izvedemo tako, da dve QCA-celici povežemo diagonalno kot kaže slika 24.3.



Slika 24.3: QCA Negator implementiran na različne načine

24.4.2 QCA majoritetna vrata

Majoritetna vrata so eden glavnih logi ˇcnih gradnikov v vezjih kvantnih celičnih avtomatov. Izhodna celica se bo polarizirala glede na večinsko polarizacijo vhodnih celic. Večinski vhod opravlja logično funkcijo s tremi vhodi. Ob predpostavki, da so vhodi A, B in C, je logična funkcija večinskega vhoda m(A, B,C) = A . B + B . C + A . C

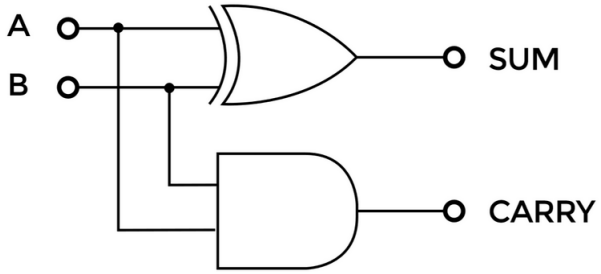


Slika 24.4: QCA Majoritetna vrata

24.5 Seštevalnik

24.5.1 Polovični seštevalnik

Polovični seštevalnik je sposoben sešteti dve binarni števili in izračunati čarry bit", če je rezultat operacije večji, kot ga lahko shrani binarni register. Shema na sliki 24.5. Tabela 24.1 predstavlja pripadajočo pravilnostno tabelo.



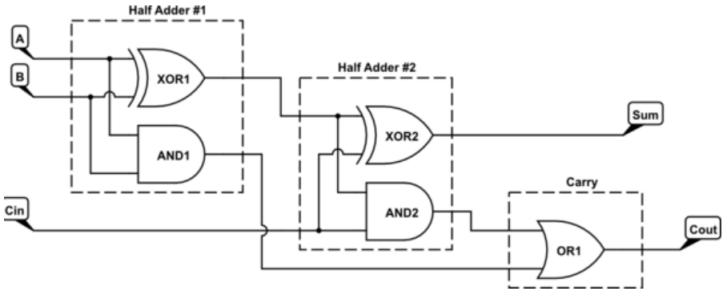
Slika 24.5: Shema Polovični seštevalnika

a	**b**	**sum**	**carry**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Tabela 24.1: Pravilnostna tabela polovičnega seštevalnika.

24.5.2 Polni seštevalnik

Polni seštevalnik je zaporedje 2n kaskadnih polovičnih seštevalnikov (n = bitna dolžina binarnih števil, ki se seštevajo). Kaskadno vezanje polovičnih seštevalnikov omogoča seštevanje posameznih bitov, iz katerih so sestavljena seštevana števila, pri čemer se upoštevajo čarry bit"– čarry bitše širi po celotnem vezju, kar ima za posledico končni čarry bit". Slika 24.6 prikazuje shemo polnega seštevalnika, Tabela 24.2 pa njegovo pripadajočo pravilnostno tabelo [197]



Slika 24.6: Shema Polovični seštevalnika

a	b	cin	sum	carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

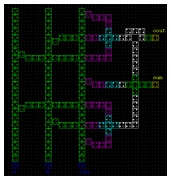
Tabela 24.2: Pravilnostna tabela polnega seštevalnika.

24.6 Metode

24.6.1 Implementacija z orodjem QCADesignerE

Za načrtovanje vezja smo uporabili program QCADesignerE. Končno vezje prikazuje slika 24.7. Vezje smo tudi simulirali in s tem preverili pravilnost delovanja

24.8. Na sliki vidimo, da vrednosti izhodnih celic glede na vrednosti vhodnih celic sovpadajo z resničnostno tabelo polnega seštevalnika 24.1.



Slika 24.7: Implementacija polnega seštevalnika

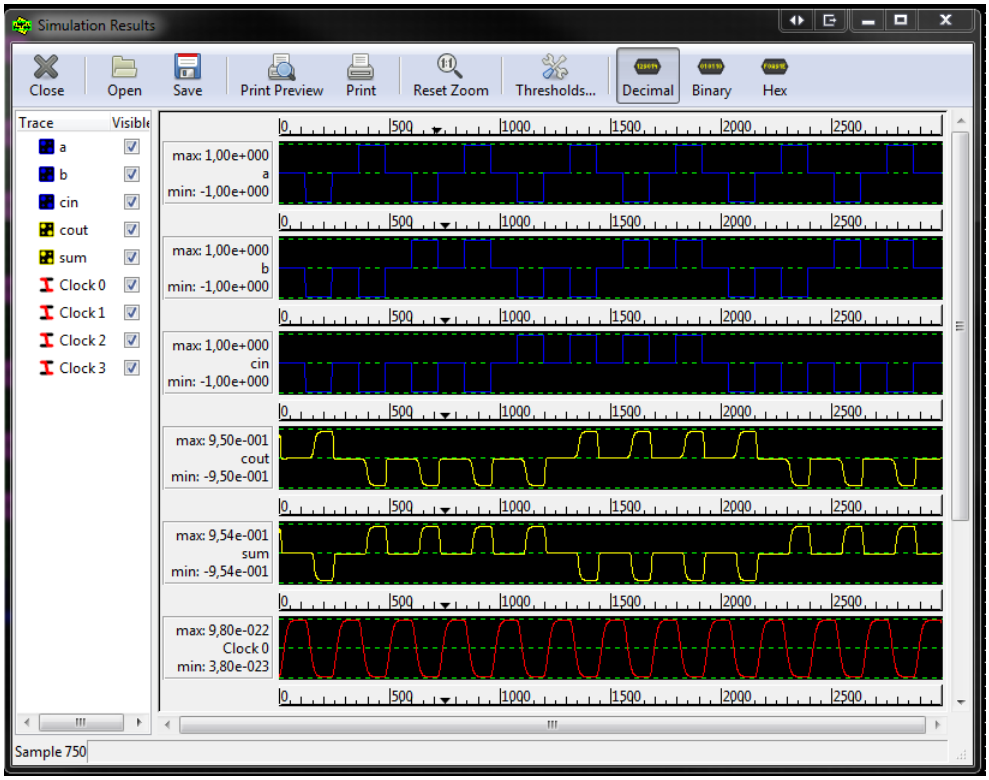
24.6.2 Implementacija z Pythonom in Qiskit

Prednost implementacije s pythonom je, da se ne rabimo zanašati na uro, da lahko lastnoročno določimo a,b in cin ter tako praktično preverimo pravilnost delovanja seštevalnika, z seštevanjem dejanskih števil [198]

Za implementacijo smo uporabili knjižnico Qisbit, ki je odprtokodni ogrodje za kvantno računalništvo, ki ga je razvil IBM in uporabnikom omogoča načrtovanje, simulacijo in zagon kvantnih vezij na resničnem kvantnem računalniku ali simulatorjih. Ponuja orodja za gradnjo algoritmov, vizualizacijo vezij in izvajanje poskusov na različnih kvantnih napravah.

24.7 Resultati

Implementacijo s QCADesignerjem smo preverili, če implementacija se ravna po pravilnostni tabeli 24.1, kot kaže slika



Slika 24.8: Rezultati simulacije QCA Designera

Implementacijo s Qiskit smo preverili preprosto tako, da smo sešteli nekaj števil, in ugotovili, če je seštevek pravilen, za testiranje smo uporabili že vgrajeno orodje unittest, ki je vgrajena knjižnica za enotno testiranje v Pythonu, ki je bila prvič predstavljena v Pythonu 2.1 leta 2001 kot del standardne knjižnice. Močno jo je navdihnil JUnit in sledi isti arhitekturi xUnit, ki se pogosto uporablja v številnih jezikih. unittest je ostal primarni okvir za testiranje v celotni seriji Python 2, vključno s Pythonom 2.7, in je še naprej v celoti podprt v sodobnem Pythonu 3. Ponuja funkcije, kot so razredi testnih primerov, trditve, nastavitve testov, odkrivanje testov in izvajalci testov, zaradi česar je strukturiran in standardiziran način za pisanje avtomatiziranih testov brez potrebe po zunanjih odvisnostih.

Spodaj vidimo primer vidimo primer našega testa

```
import unittest
class TestFullAdder(unittest.TestCase):

    def setUp(self):
        self.adder = FullAdder(bits=3)

    def test_1_plus_1(self):
        r = self.adder.add(1, 1)
        self.assertEqual(int(r, 2), 2)

    def test_2_plus_3(self):
        r = self.adder.add(2, 3)
        self.assertEqual(int(r, 2), 5)

    def test_5_plus_3(self):
        r = self.adder.add(5, 3)
        self.assertEqual(int(r, 2), 8)

    def test_7_plus_7(self):
        r = self.adder.add(7, 7)
        self.assertEqual(int(r, 2), 14)


if __name__ == "__main__":
    print("-------------------------------------------------------------------
    adder = FullAdder(bits=3)
    r = adder.add(1, 1)
    print("1 + 1 = 0b" + r + " = " + str(int(r, 2)))

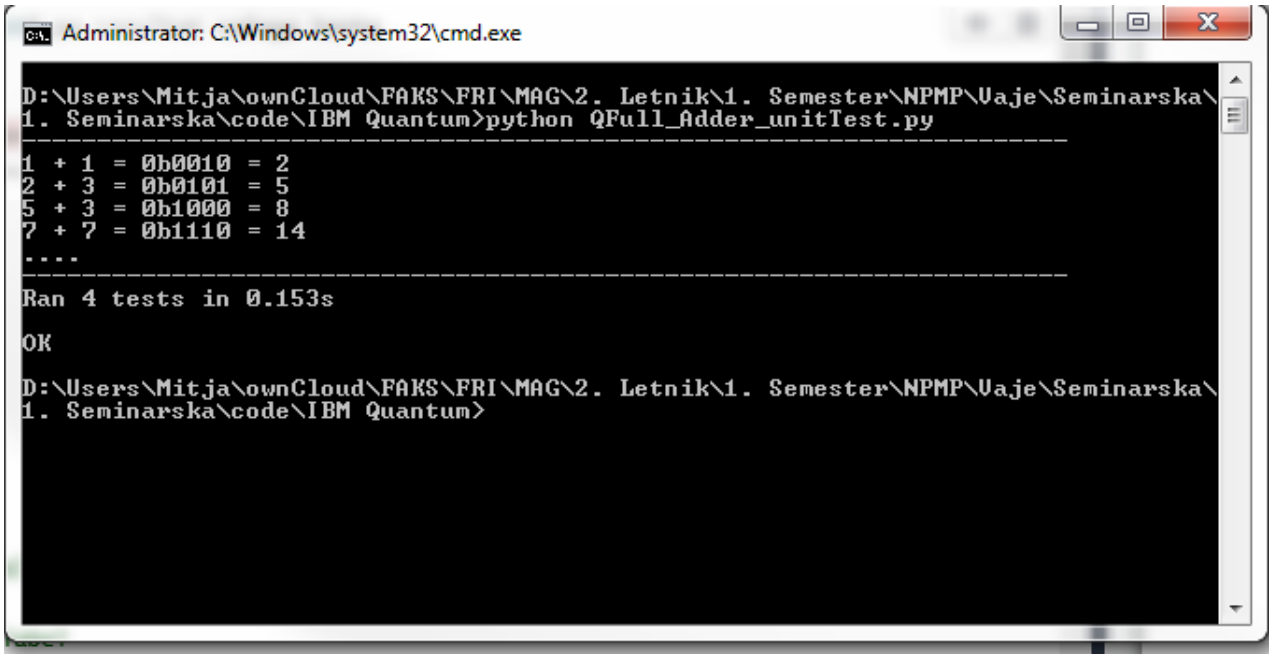
    r = adder.add(2, 3)
    print("2 + 3 = 0b" + r + " = " + str(int(r, 2)))

    r = adder.add(5, 3)
    print("5 + 3 = 0b" + r + " = " + str(int(r, 2)))
```

```
    r = adder.add(7, 7)
    print("7 + 7 = 0b" + r + " = " + str(int(r, 2)))

    unittest.main()
```

Na sliki 24.9 pa rezultat našega testa



Slika 24.9: Rezultati testiranje s knjižnico Qiskit

24.8 Zaključek

QCA predstavlja obetavno alternativo tradicionalni CMOS tehnologiji, saj omogoča gradnjo aritmetičnih vezij z nižjo porabo energije, visoko gostoto in izboljšano skalabilnostjo. V članku smo predstavili zasnovo polnega binarnega seštevalnika, ki je bil najprej implementiran in simuliran v okolju QCADesignerE. Rezultati simulacij so potrdili pravilno delovanje vezja ter skladnost z resničnostno tabelo.

Druga izvedba seštevalnika je temeljila na knjižnici Qiskit, s katero smo pripravili kvantno implementacijo in jo preverili z enotnimi testi v Pythonu. Tudi ta pristop

je pokazal pravilno delovanje in potrdil, da je možno osnovne aritmetične operacije uspešno prenesti v kvantno računsko okolje.

Opravljene implementacije dokazujejo, da QCA nudi učinkovito alternativo klasiki pri realizaciji digitalnih logičnih funkcij. Delo tako predstavlja osnovo za nadaljnje raziskave, ki bi lahko vključevale večbitne aritmetične operacije, optimizacijo prostora ter analizo odpornosti na napake, ki so v kvantnih tehnologijah še posebej pomembne.

Chapter 25
Foundational Quantum Algorithms: Grover, Deutsch–Jozsa and Shor

Miguel Serra

Abstract Quantum computing leverages superposition, interference, and entanglement to achieve speedups that are classically unattainable. This chapter presents simplified versions of three foundational algorithms (Grover's search [199], Deutsch–Jozsa [200], and Shor's [24] factoring algorithm) as key examples of quantum advantage. Using the Qiskit framework, we demonstrate how these algorithms exploit quantum mechanics to outperform classical computation in problems of function classification, unstructured search, and integer factorization. We also provide a comparative analysis of their computational complexities and discuss their implications for the future of quantum information processing.

25.1 Introduction

Quantum algorithms represent the core of quantum computing's promise to transcend classical computational limits. Unlike classical bits, which can exist only as 0 or 1, quantum bits (*qubits*) can exist in a superposition of both states. When manipulated coherently, qubits exhibit quantum interference, allowing the constructive amplification of correct results and the destructive cancellation of incorrect ones.

Three paradigmatic algorithms illustrate the power of this paradigm:

- **Grover's search algorithm**, offering a quadratic improvement in searching unstructured databases[199].
- **Deutsch–Jozsa algorithm**, the first to demonstrate exponential quantum speedup over deterministic classical computation[200].
- **Shor's factoring algorithm**, which provides an exponential speedup in integer factorization, undermining classical cryptographic assumptions[24].

25.2 Methods

In this chapter, we illustrate the functioning of three foundational quantum algorithms using simplified implementations. The code was executed in simulation using the Statevector functionality of Qiskit, producing probability distributions that are visualized in the corresponding figures. While the implementations are not fully optimized for real quantum hardware, they accurately capture the essential algorithmic behavior and allow a clear demonstration of quantum advantage.

25.2.1 Grover's Search Algorithm

25.2.1.1 Theoretical Background

Grover's algorithm addresses the problem of finding a marked element in an unstructured database of N entries, using only $O(\sqrt{N})$ oracle calls compared to $O(N)$ classically. It operates by alternating the oracle and diffusion operators to amplify the probability of the marked state [199].

25.2.1.2 Implementation

The implementation of Grover's algorithm in Qiskit begins by creating a quantum circuit with the required qubits. Hadamard gates are applied to place the qubits into a superposition, representing all possible states simultaneously. The algorithm then applies the oracle and diffusion operations (simplified here), which amplify the amplitude of the correct solution. The resulting state is simulated, and the measurement outcomes correspond to the most likely solution. The figure below shows the Qiskit code and the corresponding quantum circuit diagram

```
n = 2
grover_circuit = QuantumCircuit(n)
grover_circuit.h(range(n))

grover_circuit.cz(0, 1)

grover_circuit.h(range(n))
grover_circuit.x(range(n))
grover_circuit.h(1)
grover_circuit.cx(0, 1)
grover_circuit.h(1)
grover_circuit.x(range(n))
grover_circuit.h(range(n))

state = Statevector.from_instruction(grover_circuit)
```

Fig. 25.1: Qiskit implementation.

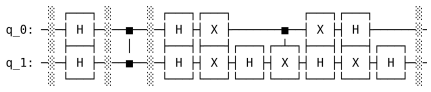


Fig. 25.2: Circuit representation.

25.2.2 The Deutsch–Jozsa Algorithm

25.2.2.1 Theoretical Background

The Deutsch–Jozsa problem asks whether a Boolean function

$$f : \{0,1\}^n \to \{0,1\}$$

is *constant* or *balanced*. Classically, this may require up to $2^{n-1}+1$ function evaluations. The quantum version resolves the problem in a *single query*, illustrating exponential speedup [200].

25.2.2.2 Implementation

The implementation starts by creating a quantum circuit with the necessary qubits. Hadamard gates prepare the superposition, the oracle encodes the function f, and the final Hadamard gates reveal whether the function is constant or balanced. Measurements then give the solution.

```
n = 3  # 3 input qubits
oracle = QuantumCircuit(n + 1)
oracle.cx(0, n)
oracle.cx(1, n)
oracle.cx(2, n)

dj = QuantumCircuit(n + 1, n)
dj.x(n)
dj.h(n)
dj.h(range(n))

dj.append(oracle, range(n + 1))

dj.h(range(n))
dj.barrier()

dj_no_measure = dj.remove_final_measurements(inplace=False)
state = Statevector.from_instruction(dj_no_measure)
```

Fig. 25.3: Qiskit implementation.

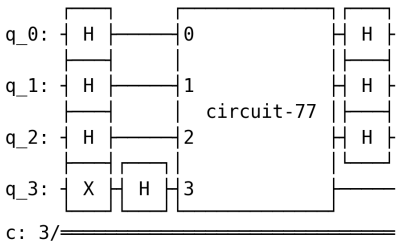


Fig. 25.4: Circuit representation.

25.2.3 Shor's Factoring Algorithm

25.2.3.1 Theoretical Background

Shor's algorithm factors a composite integer N by finding the period r of

$$f(x) = a^x \bmod N,$$

where a is coprime to N. The quantum portion of the algorithm performs a Quantum Fourier Transform (QFT) to extract r, offering exponential speedup over the best-known classical factoring algorithms [24].

25.2.3.2 Implementation

The circuit is initialized with counting and work qubits. Controlled operations encode modular exponentiation, followed by an inverse QFT on the counting qubits. Measurement reveals the period r, which can be used classically to factor N.

```
N = 15
a = 2
n_count = 2
n_work = 2
qc = QuantumCircuit(n_count + n_work)


qc.h(range(n_count))

qc.x(n_count)

for j in range(n_count):
    qc.cx(j, n_count + (j % n_work))

for i in range(n_count // 2):
    qc.swap(i, n_count - i - 1)

for j in range(n_count):
    qc.h(j)

state = Statevector.from_instruction(qc)
```

Fig. 25.5: Qiskit implementation.

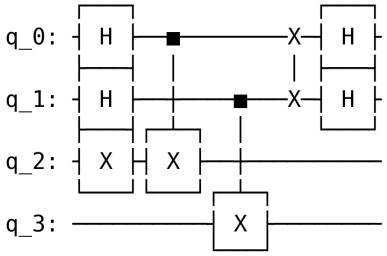


Fig. 25.6: Circuit representation.

25.3 Results

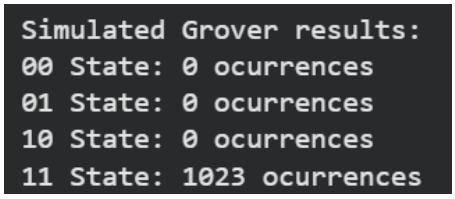
The results of the three simulated quantum algorithms show three very different behaviours, and each one revealing of the underlying implementation and goals.

Starting with Grover's algorithm, the output shows 1023 occurrences of the state 11 and zero occurrences of any other state. This is exactly what we expect from a correctly implemented Grover search: the marked element (in this case, $|11\rangle$) receives almost all the probability amplitude after the appropriate number of Grover iterations. This means the oracle, the diffusion operator, and the iteration count are all behaving as they should [199].

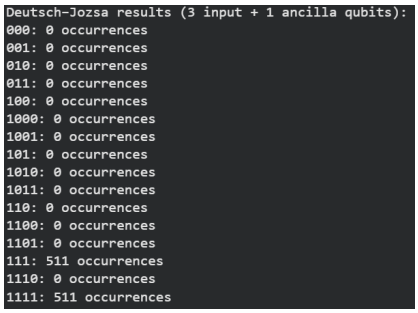
The results of the Deutsch–Jozsa algorithm show that the only measurement outcomes are 111 and 1111, each appearing 511 times. Although two different bit-lengths appear depending on how the simulator formats the output, the essential information is the same: the leading bits are 111 in every run[200]. Deutsch–Jozsa outputs $|000\ldots0\rangle$ only if the tested function is constant; any other output indicates

a balanced function. Because no occurrences of 000 appear, the function used in the test is indeed balanced, and the algorithm has correctly identified that.

In the results for the Shor All listed states appear with exactly the same number of occurrences (63 each) which means the output distribution is completely uniform. A correct implementation of Shor's algorithm should produce strong peaks at specific measurement values corresponding to multiples of the period of the modular function being analysed. A perfectly flat distribution indicates that the simulation is missing the crucial modular exponentiation step. In other words, the simulation labelled as "Shor's algorithm" is not actually performing that of Shor's algorithm, causing this distribution. The simulations demonstrate the characteristic measurement outcomes of each algorithm:



(a) Grover's algorithm results.



(b) Deutsch–Jozsa algorithm results.

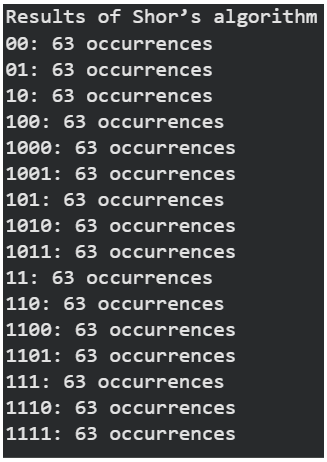


Fig. 25.7: Shor's algorithm results.

Fig. 25.8: Measurement outcomes of the three simulated quantum algorithms. Left column: Grover stacked above Deutsch–Jozsa. Right column: Shor, base aligned with Deutsch–Jozsa.

Table 25.1: Comparative performance of foundational quantum algorithms.

Algorithm	**Problem Type**	**Classical Complexity**	**Quantum Complexity**	**Speedup**
Grover	Unstructured search	$O(N)$	$O(\sqrt{N})$	Quadratic
Deutsch–Jozsa	Function classification	$O(2^n)$	$O(1)$	Exponential
Shor	Integer factorization	$O(e^{n^{1/3}})$	$O(n^3)$	Exponential

25.4 Discussion

The three algorithms examined here constitute milestones in quantum algorithmic design. They collectively demonstrate how quantum mechanical effects—superposition, entanglement, and interference—translate into computational advantage.

While Grover's algorithm has found practical relevance as a subroutine in quantum optimization and cryptanalysis[199], Deutsch–Jozsa remains primarily a theoretical demonstration[200]. Shor's algorithm poses direct implications for public-key cryptography [24], motivating the development of post-quantum cryptographic standards.

Current quantum hardware limitations restrict the full realization of these algorithms. However, as gate fidelity, coherence time, and error correction improve, these foundational methods will likely underpin future scalable quantum computation.

25.5 Conclusion

The Grover, Deutsch–Jozsa, and Shor algorithms encapsulate the essence of quantum computing's transformative potential. They illustrate the profound ways in which quantum mechanics can redefine the limits of computation, and they continue to guide the development of new algorithmic paradigms.

Future research should aim to:

- Develop fault-tolerant implementations of these algorithms.
- Explore hybrid quantum–classical frameworks for enhanced practicality.
- Extend amplitude amplification and Fourier-based techniques to emerging domains such as quantum optimization and machine learning.

Their legacy persists not merely in their mathematical formulation but in their enduring influence on quantum computational theory and technology.

Literatura

1. A. H. Majeed, "A compact and power efficient full adder-subtractor layout in qca technology," *PLoS One*, vol. 20, no. 11, p. e0335789, 2025.
2. R. Zhang, K. Walus, W. Wang, and G. Jullien, "A method of majority logic reduction for quantum cellular automata," *IEEE Transactions on Nanotechnology*, vol. 3, no. 4, pp. 443–450, 2004.
3. V. Pudi and K. Sridharan, "Efficient qca design of single-bit and multi-bit subtractors," in *2013 13th IEEE International Conference on Nanotechnology (IEEE-NANO 2013)*, pp. 1155–1158, 2013.
4. GeeksforGeeks, "4-bit binary adder-subtractor," 2025. Accessed: 2025-11-23; Last updated: 2025-07-12.
5. J. I. Reshi, F. A. Khanday, and M. T. Banday, "Optimized nanoscale adder-subtractor design architectures utilizing quantum dot cellular automata," *Russian Microelectronics*, vol. 54, no. 4, pp. 400–426, 2025.
6. F. Ahmad, S. Ahmed, V. Kakkar, G. M. Bhat, A. N. Bahar, and S. Wani, "Modular design of ultra-efficient reversible full adder-subtractor in qca with power dissipation analysis," *International Journal of Theoretical Physics*, vol. 57, no. 9, pp. 2863–2880, 2018.
7. W. contributors, "D-wave systems," 2025. Accessed: 2025-11-17.
8. P. Hauke, H. G. Katzgraber, W. Lechner, H. Nishimori, and W. D. Oliver, "Perspectives of quantum annealing: methods and implementations," *Reports on Progress in Physics*, vol. 83, no. 5, 2020.
9. A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, vol. 2, 2014.
10. F. Neukart, G. Compostella, C. Seidel, D. von Dollen, S. Yarkoni, and B. Parney, "Traffic flow optimization using a quantum annealer," 2017.
11. D.-W. Q. Inc., "The leap™ quantum cloud service — cloud platform," 2025. Accessed: 2025-11-17.
12. W. contributors, "Josephson vortex," 2025. Accessed: 2025-11-17.
13. M. Booth, A. D. King, and J. Raymond, "Solving set cover with pairs problem using quantum annealing," 2016. Preprint, accessed: 2025-11-17.
14. D.-W. Q. Inc., "General availability of advantage2 quantum computer," 2025. Accessed: 2025-11-17.
15. W. contributors, "Preprosti problem nahrbtnika," 2025. Accessed: 2025-11-17.
16. PennyLane, "Qubo: Quadratic unconstrained binary optimization," 2025. Accessed: 2025-11-17.
17. W. contributors, "Ising model," 2025. Accessed: 2025-11-17.
18. M. Booth, S. P. Reinhardt, and A. Roy, "Partitioning optimization problems for hybrid classical/quantum execution," Technical Report 14-1006A-A, D-Wave Systems Inc., 2017. Accessed: 2025-11-17.
19. W. contributors, "Tabu search," 2025. Accessed: 2025-11-21.

20. D.-W. Developers, "D-wave hybrid solver service: An overview," White Paper 14-1039A-B, D-Wave Systems Inc., 2020. Accessed: 2025-11-17.
21. S. Yarkoni, F. Neukart, E. M. G. Tagle, N. Magiera, B. Mehta, K. Hire, S. Narkhede, and M. Hofmann, "Quantum shuttle: traffic navigation with quantum computing," in *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software*, ESEC/FSE '20, p. 22–30, ACM, Nov. 2020.
22. M. A. Nielsen and I. Chuang, "Quantum computation and quantum information," 2002.
23. P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, Ieee, 1994.
24. P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, p. 1484–1509, Oct 1997.
25. G. A. et al., "Qiskit: An Open-source Framework for Quantum Computing," Jan. 2019.
26. L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.
27. M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.
28. TechTarget, "What is quantum interference and how it works?," 2023. Accessed: 2025-11-07.
29. A. EITCA, "Kako groverov algoritem zagotavlja kvadratno pospešitev v primerjavi s klasičnimi iskalnimi algoritmi," 2023. Accessed: 2025-11-05.
30. S. B. R. Chowdhury, "Qml — quantum oracle," July 2019. Accessed: 2025-11-22.
31. K. Malenko, "Posledice kvantnega računalništva," 2017.
32. "Računalniške tehnologije," 2025. GitHub repozitorij.
33. A. Ambainis, "Quantum search algorithms," *ACM Sigact News*, vol. 35, no. 2, pp. 22–35, 2004.
34. Y. Suzuki, M. Gluza, J. Son, B. H. Tiang, N. H. Ng, and Z. Holmes, "Grover's algorithm is an approximation of imaginary-time evolution," *arXiv preprint arXiv:2507.15065*, 2025.
35. G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," *arXiv preprint quant-ph/0005055*, 2000.
36. L. Grover and J. Radhakrishnan, "Is partial quantum search of a database any easier? eprint," *arXiv preprint quant-ph/0407122*, 2004.
37. A. Ciacco, F. Guerriero, and G. Macrina, "Review of quantum algorithms for medicine, finance and logistics," *Soft Computing*, vol. 29, pp. 2129–2170, 02 2025.
38. G. Laporte, "What you should know about the vehicle routing problem," *Naval Research Logistics (NRL)*, vol. 54, no. 8, pp. 811–819, 2007.
39. M. G. De Andoin, A. Bottarelli, S. Schmitt, I. Oregi, P. Hauke, and M. Sanz, "Formulation of the electric vehicle charging and routing problem for a hybrid quantum-classical search space reduction heuristic," in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, p. 5318–5323, IEEE, Sept. 2023.
40. K. Srinivasan, S. Satyajit, B. K. Behera, and P. K. Panigrahi, "Efficient quantum algorithm for solving travelling salesman problem: An ibm quantum experience," 2018.
41. R. Shang, B. Du, K. Dai, L. Jiao, A. M. G. Esfahani, and R. Stolkin, "Quantum-inspired immune clonal algorithm for solving large-scale capacitated arc routing problems," *Memetic Computing*, vol. 10, no. 1, pp. 81–102, 2018.
42. C. Hu, Y. Xia, and J. Zhang, "Adaptive operator quantum-behaved pigeon-inspired optimization algorithm with application to uav path planning," *Algorithms*, vol. 12, no. 1, 2019.
43. N. Mori and S. Furukawa, "Quantum annealing for the adjuster routing problem," *Frontiers in Physics*, vol. 11, mar 2023.
44. R. Correll, S. J. Weinberg, F. Sanches, T. Ide, and T. Suzuki, "Quantum neural networks for a supply chain logistics application," *Advanced Quantum Technologies*, vol. 6, no. 7, p. 2200183, 2023.
45. J. Jiang, D. Zhang, S. Li, and Y.-J. Liu, "Multimodal green logistics network design of urban agglomeration with stochastic demand," *Journal of Advanced Transportation*, vol. 2019, pp. 1–19, 08 2019.

46. A. Shokry and M. Youssef, "Quantum fingerprinting for heterogeneous devices localization," *Computer Communications*, vol. 204, pp. 43–51, 2023.
47. G. Malviya, B. AkashNarayanan, and J. Seshadri, "Logistics network optimization using quantum annealing," 09 2023.
48. H. Jiang, Z.-J. M. Shen, and J. Liu, "Quantum computing methods for supply chain management," *2022 IEEE/ACM 7th Symposium on Edge Computing (SEC)*, pp. 400–405, 2022.
49. P. Atchade-Adelomou, G. Alonso-Linaje, J. Albo-Canals, and D. Casado-Fauli, "qrobot: A quantum computing approach in mobile robot order picking and batching problem solver optimization," *Algorithms*, vol. 14, no. 7, 2021.
50. A. A. Bozhedarov, S. R. Usmanov, G. V. Salakhov, A. S. Boev, E. O. Kiktenko, and A. K. Fedorov, "Quantum and quantum-inspired optimization for solving the minimum bin packing problem," *Journal of Physics: Conference Series*, vol. 2701, p. 012129, feb 2024.
51. H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
52. R. Yalovetzky, P. Minssen, D. Herman, and M. Pistoia, "Nisq-hhl: Portfolio optimization for near-term quantum hardware," 10 2021.
53. S. Mugel, C. Kuchkovsky, E. Sánchez, S. Fernández-Lorenzo, J. Luis-Hita, E. Lizaso, and R. Orús, "Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks," *Phys. Rev. Res.*, vol. 4, p. 013006, Jan 2022.
54. D. Herman, R. Shaydulin, Y. Sun, S. Chakrabarti, S. Hu, P. Minssen, A. Rattew, R. Yalovetzky, and M. Pistoia, "Portfolio optimization via quantum zeno dynamics on a quantum processor," 09 2022.
55. A. G. Cadavid, I. Montalban, A. Dalal, E. Solano, and N. N. Hegade, "Efficient digitized counterdiabatic quantum optimization algorithm within the impulse regime for portfolio optimization," *Physical Review Applied*, vol. 22, Nov. 2024.
56. N. Slate, E. Matwiejew, S. Marsh, and J. B. Wang, "Quantum walk-based portfolio optimisation," *Quantum*, vol. 5, p. 513, July 2021.
57. W. F. Sharpe, "Mutual fund performance," *The Journal of Business*, vol. 39, no. 1, pp. 119–138, 1966.
58. N. Stamatopoulos, G. Mazzola, S. Woerner, and W. J. Zeng, "Towards quantum advantage in financial market risk using quantum gradient algorithms," *Quantum*, vol. 6, p. 770, July 2022.
59. S. Woerner and D. J. Egger, "Quantum risk analysis," *npj Quantum Information*, vol. 5, no. 1, p. 15, 2019.
60. D. J. Egger, R. García Gutiérrez, J. C. Mestre, and S. Woerner, "Credit risk analysis using quantum computers," *IEEE Transactions on Computers*, vol. 70, no. 12, pp. 2136–2145, 2021.
61. E. Dri, A. Aita, E. Giusto, D. Ricossa, D. Corbelletto, B. Montrucchio, and R. Ugoccioni, "A more general quantum credit risk analysis framework," *Entropy*, vol. 25, no. 4, 2023.
62. F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
63. P. Rebentrost, B. Gupt, and T. R. Bromley, "Quantum computational finance: Monte carlo pricing of financial derivatives," *Physical Review A*, vol. 98, Aug. 2018.
64. K. Kubo, K. Miyamoto, K. Mitarai, and K. Fujii, "Pricing multi-asset derivatives by variational quantum algorithms," 2022.
65. S. Chakrabarti, R. Krishnakumar, G. Mazzola, N. Stamatopoulos, S. Woerner, and W. J. Zeng, "A threshold for quantum advantage in derivative pricing," *Quantum*, vol. 5, p. 463, June 2021.
66. K. Kaneko, K. Miyamoto, N. Takeda, and K. Yoshino, "Quantum pricing with a smile: Implementation of local volatility model on quantum computer," 2020.
67. T. J. S. Durant, E. Knight, B. Nelson, S. Dudgeon, S. J. Lee, D. Walliman, H. P. Young, L. Ohno-Machado, and W. L. Schulz, "A primer for quantum computing and its applications to healthcare and biomedical research," *Journal of the American Medical Informatics Association*, vol. 31, pp. 1774–1784, 06 2024.
68. H. Tanaka, C. Mendoza, and S. Saeed, "The role of quantum computing in personalized medicine and genomic data processing," *Journal of Quantum Computing*, 02 2025.

69. Danishuddin, M. A. Haque, V. Kumar, S. N. Khan, and J.-J. Kim, "Quantum intelligence in drug discovery: Advancing insights with quantum machine learning," *Drug Discovery Today*, vol. 30, no. 10, p. 104463, 2025.
70. E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv.org*, Nov 2014.
71. M. Fingerhuth, T. Babej, and C. Ing, "A quantum alternating operator ansatz with hard and soft constraints for lattice protein folding," *arXiv.org*, Oct 2018.
72. A. Robert, P. K. Barkoutsos, S. Woerner, and I. Tavernelli, "Resource-efficient quantum algorithm for protein folding," *arXiv.org*, Aug 2019.
73. H. Mustafa, S. N. Morapakula, P. Jain, and S. Ganguly, "Variational quantum algorithms for chemical simulation and drug discovery," *2022 International Conference on Trends in Quantum Computing and Emerging Business Technologies (TQCEBT)*, pp. 1–8, 2022.
74. Q.-M. Ding, Y.-M. Huang, and X. Yuan, "Molecular docking via quantum approximate optimization algorithm," *arXiv.org*, May 2024.
75. S. Mensa, E. Sahin, F. Tacchino, P. K. Barkoutsos, and I. Tavernelli, "Quantum machine learning framework for virtual screening in drug discovery: A prospective quantum advantage," *arXiv.org*, Apr 2022.
76. F. Yan, H. Huang, W. Pedrycz, and K. Hirota, "Review of medical image processing using quantum-enabled algorithms," *Artificial Intelligence Review*, vol. 57, Sep 2024.
77. M. H. Khalaf, E. A. Adel, R. H. Mohammed, A. A. Emaduldeen, and B. L. Jasim, "Application of quantum computing in medical imaging: Revolutionizing mri and ct scan technology," *American Journal of Botany and Bioengineering*, vol. 2, p. 14–28, Apr. 2025.
78. B. T. Kiani, A. Villanyi, and S. Lloyd, "Quantum medical imaging algorithms," *arXiv.org*, Apr 2020.
79. K. Jun, "Highly accurate quantum optimization algorithm for ct image reconstructions based on sinogram patterns," *Scientific Reports*, 07 2022.
80. H. Huflage, J.-P. Grunz, A. Kunz, T. Patzer, S. Sauer, S. Christner, B. Petritsch, S. Ergün, T. Bley, and K. Luetkens, "Potential of employing a quantum iterative reconstruction algorithm for ultra-high-resolution photon-counting detector ct of the hip," *Radiography*, vol. 29, pp. 44–49, 10 2022.
81. T. Zhang, J. Zhang, T. Xue, and M. Rashid, "A brain tumor image segmentation method based on quantum entanglement and wormhole behaved particle swarm optimization," *Frontiers in Medicine*, vol. 9, p. 794126, 05 2022.
82. L. Parisi, C. Neagu, R. Ma, and F. Campean, "Quantum relu activation for convolutional neural networks to improve diagnosis of parkinson's disease and covid-19," *Expert Systems with Applications*, vol. 187, p. 115892, 09 2021.
83. M. Al-Zafar, N. Innan, A. Galib, and M. Bennai, "Brain tumor diagnosis using quantum convolutional neural networks," 01 2024.
84. R. S. Gupta, C. E. Wood, T. Engstrom, J. D. Pole, and S. Shrapnel, "A systematic review of quantum machine learning for digital health," *npj Digital Medicine*, vol. 8, May 2025.
85. J. Hoogland, O. Efthimiou, T. Nguyen, and T. Debray, "Evaluating individualized treatment effect predictions: A model-based perspective on discrimination and calibration assessment," *Statistics in medicine*, vol. 43, 08 2024.
86. B. Kaur, S. Rathi, and R. Agrawal, "Enhanced depression detection from speech using quantum whale optimization algorithm for feature selection," *Computers in Biology and Medicine*, vol. 150, p. 106122, 09 2022.
87. A. Sarkar, Z. Al-Ars, C. Almudever, and K. Bertels, "Qibam: Approximate sub-string index search on quantum accelerators applied to dna read alignment," *Electronics*, vol. 10, p. 2433, 10 2021.
88. D. Niraula, J. Jamaluddin, M. Matuszak, R. Ten Haken, and I. El Naqa, "Quantum deep reinforcement learning for clinical decision support in oncology: application to adaptive radiotherapy," *Scientific Reports*, vol. 11, 12 2021.
89. s. Moradi, C. Brandner, C. Spielvogel, D. Krajnc, S. Hillmich, R. Wille, W. Drexler, and L. Papp, "Clinical data classification with noisy intermediate scale quantum computers," *Scientific Reports*, 09 2021.

90. D. G S, J. Babu, P. B D, R. de Prado, and J. Frnda, "Quantum fruit fly algorithm and resnet50-vgg16 for medical diagnosis," *Applied Soft Computing*, vol. 136, p. 110055, 03 2023.
91. H. Gupta, H. Varshney, D. T. Sharma, N. Pachauri, and O. Verma, "Comparative performance analysis of quantum machine learning with deep learning for diabetes prediction," *Complex and Intelligent Systems*, vol. 8, 05 2021.
92. L. R. Ladd T., Jelezko F., "Quantum computers," *Nature*, 2010.
93. SPINQ, "Ultimate guide to coherence time: Everything you need to know," 2025.
94. C. O. Francesco B., Abhijit C., "Quantum computing with trapped ions: a beginner's guide," *Eur. J. Phys.*, 2023.
95. F. Flamini, M. Krumm, L. J. Fiderer, T. Müller, and H. J. Briegel, "Towards interpretable quantum machine learning via single-photon quantum walks," 2023.
96. H. J. Briegel and G. D. las Cuevas, "Projective simulation for artificial intelligence," 2013.
97. M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, "Variational quantum algorithms," *Nature Reviews Physics*, vol. 3, p. 625–644, Aug. 2021.
98. A. Ambainis, "Quantum walk algorithm for element distinctness," 2014.
99. M. Santha, "Quantum walk based search algorithms," in *Theory and Applications of Models of Computation* (M. Agrawal, D. Du, Z. Duan, and A. Li, eds.), (Berlin, Heidelberg), pp. 31–46, Springer Berlin Heidelberg, 2008.
100. M. Szegedy, "Quantum speed-up of markov chain based algorithms," in *45th Annual IEEE symposium on foundations of computer science*, pp. 32–41, IEEE, 2004.
101. F. Magniez, A. Nayak, J. Roland, and M. Santha, "Search via quantum walk," *SIAM Journal on Computing*, vol. 40, p. 142–164, Jan. 2011.
102. A. M. Childs and J. Goldstone, "Spatial search by quantum walk," *Phys. Rev. A*, vol. 70, p. 022314, Aug 2004.
103. A. Tulsi, "Faster quantum-walk algorithm for the two-dimensional spatial search," *Phys. Rev. A*, vol. 78, p. 012310, Jul 2008.
104. A. M. Childs, R. Cleve, E. Deotto, E. Farhi, S. Gutmann, and D. A. Spielman, "Exponential algorithmic speedup by a quantum walk," in *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*, pp. 59–68, 2003.
105. P. R. Giri, R. Sato, and K. Saito, "Quantum walk search based edge detection of images," *Physics Letters A*, vol. 540, p. 130391, 2025.
106. R. Portugal, *Quantum walks and search algorithms*, vol. 19. Springer, 2013.
107. W. Zhou, "Review on quantum walk algorithm," in *Journal of Physics: Conference Series*, vol. 1748, p. 032022, IOP Publishing, 2021.
108. X. Qiang, S. Ma, and H. Song, "Quantum walk computing: Theory, implementation, and application," *Intelligent Computing*, vol. 3, p. 0097, 2024.
109. C. Chandrashekar, "Discrete-time quantum walk-dynamics and applications," *arXiv preprint arXiv:1001.5326*, 2010.
110. S. Venegas-Andraca, *Quantum walks for computer scientists*. Springer Nature, 2022.
111. J. Adcock, "Discrete quantum walks and search algorithms."
112. R. Portugal, R. A. Santos, T. D. Fernandes, and D. N. Gonçalves, "The staggered quantum walk model," *Quantum Information Processing*, vol. 15, no. 1, pp. 85–101, 2016.
113. S. E. Venegas-Andraca, "Quantum walks: a comprehensive review," *Quantum Information Processing*, 2012.
114. A. M. Childs, "On the relationship between continuous- and discrete-time quantum walk," *Communications in Mathematical Physics*, 2010.
115. Z. K. Zoltán Zimborás, Mauro Faccin, J. D. Whitfield, B. P. Lanyon, and J. Biamonte, "Quantum transport enhancement by time-reversal symmetry breaking," *Scientific Reports*, 2013.
116. J. D. Whitfield, C. A. Rodr'iguez-Rosario, and A. Aspuru-Guzik, "Quantum stochastic walks: A generalization of classical random walks and quantum walks," *Physical Review A*, 2010.
117. R. Seidel, S. Bock, R. Zander, M. Petrič, N. Steinmann, N. Tcholtchev, and M. Hauswirth, "Qrisp: A framework for compilable high-level programming of gate-based quantum computers," 2024.

118. G. Coutinho and K. Guo, "Selected open problems in continuous-time quantum walks," 2024.
119. V. Kendon and C. Tamon, "Perfect state transfer in quantum walks on graphs," *Journal of Computational and Theoretical Nanoscience*, vol. 8, 04 2010.
120. C. Godsil, N. Mullin, and A. Roy, "Uniform mixing and association schemes," 2014.
121. C. Godsil, K. Guo, and M. Sobchuk, "Diagonal entries of the average mixing matrix," 2019.
122. E. A. Ali H. Majeed, Mohd Shamian Zainal, "Quantum-dot cellular automata: Review paper," *International Journal of Integrated Engineering*, vol. 11, no. 8, pp. 143–158, 2019.
123. M. Vahabi, E. Rahimi, P. Lyakhov, A. N. Bahar, K. A. Wahid, and A. Otsuki, "Novel quantum-dot cellular automata-based gate designs for efficient reversible computing," *Sustainability*, vol. 15, no. 3, p. 2265, 2023.
124. M. Mraz, "Kvantni celični avtomati," 2025.
125. A. N. Bahar, S. Waheed, N. Hossain, and M. Asaduzzaman, "A novel 3-input xor function implementation in quantum dot-cellular automata with energy dissipation analysis," *Alexandria Engineering Journal*, vol. 57, no. 2, pp. 729–738, 2018.
126. M. Askari and M. Taghizadeh, "Logic circuit design in nano-scale using quantum-dot cellular automata," *European Journal of Scientific Research*, vol. 48, pp. 516–526, 01 2011.
127. GeeksforGeeks, "Full adder – digital logic," Oct 2025. Accessed: 2025-11-19.
128. U. B. Joy, S. Chakraborty, S. Islam, H. U. Zaman, and M. Hasan, "Quantum-dot cellular automata-based full adder design: Comprehensive review and performance comparison," *Advances in Materials Science and Engineering*, pp. 1–13, 2023.
129. D. Tougaw, P. and S. Lent, C. "Logical devices implemented using quantum cellular automata," *Journal of Applied Physics*, vol. 75, no. 3, pp. 1818–1823, 1994.
130. A. Vetteth, K. Walus, V. Dimitrov, and G. Jullien, "Quantum dot cellular automata carry-look-ahead adder and barrel shifter," *IEEE Emerging Telecommunications Technologies Conference*, vol. 2, 01 2002.
131. W. Wang, K. Walus, and A. Jullien, G. "Quantum-dot cellular automata adders," in *Third IEEE Conference on Nanotechnology*, vol. 2, pp. 461–464, 2003.
132. U. B. Joy, S. Chakraborty, S. Tasnim, M. S. Hossain, A. H. Siddique, and M. Hasan, "Design of an area efficient quantum dot cellular automata based full adder cell having low latency," in *2021 2nd International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST)*, pp. 689–693, 2021.
133. S. Seyedi and N. Jafari Navimipour, "Designing a three-level full-adder based on nano-scale quantum dot cellular automata," *Photonic Network Communications*, vol. 42, 12 2021.
134. S. Seyedi and N. Jafari Navimipour, "Designing a multi-layer full-adder using a new three-input majority gate based on quantum computing," *Concurrency and Computation: Practice and Experience*, vol. 34, 10 2021.
135. D. R. Simon, "On the power of quantum computation," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1474–1483, 1997.
136. R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, p. 120–126, Feb. 1978.
137. A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," 2024.
138. S. Sheikhfaal, "Qca-based priority encoder." `https://arxiv.org/pdf/1801.09807`, 2018.
139. O. Y. Smita C. Chetti, "Priority encoder in qca design," *Global Transitions Proceedings*, 2022.
140. "Priority encoder." `https://en.wikipedia.org/wiki/Priority_encoder`, 2025.
141. K. A. A. Nuriddin Safoev, Ganiev Abdukhalil, "Design of QCA-based priority encoder," *2020 IEEE 14th International Conference on Application of Information and Communication Technologies (AICT)*, 2020.
142. "Qca designer manual." `https://www-mdp.eng.cam.ac.uk/web/CD/engapps/qcad/qcadmanual.pdf`, 2019.

143. A. Almatrood, “A novel qca design of energy-efficient three-input and/or circuit,” *Quantum Reports*, vol. 7, no. 3, p. 38, 2025.
144. C. S. Lent, P. D. Tougaw, W. Porod, and G. H. Bernstein, “Quantum cellular automata,” *Nanotechnology*, vol. 4, pp. 49–57, 1993.
145. P. D. Tougaw and C. S. Lent, “Logical devices implemented using quantum cellular automata,” *Journal of Applied Physics*, vol. 75, no. 3, pp. 1818–1825, 1994.
146. C. S. Lent and P. D. Tougaw, “A device architecture for computing with quantum dots,” *Proceedings of the IEEE*, vol. 85, no. 4, pp. 541–557, 1997.
147. F. S. Torres, R. Wille, P. Niemann, and R. Drechsler, “An energy-aware model for the logic synthesis of quantum-dot cellular automata,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 12, pp. 3031–3041, 2018.
148. R. Zhang, P. Gupta, and N. Jha, “Majority and minority network synthesis with application to qca,” *IEEE Transactions on CAD of Integrated Circuits and Systems*, vol. 26, no. 7, pp. 1233–1245, 2007.
149. M. Alharbi, G. Edwards, and R. Stocker, “The design and simulation of a reversible quantum-dot cellular automata-based arithmetic logic unit,” *Preprints*, July 2023.
150. R. K. V.Subashini, “Design and implementation of full adders using qca,” *SSRG International Journal of VLSI and Signal Processing*, 2018.
151. V. K. Sharma, “Xor gate design in qca nanotechnology using cell translation method,” *Discover Electronics*, vol. 2, p. 87, Nov 2025.
152. A. Shukla and P. Vedula, “Efficient quantum algorithm for weighted partial sums and numerical integration,” *Advanced Quantum Technologies*, vol. 8, no. 10, p. e2500084, 2025.
153. A. Shukla and P. Vedula, “An efficient quantum algorithm for preparation of uniform quantum superposition states,” *Quantum Information Processing*, vol. 23, Jan. 2024.
154. R. Laajimi, “Nanoarchitecture of quantum-dot cellular automata (qca) using small area for digital circuits,” in *Advanced Electronic Circuits - Principles, Architectures and Applications on Emerging Technologies* (M. Niu, ed.), ch. 3, London: IntechOpen, 2018.
155. GeeksforGeeks, “Latches in digital logic.” `https://www.geeksforgeeks.org/digital-logic/latches-in-digital-logic/`. Accessed: 2025-11-21.
156. Wikipedia, “Multiplexer.” `https://en.wikipedia.org/wiki/Multiplexer`. Accessed: 2025-11-21.
157. S. Seyedi, M. Darbandi, and N. J. Navimipour, “Designing an efficient fault tolerance d-latch based on quantum-dot cellular automata nanotechnology,” *Optik*, vol. 185, pp. 827–837, 2019.
158. Wikipedia, “Fredkin gate.” `https://en.wikipedia.org/wiki/Fredkin_gate`. Accessed: 2025-11-21.
159. J.-C. Jeon, “Quantum-dot ca-based fredkin gate and conservative d-latch for reliability-based information transmission on reversible computing,” *Electronics*, vol. 13, no. 14, 2024.
160. M. Tahoori, M. Momenzadeh, J. Huang, and F. Lombardi, “Defects and faults in quantum cellular automata at nano scale,” in *22nd IEEE VLSI Test Symposium, 2004. Proceedings.*, pp. 291–296, 2004.
161. D. Deutsch, “Quantum theory, the church–turing principle and the universal quantum computer,” *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, vol. 400, no. 1818, pp. 97–117, 1985.
162. E. Bernstein and U. Vazirani, “Quantum complexity theory,” in *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pp. 11–20, 1993.
163. R. de Wolf, “Quantum computing: Lecture notes,” 2023.
164. S. Rasouli Heikalabad and M. Rahimpour Gadim, “Design of improved arithmetic logic unit in quantum-dot cellular automata,” *International Journal of Theoretical Physics*, vol. 57, 06 2018.
165. B. Safaiezadeh, E. Mahdipour, M. Haghparast, S. Sayedsalehi, and M. Hosseinzadeh, “Design and simulation of qca-based 3-bit binary to gray and vice versa code converter in reversible and non-reversible mode,” *Optik*, vol. 251, p. 168464, 2022.
166. R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Commun. ACM*, vol. 21, p. 120–126, Feb. 1978.

167. IBM Quantum, "Ibm quantum experience." https://quantum.cloud.ibm.com/. accessed: 2025.
168. J. M. Pollard, "A monte carlo method for factorization," *BIT Numerical Mathematics*, vol. 15, no. 3, pp. 331–334, 1975.
169. C. Pomerance, "The quadratic sieve factoring algorithm.," in *Advances in Cryptology*, pp. 169–182, 01 1984.
170. J. Buchmann, J. Loho, and J. Zayer, "An implementation of the general number field sieve," in *Advances in Cryptology — CRYPTO' 93* (D. R. Stinson, ed.), (Berlin, Heidelberg), pp. 159–165, Springer Berlin Heidelberg, 1994.
171. J. Gambetta, "The hardware and software for the era of quantum utility is here." https://www.ibm.com/quantum/blog/quantum-roadmap-2033, Dec. 2023. Accessed: 2025.
172. V. R. Kumar, B. M. Mahan, J. C. Das, S. Kassa, and D. De, "Optimized design of digital comparator using qca," *Physica Scripta*, vol. 100, no. 6, p. 065125, 2025.
173. N. Aswathy, S. M. NM, and R. Krishnan, "Design of optimized and energy efficient single-bit comparator using quantum dot cellular automata," *Engineering Research Express*, vol. 7, no. 1, p. 015316, 2025.
174. L. BL and D. Prasad, "Quantum-dot cellular automata for arithmetic circuits: A novel approach to adders, subtractors and comparators," in *2025 3rd International Conference on Data Science and Network Security (ICDSNS)*, pp. 1–7, IEEE, 2025.
175. M. Vahabi, E. Rahimi, and P. Lyakhov, "Efficient design and implementation of a reversible switched network in quantum cellular automata technology," *Journal of King Saud University - Computer and Information Sciences*, vol. 36, no. 1, p. 101910, 2024.
176. S. F. Naz, S. Ahmed, S. Sharma, F. Ahmad, and D. Ajitha, "Fredkin gate based energy efficient reversible d flip flop design in quantum dot cellular automata," *Materials Today: Proceedings*, vol. 46, pp. 5248–5255, 2021. International Conference on Innovations in Clean Energy Technologies (ICET2020).
177. B. Safaiezadeh, L. Kettunen, and M. Haghparast, "Novel high-performance qca fredkin gate and designing scalable qca binary to gray and vice versa," *The Journal of Supercomputing*, vol. 79, pp. 7037–7060, Apr 2023.
178. F. Sill Torres, R. Wille, P. Niemann, and R. Drechsler, "An energy-aware model for the logic synthesis of quantum-dot cellular automata," *Trans. Comp.-Aided Des. Integ. Cir. Sys.*, vol. 37, pp. 3031–3041, Dec. 2018.
179. K. Walus, T. Dysart, G. Jullien, and R. Budiman, "Qcadesigner: a rapid design and simulation tool for quantum-dot cellular automata," *IEEE Transactions on Nanotechnology*, vol. 3, no. 1, pp. 26–31, 2004.
180. S. C. Brailsford, C. N. Potts, and B. M. Smith, "Constraint satisfaction problems: Algorithms and applications," *European Journal of Operational Research*, vol. 119, no. 3, pp. 557–581, 1999.
181. S. A. Cook, "The complexity of theorem-proving procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, (New York, NY, USA), p. 151–158, Association for Computing Machinery, 1971.
182. I. Rivin, I. Vardi, and P. Zimmermann, "The n-queens problem," *The American Mathematical Monthly*, vol. 101, no. 7, pp. 629–639, 1994.
183. F. Ulrich-Oltean, *Learning SAT Encodings for Constraint Satisfaction Problems*. PhD thesis, University of York, 2023.
184. N. Tamura, A. Taga, S. Kitagawa, and M. Banbara, "Compiling finite linear csp into sat," *Constraints*, vol. 14, no. 2, pp. 254–272, 2009.
185. L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.
186. R. Jozsa, "Searching in grover's algorithm," *arXiv preprint quant-ph/9901021*, 1999.
187. G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," *arXiv preprint quant-ph/0005055*, 2000.
188. A. Shukla and P. Vedula, "An efficient quantum algorithm for preparation of uniform quantum superposition states," *Quantum Information Processing*, vol. 23, p. 38, Jan. 2024.

189. M. K. Vijayan, A. P. Lund, and P. P. Rohde, "A robust w-state encoding for linear quantum optics," *Quantum*, vol. 4, p. 303, 2020.
190. W. Dür, G. Vidal, and J. I. Cirac, "Three qubits can be entangled in two inequivalent ways," *Physical Review A*, vol. 62, no. 6, p. 062314, 2000.
191. R. Jha, D. Das, A. Dash, S. Jayaraman, B. K. Behera, and P. K. Panigrahi, "A novel quantum n-queens solver algorithm and its simulation and application to satellite communication using ibm quantum experience," *arXiv: Quantum Physics*, 2018.
192. S. G. S, P. Joshi, A. Barui, and P. K. Panigrahi, "A Quantum Approach to solve N-Queens Problem," 12 2023.
193. M. Vahabi, P. Lyakhov, A. N. Bahar, A. Otsuki, and K. A. Wahid, "Novel reversible comparator design in quantum dot-cellular automata with power dissipation analysis," *Applied Sciences*, vol. 12, no. 15, 2022.
194. L. M. K. Vandersypen, M. Steffen, G. Breyta, C. S. Yannoni, M. H. Sherwood, and I. L. Chuang, "Experimental realization of shor's quantum factoring algorithm using nuclear magnetic resonance," *Nature*, vol. 414, p. 883–887, Dec. 2001.
195. E. Martín-López, A. Laing, T. Lawson, R. Alvarez, X.-Q. Zhou, and J. L. O'Brien, "Experimental realization of shor's quantum factoring algorithm using qubit recycling," *Nature Photonics*, vol. 6, p. 773–776, Oct. 2012.
196. T. Monz, D. Nigg, E. A. Martinez, M. F. Brandl, P. Schindler, R. Rines, S. X. Wang, I. L. Chuang, and R. Blatt, "Realization of a scalable shor algorithm," *Science*, vol. 351, p. 1068–1070, Mar. 2016.
197. P. K. Biswas, A. N. Bahar, M. A. Habib, and M. Abdullah-Al-Shafi, "Efficient design of feynman and toffoli gate in quantum dot cellular automata (qca) with energy dissipation analysis," *Nanoscience and Nanotechnology*, vol. 7, no. 2, pp. 27–33, 2017.
198. R. Cowburn and M. Welland, "Room temperature magnetic quantum cellular automata," *Science*, vol. 287, no. 5457, pp. 1466–1468, 2000.
199. L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing (STOC '96)*, pp. 212–219, 1996.
200. D. Deutsch and R. Jozsa, "Rapid solution of problems by quantum computation," *Proceedings of the Royal Society A*, vol. 439, no. 1907, pp. 553–558, 1992.